Prof. Santiago Badia ©

Lecture Notes on

Numerical Methods for Partial Differential Equations

Version published 27/07/26

Preface

I have created these notes for the unit MTH3340 *Numerical Methods for Partial Differential Equations*. In these lecture notes, I provide the mathematical background of finite element methods. Finite element methods rely on weak forms of partial differential equations grounded on functional analysis. Many students will likely not have much (or any) background in functional analysis. One option would be to skip these concepts and go straight to the discrete problem without discussing well-posedness at the continuous level. However, functional spaces cannot be skipped if we want to understand the convergence properties of these methods. So, I have decided to start this presentation with a *very* short and simple introduction to variational methods and functional analysis. This chapter should be accessible to students with some mathematical background. Next, I introduce the Galerkin method, analyse its error, and apply it to spectral and finite element methods in one dimension. The following chapter extends the finite element method to multiple dimensions, presenting the machinery needed in practical implementations. The last chapters go beyond coercive elliptic problems. I discuss the difficulties posed by convection-dominated problems and how stabilised finite element methods overcome them, the approximation of parabolic (time-dependent) problems via the method of lines, and, finally, how to solve efficiently the linear systems arising from all these discretisations. I have also included some *advanced* material that is not part of the unit assessment. I have marked this material with a section title in this colour. Interested students who want to explore the mathematical theory of the method in more detail can read these sections. Besides, some proofs of theorems have also been flagged as *(advanced)*, and students do not need to understand them.

We will combine the theory with computational tutorials. They can be found in this `Github repository`. I provide instructions for a local installation of the tools used in the tutorial. The tutorials use the `Gridap` software library developed by co-workers and me. This library provides tools for the numerical approximation of partial differential equations using mesh-based techniques (finite element methods in general). The tutorials and the library are written in `Julia`, a programming language that combines the expressiveness of dynamic languages like `Python` and the performance of static languages like `C++` or `FORTRAN`. `Julia` is an exciting fresh language for numerical implementations in computational and data science.

Computational mathematics is a fantastic research field at the core of computational science. It is the third pillar of science, together with experiments and theory. Finite element methods (in a broad sense) are at the core of most state-of-the-art research on numerical partial differential equations. They have applications in almost any scientific discipline since many mathematical models are multi-dimensional differential equations on complex geometries. This unit provides a broad picture of the field. Please contact me if you want more information about these techniques or research topics in this field.

It has been a formidable task to select suitable topics, decide how to present them, typeset the equations in LaTeX, and create the illustrations, tutorials, exercises, and computational material. I am positive I have made some mistakes in the process. Please inform me of typos you find and any other feedback related to this material. It will help me to improve these notes for future students. In any case, I will continuously go through this material, fix and improve things, and update it on Moodle accordingly. Please always use the latest version of the lecture notes indicated on the front page.

Santiago Badia

Melbourne, July 15, 2022

Revised in July 2026

Contents

1 Mathematical models **6**
1.1 Introduction . . . 6
1.1.1 Minimisation problem . . . 6
1.1.2 Variational formulation . . . 7
1.1.3 Differential equation . . . 8
1.2 Abstract setting . . . 9
1.2.1 Abstract minimisation . . . 10
1.2.2 Compactness . . . 14
1.2.3 Non-existence in infinite dimensions . . . 16
1.2.4 Abstract variational form . . . 17
1.2.5 Well-posedness . . . 18
1.3 Functional spaces . . . 19
1.3.1 The $L^2(\Omega)$ space . . . 19
1.3.2 Well-posedness in Hilbert spaces . . . 21
1.3.3 The direct method of the calculus of variations . . . 23
1.3.4 The $H^1(\Omega)$ space . . . 26
1.3.5 $H^1(\Omega)$ by completion . . . 28
1.3.6 Roadmap . . . 28
1.4 A multidimensional problem . . . 28
1.5 Boundary value problem . . . 31
1.6 Boundary conditions . . . 32
1.7 Further topics . . . 33
1.8 Tutorial . . . 33

2 Discretisation **35**
2.1 Galerkin method . . . 35
2.2 Spectral Galerkin methods . . . 37
2.3 Finite element methods . . . 38
2.3.1 Computing the entries of the linear system . . . 40
2.4 Error analysis of the Galerkin method . . . 42
2.4.1 Errors in the energy norm . . . 42
2.5 Approximation theory . . . 43
2.5.1 Higher order methods . . . 46
2.6 Tutorial . . . 47

3 d-dimensional finite elements **49**
3.1 The boundary value problem in weak form . . . 49
3.2 Space discretization with finite elements . . . 50
3.3 The finite element in reference and physical space . . . 50
3.3.1 The reference finite element . . . 51
3.3.2 From reference to physical spaces . . . 51
3.4 Construction of polynomial spaces . . . 54
3.4.1 Local finite element space in cubes . . . 55
3.4.2 Local finite element space in simplices . . . 59
3.4.3 Shape functions in the physical space . . . 59

3.5 Construction of the shape functions basis . . . 62
3.6 Global finite element space and conformity . . . 62
3.7 Interpolant . . . 64
3.8 Assembly and linear system . . . 64
3.9 Numerical integration . . . 65
3.10 Grad-conforming finite elements for vector fields . . . 67
3.11 Cartesian product of finite elements for multi-field problems . . . 68
3.12 Approximation properties . . . 68
3.13 Tutorials . . . 69

4 Convection-dominated problems **70**
4.1 The convection-diffusion problem . . . 70
4.2 Weak formulation and stability . . . 71
4.3 Galerkin approximation . . . 72
4.4 Artificial diffusion . . . 74
4.4.1 Error analysis for artificial diffusion . . . 74
4.5 Streamline diffusion . . . 76
4.6 SUPG stabilisation . . . 77
4.6.1 Error analysis for SUPG . . . 78
4.7 Tutorials . . . 80

5 Parabolic equations **82**
5.1 Introduction . . . 82
5.2 The heat equation . . . 82
5.3 Space discretisation using finite elements (FEs) . . . 83
5.4 Time discretisation . . . 86
5.5 Total error . . . 89
5.6 Tutorial . . . 89

6 Solving the linear system **91**
6.1 Introduction . . . 91
6.2 Preliminaries . . . 91
6.2.1 Eigenvalues and eigenvectors . . . 91
6.3 Finite Element Matrices . . . 93
6.3.1 Mass matrix . . . 93
6.3.2 Laplacian matrix . . . 94
6.4 Direct methods . . . 95
6.5 Iterative solvers . . . 96
6.5.1 Richardson method . . . 96
6.5.2 Steepest descent method . . . 98
6.5.3 Conjugate Gradient . . . 98
6.5.4 Preconditioned Conjugate Gradient . . . 99

7 Neural networks for PDE approximation **101**
7.1 The limits of linear approximation . . . 101
7.1.1 Adaptive finite elements . . . 101
7.1.2 Kolmogorov widths . . . 103
7.2 Neural networks . . . 104
7.2.1 Neural networks can emulate finite elements . . . 104
7.2.2 Higher dimensions and the need for depth . . . 106
7.3 Training . . . 108
7.4 Physics-informed neural networks . . . 109
7.4.1 The integration problem . . . 110

7.4.2 The boundary condition problem . . . 111
7.4.3 An honest assessment . . . 111
7.5 Variational losses . . . 112
7.5.1 A one-parameter tour of the loss landscape . . . 112
7.6 Inverse problems . . . 113
7.6.1 PDE-constrained optimisation . . . 113
7.6.2 Neural network parametrisations . . . 114
7.7 Summary . . . 114
7.8 Tutorial . . . 115

1 Mathematical models

1.1 Introduction

In this chapter, we will start with the statement of some mathematical models in physics that can be defined as the minimisation of a potential (energy). Then, using the calculus of variations, we will end up with its corresponding variational form. Next, the existence and uniqueness of solutions, together with the equivalence between the minimisation and variational statements, will be provided. Finally, under extra regularity assumptions, we will relate these formulations with the standard *strong* partial differential equation form. We will also discuss how we can go in the opposite direction, starting from a partial differential equation in strong form and ending up with a *weak* or variational equation or minimisation problem.

The existence issue in finite-dimensional problems is straightforward. However, for infinite-dimensional function spaces, it involves advanced mathematics; more specifically, it involves *functional analysis*. We will nevertheless try to reduce the exposition to the minimum while keeping a rigorous presentation. In this sense, we will provide some intuitive examples showing how important it is to choose suitable functional spaces to get well-posed minimisation and variational problems. We will introduce the concepts of Banach and Hilbert spaces, Cauchy sequences, completeness, and *maximal function space* with respect to a given norm. With this idea, we can naturally define some useful Lebesgue and Sobolev spaces and provide some critical properties of these spaces (without proof).

As we will see in the next chapter, finite element and (some) spectral methods rely on weak formulations of partial differential equations. Presenting these numerical methods without a thorough introduction to these problems and their relation with partial differential equations in the classical sense is not very satisfactory for maths-oriented students. This is a big difference compared to finite difference (and other collocation) methods that instead work with the strong form.

1.1.1 Minimisation problem

Let us consider a mathematical model representing a straight bar under traction (i.e., a bar in which external forces can only be in the longitudinal directions); see Figure 1.1. In this case, the physical domain that represents the bar is an interval $[a, b] \subset \mathbb{R}$ and we denote the longitudinal (or tangential) displacement of the bar particles in the original configuration by $u(x)$. Let us consider that the displacements of the bar particles are very small compared to the bar length $|b - a|$, i.e., the so-called *small displacement* assumption. Under these circumstances, the elastic energy stored by the bar is expressed by Hooke's law as

$$E(u) = \frac{1}{2}\int_a^b \kappa(x)u'(x)^2 \mathrm{d}x,$$

where $\kappa(x)$ is the Young's modulus of the bar material, which is assumed to be in the *elastic* regime. u' represents the derivative of u with respect to x. Let us assume the following *boundary conditions*, i.e., a prescribed longitudinal displacement at the end-points:

$$u(a) = u_a, \qquad u(b) = u_b, \tag{1.1}$$

where $u_a, u_b \in \mathbb{R}$, and let us consider a *forcing* term $f : [a, b] \to \mathbb{R}$. The values u_a, u_b and the function f are the *data* of the problem. Due to the *minimum energy principle*, the displacement of the bar is the function that minimises the following functional, the sum of the elastic energy and the potential energy of the external load:

$$J(u) \doteq \int_a^b (\frac{1}{2}\kappa(x)u'(x)^2 - f(x)u(x))\mathrm{d}x. \tag{1.2}$$

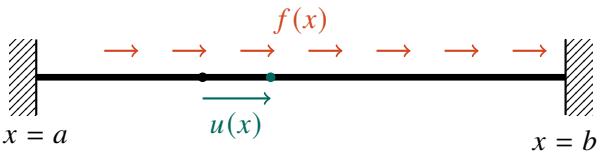


Figure 1.1: A straight bar under traction. The physical domain is the interval $[a, b]$, the longitudinal load $f(x)$ acts along the bar axis, and $u(x)$ is the longitudinal displacement of the particle originally located at x. The displacement is prescribed at both end-points.

An elastic cord with a perpendicular load $f(x)$ under the small displacement assumption satisfies the same problem; in this case $u(x)$ represents the displacement in the perpendicular direction. An analogous minimisation problem is obtained when modelling heat conduction in a bar. In this case, $u(x)$ is the temperature, $\kappa(x)$ is the heat conductivity, and $f(x)$ is the *source* term.

In any case, we have not yet stated the minimisation problem for the functional (1.2), since we have not yet defined the set of functions in which we seek the minimiser. Since we need to evaluate first derivatives of the function in (1.2), it is natural to consider that $u(x) \in C^1([a, b])$. Thus, the solution $u(x)$ reads as:

$$u \doteq \operatorname*{argmin}_{v \in C^1([a,b]) \text{ satisfying (1.1)}} J(v). \tag{1.3}$$

Thus, using physical principles (energy minimisation), we can state physical problems mathematically as minimisation problems in *infinite dimensional* spaces of functions.

1.1.2 Variational formulation

In the previous subsection, we have ended up with a minimisation problem on an infinite-dimensional space of functions. In particular, we are interested in *vector spaces* of functions that are closed under addition and multiplication by a real number.

Definition 1.1.1: Vector space

A set V of real-valued functions on a domain $\Omega \subset \mathbb{R}^d$ (where d denotes the space dimension) is a vector space if, endowed with the pointwise operations

$$(u + v)(x) \doteq u(x) + v(x), \quad (\alpha \cdot u)(x) \doteq \alpha u(x), \quad \forall x \in \Omega,$$

it is closed under these operations, i.e., $u + v \in V$ and $\alpha u \in V$ for any $u,\ v \in V$ and $\alpha \in \mathbb{R}$.

Using *calculus of variations*, we can state the minimisation problem (1.3) in a *variational* form. Assuming the existence of a global minimiser $u(x)$ for (1.3), we can now consider the variation of the functional J at u with respect to a variation $v \in C_0^1([a, b])$ times $\alpha \in \mathbb{R}$. $C_0^1([a, b])$ is the subspace of functions $C^1([a, b])$ that vanish at the end-points, i.e.,

$$C_0^1([a, b]) \doteq \{v \in C^1([a, b]) \ : \ v(a) = v(b) = 0\}.$$

Without the zero boundary conditions for the perturbation, the perturbed function $u + \alpha v$ would not satisfy the boundary conditions (1.1). Since u is a global minimiser, it naturally holds

$$J(u) \leq \Phi_v(\alpha) \doteq J(u + \alpha v) \ \ \forall v \in C_0^1([a, b]),\ \forall \alpha \in \mathbb{R}.$$

Thus, $\Phi_v(\alpha)$ has a global minimum at 0. If Φ_v is differentiable, $\Phi_v'(0) = 0$. Since u is the minimum energy configuration, any perturbation of u cannot produce a decrease of energy.

The derivative of Φ reads

$$\Phi_v'(0) = \lim_{\alpha \to 0} \frac{J(u + \alpha v) - J(u)}{\alpha}.$$

Such derivative is the so-called *directional derivative* of J at u in the direction v. In our case, expanding the square in $J(u+\alpha v)$, we obtain

$$\Phi_v(\alpha) = J(u) + \alpha \int_a^b (\kappa(x)u'(x)v'(x) - f(x)v(x))\mathrm{d}x + \frac{\alpha^2}{2}\int_a^b \kappa(x)v'(x)^2\mathrm{d}x,$$

and thus $\Phi_v'(0) = \int_a^b (\kappa u'v' - fv)\mathrm{d}x$. Enforcing $\Phi_v'(0)$ to be zero for any perturbation $v \in C_0^1([a,b])$, we get that

$$\int_a^b (\kappa(x)u'(x)v'(x) - f(x)v(x))\mathrm{d}x = 0 \ \ \forall v \in C_0^1([a,b]). \tag{1.4}$$

This expression is the *weak or variational form* of the physical problem at hand. In statics, this statement of the elastic problems above is called the *principle of virtual work*, which means that any perturbation of the equilibrium configuration requires an energy supply to the system.

So far, we have only proved one implication. A global minimiser of J, if it exists, solves the variational problem (1.4). The converse also holds, and the expansion of Φ_v above contains all we need to check it. The second derivative of Φ_v,

$$\Phi_v''(\alpha) = \int_a^b \kappa(x)v'(x)^2\mathrm{d}x \geq 0 \ \ \forall \alpha \in \mathbb{R},$$

is a non-negative constant, since $\kappa > 0$; each Φ_v is a convex parabola in α. Moreover, the second derivative is strictly positive for any $v \in C_0^1([a,b])$ with $v \not\equiv 0$. If $\int_a^b \kappa(v')^2\mathrm{d}x = 0$, then $v' \equiv 0$, so v is constant, and the zero boundary values leave $v \equiv 0$ as the only option. Let now u be a solution of the variational problem (1.4). The linear term of the expansion vanishes, and

$$J(u+\alpha v) - J(u) = \frac{\alpha^2}{2}\int_a^b \kappa(x)v'(x)^2\mathrm{d}x > 0 \ \ \forall v \in C_0^1([a,b]),\ v \not\equiv 0,\ \forall \alpha \neq 0.$$

Any admissible competitor $w \neq u$ can be written as $w = u + v$ with $v \doteq w - u \in C_0^1([a,b])$, so u is the unique global minimiser of J. Solving the minimisation problem and solving the variational problem are *equivalent*, and we can move freely between the two formulations.

1.1.3 Differential equation

We would like to relate the variational and minimisation formulations above with standard differential equations. In this process, we will observe that, to reach this form, we will need to make some additional regularity assumptions over the solution.

Before stating the key auxiliary result, let us introduce the space of *test functions* that we will use at several points of these notes.

Definition 1.1.2: The space $C_0^\infty(\Omega)$

Given a bounded open set $\Omega \subset \mathbb{R}^d$, the space $C_0^\infty(\Omega)$ is the space of infinitely differentiable functions $v : \Omega \to \mathbb{R}$ that vanish *near* the boundary. I.e., for every $v \in C_0^\infty(\Omega)$ there exists $\delta > 0$ (depending on v) such that $v(x) = 0$ for any $x \in \Omega$ at a distance smaller than δ from $\partial\Omega$.[1] In one dimension, for $\Omega = (a,b)$, the condition reads $v \equiv 0$ on $(a, a+\delta]$ and $[b-\delta, b)$.

[1] The closure of the set of points in which v is non-zero is called the *support* of v. In the literature, the condition in this definition is expressed by saying that the support of v is a *compact* subset of Ω, and $C_0^\infty(\Omega)$ (also denoted by $C_c^\infty(\Omega)$) is described as the space of smooth functions with *compact support*. We introduce the concept of compactness in the advanced Section 1.2.2.

Remark 1.1.3: Vanishing at vs. near the boundary

We note that a function in $C_0^\infty(\Omega)$ vanishes identically, together with all its derivatives, in a neighbourhood of $\partial\Omega$. Note the difference with respect to spaces like $C_0^1([a,b])$ above, in which the subscript 0 only enforces zero *values* at the end-points. E.g., the function $\sin(\pi(x-a)/(b-a))$ vanishes at the end-points of $[a,b]$ but does not vanish near them; on the contrary, a smooth *bump* function that is non-zero only in an interior subinterval belongs to $C_0^\infty((a,b))$. Vanishing near the boundary is what makes these test functions so convenient. Integration by parts can be applied any number of times without ever producing boundary terms.

Lemma 1.1.4: Fundamental lemma of calculus of variations

If a function $f \in C^0([a,b])$ *is such that*

$$\int_a^b f(x)v(x)\mathrm{d}x = 0,\ \forall v \in C_0^\infty((a,b)),$$

then $f \equiv 0$.

Proof. If $f(x_0) > 0$ at some $x_0 \in (a,b)$, by continuity $f > f(x_0)/2$ on a neighbourhood of x_0. Testing against a smooth non-negative *bump* function v supported in that neighbourhood with $v(x_0) = 1$ yields $\int_a^b f v\,\mathrm{d}x > 0$, a contradiction. The case $f(x_0) < 0$ is analogous. □

Let us assume that the solution u of the variational formulation (1.4) is in $C^2([a,b])$, $\kappa \in C^1([a,b])$, and $f \in C^0([a,b])$. Using integration by parts, we get for any $v \in C_0^1([a,b])$

$$\int_a^b \kappa(x)u'(x)v'(x)\mathrm{d}x - \int_a^b f(x)v(x)\mathrm{d}x = -\int_a^b ((\kappa u')' + f(x))v(x)\mathrm{d}x.$$

Lemma 1.1.4 leads to

$$-(\kappa u')' = f \quad \text{in } [a,b],\ \ u(a) = u_a,\ u(b) = u_b. \tag{1.5}$$

As a result, if the solution of (1.4) with the boundary conditions in (1.1) is in $C^2([a,b])$, it satisfies the two-point boundary value problem (1.5).

1.2 Abstract setting

This section considers an abstract setting for minimisation and variational problems, including the examples above. Under regularity assumptions, we show how to find their corresponding boundary value problem. We consider minimisation problems that are related to an energy functional J on a space of functions from $\Omega \subset \mathbb{R}^d$ to $\mathbb{R}$, where d is the space dimension, e.g., $d = 2, 3$. Ω is assumed to be bounded, i.e.,

$$\mathrm{diam}(\Omega) = \sup_{x,y\in\Omega} \|x - y\| < \infty,$$

and its boundary $\partial\Omega$ is piecewise smooth, i.e. it can be expressed as a smooth diffeomorphism from a reference polyhedron boundary.

Definition 1.2.1: (Bi)linear forms

Given a vector space V over the field of scalars $\mathbb{R}$, a form $\ell : V \to \mathbb{R}$ is linear if it satisfies:

$$\ell(u+v) = \ell(u) + \ell(v),\ \forall u,\ v \in V,\ \ \ell(\alpha u) = \alpha\ell(u)\ \forall u \in V,\ \alpha \in \mathbb{R}.$$

A form $a : V \times V \to \mathbb{R}$ is a bilinear form if it is linear with respect to each of its two arguments separately.

1.2.1 Abstract minimisation

We are interested in an important class of minimisation problems that covers the abovementioned examples.

Definition 1.2.2: Quadratic minimisation problem

Let us consider a functional $J : V \to \mathbb{R}$ on a vector space V that can be expressed as

$$J(v) \doteq \frac{1}{2}a(v,v) - \ell(v) + c$$

for a symmetric bilinear form $a : V \times V \to \mathbb{R}$, a linear form $\ell : V \to \mathbb{R}$ and $c \in \mathbb{R}$. The problem

$$u = \operatorname*{argmin}_{v \in V} J(v)$$

is called a *quadratic minimisation problem* on V.

We note that the space of functions with non-homogeneous boundary conditions is not a vector space but an affine space. The minimisation in an affine space can easily be transformed into a minimisation in a vector space using the *offset function* method. We consider a vector space of functions V_0 and an affine space V that can be expressed as $u_0 + V_0$, where u_0 is the so-called *offset function*. In the examples in Section 1.1, u_0 is a function in $C^1([a,b])$ that satisfies the boundary conditions (1.1) (while keeping a desired level of smoothness), whereas V_0 is a space of functions with homogeneous (zero) boundary conditions, i.e., $V_0 = C_0^1([a,b])$. Given J, V_0 and u_0, the minimisation problem in V can be expressed as follows. We have

$$\begin{aligned} J(v + u_0) &= \frac{1}{2}a(v + u_0, v + u_0) - \ell(v + u_0) + c \\ &= \frac{1}{2}a(v,v) + a(v,u_0) - \ell(v) + \frac{1}{2}a(u_0,u_0) - \ell(u_0) + c \doteq \tilde{J}(v) \end{aligned}$$

where $\tilde{J}$ is a quadratic functional too. Thus,

$$\operatorname*{argmin}_{u \in u_0 + V_0} J(u) = u_0 + \operatorname*{argmin}_{v \in V_0} \tilde{J}(v).$$

As a result, we can restrict to quadratic minimisation problems on vector spaces without loss of generality. Now, let us consider the well-posedness of the quadratic minimisation problem.

Definition 1.2.3: Positive definiteness

A symmetric bilinear form $a : V_0 \times V_0 \to \mathbb{R}$ on a real vector space V_0 is *positive semi-definite* if

$$a(u,u) \geq 0 \quad \forall u \in V_0.$$

Moreover, it is *positive definite* if

$$a(u,u) > 0 \quad \forall u \in V_0 \setminus \{0\}.$$

Lemma 1.2.4: Necessary condition for existence of a global minimum

If the quadratic minimisation problem in Definition 1.2.2 has a solution, then its bilinear form $a : V_0 \times V_0 \to \mathbb{R}$ must be positive semi-definite.

Proof. If the bilinear form is not positive semi-definite, we can pick a $u \in V_0$ such that $a(u,u) < 0$. Thus, $J(\alpha u) = 1/2\alpha^2 a(u,u) - \alpha\ell(u) + c$ and $\lim_{\alpha \to \infty} J(\alpha u) = -\infty$. □

Lemma 1.2.5: Sufficient condition for uniqueness of the global minimum

If the quadratic minimisation problem in Definition 1.2.2 has a solution and its bilinear form a is positive definite, then the solution is unique.

Proof. Let us assume that there exist two solutions $u,\ v \in V_0$ such that $u \neq v$. $\Phi(\alpha) \doteq J(\alpha u + (1-\alpha)v)$ has two distinct global minima at $\alpha = 0$ and $\alpha = 1$. Besides, $\Phi(\alpha) = \alpha^2/2a(u-v, u-v)+$ lower order terms and $a(u-v, u-v) > 0$. Thus, Φ is a non-degenerate parabola that opens up and can only have a minimum at its vertex. It proves the result by contradiction. □

If a is positive semi-definite (but not positive definite), solutions are not unique in general; two solutions can only differ by an element of the kernel of a, i.e., two solutions $u,\ v \in V_0$ must satisfy $a(u-v, u-v) = 0$.

Let us introduce some additional concepts that will allow us to prove more necessary conditions for existence.

Definition 1.2.6: Norm on a vector space

A norm $\|\cdot\|$ on a vector space V is a map $\|\cdot\| : V \to \mathbb{R}_+$ such that

- Positive definiteness: $\|v\| = 0 \iff v = 0$, for $v \in V$,
- Absolute homogeneity: $\|\alpha v\| = |\alpha|\|v\|,\ \forall \alpha \in \mathbb{R},\ \forall v \in V$,
- Triangle inequality: $\|v + w\| \leq \|v\| + \|w\|,\ \forall v,\ w \in V$.

Definition 1.2.7: Energy norm

A symmetric positive definite bilinear form $a : V \times V \to \mathbb{R}$ induces the *energy norm*

$$\|u\|_a \doteq a(u,u)^{1/2}.$$

The norm properties of $\|\cdot\|_a$ are not entirely obvious; the triangle inequality follows from the next classical result, which we will invoke repeatedly in these notes. We also note that the definiteness required in Definition 1.2.6 is exactly the positive definiteness of a, and homogeneity follows from bilinearity. Thus, $\|\cdot\|_a$ is a norm *if and only if* the symmetric form a is positive definite; no further assumptions are needed.

Lemma 1.2.8: Cauchy–Schwarz inequality

Let $a : V \times V \to \mathbb{R}$ be a symmetric positive semi-definite bilinear form on a real vector space V. Then,

$$|a(u,v)| \leq a(u,u)^{1/2}\, a(v,v)^{1/2}, \qquad \forall u,\ v \in V.$$

Proof. For any $t \in \mathbb{R}$, positive semi-definiteness and bilinearity yield

$$0 \leq a(u+tv, u+tv) = a(u,u) + 2t\, a(u,v) + t^2 a(v,v).$$

The right-hand side is a quadratic polynomial in t that never becomes negative, so its discriminant must satisfy $4a(u,v)^2 - 4a(u,u)a(v,v) \leq 0$, which proves the result. □

Definition 1.2.9: Continuity of (bi)linear forms

Given a vector space V with norm $\|\cdot\|$, a linear form $\ell : V \to \mathbb{R}$ is *continuous* or *bounded* if

$$\exists C > 0 \ :\ |\ell(v)| \leq C\|v\|\ \forall v \in V.$$

A bilinear form $a : V \times V \to \mathbb{R}$ is *continuous* if it is bounded, that is if

$$\exists K > 0 \ : \ |a(u,v)| \leq K\|u\|\|v\|, \ \forall u, \ v \in V.$$

We note that continuity of the bilinear form that provides the energy norm can readily be checked for $K = 1$ by using the Cauchy-Schwarz inequality in Lemma 1.2.8.

Continuity bounds the bilinear form from above. The companion notion bounds it from below.

Definition 1.2.10: Coercivity

Given a vector space V with norm $\|\cdot\|$, a bilinear form $a : V \times V \to \mathbb{R}$ is *coercive* if

$$\exists \alpha > 0 \ : \ a(v,v) \geq \alpha\|v\|^2 \ \ \forall v \in V.$$

A functional $J : V \to \mathbb{R}$ is *coercive* if $J(v) \to \infty$ as $\|v\| \to \infty$.

Coercivity is a statement about how the energy norm compares with the norm of the space. Two norms on V are *equivalent* if each one bounds the other up to a constant; equivalent norms have the same convergent and the same Cauchy sequences, so they are interchangeable for all the purposes of these notes.

Lemma 1.2.11: Coercivity and norm equivalence

Let V be a vector space with norm $\|\cdot\|$ and let $a : V \times V \to \mathbb{R}$ be a symmetric positive semi-definite bilinear form. Then, a is continuous and coercive with respect to $\|\cdot\|$ if and only if there exist constants $0 < \alpha \leq K$ such that

$$\alpha\|v\|^2 \leq a(v,v) \leq K\|v\|^2 \ \ \forall v \in V.$$

In this case, a is positive definite and the energy norm $\|\cdot\|_a$ is equivalent to $\|\cdot\|$.

Proof. The lower bound is the definition of coercivity, and taking $u = v$ in the definition of continuity gives the upper bound. Conversely, given the two-sided bound, coercivity holds by definition, and the Cauchy–Schwarz inequality in Lemma 1.2.8 gives $|a(u,v)| \leq a(u,u)^{1/2}a(v,v)^{1/2} \leq K\|u\|\|v\|$, i.e., continuity. Positive definiteness follows from $a(v,v) \geq \alpha\|v\|^2 > 0$ for $v \neq 0$, and the norm equivalence is the two-sided bound after taking square roots. □

In words, coercivity says that the energy norm is *at least as strong* as the norm of the space, and continuity says that it is *at most as strong*. In particular, coercivity with respect to any norm implies positive definiteness, so it is a practical, quantitative criterion for definiteness. The converse is false; positive definiteness alone does not make the energy norm comparable to the norm of the space, as the following example shows.

Example 1.2.12: A positive definite form that is not coercive

Let $V \doteq C^1_0([0,1])$, endowed with the norm $\|v\|^2 \doteq \int_0^1 (v^2 + (v')^2)\,\mathrm{d}x$, which measures the function *and* its derivative (we will meet it again as the H^1 norm), and let $a(u,v) \doteq \int_0^1 uv\,\mathrm{d}x$. The form a is positive definite, so $\|\cdot\|_a$ is a norm, but a strictly weaker one. For $v_n(x) \doteq \frac{1}{n}\sin(n\pi x)$, a direct computation gives

$$a(v_n, v_n) = \frac{1}{2n^2} \to 0, \qquad \|v_n\|^2 = \frac{1}{2n^2} + \frac{\pi^2}{2} \geq \frac{\pi^2}{2},$$

so no $\alpha > 0$ can satisfy $a(v_n, v_n) \geq \alpha\|v_n\|^2$; the form is not coercive. The oscillations of v_n are invisible to the energy norm but not to the norm of V.

This weakness is not a curiosity; it is precisely the pathology of Example 1.2.18, which involves the same

bilinear form. When the energy norm is strictly weaker than the norm of the space, minimising sequences that are tame in the energy norm can be wild in the norm of V, and existence in V can fail; we will come back to this point when we discuss completions. Coercivity with respect to the right norm rules this out. E.g., the first Poincaré–Friedrichs inequality (Theorem 1.4.1) will provide the coercivity of the Dirichlet bilinear form with respect to the $H^1(\Omega)$ norm, so that $|\cdot|_{H^1(\Omega)}$ is not only a norm on $H^1_0(\Omega)$ (Corollary 1.4.3) but a norm equivalent to the full $H^1(\Omega)$ norm. Finally, we note that a symmetric positive definite bilinear form is trivially coercive and continuous with respect to its own energy norm, with the best constants $\alpha = K = 1$. Coercivity is a genuine requirement only when the norm of the space is fixed beforehand and differs from the energy norm, which will be the situation in the next chapters.

The notions of coercivity for bilinear forms and for functionals are also related. For a coercive bilinear form and a continuous linear form, the quadratic functional of Definition 1.2.2 is coercive, since $J(v) \geq \frac{\alpha}{2}\|v\|^2 - C\|v\| \to \infty$ as $\|v\| \to \infty$.

It is essential to prove that the potential J is bounded from below to have a well-posed minimisation problem.

Lemma 1.2.13: Boundedness from below

The quadratic functional J in Definition 1.2.2 with a positive definite bilinear form a is bounded from below *in V_0 if and only if the linear form ℓ is continuous in V_0 with respect to the energy norm $\|\cdot\|_a$.*

Proof. If a is positive definite and ℓ is continuous, using the generalised Young's inequality

$$ab \leq \frac{a^2}{2\epsilon} + \frac{\epsilon b^2}{2}, \ a, \ b \in \mathbb{R}, \ \epsilon > 0,$$

we readily get

$$J(u) = 1/2a(u,u) - \ell(u) \geq 1/2\|u\|_a^2 - C\|u\|_a \geq -1/2C^2$$

for any u in V_0.

In the other direction, let us assume that J is bounded from below and ℓ is not continuous. For every $n \in \mathbb{N}$, we can pick $u_n \in V_0$ such that

$$\ell(u_n) \geq n\|u_n\|_a.$$

By re-scaling $u_n \leftarrow \frac{u_n}{\|u_n\|_a}$ we can assume that $\|u_n\|_a = 1$, thus

$$J(u_n) \leq 1/2 - n \to -\infty, \text{ as } n \to \infty.$$

Thus, J cannot be bounded below, leading to a contradiction. It proves the result. □

So far, we have found necessary conditions for existence and uniqueness of solutions. Assuming that the vector space V_0 is finite dimensional, existence can readily be proven; the quadratic minimisation functional in finite dimension is a non-degenerate paraboloid opening up, for which there is a unique global minimum at its vertex.

Theorem 1.2.14: Existence and uniqueness of a minimiser in finite dimension

If the quadratic minimisation problem Definition 1.2.2 is defined on a real vector space V_0 of finite dimension and involves a positive definite symmetric bilinear form a and a continuous linear functional ℓ, then the problem has a solution and it is unique.

Proof. If the vector space V_0 has a finite dimension, we can define an ordered basis $\{b^1, \ldots, b^N\}$ for it, and thus V_0 is isomorphic to $\mathbb{R}^N$. Moreover, since a is a symmetric positive definite bilinear form, we can orthonormalise the basis with respect to a by the Gram–Schmidt process, so that $a(b^i, b^j)$ equals 1 if $i = j$ and 0 otherwise. Every $v \in V_0$ can be written as $v = \sum_{i=1}^N \vec{v}_i b^i$ for a unique coefficient vector $\vec{v} \in \mathbb{R}^N$, and

orthonormality gives

$$\|v\|_a^2 = a(v,v) = \sum_{i,j=1}^{N} \vec{v}_i \vec{v}_j \, a(b^i, b^j) = |\vec{v}|^2,$$

where $|\cdot|$ denotes the Euclidean norm; the map $v \mapsto \vec{v}$ is an *isometric isomorphism* between V_0, endowed with the energy norm, and $\mathbb{R}^N$, endowed with the Euclidean norm. Using the bilinearity of a and the linearity of ℓ, the functional reads

$$J(v) = \frac{1}{2}|\vec{v}|^2 - \vec{f}^T\vec{v} + c, \qquad \vec{f}_i \doteq \ell(b^i).$$

Let $u \in V_0$ be the function with coefficient vector $\vec{u} \doteq \vec{f}$, i.e., $u = \sum_{i=1}^{N} \ell(b^i)\, b^i$; no matrix appears, and the solution is explicit. Completing the square, we obtain, for any $\vec{v} \in \mathbb{R}^N$,

$$J(v) - J(u) = \frac{1}{2}|\vec{v} - \vec{u}|^2 > 0, \qquad \text{unless } \vec{v} = \vec{u}.$$

Thus, u is the unique global minimiser. □

For a general, non-orthonormalised basis, the same computation leads to the linear system of equations $\boldsymbol{A}\vec{u} = \vec{f}$, where $\boldsymbol{A}_{ij} \doteq a(b^j, b^i)$ is a symmetric positive definite (thus non-singular) matrix. This is the situation encountered in practice in the following chapters, where orthonormalising the basis is not computationally affordable; existence and uniqueness then follow from the invertibility of $\boldsymbol{A}$.

The previous proof is purely algebraic. An alternative, more geometrical, proof of this theorem is discussed next, in the advanced Section 1.2.2, together with the reason why neither argument extends to infinite dimensions.

More details on the concepts in the previous proof can be found later, when we discuss finite dimensional discretisations of infinite dimensional problems. Unfortunately, sufficient conditions for existence in infinite dimensions are more elusive and are strongly related to the right choice of the vector space V_0. It must be *large enough* for existence but *small enough* for uniqueness.

1.2.2 Compactness

This section is intended for advanced readers and is not part of the unit assessment. We introduce the concept of compactness, connect it with the space of test functions in Definition 1.1.2, and provide an alternative proof of Theorem 1.2.14. We also show why such a proof cannot work in infinite dimensions, which further motivates the notion of completeness introduced in Section 1.3.

Definition 1.2.15: Compact set

Let V be a normed vector space. A set $K \subset V$ is *(sequentially) compact* if every sequence of elements of K admits a subsequence that converges to a limit that belongs to K.

Theorem 1.2.16: Heine–Borel theorem

A set $K \subset \mathbb{R}^d$ is compact if and only if it is closed and bounded.

We do not prove this classical result, but we note that both conditions are clearly necessary. An unbounded set contains sequences whose norm blows up, and no subsequence of them can converge. A non-closed set admits sequences converging to a limit outside the set, and every subsequence converges to the same (external) limit.

A useful intuition is the following. Boundedness prevents sequences from going to infinity. Closedness prevents limits from leaving the set. Compactness gives both; by the Heine–Borel theorem, boundedness and closedness are exactly what compactness means in $\mathbb{R}^d$. Sequences can neither blow up nor escape, so they must accumulate at points of the set. The top panels of Figure 1.2 depict the three situations.

In PDE theory, compactness is one of the main mechanisms that converts boundedness estimates into existence of solutions. One often cannot prove directly that a sequence, e.g., a minimising sequence, converges; compactness allows us to extract a convergent subsequence, and its limit is the candidate solution. Let us see this mechanism at work, using it to prove Theorem 1.2.14 in an alternative, more geometrical way that does not use the algebraic structure of the problem. The proof relies on the following classical result, which we state without proof.

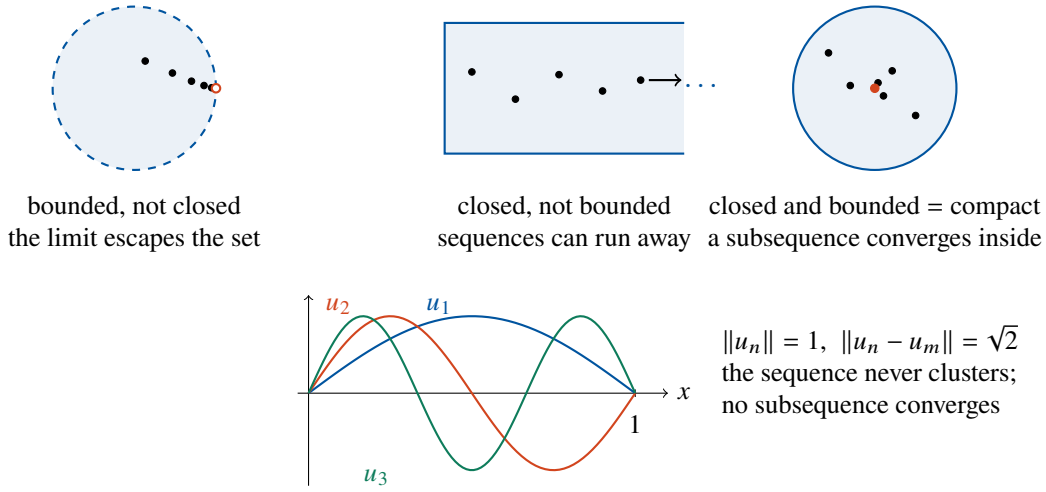


Figure 1.2: Compactness at a glance. Top left, a bounded but not closed set; a sequence can converge to a point outside the set. Top middle, a closed but unbounded set; a sequence can run away to infinity. Top right, a closed and bounded set of $\mathbb{R}^d$ is compact; every sequence accumulates at a point of the set. Bottom, in infinite dimensions closedness and boundedness are not enough; the functions $u_n = \sqrt{2}\sin(n\pi x)$ live on the unit sphere of the L^2 norm but keep mutual distance $\sqrt{2}$.

Theorem 1.2.17: Weierstrass extreme value theorem

Let K be a compact set of a normed vector space and let $f : K \to \mathbb{R}$ be continuous on K. Then, f attains its minimum and its maximum on K, i.e., there exist $x_{\min},\ x_{\max} \in K$ such that

$$f(x_{\min}) \le f(x) \le f(x_{\max}) \quad \forall x \in K.$$

Its proof is precisely a clustering argument. A sequence approaching the infimum clusters at a point of K by compactness, and the continuity of f turns the cluster point into a minimiser.

Alternative proof of Theorem 1.2.14. We prove existence; uniqueness follows from Lemma 1.2.5. As in the algebraic proof, we take a basis $\{b^1, \dots, b^N\}$ of V_0 orthonormalised with respect to a, so that $\|v\|_a = |\vec{v}|$ and we can work with the coefficient representatives $\vec{v} \in \mathbb{R}^N$, using the isometric isomorphism between V_0 and $\mathbb{R}^N$ established there. The functional J is a quadratic polynomial in $\vec{v}$, thus continuous, and the continuity of ℓ (with respect to the energy norm) reads $|\ell(v)| \le C\|v\|_a = C|\vec{v}|$. We obtain, for any $v \in V_0$,

$$J(v) = \frac{1}{2}a(v,v) - \ell(v) + c \ge \frac{1}{2}|\vec{v}|^2 - C|\vec{v}| + c \to \infty \ \text{ as } \ |\vec{v}| \to \infty.$$

This property is the coercivity of J introduced in Definition 1.2.10. Thus, there is a radius R large enough such that $J(v) > J(0)$ for any $|\vec{v}| > R$. The ball $\bar{B}_R \doteq \{\vec{v} \in \mathbb{R}^N \ : \ |\vec{v}| \le R\}$ is closed and bounded, and thus compact by the Heine–Borel theorem (Theorem 1.2.16). By the Weierstrass theorem, the continuous function $\vec{v} \mapsto J(v)$ attains its minimum over $\bar{B}_R$ at some $\vec{u} \in \bar{B}_R$, i.e., at $u \doteq \sum_{i=1}^N \vec{u}_i b^i \in V_0$. This minimum is global; for $|\vec{v}| > R$, it holds $J(v) > J(0) \ge J(u)$. Figure 1.3 depicts the geometry of the argument. □

Neither of the two proofs of Theorem 1.2.14 extends to infinite dimensions. The algebraic proof requires

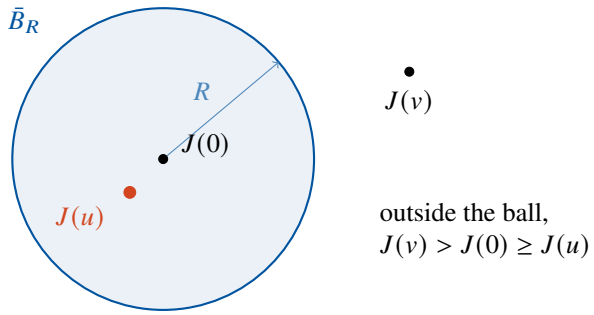


Figure 1.3: The geometry of the alternative proof. The coercivity of J confines the minimisation to the compact ball $\bar{B}_R \subset \mathbb{R}^N$ of coefficient vectors, where the Weierstrass theorem provides a minimiser u (in orange) with $J(u) \leq J(0)$; any v outside the ball satisfies $J(v) > J(0) \geq J(u)$.

a finite basis, in which the solution is written explicitly. The geometrical proof requires the compactness of closed balls, which fails in infinite dimensions. This is a genuine finite- vs infinite-dimensional divide. In a discrete space, i.e., a finite-dimensional space like those used for numerical approximation in the next chapters, a basis identifies the space with $\mathbb{R}^N$, and closed bounded sets, e.g., the closed unit ball, are always compact by the Heine–Borel theorem. Let us check this failure in the space $C^0([0,1])$ endowed with the norm

$$\|v\| \doteq \left(\int_0^1 v(x)^2 \, \mathrm{d}x \right)^{1/2}$$

(this norm is the L^2 norm introduced in Section 1.3). Consider the sequence $u_n(x) \doteq \sqrt{2}\sin(n\pi x)$, $n \in \mathbb{N}$. A direct computation shows that $\|u_n\| = 1$ for any n, whereas $\|u_n - u_m\|^2 = 2$ for $n \neq m$ (check it). All the terms of the sequence belong to the closed unit ball, but no subsequence can converge, since its terms never get closer to each other. Thus, the closed unit ball is not compact with respect to this norm, in stark contrast with the discrete case; see the bottom panel of Figure 1.2.

To close this section, let us make a different use of compactness and connect it with the space of test functions in Definition 1.1.2. The closure of the set of points in which a function v is non-zero is called the *support* of v. For $v \in C_0^\infty(\Omega)$ and a bounded Ω, the support is closed (it is a closure), it is bounded (it is a subset of a bounded set), and it stays at a distance $\delta > 0$ from $\partial\Omega$. Thus, the support is a compact subset of Ω, and $C_0^\infty(\Omega)$ is the space of smooth functions with *compact support* in Ω, which is the standard definition in the literature.

1.2.3 Non-existence in infinite dimensions

The loss of compactness identified above is not a mere technicality. In this section, we show that it makes the existence of minimisers genuinely fail in infinite dimensions, even for positive definite quadratic functionals and without any boundary condition. We exhibit a simple problem of this kind, we identify the mechanism behind the failure, and we diagnose it as a mismatch between the space of functions and the norm inherited from the bilinear form. The example also anchors the discussion of completeness in Section 1.3.

The trial space must be *large enough* to contain the minimiser; if it is too small, existence fails. The following example shows the issue.

Example 1.2.18: Non-existence of positive definite quadratic minimisation problems

Let us consider the space $V_0 \doteq C^0([-1,1])$, without boundary conditions, and the discontinuous target

$$g(x) \doteq \begin{cases} 0, & x \in [-1,0], \\ 1, & x \in (0,1]. \end{cases}$$

We seek the global minimiser of the positive definite quadratic minimisation problem with

$$a(u,v) \doteq \int_{-1}^{1} u(x)v(x)\mathrm{d}x, \qquad \ell(v) \doteq \int_{-1}^{1} g(x)v(x)\mathrm{d}x = \int_{0}^{1} v(x)\mathrm{d}x,$$

that is, of the functional

$$J(u) \doteq \int_{-1}^{1} \left(\frac{1}{2}u^2(x) - g(x)u(x)\right)\mathrm{d}x = \frac{1}{2}\int_{-1}^{1} (u(x)-g(x))^2\,\mathrm{d}x - \frac{1}{2} \geq -\frac{1}{2}.$$

The value $-\frac{1}{2}$ is the infimum of J over V_0. Indeed, the continuous ramps

$$u_n(x) \doteq \begin{cases} 0, & x \in [-1,0], \\ nx, & x \in (0,1/n], \\ 1, & x \in (1/n,1], \end{cases} \qquad \text{satisfy} \qquad J(u_n) = \frac{1}{6n} - \frac{1}{2} \longrightarrow -\frac{1}{2}$$

as $n \to \infty$ (check it). However, the infimum is not attained. If $J(u) = -\frac{1}{2}$ for some $u \in V_0$, then $\int_{-1}^{0} u^2\,\mathrm{d}x = \int_{0}^{1}(u-1)^2\,\mathrm{d}x = 0$, and a continuous non-negative integrand with vanishing integral is identically zero, as in the positive definiteness of the bilinear form a. Thus, $u \equiv 0$ on $[-1,0)$ and $u \equiv 1$ on $(0,1]$, which is impossible at $x = 0$ by continuity. There is no global minimiser. The infimum cannot be attained in the space; the minimising sequence *escapes* from it, and no boundary condition is involved.

Remark 1.2.19: A mismatch between the space and the norm

The pathology of Example 1.2.18 can be restated in the language of Section 1.2.2. The space $C^0([-1,1])$ and the norm inherited from its bilinear form, i.e., the energy norm $\|v\|_a = (\int_{-1}^{1} v^2\mathrm{d}x)^{1/2}$, are mismatched; the norm is too weak for the space. In particular, closed bounded sets of $C^0([-1,1])$ are not compact with respect to this norm, and the minimising sequence $\{u_n\}_{n\in\mathbb{N}}$ above provides a counterexample. It is bounded, since $\|u_n\|_a \leq 1$, but no subsequence can converge in $C^0([-1,1])$, because the whole sequence converges in the norm to g, which does not belong to the space. The Weierstrass mechanism is thus unavailable, and existence indeed fails.

Without compactness, the Weierstrass argument cannot be invoked, and the existence of minimisers in infinite dimensions requires a different mechanism. This is precisely the role of *completeness* in Section 1.3. In the proof of Theorem 1.3.9, the minimising sequence is shown to be a Cauchy sequence, and completeness provides the limit. In a sense, completeness replaces compactness as the property that prevents minimising sequences from escaping the space.

Remark 1.2.20: Two possible fixes

The mismatch admits two possible fixes. The first one is to consider a stronger norm for the space. This is not possible in our setting; the norm is not ours to choose, since the energy norm is determined by the minimisation problem itself. The second one is to change the space in which we minimise, enlarging it so that limits of sequences cannot leave the space. That is the whole point of what comes next (Section 1.3).

1.2.4 Abstract variational form

In this section, we provide an abstract definition of variational forms. In the most general case, a linear variational problem is stated as follows.

Definition 1.2.21: Variational problem

A variational problem reads as:

$$u \in V \ : \ a(u,v) = \ell(v), \quad \forall v \in V_0$$

where V_0 is a vector space of functions, V is an affine space of functions, $a : V \times V_0 \to \mathbb{R}$ is a bilinear form and $\ell : V_0 \to \mathbb{R}$ is a linear form.

In Definition 1.2.21, the space V in which we seek the solution is the *trial space* (affine space) and the space V_0 of admissible test functions is the *test space* (vector space). Variational problems can also be stated in terms of vector spaces by using the fact that $V = u_0 + V_0$ for an arbitrary *offset function* $u_0 \in V$. The abstract variational problem in Definition 1.2.21 can be written as:

$$\tilde{u} \in V_0 : \ a(\tilde{u},v) = \ell(v) - a(u_0,v) \ \ \forall v \in V_0, \ \ u = \tilde{u} + u_0.$$

Thus, we can restrict ourselves to variational problems in which the trial and test spaces are the same vector space without loss of generality.

Let us consider now the relationship between minimisation and variational problems. Let us restrict ourselves to quadratic minimisation problems. Let us *assume the existence* of a global minimiser u. Using the calculus of variations (as described above), and the symmetry of a, we can compute the directional derivative of the functional as

$$\begin{aligned}\lim_{\alpha\to 0} \frac{J(u+\alpha v) - J(u)}{\alpha} &= \lim_{\alpha\to 0} \frac{\alpha a(u,v) + \frac{\alpha^2}{2} a(v,v) - \alpha \ell(v)}{\alpha} \\ &= a(u,v) - \ell(v), \quad \forall v \in V_0.\end{aligned}$$

Enforcing that such derivative is equal to zero because u is a minimum of the functional, we end up with a *linear* variational problem. As a result, *the global minimiser u of a quadratic functional is the solution of a linear variational problem*.

Conversely, a linear variational problem admitting a solution u and involving a symmetric positive semi-definite bilinear form a defines a quadratic minimisation problem on V_0 with the functional $J(v) = \frac{1}{2}a(v,v) - \ell(v)$, and u is a global minimiser of J. Indeed, for any $v \in V_0$, using the variational equation and the symmetry of a,

$$J(u+v) - J(u) = a(u,v) - \ell(v) + \frac{1}{2}a(v,v) = \frac{1}{2}a(v,v) \geq 0.$$

This computation is the abstract version of the argument in Section 1.1.2. In the notation used there, $\Phi_v''(\alpha) = a(v,v)$ for every α, so each Φ_v is a convex parabola. If a is positive definite, the inequality above is strict for $v \neq 0$, the parabolas are strictly convex, and the solution of the variational problem is the *unique* global minimiser. Under existence, the quadratic minimisation problem and the linear variational problem are thus equivalent.

1.2.5 Well-posedness

In the previous sections, we have proved some necessary conditions for existence of solutions of a quadratic minimisation problem and, in turn, its corresponding linear variational problem. Whereas existence is not complicated in finite dimensional vector spaces, the situation is far more subtle in infinite dimensions. If a global minimiser u exists for the quadratic minimisation problem, its energy norm $\|u\|_a$ must be bounded. It leads to the following necessary condition.

Corollary 1.2.22: Necessary continuity of the linear form

If there is a global minimiser for the quadratic minimisation problem in Definition 1.2.2 with a symmetric positive definite bilinear form a, then the linear form ℓ must be continuous.

Proof. Any minimiser $u \in V_0$ of the quadratic minimisation problem satisfies the variational problem in Definition 1.2.21. Thus, for any $v \in V_0$, it holds:

$$|\ell(v)| = |a(u,v)| \leq \|u\|_a \|v\|_a \leq C\|v\|_a,$$

for $C \doteq \|u\|_a < \infty$, where we have used the Cauchy-Schwarz inequality in Lemma 1.2.8. □

In Section 1.3, we show how to *complete* functional spaces so that one can prove the existence of solutions for a given quadratic minimisation problem.

1.3 Functional spaces

The motivation of this section is to define functional spaces for which the variational (or minimisation) problems above have solutions. We present here some functional spaces that solve the question of existence for some simple quadratic minimisation functionals. The broad idea is to define the largest space of functions for which the bilinear form *a is well defined* and that satisfy *suitable* boundary conditions.

As commented above, a minimiser $u : \Omega \to \mathbb{R}$ of the quadratic minimisation problem must have bounded energy, i.e., $a(u,u) < \infty$. Thus, we define the space V in terms of J as follows:

$$V \doteq \{v : \Omega \to \mathbb{R} \ : \ a(v,v) < \infty\}. \tag{1.6}$$

It is the so-called *maximal functional space* on which J is defined.

1.3.1 The $L^2(\Omega)$ space

Let us consider the potential

$$J_0(u) \doteq 1/2 \int_\Omega |u(x)|^2 \mathrm{d}x.$$

The *maximal functional space* with respect to J_0 leads to

$$V_0 \doteq \{v : \Omega \to \mathbb{R} : \int_\Omega |v(x)|^2 \mathrm{d}x < \infty\}. \tag{1.7}$$

Definition 1.3.1: $L^2(\Omega)$ space

The function space (1.7) is the space of *square-integrable* functions on Ω, which is denoted by $L^2(\Omega)$. It is a normed space for

$$\|v\|_0 \doteq \|v\|_{L^2(\Omega)} \doteq \left(\int_\Omega |v(x)|^2 \mathrm{d}x \right)^{1/2}.$$

We note that boundary values of $L^2(\Omega)$ functions are not well defined, as the following example illustrates.

Example (advanced) 1.3.2: Boundary conditions cannot be imposed in $L^2(\Omega)$

Let us consider a function in $u \in C^0([0,1]) \subset L^2((0,1))$. Now, let us consider a sequence of perturbations $\{\tilde{u}_n\}_{n\in\mathbb{N}}$ of this function for arbitrary values v_0 and v_1:

$$\tilde{u}_n \doteq \begin{cases} u(x) + (1-nx)(v_0 - u(0)), & 0 \leq x \leq 1/n, \\ u(x), & 1/n < x < 1 - 1/n \\ u(x) - n(1-1/n-x)(v_1 - u(1)), & 1 - 1/n < x \leq 1. \end{cases}$$

Clearly $\tilde{u}_n(0) = v_0$, $\tilde{u}_n(1) = v_1$ for any $n \in \mathbb{N}$, $n \geq 2$; see Figure 1.4. On the other hand, we obtain after integration $\|\tilde{u}_n - u\|^2_{L^2((0,1))} = \frac{1}{3n}\left((v_0 - u(0))^2 + (v_1 - u(1))^2\right) \to 0$ as $n \to \infty$.

Thus, we can find functions arbitrarily close to u in the L^2-norm that satisfy whatever boundary condition. Thus, the L^2-norm is not strong enough to *feel* the boundary conditions. It means that the

subspace V of continuous functions vanishing at the end-points is not *closed* in $L^2((0,1))$, i.e., we can find functions that are not in V but can be arbitrarily well approximated by functions in V (as in the example above). Another way to express this situation is saying that the linear operator that takes a continuous function and returns its value at one of the end-points is not continuous with respect to the L^2-norm.

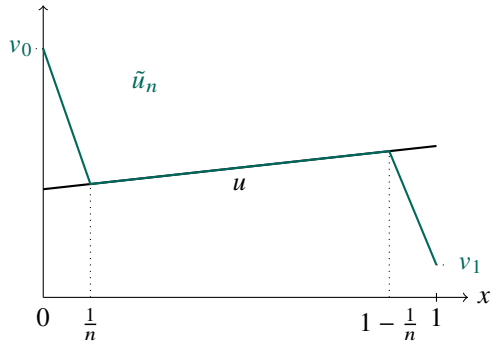


Figure 1.4: The perturbations $\tilde{u}_n$ of Advanced Example 1.3.2 attain arbitrary boundary values v_0, v_1 while converging to u in the L^2-norm. The boundary layers of width $\frac{1}{n}$ contribute less and less to the norm as $n \to \infty$.

Definition 1.3.3: Cauchy sequence

Consider a normed vector space V equipped with the norm $\|\cdot\|$. A sequence $\{v_n\}_{n\in\mathbb{N}}$ of elements of V is called a *Cauchy sequence* if

$$\forall \epsilon > 0 : \exists n = n(\epsilon) \in \mathbb{N} \ : \ \|v_k - v_m\| \leq \epsilon, \ \forall k, m \geq n.$$

It is straightforward to check that every convergent sequence is a Cauchy sequence. However, the equivalence between Cauchy and convergent sequences is only true for a particular type of spaces of paramount importance for the statement of well-posed variational problems.

Definition 1.3.4: Banach space

A normed vector space is called *complete* if every Cauchy sequence converges. A complete normed vector space is called a *Banach space*[2].

Definition 1.3.5: Inner product

An *inner product* $(\cdot,\cdot) : V \times V \to \mathbb{R}$ in a vector space V is a symmetric positive definite bilinear form in V.

[2] An attentive reader will notice that when defining the Cauchy sequence convergence using a norm, there is no difference between a complete and a Banach space. But in general, Cauchy sequences convergence is defined using a *distance* function $v_k, v_m \to d(v_k, v_m)$, so complete spaces need to be *metric* spaces but not necessarily normed spaces. The notion of distance is weaker than a norm, and a norm always defines a distance by $d(u,v) \doteq \|u - v\|$.

Definition 1.3.6: Hilbert space

If a Banach space is endowed with a norm that is the energy norm with respect to an inner product, it is called a *Hilbert space*.

Let us list some examples of Banach and Hilbert spaces. The set of real numbers $\mathbb{R}$ with the modulus norm is complete (this is in fact an axiom, the *axiom of completeness*, which states that the real line has no gaps). Finite dimensional normed spaces are also complete.

Remark 1.3.7: Completeness depends on the norm

We note that the definition of completeness depends not only on the set but also on the distance (norm) being used, i.e., on the metric space. For instance, the space $C^0(\bar{\Omega})$ for $\Omega \subset \mathbb{R}$ bounded (thus $\bar{\Omega}$ closed and bounded) endowed with the supremum norm $\|\cdot\|_\infty$, where $\|u\|_\infty \doteq \sup_{x\in\bar{\Omega}} |u(x)|$, is complete. But we have seen that $C^0(\bar{\Omega})$ with the L^2 norm is not complete. Intuitively, completeness tells us there are no *missing* points in our set (in the interior or boundary).

Remark 1.3.8: $L^2(\Omega)$ is a Hilbert space

The space $L^2(\Omega)$ is complete with respect to the norm $\|\cdot\|_{L^2(\Omega)}$; we state this property without proof. Moreover, the L^2 norm is the energy norm of the inner product $(u,v)_{L^2(\Omega)} \doteq \int_\Omega uv\,\mathrm{d}x$, so $L^2(\Omega)$ is a Hilbert space. The advanced Theorem 1.3.13 below defines $L^2(\Omega)$ equivalently as a completion, for which completeness holds by construction.

So you have to understand that, when we talk, e.g., about $L^2(\Omega)$, we are not just defining the set of functions but also the norm for these functions, which in turn provides a distance $d(u,v) = \|u-v\|_{L^2(\Omega)}$.

Completeness is the essential ingredient that is needed to prove the existence of a minimiser for the quadratic minimisation problem in Definition 1.2.2 (and thus, a solution of the variational problem in Definition 1.2.21). We provide more details for the interested reader in the following (advanced) section. Intuitively, we want to design a sequence of functions that get closer and closer to the functional infimum and be sure that this limit (the solution) does exist and belongs to our space.

1.3.2 Well-posedness in Hilbert spaces

Theorem 1.3.9: Existence and uniqueness of solutions in Hilbert spaces

Let us consider a Hilbert space V_0 with inner product $(\cdot,\cdot)_V$ and norm $\|\cdot\|_V$, a symmetric bilinear form $a : V_0 \times V_0 \to \mathbb{R}$ that is continuous and coercive with respect to $\|\cdot\|_V$, and a continuous linear functional $\ell : V_0 \to \mathbb{R}$. The corresponding quadratic minimisation problem

$$u = \operatorname*{argmin}_{v\in V_0} J(v),\quad J(v) \doteq 1/2a(v,v) - \ell(v)$$

has a unique solution.

Proof. By the continuity and coercivity of a, the energy norm $\|\cdot\|_a$ is equivalent to $\|\cdot\|_V$ (Lemma 1.2.11); in particular, ℓ is continuous with respect to $\|\cdot\|_a$ too. By Lemma 1.2.13, the quadratic functional J is bounded below. Thus, we can define a sequence $\{v_n\}_{n\in\mathbb{N}}$ such that

$$|J(v_n) - \mu| \le 1/n,\quad \mu \doteq \inf_{v\in V_0} J(v).$$

On the other hand, due to the bilinearity of a, we obtain

$$\begin{aligned}&\frac{1}{2}(J(v)+J(w))-J(1/2(v+w))\\&=1/4\,(a(v,v)+a(w,w)-2a(1/2(v+w),1/2(v+w)))\\&=1/8\|v-w\|_a^2.\end{aligned}$$

Clearly, $J(1/2(v+w))\geq\mu$, which combined with the previous results leads to

$$\begin{aligned}1/8\|v_k-v_m\|_a^2&\leq 1/2(J(v_k)+J(v_m))-\mu\\&\leq 1/2(1/k+1/m)\leq\max\{1/k,1/m\}.\end{aligned}$$

Thus, $\{v_n\}_{n\in\mathbb{N}}$ is a Cauchy sequence with respect to the energy norm. Using again the equivalence of norms, which is a consequence of the continuity and coercivity of a, the sequence is also a Cauchy sequence with respect to $\|\cdot\|_V$, and

$$u\doteq\lim_{n\to\infty}v_n\in V_0$$

due to completeness of V_0 with respect to its own norm. Since J is a continuous functional in V_0 (both a and ℓ are continuous, by assumption), we have that

$$J(u)=J(\lim_{n\to\infty}v_n)=\lim_{n\to\infty}J(v_n)=\mu.$$

Thus, $u\in V_0$ is a global minimiser of the problem at hand. Uniqueness is proved using Lemma 1.2.5. □

Remark 1.3.10: Compactness vs. completeness

The proof deserves a second reading, since it clarifies the division of labour between the hypotheses and connects with the discussion in Section 1.2.2. In the compactness proof of Theorem 1.2.14, compactness performs two tasks at once. It forces a minimising sequence to cluster, extracting a convergent subsequence, and it prevents the limit from leaving the set. In infinite dimensions, compactness is not available, and the proof above reassigns the two tasks. The key inequality

$$\frac{1}{8}\|v_k-v_m\|_a^2\leq\frac{1}{2}\,(J(v_k)+J(v_m))-\mu$$

holds by pure bilinearity, and its right-hand side vanishes along any minimising sequence. Any two members of a minimising sequence are forced to be close to each other. The clustering task is thus performed by the parabolic structure of the quadratic functional, not by the space; it is even performed better, since the whole sequence is Cauchy and no subsequence extraction is needed. Completeness performs the second task. A Cauchy sequence piles up, but only in a space without holes does it pile up on a point of the space. In the language of the intuition in Section 1.2.2, the quadratic structure plays the role of boundedness, since minimising sequences cannot spread out, and completeness plays the role of closedness, since their limits cannot leave the space. Example 1.2.18 is exactly a Cauchy minimising sequence in an incomplete space; the sequence clusters, and the point it clusters towards is missing.

This bookkeeping also shows that the mechanism is tailored to quadratic minimisation. Without the parabolic structure, minimising sequences are not Cauchy in general, and completeness has nothing to close. The general mechanism behind existence results in the calculus of variations replaces norm compactness by *weak* compactness, a weaker notion of convergence for which the closed balls of a Hilbert space are compact again, at the price of replacing the continuity of J by a one-sided substitute (lower semicontinuity). We present this *direct method of the calculus of variations* in Section 1.3.3.

We can observe that the problem in Example 1.2.18 was ill-posed in $C^0([-1,1])$ but it is well-posed in $L^2((-1,1))$ (with minimiser g). The space $C^0([-1,1])$ was *too small*; it did not include the limits of

Cauchy sequences, i.e., it was not complete. In fact, a quadratic minimisation problem can always be *fixed* by *completing* the vector space, which means to augment the space by including also all the potential limits of Cauchy sequences in it.

Definition 1.3.11: Dense space

A subset $W \subset V$ is *dense* in a normed vector space V if V is the union of W and all the limits of Cauchy sequences in W.

Theorem 1.3.12: Completion of a normed vector space

For every normed space V_0 there is a unique complete vector space $\tilde{V}_0$ (up to isomorphisms) that contains V_0 as a dense subspace.

These results provide a constructive way to create well-posed minimisation problems. We start with an admissible space V_0 of functions with bounded energy (see (1.6)). Next, we consider the completion $\tilde{V}_0$ of V_0 with respect to the energy norm. $\tilde{V}_0$ is complete by definition, and thus, the problem is well-posed in $\tilde{V}_0$ by Theorem 1.3.9, taking $(\cdot,\cdot)_V = a$; a symmetric positive definite form is continuous and coercive with respect to its own energy norm, with $\alpha = K = 1$. For instance, in the case of Example 1.2.18, we should consider the completion of $C^0([-1,1])$ with respect to the norm induced by $a(u,v) = \int_{-1}^{1} u(x)v(x)\mathrm{d}x$, i.e., the L^2-norm. On the other hand, we know that this problem is well-posed in $L^2((-1,1))$. The following result reconciles these two observations, showing that the completion is precisely $L^2((-1,1))$. Note also that completion can *wash out* boundary conditions. The completion of $C_0^0(\Omega)$ is the same space $L^2(\Omega)$, consistently with Advanced Example 1.3.2; the L^2-norm cannot retain them.

Theorem 1.3.13: $L^2(\Omega)$ as completion of $C^0(\Omega)$

Given a bounded domain $\Omega \subset \mathbb{R}^d$, the completion of $C^0(\bar{\Omega})$ (or of $C_0^0(\Omega)$) with respect to $\|\cdot\|_{L^2(\Omega)}$ is the space $L^2(\Omega)$.

Coercivity tells us when the completion step is harmless. If V_0 is already complete with respect to a norm $\|\cdot\|$ and a is continuous and coercive with respect to it, the energy norm is equivalent to $\|\cdot\|$ by Lemma 1.2.11; V_0 is then also complete with respect to the energy norm, and the minimisation problem is well-posed in V_0 itself by Theorem 1.3.9. No completion is needed, and no boundary conditions are washed out. Completion enlarges the space precisely when the energy norm is strictly weaker than the norm of the space, as in Example 1.2.12.

1.3.3 The direct method of the calculus of variations

This section is intended for very advanced readers, going beyond the other advanced sections, and it is not part of the unit assessment. It can be skipped without any loss of continuity. As announced in Remark 1.3.10, the Cauchy sequence mechanism behind Theorem 1.3.9 is tailored to quadratic problems. There is a second, far more general existence mechanism, which recovers the compactness argument of Section 1.2.2 by weakening the notion of convergence. We build the required ingredients first, and we close with a general existence and uniqueness theorem that contains the quadratic case.

Definition 1.3.14: Weak convergence

Let V be a normed vector space. A sequence $\{v_n\}_{n\in\mathbb{N}} \subset V$ *converges weakly* to $v \in V$, written $v_n \rightharpoonup v$, if

$$\varphi(v_n) \to \varphi(v) \quad \text{as} \quad n \to \infty, \quad \text{for every continuous linear functional } \varphi : V \to \mathbb{R}.$$

Norm convergence implies weak convergence, since $|\varphi(v_n) - \varphi(v)| \le C\|v_n - v\|$ by continuity; the converse fails in infinite dimensions, as the example below shows. In a Hilbert space with inner product a, every functional of the form $a(\cdot, w)$ is linear and continuous by the Cauchy–Schwarz inequality, so $v_n \rightharpoonup v$ implies $a(v_n, w) \to a(v, w)$ for all w. The converse implication also holds, since every continuous linear functional on a Hilbert space is of this form; this is the Riesz representation theorem, which we will meet in Theorem 1.4.5. Thus, in a Hilbert space, weak convergence is exactly the convergence of the inner products against every fixed element of the space.

Example 1.3.15: The sine sequence converges weakly to zero

Let us revisit the sequence $u_n(x) = \sqrt{2}\sin(n\pi x)$ of Section 1.2.2, now in the Hilbert space $L^2((0,1))$ with the inner product $a(v, w) = \int_0^1 vw \,\mathrm{d}x$. We computed there that $\|u_n\|_a = 1$ and $\|u_n - u_m\|_a^2 = 2$ for $n \neq m$; expanding the square, $2 - 2a(u_n, u_m) = 2$, so the sequence is *orthonormal*, $a(u_n, u_m) = 0$ for $n \neq m$. For any fixed w and any $N \in \mathbb{N}$, orthonormality gives

$$0 \le \Big\| w - \sum_{n=1}^{N} a(w, u_n)\, u_n \Big\|_a^2 = \|w\|_a^2 - \sum_{n=1}^{N} a(w, u_n)^2,$$

i.e., the partial sums of the series $\sum_n a(w, u_n)^2$ are bounded by $\|w\|_a^2$; this is the so-called Bessel inequality. A series of non-negative terms with bounded partial sums converges, and the terms of a convergent series vanish. Thus, $a(w, u_n) \to 0 = a(w, 0)$ for every w, and $u_n \rightharpoonup 0$. The sequence has no limit in norm, but it does have a weak limit. We also note that $\|u_n\|_a = 1$ does not converge to $\|0\|_a = 0$; the norm of the weak limit can be strictly smaller than the limit of the norms.

The example suggests that weak convergence is the notion for which bounded sequences cluster. This is the content of the following deep result, which we state without proof; it replaces the Heine–Borel theorem in infinite dimensions.

Theorem 1.3.16: Weak compactness of bounded sequences

Every bounded sequence in a Hilbert space admits a weakly convergent subsequence.

Compactness is recovered, but with respect to a weaker notion of convergence, and this weakening has a price. The functionals we minimise are continuous with respect to the norm, not with respect to weak convergence; the sine sequence converges weakly to zero while its norm stays at one. What survives is a one-sided version of continuity, and it is all we need.

Definition 1.3.17: Weak lower semicontinuity

A functional $J : V \to \mathbb{R}$ on a normed vector space V is *weakly (sequentially) lower semicontinuous* if

$$J(v) \le \liminf_{n\to\infty} J(v_n) \quad \text{whenever} \quad v_n \rightharpoonup v.$$

The value of J can drop in the weak limit, but it cannot jump up. For minimisation, this is the favourable direction; limits can only lose energy.

Lemma 1.3.18: Weak lower semicontinuity of quadratic functionals

The quadratic functional $J(v) = \frac{1}{2}a(v, v) - \ell(v)$, with a a continuous symmetric positive semi-definite bilinear form on a Hilbert space V_0 and ℓ continuous, is weakly lower semicontinuous.

Proof. Let $v_n \rightharpoonup v$. The linear part passes to the limit, $\ell(v_n) \to \ell(v)$, by the definition of weak convergence. For the quadratic part, testing the weak convergence against v itself (the map $w \mapsto a(w, v)$ is a continuous linear functional, by the continuity of a) and using the Cauchy–Schwarz inequality,

$$\|v\|_a^2 = a(v, v) = \lim_{n\to\infty} a(v_n, v) \leq \liminf_{n\to\infty} \|v_n\|_a \, \|v\|_a,$$

so $\|v\|_a \leq \liminf_{n\to\infty} \|v_n\|_a$ (the inequality is trivial if $\|v\|_a = 0$). Squaring preserves the inequality between non-negative numbers, and thus

$$J(v) = \frac{1}{2}\|v\|_a^2 - \ell(v) \leq \liminf_{n\to\infty} \frac{1}{2}\|v_n\|_a^2 - \lim_{n\to\infty} \ell(v_n) = \liminf_{n\to\infty} J(v_n),$$

which proves the result. □

We can now state the main result of this section. We stress that the functional is *not* assumed to be quadratic.

Theorem 1.3.19: Existence and uniqueness by the direct method

Let V_0 be a Hilbert space with norm $\|\cdot\|$ and let $J : V_0 \to \mathbb{R}$ be a functional that is

(i) coercive (Definition 1.2.10), and

(ii) weakly sequentially lower semicontinuous.

Then, J attains its global minimum, i.e., there exists $u \in V_0$ such that $J(u) = \inf_{v\in V_0} J(v)$. If, in addition, J is strictly convex at midpoints, i.e.,

$$J\left(\frac{v+w}{2}\right) < \frac{1}{2}\left(J(v) + J(w)\right) \quad \forall v, w \in V_0,\ v \neq w,$$

then the minimiser is unique.

Proof. Let $\mu \doteq \inf_{v\in V_0} J(v) \in [-\infty, \infty)$ and let $\{v_n\}_{n\in\mathbb{N}}$ be a minimising sequence, $J(v_n) \to \mu$. The sequence is bounded; otherwise, a subsequence would satisfy $\|v_n\| \to \infty$, and coercivity would give $J(v_n) \to \infty$, which contradicts $J(v_n) \to \mu < \infty$. By Theorem 1.3.16, there is a subsequence $v_{n_k} \rightharpoonup u \in V_0$. By weak lower semicontinuity,

$$J(u) \leq \liminf_{k\to\infty} J(v_{n_k}) = \mu.$$

Since $J(u)$ is a real number, $\mu > -\infty$, and, since μ is the infimum, $J(u) = \mu$; the infimum is attained at u. For uniqueness, let $u \neq w$ be two minimisers. Strict convexity at midpoints gives

$$J\left(\frac{u+w}{2}\right) < \frac{1}{2}\left(J(u) + J(w)\right) = \mu,$$

which contradicts the definition of μ. □

The proof follows the compactness pattern of Section 1.2.2 step by step. Coercivity confines the minimising sequence to a bounded set; weak compactness extracts a convergent subsequence; lower semicontinuity guarantees that the weak limit is a minimiser. The two tasks that norm compactness cannot perform in infinite dimensions are performed by the pair weak compactness plus weak lower semicontinuity. The quadratic well-posedness result is now a particular case.

Second proof of Theorem 1.3.9, by the direct method. We verify the hypotheses of Theorem 1.3.19 for $J(v) = \frac{1}{2}a(v, v) - \ell(v)$ with the energy norm $\|\cdot\|_a$. Working with the energy norm is legitimate; by Lemma 1.2.11, it is equivalent to $\|\cdot\|_V$, so V_0 is also complete with respect to it. Coercivity follows from

the continuity of ℓ and the generalised Young inequality, as in the proof of Lemma 1.2.13,

$$J(v) \geq \frac{1}{2}\|v\|_a^2 - C\|v\|_a \geq \frac{1}{4}\|v\|_a^2 - C^2 \to \infty \text{ as } \|v\|_a \to \infty.$$

Weak lower semicontinuity is Lemma 1.3.18. Strict convexity at midpoints is the identity from the first proof of Theorem 1.3.9,

$$\frac{1}{2}\left(J(v) + J(w)\right) - J\left(\frac{v+w}{2}\right) = \frac{1}{8}\|v-w\|_a^2 > 0 \text{ for } v \neq w.$$

Thus, Theorem 1.3.19 provides existence and uniqueness. □

Remark 1.3.20: Beyond quadratic problems

Nothing quadratic appears in Theorem 1.3.19; the quadratic structure only entered when checking its hypotheses. It can be proved that every convex and continuous functional is weakly lower semicontinuous, so the theorem covers the nonlinear convex energies that appear, e.g., in nonlinear elasticity. This three-step scheme is called the *direct method of the calculus of variations*, and it is the standard existence mechanism for nonlinear variational problems. Compared with the Cauchy sequence argument in the first proof of Theorem 1.3.9, it is far more general, but its conclusion is weaker; it provides the weak convergence of a subsequence, whereas the quadratic argument provides the norm convergence of the whole minimising sequence.

1.3.4 The $H^1(\Omega)$ space

Definition 1.3.21: Gradient of a function

Given a function $f : \Omega \subset \mathbb{R}^d \to \mathbb{R}$, we define its gradient as $\boldsymbol{\nabla} f(x) \doteq [\partial f/\partial x_1(x), \ldots, \partial f/\partial x_d(x)]^T \in \mathbb{R}^d$ for $x \in \Omega$.

We can now proceed analogously for the positive semi-definite bilinear form

$$a(u,v) \doteq \int_\Omega \boldsymbol{\nabla} u(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x.$$

The corresponding maximal functional set for the semi-norm induced by this bilinear form reads

$$V_0 \doteq \{v : \Omega \to \mathbb{R} \ : \ v = 0 \text{ on } \partial\Omega, \ \int_\Omega |\boldsymbol{\nabla} v(x)|^2 \mathrm{d}x < \infty\}. \tag{1.8}$$

Definition 1.3.22: Sobolev space

A Sobolev space is a vector space of functions endowed with a norm that combines L^p norms of the function itself and its derivatives up to a given order. The $L^p(\Omega)$ space, $1 \leq p < \infty$, is the space of functions such that $\left(\int_\Omega |v|^p \mathrm{d}x\right)^{\frac{1}{p}} < \infty$.

Definition 1.3.23: Sobolev space $H_0^1(\Omega)$

The function space (1.8) is the space of *square integrable* functions with *square integrable* gradients on Ω that vanish on $\partial\Omega$, which is denoted by $H_0^1(\Omega)$. It is a normed space for

$$|v|_{H^1(\Omega)} \doteq \left(\int_\Omega |\boldsymbol{\nabla} v(x)|^2 \mathrm{d}x\right)^{1/2}.$$

We note that we have started considering the space $H^1_0(\Omega)$, in which the zero subscripts means zero value on $\partial\Omega$. The control provided by the semi-norm $|\cdot|_{H^1(\Omega)}$ is enough to make boundary conditions meaningful.[3] For instance, if we consider Advanced Example 1.3.2, we can observe that $|u - \tilde{u}_n|^2_{H^1((0,1))} = n\left((v_0 - u(0))^2 + (v_1 - u(1))^2\right) \to \infty$ as $n \to \infty$, unless $v_0 = u(0)$ and $v_1 = u(1)$. Thus, the H^1-semi-norm *feels* the boundary value, since it cannot be changed without changing the energy of the function.

Now, we want to consider more general boundary values, not just zero on $\partial\Omega$. To get a new norm, we eliminate the boundary condition and add an additional term, the L^2 norm $\|\cdot\|_0$, to the semi-norm $|\cdot|_{H^1(\Omega)}$.

Definition 1.3.24: The $H^1(\Omega)$ space

The *Sobolev* space

$$H^1(\Omega) \doteq \{v \in L^2(\Omega) \ : \ \int_\Omega |\nabla v(x)|^2 \mathrm{d}x < \infty\}$$

is a normed space with

$$\|v\|^2_{H^1(\Omega)} \doteq \|v\|^2_0 + |v|^2_{H^1(\Omega)}.$$

$H^1(\Omega)$ is the maximal function space with respect to the norm $\|v\|_{H^1(\Omega)}$. We note that $|\cdot|$ is a semi-norm in $H^1(\Omega)$ because $|v| = 0$ for v a non-zero constant, violating the definiteness condition. However, it is a norm, e.g., in the subspace of functions in $H^1(\Omega)$ with zero mean value.

Remark 1.3.25: $H^1(\Omega)$ and $H^1_0(\Omega)$ are Hilbert spaces

As $L^2(\Omega)$, these Sobolev spaces are complete; we state this property without proof (the advanced Theorem 1.3.27 below defines them as completions, for which completeness holds by construction). The space $H^1(\Omega)$ is complete with respect to $\|\cdot\|_{H^1(\Omega)}$, which is the energy norm of the inner product $(u, v)_{H^1(\Omega)} \doteq \int_\Omega uv + \nabla u \cdot \nabla v \, \mathrm{d}x$, and $H^1_0(\Omega)$ is complete with respect to $|\cdot|_{H^1(\Omega)}$, which is the energy norm of the inner product $\int_\Omega \nabla u \cdot \nabla v \, \mathrm{d}x$ (a norm on $H^1_0(\Omega)$ thanks to the boundary conditions). Thus, both are Hilbert spaces, which is precisely what Theorem 1.3.9 requires. In particular, the completeness of $H^1_0(\Omega)$ and Theorem 1.3.9 are all we need to prove the existence and uniqueness of a solution for the model problem of Section 1.1.1, once the continuity of the linear form is verified; we do so in Section 1.4.

Example 1.3.26: Piecewise continuous functions in $H^1(\Omega)$

We observe that the space $H^1(\Omega)$ includes functions that do not posses *classical derivatives*, i.e., they are not differentiable at all points. For instance, let us consider a *piecewise* function in $[0, 1]$

$$u(x) = \begin{cases} 2x, & 0 < x < 1/2, \\ 2(1-x), & 1/2 \le x < 1. \end{cases}$$

This function belongs to $H^1(\Omega)$ since

$$|u|^2_{H^1(\Omega)} = \int_0^1 |\nabla u(x)|^2 \mathrm{d}x = 4 < \infty.$$

In general, it can be checked that the space $C^1_{\mathrm{pw}}(\bar{\Omega})$ of *continuous* functions with piecewise continuous first derivatives is a subset of $H^1(\Omega)$. Global continuity is essential. A function with a jump, e.g., the step function $u = 0$ for $x < \frac{1}{2}$ and $u = 1$ for $x \ge \frac{1}{2}$, is *not* in $H^1((0, 1))$, even though its pointwise derivative vanishes wherever it is defined.

[3] Let us remark that $|\cdot|_{H^1(\Omega)}$ is in fact a norm for the space $H^1_0(\Omega)$ thanks to the homogeneous boundary conditions.

1.3.5 $H^1(\Omega)$ by completion

Analogously as $L^2(\Omega)$, the $H^1(\Omega)$ space can be defined by completion.

Theorem 1.3.27: $H^1(\Omega)$ and $H^1_0(\Omega)$ by completion

Given a piecewise smooth domain Ω, the space $H^1(\Omega)$ is the completion of $C^\infty(\Omega)$ with respect to the norm $\|\cdot\|_{H^1(\Omega)}$. For a bounded domain $\Omega \subset \mathbb{R}^d$, $H^1_0(\Omega)$ is the completion of $C^\infty_0(\Omega)$ with respect to the semi-norm $|\cdot|_{H^1(\Omega)}$.

Thus, the space of smooth functions $C^\infty(\Omega)$ and $C^\infty_0(\Omega)$ are dense in $H^1(\Omega)$ and $H^1_0(\Omega)$, respectively.

1.3.6 Roadmap

Let us summarise the constructive *recipe* developed in this section to obtain well-posed minimisation (and variational) problems:

1. Given the energy functional J with bilinear form a, consider the *maximal functional space*, i.e., the functions with bounded energy (and suitable boundary conditions);
2. *complete* this space with respect to the norm induced by a (this step gives the Lebesgue and Sobolev spaces above);
3. check the *continuity of the linear form ℓ* with respect to that norm.

With these ingredients, existence and uniqueness follow from Theorem 1.3.9. In the next section, we apply this recipe to a multi-dimensional model problem.

1.4 A multidimensional problem

At this point, we are in position to consider the multi-dimensional version of the model problem in (1.3), which models the normal displacement u of a membrane under a normal external pressure f and prescribed displacement on the boundary; we consider $\kappa = 1$ and zero boundary conditions for simplicity. This problem is represented by the quadratic minimisation problem with the functional

$$J(u) = \int_\Omega \left(\frac{1}{2} |\nabla u(x)|^2 - f(x)u(x) \right) \mathrm{d}x.$$

Thus, the energy norm of this problem is $\|\cdot\|_a^2 \doteq a(u,u)$ for

$$a(u,v) \doteq \int_\Omega \nabla u(x) \cdot \nabla v(x) \mathrm{d}x,$$

whereas the corresponding linear form reads

$$\ell(u) \doteq \int_\Omega f(x)u(x)\mathrm{d}x.$$

Thus, we consider the maximal functional space for this energy, i.e., the space of functions with bounded energy, which we now know is $H^1(\Omega)$. The subspace of functions in $H^1(\Omega)$ that satisfy zero boundary conditions is $H^1_0(\Omega)$; thus, this will be the test space.

As proved in Corollary 1.2.22, the continuity of the linear form is necessary for the existence of a global minimiser. Assuming that $f \in L^2(\Omega)$, using the Cauchy-Schwarz inequality we readily get:

$$\int_\Omega f(x)v(x)\mathrm{d}x \le \left(\int_\Omega |f(x)|^2 \right)^{1/2} \left(\int_\Omega |v(x)|^2 \right)^{1/2} = \|f\|_0 \|v\|_0, \ v \in H^1_0(\Omega),$$

where $\|f\|_0 < \infty$. We note that the continuity must be in terms of the energy (semi-)norm $|\cdot|_{H^1(\Omega)}$ but the above result is bounded with respect to the norm $\|\cdot\|_0$. The following classical inequality solves this issue.

Theorem 1.4.1: First Poincaré-Friedrichs inequality

Given a bounded domain $\Omega \subset \mathbb{R}^d$, it holds

$$\|u\|_0 \leq \operatorname{diam}(\Omega)\|\nabla u\|_0, \quad \forall u \in H_0^1(\Omega).$$

Proof (advanced). We can prove this result by relying on the fact that smooth functions in $C_0^\infty(\Omega)$ are dense in $H_0^1(\Omega)$ (see Theorem 1.3.27). If the result holds for these smooth functions, it readily holds for $H_0^1(\Omega)$, using the definition of density; such strategy is common in functional analysis and is called a *density argument*. We show the result for $d = 1$ and $\Omega = (0, 1)$ for the sake of simplicity; note that $\operatorname{diam}(\Omega) = 1$ in this case. Using the fundamental theorem of calculus and $u(0) = 0$ for $u \in C_0^\infty(\Omega)$, we have:

$$u(x) = \int_0^x u'(s)\mathrm{d}s, \ 0 \leq x \leq 1, \ \forall u \in C_0^\infty(\Omega).$$

The Cauchy–Schwarz inequality applied to the inner integral yields

$$\begin{aligned}\|u\|_0^2 &= \int_0^1 \left|\int_0^x u'(s)\,\mathrm{d}s\right|^2 \mathrm{d}x \leq \int_0^1 \left(\int_0^x 1\,\mathrm{d}s\right)\left(\int_0^x |u'(s)|^2\,\mathrm{d}s\right)\mathrm{d}x \\ &\leq \int_0^1 x\,\|u'\|_0^2\,\mathrm{d}x = \frac{1}{2}\|u'\|_0^2 \leq \|u'\|_0^2,\end{aligned}$$

where we have used that $\int_0^x |u'(s)|^2\,\mathrm{d}s \leq \|u'\|_0^2$ and $x \leq 1$. Taking square roots proves the result. □

It leads to the following result.

Corollary 1.4.2: Admissible forcing term

The linear functional $\int_\Omega f(x)u(x)\mathrm{d}x$ with $f \in L^2(\Omega)$ is continuous in $H_0^1(\Omega)$ with respect to the semi-norm $|\cdot|_{H^1(\Omega)}$.

Proof. Combining the previous bound with the Poincaré–Friedrichs inequality, $\int_\Omega f(x)u(x)\,\mathrm{d}x \leq \|f\|_0\|u\|_0 \leq \operatorname{diam}(\Omega)\|f\|_0\,|u|_{H^1(\Omega)}$. □

Corollary 1.4.3: $|\cdot|_{H^1(\Omega)}$ is a norm in $H_0^1(\Omega)$

The semi-norm $|\cdot|_{H^1(\Omega)}$ is a norm in $H_0^1(\Omega)$.

Proof. Due to the Poincaré-Friedrichs inequality in Section 1.4, we have that

$$\|u\|_{H^1(\Omega)} \leq (\operatorname{diam}(\Omega) + 1)|u|_{H^1(\Omega)}.$$

In particular, the only non-obvious norm property, definiteness, holds: $|u|_{H^1(\Omega)} = 0$ implies $\|u\|_{H^1(\Omega)} = 0$ and thus $u = 0$. □

Now, let us consider non-homogeneous boundary conditions

$$u(x) = g(x), \ \forall x \in \partial\Omega,$$

where $g : \partial\Omega \to \mathbb{R}$ is the boundary value to be prescribed on the boundary. Let us define the trial space $H_g^1(\Omega) \doteq \{v \in H^1(\Omega) \ : \ v = g \ \text{ on } \partial\Omega\}$. The way we understand the equality in the boundary conditions is also weak, and the regularity assumptions over g and why this boundary condition is meaningful are out of the scope of this course. The interested student can look for *trace theorems in Sobolev spaces* for more information.

Taking the directional derivatives of the quadratic functional and enforcing them to be zero, we end up with the following variational formulation:

$$u \in H^1_g(\Omega) \ : \ \int_\Omega \boldsymbol{\nabla} u(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x = \int_\Omega f(x) v(x) \mathrm{d}x, \ \forall v \in H^1_0(\Omega).$$

As commented after Definition 1.2.2, this problem can also be stated using the *offset function* method. Pick a function $u_0 \in H^1_g(\Omega)$, and re-state the problem as:

$$\begin{aligned} \delta u \in H^1_0(\Omega) \ : \ & \int_\Omega \boldsymbol{\nabla} \delta u(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x \\ = & \int_\Omega f(x) v(x) - \int_\Omega \boldsymbol{\nabla} u_0(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x, \ \forall v \in H^1_0(\Omega), \end{aligned} \tag{1.9}$$

and return $u = u_0 + \delta u$. It is straightforward to check that the additional right-hand side term due to boundary conditions is continuous, since $u_0 \in H^1(\Omega)$.

Clearly, this problem is a linear variational problem with $V_0 \doteq H^1_0(\Omega)$,

$$a(u, v) \doteq \int_\Omega \boldsymbol{\nabla} u(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x,$$

and

$$\ell(v) \doteq \int_\Omega f(x) v(x) \mathrm{d}x - a(u_0, v)$$

for $f(x) \in L^2(\Omega)$ and $u_0 \in H^1_g(\Omega)$, where ℓ is continuous in $H^1_0(\Omega)$. As a result, the energy norm of the problem is $\| \cdot \|_a \doteq | \cdot |_{H^1(\Omega)}$, which is in fact a norm in $H^1_0(\Omega)$ due to the first Poincare-Friedrichs inequality.

Remark 1.4.4: Rougher forcing terms and the dual space $H^{-1}(\Omega)$

We note that the regularity $f(x) \in L^2(\Omega)$ is sufficient for well-posedness but not necessary. Instead, we could just consider $f : H^1_0(\Omega) \to \mathbb{R}$ to be a linear and continuous functional, i.e., $|f(v)| \leq C \|v\|_{H^1(\Omega)}$ for any $v \in H^1_0(\Omega)$ and a constant C independent of v; the application of f to v can still be symbolically represented using integration, i.e., $f(v) = \int_\Omega f(x) v(x) \mathrm{d}x$. The vector space of these bounded functionals is the *dual* space of $H^1_0(\Omega)$ and is usually denoted by $H^{-1}(\Omega)$. It is a Banach space (normed and complete) with norm

$$\|f\|_{H^{-1}(\Omega)} \doteq \sup_{v \in H^1_0(\Omega) \setminus \{0\}} \frac{f(v)}{\|v\|_{H^1(\Omega)}}.$$

We have the following significant result showing that *primal* and dual spaces are *isometrically isomorphic*.

Theorem 1.4.5: Riesz representation theorem

For a linear continuous functional $\ell : V \to \mathbb{R}$ in a real Hilbert space V endowed with the inner product $a(\cdot, \cdot)$ and corresponding norm $\| \cdot \|_a$, there exists a unique $u \in V$ such that

$$a(u, v) = \ell(v), \ \forall v \in V, \qquad \|u\|_a = \sup_{v \in V \setminus \{0\}} \frac{\ell(v)}{\|v\|_a} \doteq \|\ell\|_{a'}.$$

Proof. The existence and uniqueness of a solution for the linear variational problem has already been proved above. Concerning the second part, the one related to the equivalence between the primal norm of the solution and the dual norm of the linear form, we proceed as follows. First, due to the Cauchy-Schwarz inequality, we have $|\ell(v)| = |a(u, v)| \leq \|u\|_a \|v\|_a$ and thus $\sup_{v \in V \setminus \{0\}} \frac{\ell(v)}{\|v\|_a} \leq \|u\|_a$. The supremum is

attained (for $u \neq 0$; the case $u = 0$ is trivial) since $\ell(u) = \|u\|_a^2$. It proves the theorem. □

Corollary 1.4.6: Well-posedness of a 2nd order elliptic problem

The variational formulation (1.9) has a unique solution for $f \in H^{-1}(\Omega)$ and $u_0 \in H^1(\Omega)$.

Example 1.4.7: Load force

Let us consider the unit interval $[0, 1]$. The Dirac delta $\delta_s(v) \doteq v(s)$ for $0 < s < 1$ is a linear functional that cannot be represented as $\int_0^1 f(x)v(x)\mathrm{d}x$ for any function $f \in L^2((0, 1))$. On the other hand, it is possible to check that in 1D the functions in $H_0^1(\Omega)$ are continuous and thus, $\delta_s \in H^{-1}((0, 1))$. It has some physical implications, e.g., one can consider the elastic bar problem with a point load, which is not meaningful in the strong form of the problem.

1.5 Boundary value problem

In this section, we will go from the variational formulation to its corresponding boundary value problem and its corresponding partial differential equation. As we did in the introduction for a 1D problem, it will require some regularity assumptions. The key to transforming the variational form into a partial differential equation is *integration by parts*. We need the following standard results.

Lemma 1.5.1: Product rule

For all $\boldsymbol{\psi} \in C^1(\bar{\Omega})^d$, $v \in C^1(\bar{\Omega})$, it holds

$$\boldsymbol{\nabla} \cdot (\boldsymbol{\psi} v) = v \boldsymbol{\nabla} \cdot (\boldsymbol{\psi}) + \boldsymbol{\psi} \cdot \boldsymbol{\nabla} v.$$

Theorem 1.5.2: Gauss theorem

Let us represent with $\boldsymbol{n} : \partial\Omega \to \mathbb{R}^d$ the outwards normal vector field on $\partial\Omega$. It holds:

$$\int_\Omega \boldsymbol{\nabla} \cdot \boldsymbol{\psi}(x)\mathrm{d}x = \int_{\partial\Omega} \boldsymbol{\psi}(s) \cdot \boldsymbol{n}(s)\mathrm{d}s, \quad \forall \boldsymbol{\psi} \in C^1(\bar{\Omega})^d.$$

where $\mathrm{d}s$ denotes integration with respect to the surface measure.

Theorem 1.5.3: Green's first formula

For all $\boldsymbol{\psi} \in C^1(\bar{\Omega})^d$, $v \in C^1(\bar{\Omega})$, it holds:

$$\int_\Omega \boldsymbol{\psi}(x) \cdot \boldsymbol{\nabla} v(x)\mathrm{d}x = -\int_\Omega \boldsymbol{\nabla} \cdot (\boldsymbol{\psi}(x))v(x)\mathrm{d}x + \int_{\partial\Omega} \boldsymbol{\psi}(s) \cdot \boldsymbol{n}(s)v(s)\mathrm{d}s.$$

At this point, if we assume that $u \in C^2(\bar{\Omega})$ and thus, $\boldsymbol{\nabla} u \in C^1(\bar{\Omega})$, we can use Green's first formula to get:

$$\begin{aligned}&\int_\Omega \boldsymbol{\nabla} u(x) \cdot \boldsymbol{\nabla} v(x)\mathrm{d}x \\ &\quad = -\int_\Omega \boldsymbol{\nabla} \cdot (\boldsymbol{\nabla} u(x))v(x)\mathrm{d}x, \quad \forall v \in C_0^1(\bar{\Omega}),\end{aligned}$$

using the fact that the boundary terms vanish because $v = 0$ on $\partial\Omega$. As a result, using the variational formulation (since $C_0^1(\bar{\Omega}) \subset H_0^1(\Omega)$) and assuming that $f \in C^0(\Omega)$, we get

$$\int_\Omega (-\boldsymbol{\nabla} \cdot (\boldsymbol{\nabla} u(x)) - f(x))v(x)\mathrm{d}x = 0, \ \forall v \in C_0^1(\Omega).$$

Lemma 1.5.4: Fundamental lemma of calculus of variations (multi-dimensional)

If a function $f \in C^0(\Omega)$ *is such that*

$$\int_\Omega f(x)v(x)\mathrm{d}x = 0,\ \forall v \in C_0^\infty(\Omega),$$

then $f(x) = 0$ *for any* $x \in \Omega$.

As a result, if the solution of the variational formulation is in $C^2(\Omega)$, it satisfies the following boundary value problem:

$$-\Delta u = f \quad \text{in } \Omega, \quad u = g \quad \text{on } \partial\Omega. \tag{1.10}$$

Δ defined as $\Delta v \doteq \boldsymbol{\nabla} \cdot (\boldsymbol{\nabla} v)$ is the so-called *Laplacian* operator.

Solutions of (1.10) are called *classical (or strong)* solutions. If they exist, they are also solutions of the variational formulation. However, the extra regularity assumption requires additional smoothness over the data (not only over the forcing term, which has been made explicit above, but also the domain Ω and the boundary value g). Summarising, the variational formulations have been proved to be well-posed, i.e., they have a unique solution in the maximal function space. The corresponding boundary value problem (1.10) is not well-posed in general because it requires additional smoothness assumptions over the solution that are not true in general.

The general strategy to obtain the boundary value problem out of a variational formulation is to use integration by parts to transfer all derivatives over test functions to the trial functions, as above. After this process, we end up with the following re-statement of the variational equation in the following form

$$u \in V \ : \ \int T(u)v\mathrm{d}x = 0,\ \forall v \in V.$$

Remark 1.5.5: The Euler–Lagrange equation

We note that the integral could involve bulk and surface integration. One can ask whether it is legitimate to do integration by parts for the space V. In fact, it is related to the fact that we consider derivatives *in a weak (or distributional) way*, but we do not want to go further into these concepts. When we want $T(u)$ to have a classical pointwise sense, we need to enforce the additional regularity over u. The equation $T(u) = 0$ is the so-called *Euler-Lagrange* equation of the underlying functional.

1.6 Boundary conditions

So far, we have always enforced the value of the unknown $u : \Omega \to \mathbb{R}$ on the whole domain boundary $\partial\Omega$: $u = g$ on $\partial\Omega$, where we can assume that $g \in C^0(\Omega)$. These are the so-called Dirichlet boundary conditions or *essential* boundary conditions. But we can consider other types of boundary conditions too. For instance, in the model for an elastic membrane above, we could instead consider that part of $\partial\Omega$ is not clamped, and instead, we provide the surface load on that boundary. Analogously, the model above also represents heat conduction. Instead of just imposing the temperature on the whole boundary, we could prescribe the heat flux in part of the boundary. Let us see how these physically relevant boundary conditions are described in our mathematical formulations above.

Let us consider $\Gamma_0 \subset \partial\Omega$ with $\partial\Omega \setminus \Gamma_0$ non empty, and enforce the Dirichlet boundary condition $u = g$ on Γ_0. On the other hand, let us assume that there exists an offset function $u_0 \in H^1(\Omega)$ that satisfies these boundary conditions. Using the offset function method, $u - u_0 \in V_0 \doteq \{v \in H^1(\Omega) \ : \ v = 0 \ \text{ on } \Gamma_0\}$; clearly, V_0 is a vector space. We also note that, for the resulting problem to be well-posed, the Dirichlet boundary Γ_0 must be non-trivial (e.g., of positive surface measure). The Poincaré–Friedrichs inequality in Section 1.4 then extends to functions vanishing only on Γ_0, a result we state without proof. On the other hand, we can

now define a function $h : \partial\Omega \setminus \Gamma_0 \to \mathbb{R}$ and state the following linear variational problem:

$$\begin{aligned}\delta u \in V_0 \ : \ & \int_\Omega \boldsymbol{\nabla}\delta u(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x \\ & = -\int_\Omega \boldsymbol{\nabla} u_0(x) \cdot \boldsymbol{\nabla} v(x) \mathrm{d}x \\ & \quad + \int_\Omega f(x) v(x) \mathrm{d}x + \int_{\partial\Omega\setminus\Gamma_0} h(s) v(s) \mathrm{d}s, \ \forall v \in V_0.\end{aligned}$$

h is a prescribed flux on the Neumann boundary $\partial\Omega \setminus \Gamma_0$ (e.g., a heat flux in thermal problems or surface tension in fluid/solid mechanics), and it is data (as f).

Now, since the test functions do not vanish anymore on $\partial\Omega$, we can add boundary *loads*. In order to keep a linear continuous form, the only thing that we need is the boundedness of $h(v) \doteq \int_{\partial\Omega\setminus\Gamma_0} h(s)v(s)\mathrm{d}s$, i.e., $|h(v)| \leq C\|v\|_{H^1(\Omega)}$ for any $v \in V_0$; e.g., $h \in L^2(\partial\Omega \setminus \Gamma_0)$ suffices, a fact related to the trace theorems mentioned above.

Now, we are going to use the procedure defined above to recover the boundary value problem associated to this linear variational problem. Let us assume that $u \in C^2(\bar{\Omega})$, $f \in C^0(\Omega)$, and $h \in C^0(\partial\Omega \setminus \Gamma_0)$. We also need to assume that $\partial\Omega$ is such that its outward normal vector field $\boldsymbol{n}$ is in $C^0(\partial\Omega)^d$. Using first Green's formula, now keeping (part of) the boundary terms, we obtain

$$\int_\Omega (-\Delta u(x) - f(x))\, v(x) \mathrm{d}x + \int_{\partial\Omega\setminus\Gamma_0} (\boldsymbol{n} \cdot \boldsymbol{\nabla} u(s) - h(s)) v(s) \mathrm{d}s = 0, \ \forall v \in C^1_{\Gamma_0}(\Omega).$$

where $C^1_{\Gamma_0} \doteq \{v \in C^1(\Omega) \ : \ v = 0 \ \text{ on } \Gamma_0\}$. Using the fundamental lemma of calculus of variations twice (first with test functions vanishing on the whole boundary, which recovers the equation in Ω, and next with test functions that do not vanish on the Neumann boundary, which recovers the flux condition), we get that u satisfies the following boundary value problem:

$$-\Delta u = f \ \text{ in } \Omega, \quad \boldsymbol{n} \cdot \boldsymbol{\nabla} u = h \ \text{ on } \partial\Omega \setminus \Gamma_0, \quad u = g \ \text{ on } \Gamma_0.$$

Thus, we have seen how we can define so-called *Neumann (or natural)* boundary conditions for variational formulations, i.e., via a boundary source or load term, and how they transform into a flux boundary condition in the corresponding boundary value problem.

1.7 Further topics

In the previous presentation, I have not considered some crucial concepts that are out of the scope of this course. For instance, I have not discussed Lebesgue integration, Lebesgue measure, or a measurable function. I have also omitted the concept of weak derivative and I have not been rigorous about boundary conditions; I have not explained why one can define the trace of a function in $H^1(\Omega)$, in which sense that equality is understood, or the regularity required on the boundary data g for the problem to be well-posed. It would involve introducing trace theorems, which are also out of the scope of this course. In any case, the reader can do an Internet search for these keywords and find lots of material discussing these issues. A complete and rigorous treatment of all these topics can be found in the first volume of the *Finite Elements* series by A. Ern and J.-L. Guermond (Springer, 2021).

1.8 Tutorial

1. Let us consider the problem: find u such that $-u''(x) = \delta_0$ (where δ_0 is the Dirac delta at $x = 0$, i.e., $\delta_0(v) = v(0)$) in $(-1, 1)$ and $u(-1) = u(1) = 0$. Do you think that this problem is meaningful in strong (classical) pointwise form? (Hint: Check the regularity of the forcing term (right-hand side of the PDE).)

State the weak form of the problem. Can you find the solution to this problem? Hint: Use the fact that $H_0^1((-1,1)) \subset C_0^0([-1,1])$. Try to find possible solutions to this problem on $[-1,0]$ and $[0,1]$ separately. Enforce the continuity of the solution (since the function must belong to H^1) at $x = 0$ to end up with a unique solution.)

2. Show that pointwise evaluation of $L^2((0,1))$ functions (i.e., the evaluation at a point in $(0,1)$ of these functions) is not continuous/bounded in general. Hint: Find a counterexample by building a sequence of functions in $L^2((0,1))$ (i.e., the integral of their square is bounded) whose value at a given point (e.g., at 0) is not bounded.

3. We want to solve the following problem: find u such that

$$-\boldsymbol{\nabla}\boldsymbol{\nabla}\cdot\boldsymbol{u} + \boldsymbol{u} = \boldsymbol{f} \text{ in } \Omega \subset \mathbb{R}^3,$$

for the boundary conditions $\boldsymbol{u}\cdot\boldsymbol{n} = 0$ on $\partial\Omega$. Can you obtain the weak formulation of this problem? Which functional space would you consider as trial/test space? (Hint: Use the definition of maximal functional space)

Which are the natural boundary conditions for this problem? (Hint: Leave free (no Dirichlet boundary conditions) part of the boundary, use integration by parts, and check the boundary term that arises.)

4. We want to solve the following minimisation problem: find u that minimises the functional

$$J(u) \doteq 1/2\int_\Omega |\alpha(x)^{1/2}u(x)|^2\mathrm{d}x + 1/2\int_\Omega |\beta(x)^{1/2}\boldsymbol{\nabla}u(x)|^2\mathrm{d}x - \int_\Omega f(x)u(x)\mathrm{d}x,$$

with zero boundary conditions. Can you state the variational formulation? Can you provide *sufficient* conditions on α, β for the problem to be well-posed? (Hint: Consider the Poincaré inequality.)

2 Discretisation

In the previous chapter, we have presented different ways to look at mathematical models that involve partial differential equations. We have observed that, for the elliptic problems, we can state the same problem as a minimisation, a variational, or a boundary value problem. We have also observed that these formulations are equivalent only under particular circumstances, e.g., the boundary value problem requires additional regularity. Finally, we have shown the well-posedness of the variational (and minimisation) problems and that the choice of suitable functional space is essential.

We have learned that variational formulations allow one to understand the problem in a weaker (non-pointwise) sense, which allows one to solve more general problems. It has clear *practical* implications. For example, the problem of a clamped elastic structure in an L-shaped domain does not have a solution in the classical sense since the stresses in the inner corner are infinite.

Even though the variational formulations have excellent properties, it is generally impossible to find an analytical solution. These problems are infinite-dimensional and cannot be solved with computers; a computer can only perform floating point operations with finite precision. As a result, it has motivated whole fields of mathematics, namely numerical analysis and scientific computing. These fields are about the design and analysis of *approximations* of mathematical models, with applications in almost any scientific discipline. They are part of a broad scientific area called *computational science*, which has become the third pillar of science after traditional theory and experimentation.

This course focuses on the numerical approximation of partial differential equations. The idea is simple. We want to get the most accurate approximation of the solution of a partial differential equation at the minimum computational cost. The spectral and finite element methods that we will study in this course rely on the variational formulation of a mathematical model. It is a clear difference compared to finite difference methods that work on the strong form. It also motivates *why* we have started this course with an introduction to *variational forms and functional spaces.*

We start this chapter with an abstract discretisation framework (the Galerkin method) that uses finite-dimensional approximations of our variational problem. Next, we will consider two different ways to generate *accurate* finite-dimensional spaces, namely *spectral Galerkin* methods and *finite element* methods. For the time being, we will restrict these constructions to one-dimensional problems since their multi-dimensional generalisation requires some technicalities addressed in the next chapter.

We will perform a numerical analysis of the Galerkin method, showing that the error depends on the discrete space approximability properties. After proving some approximability results, we will obtain error estimates for the finite element method in one dimension.

2.1 Galerkin method

The idea of Galerkin methods is quite simple. Let us recall an abstract variational formulation:

$$u \in V_g \ : \quad a(u,v) = \ell(v), \quad \forall v \in V_0. \tag{2.1}$$

As commented above, this problem is infinite dimensional. In order to *solve* this problem, let us consider *finite dimensional vector subspaces* $V_{N,0} \subset V_0$ (the vector space with homogeneous boundary conditions) and $V_N \subset V$ (without any Dirichlet boundary conditions), where $N \doteq \dim(V_N)$. Let us also consider the affine space $V_{N,g} \subset V_N$ of discrete functions satisfying the Dirichlet boundary conditions. Now, we can solve the following problem.

Definition 2.1.1: Galerkin approximation

Let us consider the variational formulation in (2.1) and subspaces $V_{N,0} \subset V_0$ and $V_N \subset V$. The Galerkin approximation of (2.1) in $V_{N,0}$ reads:

$$u_N \in V_{N,g} \; : \quad a(u_N, v) = \ell(v), \quad \forall v \in V_{N,0}. \tag{2.2}$$

We can readily check that if the variational formulation is linear and well-posed (which now we can check using the results in the previous chapter), the Galerkin approximation is also well-posed (see Theorem 1.2.14). We note that the Galerkin problem in (2.2) is also called the *Galerkin projection* of the problem. The previous problem can be understood as solving (2.1) in the subspace V_N.

Analogously, we could consider the discrete version of the quadratic minimisation problem in Definition 1.2.2. Besides, for non-homogeneous boundary conditions, we can analogously use at the discrete level the offset function method commented above.

$V_{N,0}$ and V_N are real vector spaces of finite dimension, for which we can pick a *basis*.

Definition 2.1.2: Basis of a finite dimensional vector space

Given a finite dimensional real vector space V_M, the set $\{b_1, \ldots, b_M\} \subset V_M$, $M \in \mathbb{N}$ is a basis of V_M if $\forall v \in V_M$ there is a unique set of coefficients $\{v_i\}_{i=1}^M \subset \mathbb{R}$ such that $v = v_1 b_1 + \ldots + v_M b_M$. M is equal to the dimension of V_M.

A finite dimensional functional space V_N of dimension N is isomorphic to $\mathbb{R}^N$. Such an isomorphism can be defined by considering an *ordered* basis and mapping functions in V_N to their unique vector of coefficients $\boldsymbol{\mu} \in \mathbb{R}^N$.

To consider the linear system related to the Galerkin problem, let us first apply the offset function method to the Galerkin formulation Definition 2.1.1.

Definition 2.1.3: Galerkin method with offset function

Let us consider the Galerkin problem in Definition 2.1.1. Let us pick an offset function $u_{N,0} \in V_{N,g} \subset V_N$. The solution of the Galerkin problem reads as $u_N = u_{N,0} + \delta u_N$, where

$$\delta u_N \in V_{N,0} \; : \; a(\delta u_N, v_N) = \tilde{\ell}(v_N) \doteq \ell(v_N) - a(u_{N,0}, v_N), \;\; \forall v_N \in V_{N,0}. \tag{2.3}$$

Lemma 2.1.4: Galerkin system as linear system

We consider a basis $\mathcal{B} \doteq \left\{b_N^1, \ldots, b_N^{N_0}\right\}$ *of* $V_{N,0}$*, where clearly* $N_0 \doteq \dim(V_{N,0})$*. The Galerkin problem in (2.3) is equivalent to the linear system: find* $\delta u_N = \sum_{i=1}^{N_0} \mu_i b_N^i$ *with*

$$\begin{aligned} \boldsymbol{\mu} \in \mathbb{R}^{N_0} \quad : \quad & \boldsymbol{A}\boldsymbol{\mu} = \boldsymbol{f}, \quad \textit{where} \\ & \boldsymbol{A} \in \mathbb{R}^{N_0 \times N_0}, \quad \boldsymbol{A}_{ij} \doteq a(b_N^j, b_N^i), \quad i, j \in \{1, \ldots, N_0\}, \\ & \boldsymbol{f} \in \mathbb{R}^{N_0}, \quad \boldsymbol{f}_i \doteq \tilde{\ell}(b_N^i), \quad i \in \{1, \ldots, N_0\}. \end{aligned}$$

Proof. Let us consider the solution $\delta u_N \in V_{N,0}$ of (2.3), and its unique expression $\delta u_N = \sum_{i=1}^{N_0} \mu_i b_N^i$. Using (2.3) with this expression of δu_N as a linear combination of elements in $\mathcal{B}$ and using as test function the elements of the basis, we get

$$a\left(\sum_{j=1}^{N_0} \mu_j b_N^j, b_N^i\right) = \boldsymbol{A}_{ij}\boldsymbol{\mu}_j = \tilde{\ell}(b_N^i) = \boldsymbol{f}_i, \quad \forall i \in \{1, \ldots, N_0\}. \tag{2.4}$$

Thus, it solves the linear system. On the other hand, for any $v \in V_{N,0}$, we can write it as $v = \sum_{i=1}^{N_0} v_i b_N^i$. Multiplying equation i in (2.4) times v_i and adding up for $i = 1, \dots, N_0$, we readily check that the variational equation (2.3) holds. □

Remark 2.1.5: The choice of basis

We note that the choice of the basis $\mathcal{B}$ *does not affect* the solution u of (2.2) but it does affect the matrix $\boldsymbol{A}$, the right-hand side $\boldsymbol{f}$ and the solution vector $\boldsymbol{\mu}$. In short, it determines the isomorphism between $V_{N,0}$ and $\mathbb{R}^{N_0}$.

2.2 Spectral Galerkin methods

Spectral Galerkin methods approximate the infinite-dimensional space V (and V_0) by the space of polynomials up to a given order (with zero boundary values in the case of V_0). For simplicity, let us consider that $\Omega \subset \mathbb{R}$, even though the generalisation to d-cubes does not require new ingredients. We choose V_N to be the set of polynomials up to order p in $\mathbb{R}$, which is denoted by $\mathcal{P}_p(\mathbb{R}) \doteq \operatorname{span}\{1, x, \dots, x^p\}$. In order to enforce boundary conditions, we consider $V_{N,0} \doteq \mathcal{P}_p(\mathbb{R}) \cap C_0^0(\overline{\Omega})$. We note that the space of polynomials $\mathcal{P}_p(\mathbb{R})$ is a vector space of dimension $p+1$. On the other hand, the imposition of the zero boundary conditions at the end-points of Ω reduces the dimension by two, since they involve two independent restrictions on such space. As a result $N \doteq p+1$ and $N_0 \doteq p-1$.

Example 2.2.1: Choice of the basis in finite dimension

As commented in the previous section, the choice of the basis for our finite dimensional space does not affect the solution. This assertion would certainly be true if computations were performed with exact arithmetic. However, this is not the case with computers, which can only perform floating point operations up to finite precision. Thus, in practice, the choice of the basis *can dramatically affect the solution* due to rounding errors. One example is the basis for the polynomial spaces. For instance, one can easily check that, for $\Omega = [-1, 1]$,

$$V_{N,0} = \operatorname{span}\left\{1 - x^2, x(1-x^2), \dots, x^{p-2}(1-x^2)\right\}.$$

However, such monomial-like basis is highly ill-conditioned and useless for high values of p. Instead, one must consider other polynomial bases like Legendre polynomials. In any case, we are not going to explore this issue here.

In practical applications, the right-hand side will have a general expression. The matrix can also involve physical parameters, e.g., a non-constant heat conductivity or Young's modulus. So, we want a general procedure for numerical integration of functions in terms of floating point operations that can easily be implemented in computers. Thus, we can replace integrals by an m-point quadrature formula on $\Omega \doteq [a, b] \subset \mathbb{R}$, $m \in \mathbb{N}$,

$$\int_a^b f(x)\mathrm{d}x \approx Q_m^{[a,b]}(f) \doteq \sum_{j=1}^{m} \omega_j^m f(\xi_j^m),$$

where ω_j^m are the quadrature *weights* and ξ_j^m are the quadrature *points*. The optimal choice for these weights and quadratures in the domain $[-1, 1]$ is the m-point Gauss quadrature, which is exact up to polynomials of degree $2m-1$. We note that we can write the integral over a general interval $[a, b]$ as

$$\int_a^b f(x)\mathrm{d}x = \frac{b-a}{2} \int_{-1}^{1} f\left(\frac{a+b}{2} + \frac{b-a}{2}\xi\right)\mathrm{d}\xi,$$

and apply the Gauss quadrature formula. This way, we can state the spectral Galerkin method in terms of floating point operations only and thus, solve it using computers.

Non-homogeneous boundary conditions in the spectral Galerkin method can be applied as follows. Given the boundary condition

$$u(a) = u_a, \quad u(b) = u_b$$

on the end-points of $[a, b]$, we need to find a polynomial that *goes through* these two points, i.e., that interpolates these values. For two points, we need at least a polynomial in $\mathcal{P}_1(\mathbb{R})$. So, let us consider the *unique* first order interpolation polynomial for the data (a, u_a), (b, u_b) (thus satisfying the boundary conditions), which has the following expression

$$u_0(x) = \frac{bu_a - au_b}{b-a} + \frac{u_b - u_a}{b-a}x.$$

In the next definition, we state the spectral Galerkin method.

Definition 2.2.2: Spectral Galerkin method with offset function

Let us consider the continuous problem in (2.1) where a is a bilinear form. Let us consider the domain $\Omega \doteq [a, b] \subset \mathbb{R}$ and the boundary conditions $u(a) = u_a$ and $u(b) = u_b$. We define the Spectral Galerkin approximation u_p of order $p \in \mathbb{N}^+$ as follows. First, we define the offset function

$$u_{p,0}(x) = \frac{bu_a - au_b}{b-a} + \frac{u_b - u_a}{b-a}x, \qquad x \in [a, b].$$

Let us define $\mathcal{V}_p \doteq \mathcal{P}_p([a, b]) \cap C_0^0([a, b])$. Next, we compute

$$\delta u_p \in \mathcal{V}_p \ : \ a(\delta u_p, v_p) = \ell(v_p) - a(u_{p,0}, v_p), \qquad \forall v_p \in \mathcal{V}_p.$$

The Spectral Galerkin approximation finally reads $u_p \doteq u_{p,0} + \delta u_p$.

We note that we have not only defined a finite dimensional space but a *family* of spaces parameterised with the order p, i.e., $\{\mathcal{V}_p\}_{p=1}^{\infty}$. In our simulations, we will certainly pick an order. Still, it will be interesting to consider how the solution improves as we increase that order, i.e., to perform a *convergence* analysis of the solution.

2.3 Finite element methods

Spectral methods make use of global polynomials in the domain Ω to build the finite dimensional spaces in the Galerkin formulation. Instead, finite element methods consider a partition of the domain into pieces (*elements or cells* in a *mesh*) and globally continuous functions that are polynomials inside the cells, i.e., *piecewise* polynomial continuous functions.

The first ingredient in a finite element method is the *mesh*, i.e., a partition of the domain Ω. As for the spectral Galerkin method, we consider a 1D problem with $\Omega \doteq [a, b]$.

Definition 2.3.1: Mesh in $\Omega \subset \mathbb{R}$

Given a domain $\Omega \doteq [a, b] \subset \mathbb{R}$, $M \in \mathbb{N}$, and a set of *nodes*

$$\Xi_M \doteq \{a \doteq x_0 < x_1 < \ldots < x_{M-1} < x_M \doteq b\},$$

we can define a mesh $\mathcal{M}_M$ of Ω as

$$\mathcal{M}_M \doteq \{(x_{j-1}, x_j), \ : \ 1 \leq j \leq M\}.$$

The open sub-intervals (x_{j-1}, x_j) are the *cells* of $\mathcal{M}_M$, with cell *size* $h_j \doteq |x_j - x_{j-1}|$. The mesh *width* is $h \doteq \max_{j \in \{1,\ldots,M\}} h_j$. In the particular case in which all cells have the same size, i.e., $h_j \doteq h$ for any $j \in \{1, \ldots, M\}$, the mesh is called *uniform*.

On top of the 1D mesh, let us create a finite dimensional space $V_{N,0}$. To do that, we define the following function.

Definition 2.3.2: Hat function in 1D linear finite elements

Given a mesh $\mathcal{M}_{N+1}$, for every interior node x_j, $j = 1, \dots, N$, we define its corresponding hat function b_N^j as

$$b_N^j \doteq \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}}, & x_{j-1} \le x < x_j, \\ 1 - \frac{x - x_j}{x_{j+1} - x_j}, & x_j \le x < x_{j+1}, \\ 0 & \text{otherwise}, \end{cases}$$

At the end-points, i.e., $j \in \{0, N+1\}$, we consider the hat functions as

$$b_N^0(x) \doteq \begin{cases} 1 - \frac{x-a}{h_1} & a \le x \le x_1 \\ 0 & \text{otherwise}, \end{cases}$$
$$b_N^{N+1}(x) \doteq \begin{cases} 1 - \frac{b-x}{h_{N+1}} & x_N \le x \le b, \\ 0 & \text{otherwise}. \end{cases}$$

The linear finite element space associated with $\mathcal{M}_{N+1}$ is defined as

$$V_N \doteq \operatorname{span}\left\{b_N^0, b_N^1, \dots, b_N^{N+1}\right\}, \qquad V_{N,0} \doteq \operatorname{span}\left\{b_N^1, \dots, b_N^N\right\}.$$

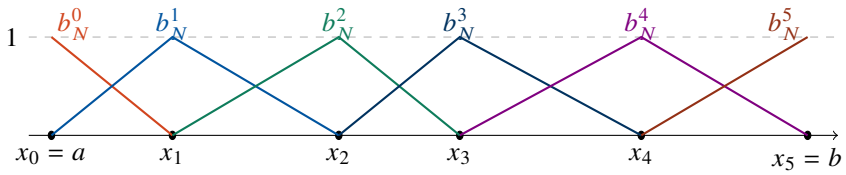


Figure 2.1: The hat function basis of the linear finite element space V_N on a non-uniform mesh of $[a, b]$, here with $N + 1 = 5$ cells ($N = 4$ interior nodes). Each interior hat function takes the value one at its node, vanishes at all other nodes, and is supported on the two cells that contain its node; the end-point functions are supported on one cell.

It is easy to check that functions in $V_{N,0}$ vanish at the end-points. We can also observe that the hat functions are continuous and piecewise linear polynomials; see Figure 2.1. In fact, the hat function b_N^j associated to node x_j is equal to zero at all cells of the mesh $\mathcal{M}_{N+1}$ but the ones that *contain* x_j, i.e., $[x_{j-1}, x_j]$ and $[x_j, x_{j+1}]$. In these two cells, the function is a first order polynomial, reason why it is called a linear finite element space. We can also re-state the global finite element spaces as

$$V_N \doteq \left\{v \in C^0([a,b]) \,:\, v|_{[x_{j-1},x_j]} \in \mathcal{P}_1([x_{j-1}, x_j]),\ j = 1, \dots, N+1\right\}, \tag{2.5}$$
$$V_{N,0} \doteq V_N \cap C_0^0([a,b]).$$

On the other hand, we can also check that $b_N^j(x_i) = \delta_{ij}$, i.e., hat functions take the value one in their corresponding node and zero at all other nodes. In the finite element method, the basis $\mathcal{B}_N$ that satisfies this property is the basis of *shape functions*, which we will define below.

We also know from the first chapter that the hat functions are in $H^1(\Omega)$. Thus, $V_N \subset H^1((a,b))$ and $V_{N,0} \subset H_0^1((a,b))$. The derivative of the interior hat function b_N^j takes the following value

$$\frac{\mathrm{d}b_N^j}{\mathrm{d}x}(x) \doteq \begin{cases} \frac{1}{h_j}, & x_{j-1} \le x < x_j \\ -\frac{1}{h_{j+1}}, & x_j \le x \le x_{j+1} \\ 0, & \text{otherwise}. \end{cases} \tag{2.6}$$

We proceed similarly for the end-point shape functions. Let us remark that *finite element spaces are unsuitable if we require a pointwise well-defined (classical) derivative*. The fact that we will only need our space to be a subset of $H^1(\Omega)$ is what allows us to use finite element spaces (for partial differential equations that involve at most second-order derivatives in their strong form).

Definition 2.3.3: Finite element method with offset function

Let us consider the continuous problem in (2.1) where a is a bilinear form. Let $\Omega \doteq [a, b] \subset \mathbb{R}$ and the boundary conditions $u(a) = u_a$ and $u(b) = u_b$. Given a mesh $\mathcal{M}_{N+1}$, we build the finite element spaces V_N and $V_{N,0}$ in (2.5) . With these ingredients, we define the finite element approximation u_N as follows. First, we define the offset function

$$u_{N,0}(x) = u_a b_N^0 + u_b b_N^{N+1}.$$

Next, we compute

$$\delta u_N \in V_{N,0} \ : \ a(\delta u_N, v_N) = \ell(v_N) - a(u_{N,0}, v_N), \ \forall v_N \in V_{N,0}.$$

The finite element approximation finally reads $u_N \doteq u_{N,0} + \delta u_N$.

2.3.1 Computing the entries of the linear system

In this section, we will apply the finite element method to an elementary problem computing by hand the matrix and right-hand side.

Example 2.3.4: 1D model problem

We consider our 1D model problem introduced in the previous chapter:

$$u \in H_0^1((a,b)) : \int_a^b \kappa(x) \frac{\mathrm{d}u}{\mathrm{d}x}(x) \frac{\mathrm{d}v}{\mathrm{d}x}(x) \mathrm{d}x = \int_a^b f(x) v(x) \mathrm{d}x,$$

$\forall v \in H_0^1((a,b))$. We consider homogeneous boundary conditions, $f(x) = 1$ and $\kappa(x) = 1$ for simplicity. We want to compute the system matrix for a mesh with $N + 1$ cells.

Using Lemma 2.1.4, the entries of the matrix and right-hand side using the Galerkin method applied with the 1D linear finite element space read

$$\boldsymbol{A}_{ij} = \int_a^b \frac{\mathrm{d}b_N^j}{\mathrm{d}x}(x) \frac{\mathrm{d}b_N^i}{\mathrm{d}x}(x) \mathrm{d}x.$$

Now, using the expression of the derivatives of the hat functions in (2.6), we get

$$\boldsymbol{A}_{ij} \doteq \begin{cases} -\frac{1}{h_{i+1}}, & \text{if } j = i+1 \\ -\frac{1}{h_i}, & \text{if } j = i-1 \\ \frac{1}{h_i} + \frac{1}{h_{i+1}}, & \text{if } j = i \\ 0 & \text{otherwise,} \end{cases}$$

for $i, \ j = 1, \ldots, N$. An extremely important property of finite element methods is their *sparsity*; the entries

$\boldsymbol{A}_{ij}$ of the matrix are equal to zero if $|i-j| \geq 2$. Thus, the system matrix will read:

$$\begin{bmatrix} \frac{1}{h_1}+\frac{1}{h_2} & -\frac{1}{h_2} & 0 & \dots & & 0 \\ -\frac{1}{h_2} & \frac{1}{h_2}+\frac{1}{h_3} & -\frac{1}{h_3} & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \ddots & & 0 \\ \vdots & & & & & \vdots \\ 0 & \ddots & \ddots & 0 & -\frac{1}{h_N} & \frac{1}{h_N}+\frac{1}{h_{N+1}} \end{bmatrix}$$

As commented above, we need a numerical quadrature to integrate the right-hand side. For simplicity, let us consider the trapezoidal rule to perform the numerical integration at every cell of the mesh, i.e.,

$$\int_{x_{i-1}}^{x_i} g(x)\mathrm{d}x \approx \frac{1}{2}h_i(g(x_{i-1})+g(x_i)).$$

Assuming that we have a subroutine that allows us to evaluate the forcing term at a point, we can compute the entries of the right-hand side for a unit force as follows:

$$\boldsymbol{f}_j = \int_a^b b_N^j(x)\mathrm{d}x = \frac{h_j+h_{j+1}}{2}, \qquad j=1,\dots,N.$$

In the particular case in which the mesh is uniform with a mesh size h, the linear system reads:

$$\begin{bmatrix} 2 & -1 & 0 & \dots & & 0 \\ -1 & 2 & -1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \ddots & & 0 \\ \vdots & & & & & \vdots \\ 0 & \ddots & \ddots & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_N \end{bmatrix} = h^2 \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$$

The systems that arise from finite element and finite difference schemes are identical for a 1D uniform mesh and a trapezoidal rule for the right-hand side. However, this is not the case for general meshes or $d > 1$, as we will see.

Now, let us consider the non-homogeneous boundary conditions $u(a) = u_a$ and $u(b) = u_b$. Let us note that the end-point hat functions are not being used for solving the problem with homogeneous boundary conditions; they are essential to pick a suitable offset function. As commented above, we take $u_0 \doteq u_a b_N^0 + u_b b_N^{N+1}$. It is easy to check that u_0 satisfies the right boundary conditions. The main reason why we use such a technique for finite element methods is to keep *sparsity*. With this expression, the additional term to be added to the right hand side is

$$\begin{aligned} -\int_a^b \frac{\mathrm{d}b_N^j}{\mathrm{d}x}\frac{\mathrm{d}u_0}{\mathrm{d}x}\mathrm{d}x &= -u_a\int_a^b \frac{\mathrm{d}b_N^j}{\mathrm{d}x}\frac{\mathrm{d}b_N^0}{\mathrm{d}x}\mathrm{d}x - u_b\int_a^b \frac{\mathrm{d}b_N^j}{\mathrm{d}x}\frac{\mathrm{d}b_N^{N+1}}{\mathrm{d}x}\mathrm{d}x \\ &\doteq \begin{cases} \frac{u_a}{h_1} & j=1 \\ \frac{u_b}{h_{N+1}} & j=N \\ 0 & j\in\{2,\dots,N-1\}. \end{cases} \end{aligned}$$

On the contrary, if we used the polynomial offset function of the spectral Galerkin method, all the entries in the right-hand side would be different from zero. It would imply the computation of terms in all the cells of the mesh, which is a computational cost that we can avoid. On the other hand, as we will see, the finite element approach is a practical way to impose Dirichlet boundary conditions for general geometries in 2D and 3D, whereas it is not possible to do so with global polynomials.

2.4 Error analysis of the Galerkin method

Let us consider our linear variational problem

$$u \in V_0 \ : \ a(u,v) = \ell(v), \qquad \forall v \in V_0,$$

where we recall that V_0 is a real vector space of functions from a physical domain Ω to $\mathbb{R}$.

Assumption 2.4.1: Requirements for a well-posed linear variational problem

We assume $a : V_0 \times V_0 \to \mathbb{R}$ to be a bilinear symmetric and positive definite form and $\ell : V_0 \to \mathbb{R}$ a continuous linear form with respect to the energy norm $\|\cdot\|_a$. Furthermore, V_0 endowed with the inner product provided by a is a Hilbert space (it is complete).

Under these assumptions, we already know from Theorem 1.3.9 that there exists a unique solution $u \in V_0$. Now, let us consider the Galerkin discretisation of this problem, using as trial and test space the vector space $V_{N,0} \subset V_0$, $\dim V_{N,0} < \infty$. The Galerkin approximation reads:

$$u \in V_{N,0} \ : \ a(u,v) = \ell(v), \qquad \forall v \in V_{N,0}.$$

The existence and uniqueness of the discrete problem is a direct consequence of the continuous counterpart since the discrete problem inherits the requirements in Assumption 2.4.1 above (even though we already observed that the proof for the discrete case is much simpler). The remaining point in the Galerkin formulations is the definition of the discrete space $V_{N,0}$; we have learned two ways to define such space in 1D, using either the spectral Galerkin method or the finite element method.

Now, the essential questions that arise are: What is the error we are committing with our discrete scheme? How does this error change with N? In the following, we will derive error bounds for the abstract Galerkin problem above. As we will see, the dependence of the error with respect to N does (naturally) depend on the particular construction of the discrete space $V_{N,0}$, e.g., whether we are using a spectral Galerkin or a finite element method, etc.

2.4.1 Errors in the energy norm

We want to analyse the error committed by the Galerkin method. A natural way to look at this problem is to find bounds for the norm of the error function $u - u_N$. The easiest choice for the error norm is to pick the energy norm. First, let us state what we know about u and u_N. These functions are solution of the continuous and Galerkin problems:

$$\begin{aligned} a(u,v) &= \ell(v), \qquad \forall v \in V_0, \\ a(u_N,v_N) &= \ell(v_N), \qquad \forall v_N \in V_{N,0}. \end{aligned}$$

Thus, using the linearity of the bilinear form and the fact that $V_{N,0} \subset V_0$, we can also pick $v_N \in V_{N,0}$ as test function in the continuous problem. Choosing the same test function in both problems and subtracting, we readily get:

$$a(u - u_N, v_N) = 0, \qquad \forall v_N \in V_{N,0}. \tag{2.7}$$

This result is called *Galerkin orthogonality*; the error function is orthogonal to the discrete space $V_{N,0}$ with respect to the a-inner product.

It tells us that the Galerkin solution is the best possible solution in $V_{N,0}$ with respect to the energy norm. For any $v_N \in V_{N,0}$ we have

$$\begin{aligned} \|u - v_N\|_a^2 &= \|u - u_N\|_a^2 + \|u_N - v_N\|_a^2 + 2a(u - u_N, u_N - v_N) \\ &= \|u - u_N\|_a^2 + \|u_N - v_N\|_a^2. \end{aligned}$$

Theorem 2.4.2: Cea's lemma

Let us assume that the boundary conditions on the Dirichlet boundary $u = g$ on Γ_0 can be exactly imposed with the discrete space V_N, i.e., we can pick $u_{N,g} \in V_N$ such that $u_{N,g} = g$ on Γ_0. The Galerkin discretisation in Definition 2.1.3 under Assumption 2.4.1 satisfies

$$\|u - u_N\|_a = \inf_{v_N \in V_{N,g}} \|u - v_N\|_a.$$

Proof. First, we note that due to the assumptions in the theorem, we can pick the same offset function for both the continuous and discrete problem, thus having exactly the same right-hand side. As a result, we can use the orthogonality in (2.7). Using the bilinearity of a, we readily get

$$\|u - u_N\|_a^2 = a(u - u_N, u - u_N) = a(u - v_N, u - u_N) + a(v_N - u_N, u - u_N)$$

for any $v_N \in V_{N,g}$, so that $v_N - u_N \in V_{N,0}$. The last term in this expression vanishes due to the orthogonality of the Galerkin projection. Thus, using the Cauchy-Schwarz inequality we readily get

$$\|u - u_N\|_a^2 \leq \|u - v_N\|_a \cdot \|u - u_N\|_a.$$

Dividing this expression by $\|u - u_N\|_a$, and taking the infimum with respect to v_N, we prove the result. □

This result gives us an essential piece of information. The Galerkin solution is the best possible solution in the trial space, measured in terms of the energy norm error. As a result, we only need to check how accurately functions in $V_{N,0}$ can approximate the continuous solution.

Cea's lemma also tells us that if we consider two discrete spaces $V_{N,0}$ and $V_{M,0}$ such that $V_{N,0} \subset V_{M,0}$, then the error $\|u - u_M\|$ cannot be larger than $\|u - u_N\|$, since

$$\inf_{v_M \in V_{M,0}} \|u - v_M\|_a \leq \inf_{v_N \in V_{N,0}} \|u - v_N\|_a.$$

For spectral Galerkin methods, the solution improves as we increase the order since $V_p \subset V_{p+1}$. For finite element methods, it also holds when *refining* the mesh $\mathcal{M}_{N+1}$; in 1D, it implies splitting every cell into two cells to get $\mathcal{M}_{2N+2}$. Thus, $V_{N,0} \subset V_{2N+1,0}$. As a result, we can obtain more accurate approximations of our problem by enlarging the discrete space in the Galerkin method. In any case, we still do not know how fast the error is reduced in terms of N.

2.5 Approximation theory

As we have commented above, error bounds on the Galerkin approximation can be attained if we can determine how accurately the discrete space can approximate functions in the continuous space. Approximation theory answers this question. First, we will introduce the concept of *interpolation operator*, which takes continuous functions and provides discrete functions. Then, we will use this interpolation operator to determine the *interpolation error* when using piecewise polynomial (finite element) spaces. It will provide a bound for the error of the Galerkin solution by the results in the previous section, i.e., Cea's lemma.

Definition 2.5.1: Interpolation operator for 1D finite elements

Given the linear 1D finite element space

$$V_N = \left\{ v \in C^0([a,b]) \ : \ v|_{[x_{i-1},x_i]} \in \mathcal{P}_1([x_{i-1},x_i]),\ i = 1,\dots,N+1 \right\},$$

we define the interpolation operator $\pi : V \to V_N$ as follows:

$$\pi(v) \doteq \sum_{i=0}^{N+1} v(x_i) b_N^i,$$

where b_N^i stands for the hat functions in Definition 2.3.2.

We can see that this operator is an interpolation operator. It has sense in 1D for $V = H^1((a,b))$ because $C^0([a,b]) \subset H^1((a,b))$ and thus, the functions have pointwise sense. Thus, given a function $v \in V$, to compute its interpolant $\pi(v) \in V_N$, we must evaluate v at the mesh nodes. These functionals (pointwise evaluations $v \mapsto v(x_i)$ for $i = 0, \dots, N+1$) are the so-called *degrees of freedom* of the finite element space V_N.

Definition 2.5.2: Degrees of freedom

Let us consider a set of $\{\sigma_i\}_{i=0}^{N+1}$ linear functionals in V_N. We say that $\{\sigma_i\}_{i=0}^{N+1}$ is an admissible basis of *degrees of freedom* if the operator $I : V_N \to \mathbb{R}^{N+2}$ such that

$$I(v) \doteq [\sigma_0(v), \sigma_1(v), \dots, \sigma_{N+1}(v)]^T$$

is a bijection.

Definition 2.5.3: Shape functions

Given a basis of degrees of freedom, we say that a basis $\{b_N^0, \dots, b_N^{N+1}\}$ of V_N is the basis of *shape functions* if $\sigma_i(b_N^j) = \delta_{ij}$ for $i,\ j = 0, \dots, N+1$.

We can infer from this definition that there is a unique basis of shape functions. Furthermore, the shape functions in Definition 2.3.2 are this basis for the finite element space V_N.

Now, let us prove some key interpolation theory results. We prove stability first and error bounds next.

Lemma 2.5.4: Continuity of the 1D interpolant

The map $\pi : H^1(\Omega) \to V_N \subset H^1(\Omega)$ is continuous (bounded) uniformly with respect to the mesh width h for a bounded $\Omega \subset \mathbb{R}$.

Proof. First, let us note that $V_N \subset H^1(\Omega)$ is a consequence of the fact that hat functions are in $H^1(\Omega)$. Besides, functions in $H^1(\Omega)$ for a bounded $\Omega \subset \mathbb{R}$ are continuous in one dimension. Thus, for any $v \in H^1(\Omega)$ and $x,\ y \in \overline{\Omega}$, we have:

$$|v(y) - v(x)| \leq \int_x^y |v'(s)| \mathrm{d}s \leq |y-x|^{\frac{1}{2}} |v|_{H^1(\Omega)}, \tag{2.8}$$

where we have used the Cauchy-Schwarz inequality. Let us pick x as the point in which $|v|$ reaches its minimum on $\overline{\Omega}$. We readily have that $|v(x)| \leq |\Omega|^{-\frac{1}{2}} \|v\|_{L^2(\Omega)}$; $|\Omega|$ denotes the size (length) of the domain. Thus, we have, by the triangle inequality:

$$\|v\|_{\infty,\Omega} \doteq \sup_{y \in \Omega} |v(y)| \leq |\Omega|^{-\frac{1}{2}} \|v\|_{L^2(\Omega)} + |\Omega|^{\frac{1}{2}} |v|_{H^1(\Omega)}. \tag{2.9}$$

Therefore, pointwise evaluations are bounded in $H^1(\Omega)$. As a result, $\pi : H^1(\Omega) \to H^1(\Omega)$ is a bounded operator in $H^1(\Omega)$. Now, we can easily check that $\pi(v)'|_{[x_{i-1},x_i]} = \frac{v(x_i) - v(x_{i-1})}{h_i}$. Using the result (2.8) for $\Omega \doteq [x_{i-1}, x_i]$, we get

$$|\pi(v)|_{H^1([x_{i-1},x_i])} = h_i^{-\frac{1}{2}} |v(x_i) - v(x_{i-1})| \leq |v|_{H^1([x_{i-1},x_i])}.$$

Adding up for all cells, we get

$$|\pi(v)|_{H^1(\Omega)} \leq |v|_{H^1(\Omega)}.$$

On the other hand,

$$\|\pi(v)\|_{L^2(\Omega)} \leq |b-a|^{\frac{1}{2}} \|\pi(v)\|_{\infty,\Omega}, \qquad \|\pi(v)\|_{\infty,\Omega} \leq \|v\|_{\infty,\Omega}$$

Using these expressions in (2.9) for the domain Ω, we get:

$$\|\pi(v)\|_{L^2(\Omega)} \leq \|v\|_{L^2(\Omega)} + |b-a||v|_{H^1(\Omega)} \leq c\|v\|_{H^1(\Omega)}.$$

As a result, $\|\pi(v)\|_{H^1(\Omega)} \leq c\|v\|_{H^1(\Omega)}$, and thus a continuous (bounded) operator with a constant that is independent of the mesh size h. □

The error estimates below require some regularity over the solution.

Definition 2.5.5: The $H^2(\Omega)$ space

The space $H^2(\Omega)$ is the subspace of functions in $H^1(\Omega)$ whose second derivatives are in $L^2(\Omega)$. The corresponding norm reads:

$$\|v\|_{H^2(\Omega)} \doteq \|v\|_{H^1(\Omega)} + |\nabla v|_{H^1(\Omega)}, \qquad |v|_{H^2(\Omega)} \doteq |\nabla v|_{H^1(\Omega)}.$$

Lemma 2.5.6: Error bounds for the 1D interpolant

For all $v \in H^2(\Omega)$, the interpolant π in a mesh with mesh width h satisfies:

$$\|v - \pi(v)\|_{L^2(\Omega)} \leq h^2 |v|_{H^2(\Omega)}, \qquad |v - \pi(v)|_{H^1(\Omega)} \leq h|v|_{H^2(\Omega)}.$$

Proof. Let us consider the cell $[x_{i-1}, x_i]$ and $(v - \pi(v))|_{[x_{i-1},x_i]}$. Using the fact that $v - \pi(v)$ vanishes at the end-points of the interval, we can make use of the First Poincaré-Friedrichs inequality to bound the $L^2((x_{i-1}, x_i))$ norm by the $H^1((x_{i-1}, x_i))$ seminorm (which is a norm in $H^1_0((x_{i-1}, x_i))$). (Alternatively, you can just use (2.8).) In this case, the diameter of $((x_{i-1}, x_i))$ is h. We get

$$\|v - \pi(v)\|_{L^2((x_{i-1},x_i))} \leq h_i |v - \pi(v)|_{H^1((x_{i-1},x_i))}. \tag{2.10}$$

Now, let us consider the function $v' - \pi(v)'$. Due to the mean-value theorem, we know this function vanishes at some point in the cell (this is in fact all we need to be able to use the Poincaré-Friedrichs inequality, see (2.8)). On the other hand, $\pi(v)'' = 0$ on the cell since it is a first-order polynomial. Thus, we can use the previous bound also for the derivative and obtain

$$\|v - \pi(v)\|_{H^1((x_{i-1},x_i))} \leq h_i |v|_{H^2((x_{i-1},x_i))}.$$

Combining these results and adding them up for all cells in the mesh, we prove the results in the lemma. □

The following corollary is a direct consequence of the previous approximation error estimates and Cea's lemma.

Corollary 2.5.7: Error estimates for the finite element solution in 1D

Under the conditions in Cea's lemma, assuming that the solution satisfies $u \in H^2(\Omega)$, and for $\Omega \subset \mathbb{R}$, a linear finite element space V_N, and a mesh with mesh width h, the following a priori *error bound holds:*

$$\|u - u_N\|_{H^1(\Omega)} \leq Ch|u|_{H^2(\Omega)}.$$

Results with respect to the $L^2(\Omega)$ norm also hold:

$$\|u - u_N\|_{L^2(\Omega)} \leq Ch^2|u|_{H^2(\Omega)},$$

where we have weakened the norm of the error estimate and in turn, improved the convergence order. We will not prove this result since it involves *duality* arguments that are out of the scope of this course.

2.5.1 Higher order methods

Now that we have learned how to construct and analyse linear finite element methods, we can go one step further. We can now consider at every cell of the mesh a polynomial of an arbitrary order p. Given a $p \in \mathbb{N}$ and a mesh $\mathcal{M}_{N+1}$ of the domain $\Omega \doteq [a, b]$, we define the p-th order finite element space as:

$$V_N^p \doteq \left\{ v \in C^0([a,b]) \ : \ v|_{[x_{i-1},x_i]} \in \mathcal{P}_p([x_{i-1},x_i]),\ i = 1,\dots,N+1 \right\}.$$

We must define its degrees of freedom and associated basis of shape functions. It is clear that using only the mesh nodes of the *linear* mesh $\mathcal{M}_{N+1}$ is not enough to uniquely define elements of V_N^p. We need to add more degrees of freedom. To do that, let us consider the following set of DOFs

$$\begin{aligned} \sigma_i^1 &\doteq v(x_i), && i = 0,\dots,N+1, \\ \sigma_i^k &\doteq v\left(x_{i-1} + \frac{(x_i - x_{i-1})k}{p}\right), && i = 1,\dots,N+1,\ k = 1,\dots,p-1 \end{aligned} \tag{2.11}$$

Clearly, for linear finite elements ($p = 1$) we recover the previous definition. For *quadratic* elements ($p = 2$), we have to add one additional node at the mid-point of each cell. For an element of order p, we add $p - 1$ equidistant nodes at each cell of the mesh. It is easy to check that this set of degrees of freedom define a bijective map between V_N^p and $\mathbb{R}^{N_p}$, with $N_p = p(N + 1) + 1$.

Concerning the shape functions, we still keep locality properties. The shape functions associated to *interior* degrees of freedom (the high order ones) σ_i^k vanish in all cells but the cell $[x_{i-1}, x_i]$. At such cell, shape functions are zero on the cell boundary and all interior nodes, i.e., $x_{i-1} + \frac{k(x_i - x_{i-1})}{p}$ for $k = 1,\dots,p-1$, except one. E.g., in the case of quadratic elements, there is only one interior node, one interior degree of freedom, and its corresponding shape function is a parabola that vanishes on the end-points of the cell and takes the value one at the mid-point. The shape functions associated to interior *vertex* nodes have support on two cells, e.g., the one related to σ_i^1 for $i = 1,\dots,N$ has support in cells $[x_{i-1}, x_i]$ and $[x_i, x_{i+1}]$. It takes the value one at its node and vanishes at the other end-points and interior nodes of these two cells.

High-order polynomials are worth the price (in fact, they are much better than mesh refinement in terms of accuracy vs. number of degrees of freedom) as soon as the solution of the problem at hand is regular enough. For example, we can now extend the definition of the interpolant π^p to arbitrary order by simply using the set of degrees of freedom in (2.11). The following error estimates provide us essential information.

Lemma 2.5.8: Error bounds for the 1D interpolant (arbitrary order)

Given a finite element space of order p, i.e., V_N^p, on a mesh with mesh width h, the interpolant π^p satisfies:

$$\|v - \pi^p(v)\|_{L^2(\Omega)} \leq Ch^{p+1}|v|_{H^{p+1}(\Omega)}, \qquad |v - \pi^p(v)|_{H^1(\Omega)} \leq Ch^p|v|_{H^{p+1}(\Omega)},$$

for any $v \in H^{p+1}(\Omega)$, with a constant C independent of h and v.

Proof. The proof of this result is analogous to the one of the linear case. The idea is to obtain a similar result as (2.10) for all derivatives up to order p of the error function $v - \pi^p(v)$. Next, we use the same result for its $p + 1$ derivative and observe that the $p + 1$ derivative of the interpolant vanishes at each cell. □

We can readily combine these interpolation errors with Cea's lemma to get the following result.

Corollary 2.5.9: Error estimates for the finite element solution in 1D

Under the conditions in Cea's lemma and assuming that $\Omega \subset \mathbb{R}$ *for a finite element space* V_N^p *of order* p *in a mesh with mesh width* h *and* $u \in H^{q+1}(\Omega)$ *for* $0 < q \leq p$ *, the following* a priori *error bound holds:*

$$\|u - u_N\|_{H^1(\Omega)} \leq Ch^q |u|_{H^{q+1}(\Omega)}.$$

2.6 Tutorial

1. We want to use a third-order finite element method in 1D and integrate the matrix corresponding to the following bilinear form

$$a(u, v) \doteq \int_a^b u(x)v(x) + \nabla u(x) \cdot \nabla v(x) \mathrm{d}x$$

exactly. Let us consider a Gauss quadrature for the integration in every mesh cell. How many points would we need in the Gauss quadrature at every cell? (Hint: Which is the maximum order of the polynomials we want to integrate in each spatial dimension?)

2. Can you compute the system matrix of the bilinear form

$$a(u, v) \doteq \int_a^b \kappa(x)u'(x)v'(x) \mathrm{d}x,$$

in a 1D mesh, in terms of the cell mesh sizes h_i. Do a diagram as in the finite element matrix above, providing all the information needed to determine the matrix entirely. Use the trapezoidal rule for the integration in each cell.

3. In a finite element code, students are told to show the error between the exact and the finite element solution (using the Galerkin method) in terms of the square of the energy norm, i.e., $\|u - u_N\|_a^2$. One student has implemented instead $\|u\|_a^2 - \|u_N\|_a^2$. Can you tell me which error they are committing? In other words, what is the value of $\|u - u_N\|_a^2 - \|u\|_a^2 + \|u_N\|_a^2$.

4. Let us consider a linear finite element approximation of a problem with an analytical solution in $C^\infty(\Omega)$, with uniform mesh with N elements. We want a more accurate solution. We can consider refining the mesh using *bisection*, i.e., split every cell into two cells, to get a $2N$ mesh or to consider a second order finite element space on the same mesh. What is going to be more effective in terms of error reduction? Quantify it.

Convergence plots

In the tutorials for the unit, which can be found in this Github repos, we evaluate how the error decreases when refining the mesh (h-refinement) and increasing the order (p-refinement). The primary motivation is to observe how the error behaves in terms of degrees of freedom, i.e., the size of the corresponding linear system of equations, which is a measure of the computational cost.

Let us start with linear finite elements on a 1D uniform mesh with N cells. As a result, the mesh width is $\frac{1}{N}$. In this case, the number of degrees of freedom $\sharp \doteq N - 2$. Thus, using the results in Corollary 2.5.9, if our solution is in $H^2(\Omega)$, we have:

$$e_N \doteq \|u - u_N\|_a \leq \frac{C_1}{N},$$

for some constant $C_1 > 0$, or using Landau notation, $e_N \lesssim O\left(N^{-1}\right)$. We can now apply the log function on both sides of this inequality. We readily get

$$\log e_N \leq \log C_1 - \log N = C_2 - \log N.$$

If we consider now finite elements of order p on the same mesh, we can readily check that $\sharp \doteq pN - 1$, whereas $h = \frac{1}{N}$. Using the error bounds in Corollary 2.5.9, and *assuming that* $u \in H^{p+1}(\Omega)$, we obtain

$$e_{N,p} \doteq \|u - u_N\|_a \leq \frac{C_3}{N^p}.$$

Again, if we compute the log function at both sides of the inequality, we get

$$\log e_{N,p} \leq \log C_3 - p \log N = C_4 - p \log N.$$

With these bounds, we can consider the following refinements strategies:

- h-refinement: This refinement involves increasing the cells in our mesh (e.g., using bisection) and keeping the polynomial order p. E.g., using uniform refinement we multiply the number of degrees of freedom $\sharp$ by 2 and reduce the error by a factor $\frac{1}{2}$.

  In this situation, we have $\sharp \approx pN$ and

  $$\log e_{N,p} \leq C_4 - p \log \sharp + p \log p = C_5 - p \log \sharp.$$

  Thus, $\log e_{N,p}$ must be reduced at least linearly with slope $-p$ in terms of $\log \sharp$. As an example, using bisection, increasing $\sharp$ by two, we reduce the error by two. This kind of convergence is called *algebraic* convergence.

- p-refinement: This refinement consists in fixing the mesh and increasing the polynomial order in the finite element method. Using the fact that $\sharp \approx pN$, we have that

  $$\log e_{N,p} \leq C_4 - p \log N = C_4 - \frac{\log N}{N} \sharp.$$

  Thus, $\log e_{N,p}$ is reduced at least linearly in terms of $\sharp$.

As a result, *if the solution is smooth enough*, p-refinement is much more effective than h-convergence. Whereas p-refinement reduces $\log e_{N,p}$ linearly with the number of degrees of freedom $\sharp$, h-refinement does it with respect to $\log \sharp$. The difference is huge!

3 d-dimensional finite elements

At this point, we have already introduced the Galerkin method for the numerical approximation of partial differential equations. The Galerkin method requires finite dimensional subspaces of the (infinite dimensional) functional spaces in which our weak formulation is well-posed. In order to do that, we have built finite element methods for 1D problems. In this chapter we are going to extend to arbitrary dimensions the definition of finite element spaces.

3.1 The boundary value problem in weak form

We are interested in problems governed by partial differential equations (PDEs) posed in a physical domain $\Omega \subset \mathbb{R}^d$ with boundary $\Gamma \doteq \partial\Omega$. In practice $d = 2, 3$ but we are also interested in $d > 3$ for some particular applications. Let us consider a differential operator A, e.g., the Laplace operator $-\Delta$, and a force term $f : \Omega \to \mathbb{R}$. Let us also consider a partition of Γ into a Dirichlet boundary Γ_D and a Neumann boundary Γ_N, and the corresponding boundary data $u_D : \Gamma_D \to \mathbb{R}$ and $g_N : \Gamma_N \to \mathbb{R}$. The boundary value problem reads as follows: find $u(\boldsymbol{x})$ such that

$$\begin{aligned} Au(\boldsymbol{x}) &= f(\boldsymbol{x}) \quad \text{in } \Omega, \\ B_D u(\boldsymbol{x}) &= u_D(\boldsymbol{x}) \quad \text{on } \Gamma_D, \\ B_N u(\boldsymbol{x}) &= g_N(\boldsymbol{x}) \quad \text{on } \Gamma_N. \end{aligned} \tag{3.1}$$

The operator B_D is a trace operator[1] and B_N is the flux operator.[2] Other boundary conditions, e.g., Robin (mixed) conditions can also be considered. We assume the unknown $u(\boldsymbol{x})$ in (3.1) can be a scalar, vector, or tensor field. (The case of multi-field problems is considered in Sect. 3.11.)

For finite element analysis, (3.1) must be understood in a weak sense. The weak formulation can be stated in an abstract setting. Let us consider an abstract problem determined by a Hilbert space $\mathcal{X}$ (*trial space*), a Hilbert space $\mathcal{X}$ (*test space*), a continuous bilinear form $a : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, and a continuous linear form $\ell : \mathcal{X} \to \mathbb{R}$. The abstract problem is stated as: find $u \in \mathcal{X}$ such that

$$a(u, v) = \ell(v), \qquad \text{for any } v \in \mathcal{X}. \tag{3.2}$$

The link between the weak and strong formulations has already been described in the previous chapters. Without mathematical rigor, one can simply consider integration by parts (assuming continuous functions) to transfer derivatives from the trial to the test function. E.g., for the Laplace operator, the bilinear form reads $a(u, v) \doteq \int_\Omega \boldsymbol{\nabla} u \cdot \boldsymbol{\nabla} v \mathrm{d}\Omega$. Furthermore, homogeneous Dirichlet boundary conditions, i.e., $u = 0$ on Γ_D, are usually enforced in a strong way; the functions in $\mathcal{X}$ and $\mathcal{X}$ satisfy these boundary conditions. The extension to non-homogeneous boundary conditions can be done using the offset function method. One can arbitrarily define Eu_D, an extension to $H^1(\Omega)$ of the Dirichlet data u_D, i.e. verifying $Eu_D = u_D$ on Γ_D. Next, we define the function $u_0 \doteq u - Eu_D$ with zero trace on Γ_D and solve (3.2) for u_0 with the right-hand side

$$\ell(v) - a(Eu_D, v).$$

Let us consider one classical example.

[1] The trace operator tells us which are the right boundary conditions to be imposed on the boundary and in which sense do they have sense. E.g., for $H^1(\Omega)$, we can impose the full trace of the unknown. The space of traces is $H^{\frac{1}{2}}(\partial\Omega)$.

[2] We have already seen how to determine the Neumann or natural boundary conditions using integration by parts (assuming smooth trial and test functions). E.g., for problems posed in $H^1(\Omega)$, the fluxes are to be understood in $H^{-\frac{1}{2}}(\Gamma_N)$, the dual of the trace space.

Example 3.1.1: Poisson problem

Let us consider the Poisson problem $-\boldsymbol{\nabla}\cdot\boldsymbol{\kappa}\boldsymbol{\nabla}u = f$ with $u = u_{\mathrm{D}}$ on Γ_{D} and $\boldsymbol{n}\cdot\boldsymbol{\kappa}\boldsymbol{\nabla}u = g_{\mathrm{N}}$ on Γ_N; $\boldsymbol{n}$ is the outward normal. Let us assume that $\boldsymbol{\kappa} \in L^\infty(\Omega)^{d\times d}$ is symmetric and uniformly positive definite, $f \in H^{-1}(\Omega)$, $g_{\mathrm{N}} \in H^{-\frac{1}{2}}(\Gamma_{\mathrm{N}})$, and $u_{\mathrm{D}} \in H^{\frac{1}{2}}(\Gamma_{\mathrm{D}})$. Let us also consider an extension $Eu_{\mathrm{D}} \in H^1(\Omega)$ such that $Eu_{\mathrm{D}} = u_{\mathrm{D}}$ on Γ_{D}. The weak form of the problem reads as: find $u_0 \in H^1_{\Gamma_{\mathrm{D}}}(\Omega)$ (the subspace of functions in $H^1(\Omega)$ that vanish on Γ_{D}) such that

$$\int_\Omega \boldsymbol{\kappa}\boldsymbol{\nabla}u_0\cdot\boldsymbol{\nabla}v\mathrm{d}\Omega = \int_\Omega fv\mathrm{d}\Omega + \int_{\Gamma_{\mathrm{N}}} g_{\mathrm{N}}v\mathrm{d}\Gamma - \int_\Omega \boldsymbol{\kappa}\boldsymbol{\nabla}Eu_{\mathrm{D}}\cdot\boldsymbol{\nabla}v\mathrm{d}\Omega,$$

for any $v \in H^1_{\Gamma_{\mathrm{D}}}(\Omega)$. The solution is $u \doteq u_0 + Eu_{\mathrm{D}}$.

3.2 Space discretization with finite elements

Problem (3.2) is an infinite-dimensional problem. In order to end up with a computable one, we must introduce finite-dimensional subspaces with some approximability properties. We restrict ourselves to *conforming* finite element schemes, which satisfy $\mathcal{X}_h \subset \mathcal{X}$. The discrete problem reads as: find $u_h \in \mathcal{X}_h$ such that

$$a(u_h, v_h) = \ell(v_h), \qquad \text{for any } v_h \in \mathcal{X}_h. \tag{3.3}$$

This is the *Galerkin* problem. One can also define the affine operator

$$\mathcal{F}_h(u_h) = a_h(u_h,\cdot) - \ell_h(\cdot) \in \mathcal{X}_h'. \tag{3.4}$$

We note that for a given u_h this is a bounded linear functional that takes test functions in $\mathcal{X}_h$ and returns real values, thus in its dual space $\mathcal{X}_h'$. Thus, we can state (3.3) as: find $u_h \in \mathcal{X}_h$ such that $\mathcal{F}_h(u_h) = 0$.

In order to define finite element spaces, we require a triangulation $\mathcal{T}_h$ of the domain Ω into a set $\{K\}$ of *cells*. This triangulation is assumed to be conforming, i.e., for two neighbour cells K^+, $K^- \in \mathcal{T}_h$, the intersection of their closures $\bar{K}^+ \cap \bar{K}^-$ is either empty or a *whole* k-face ($k < d$) of both cells.[3] Thus, for every element $K \in \mathcal{T}_h$, we assume that there is a reference cell $\hat{K}_K$ and a diffeomorphism (smooth bijective map) $\boldsymbol{\Phi}_K : \hat{K} \to K$. In what follows, we usually use the notation $\hat{\boldsymbol{x}} \doteq \boldsymbol{\Phi}_K^{-1}(\boldsymbol{x})$ (see Figure 3.2 for a conforming mesh and the geometrical map definition).

Next, we will see the standard procedure for building finite element functional spaces. They rely on a *reference cell* functional space as follows:

1. We define a functional space in the reference cell $\hat{K}$;
2. We define a set of functions in the physical cell K via function and geometrical maps;
3. We define the global space as the assembly of cell-based spaces plus continuity constraints between cells.

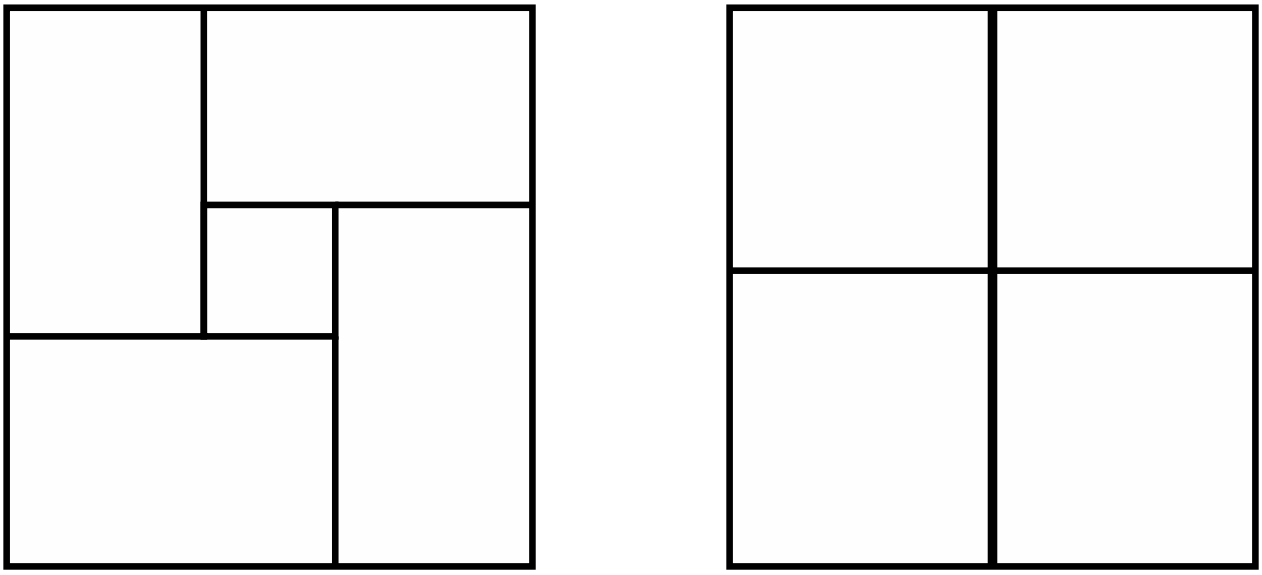
In order to present this process, we introduce the concept of reference finite element, finite element, and finite element space, respectively.

3.3 The finite element in reference and physical space

Using the abstract definition of *Ciarlet*, a finite element is represented by the triplet $\{K, \mathcal{V}, \Sigma\}$, where:

1. K is a compact, connected, Lipschitz subset of $\mathbb{R}^d$,

[3] We note that k-face refers to a geometrical entity for a *polytope*. For a three-dimensional polytope (e.g., a hexahedron or tetrahedron) the cell itself is a 3-face, the faces are 2-faces, the edges are 1-faces and vertices are 0-faces. In finite element methods, the cells can be mapped to a particular type of mapping over a set of admissible geometries (polytopes).



Non-conforming Conforming

Figure 3.1: A non-conforming mesh on the left-hand side; the intersection of the closure of cells is not always the closure of a vertex or edge in all cells. The mesh on the right-hand side is conforming.

2. $\mathcal{V}$ is a vector space of functions,

3. and Σ is a set of linear functionals that form a basis for the dual space $\mathcal{V}'$.

The elements of Σ are the so-called degrees of freedom (DOFs) of the finite element. We denote the number of DOFs as n_Σ. The DOFs can be written as σ_i for $i \in \mathcal{N}_\Sigma \doteq \{1, \dots, n_\Sigma\}$. We can also define a basis $\{b^1, \dots, b^{n_\Sigma}\}$ for $\mathcal{V}$. In particular, we are interested in the basis $\{\phi^i\}_{i \in \mathcal{N}_\Sigma}$ for $\mathcal{V}$ such that $\sigma_i(\phi^j) = \delta_{ij}$ for $i, j \in \mathcal{N}_\Sigma$. These functions are the so-called *shape functions* of the finite element, and there is a one-to-one mapping between shape functions and DOFs.

3.3.1 The reference finite element

In the reference space, we build *reference* finite elements $(\hat{K}, \hat{\mathcal{V}}, \hat{\Sigma})$ as follows. First, we consider a bounded set of possible cell geometries, denoted by $\hat{K}$.[4] On $\hat{K}$, we build a functional space $\hat{\mathcal{V}}$ and a set of DOFs $\hat{\Sigma}$. In this chapter, we will use polynomials spaces for $\hat{\mathcal{V}}$ and the degrees of freedom will be the evaluation of the functions at a set of nodes (points). We consider some examples of reference finite elements later on.

In Figure 3.3 we can see on the left a square bilinear reference finite element. The reference cell is $\hat{K} \doteq (0, 1)^2$. We define as a basis the so-called bilinear polynomials spanned by $\hat{\mathcal{V}} \doteq \text{span}\{1, \hat{x}, \hat{y}, \hat{x}\hat{y}\}$. If we represent the four vertices of $\hat{K}$ with $\{\hat{\boldsymbol{x}}_1, \hat{\boldsymbol{x}}_2, \hat{\boldsymbol{x}}_3, \hat{\boldsymbol{x}}_4\}$, the space of degrees of freedom $\Sigma \doteq \{\hat{\sigma}_1, \hat{\sigma}_2, \hat{\sigma}_3, \hat{\sigma}_4\}$ are the node evaluations, i.e., $\hat{\sigma}_i(\hat{v}) \doteq \hat{v}(\hat{\boldsymbol{x}}_i)$.

As stated above, one can check that Σ is a basis of $\mathcal{V}'$ (the dual space of $\mathcal{V}$, which is the space of linear functionals from $\mathcal{V}$ to $\mathbb{R}$). Note that two bilinear polynomials that have the same nodal values are the same. Thus, the map $v \mapsto [\hat{\sigma}_1(v), \dots, \hat{\sigma}_4(v)]^T$ is injective between spaces of the same dimension (four), and thus a bijection.

3.3.2 From reference to physical spaces

In the physical space, the finite element triplet $(K, \mathcal{V}, \Sigma)$ on a mesh cell $K \in \mathcal{T}_h$ relies on:

[4] Admissible geometries are a segment in one-dimension, triangles and quadrilaterals in two-dimensions, and tetrahedra and hexahedra in three-dimensions.

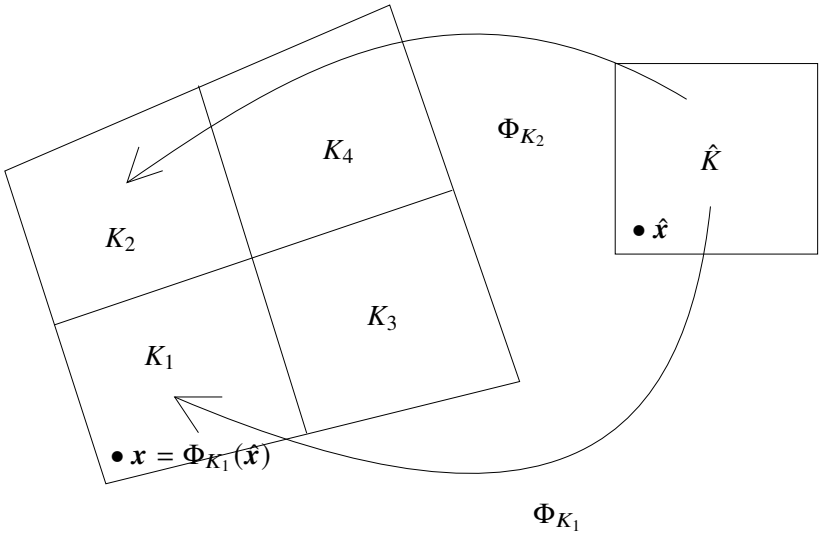


Figure 3.2: In this plot, we show a finite element mesh of 4 quadrilateral cells in the physical space on the left, and the corresponding reference squared cell $\hat{K}$ on the right. For every cell in the physical space K_i , we have a map that takes points $\hat{\boldsymbol{x}} \in \hat{K}$ and returns points $\boldsymbol{x} \in K_i$. These maps are assumed to be smooth and bijective, otherwise the mesh is not suitable for finite element analysis.

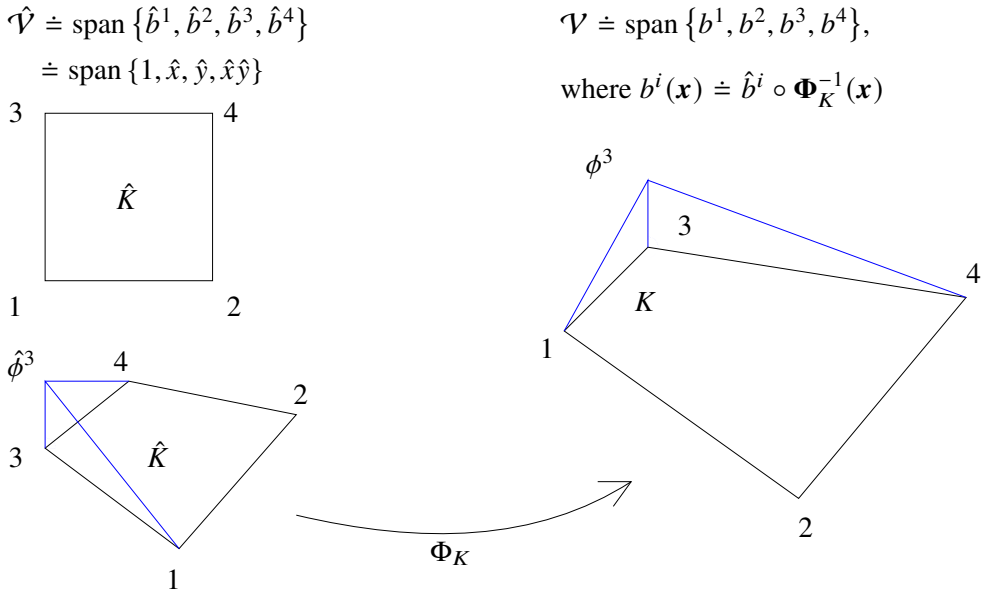


Figure 3.3: In this plot, we show the map from the reference cell $\hat{K}$ to a physical cell K. We consider a linear finite element, and show the indexing for its vertices/nodes. We have a basis for the Lagrangian reference finite element space $\hat{\mathcal{V}} \doteq \{1, \hat{x}, \hat{y}, \hat{x}\hat{y}\}$ and define the one in the reference finite element space from this one and the geometrical map $\mathcal{V}$. For node 3, we plot the shape function associated to the node both in the reference space $\hat{\phi}^3(\hat{\boldsymbol{x}})$ and the physical space $\phi^3(\boldsymbol{x}) = \hat{\phi}^3 \circ \boldsymbol{\Phi}_K^{-1}(\boldsymbol{x})$.

1. a reference finite element $(\hat{K}, \hat{\mathcal{V}}, \hat{\Sigma})$;
2. a geometrical mapping $\boldsymbol{\Phi}_K$ such that $K \doteq \boldsymbol{\Phi}_K(\hat{K})$;[5]
3. a linear bijective function mapping $\hat{\Psi}_K : \hat{\mathcal{V}} \to \hat{\mathcal{V}}$.[6]

With these ingredients, we construct the physical finite element $(K, \mathcal{V}, \Sigma)$ as follows.

1. The geometry is $K \doteq \boldsymbol{\Phi}_K(\hat{K})$;
2. The functional space in the physical space is defined as

$$\mathcal{V} \doteq \{v = \Psi_K(\hat{v}) : \hat{v} \in \hat{\mathcal{V}}\};$$

where $\Psi_K : \hat{\mathcal{V}} \to \mathcal{V}$ is defined as

$$\Psi_K \doteq \hat{\Psi}_K(\hat{v}) \circ \boldsymbol{\Phi}_K^{-1}.$$

Again, in the case of the Lagrangian elements it is just $\Psi_K(\hat{v}) \doteq \hat{v} \circ \boldsymbol{\Phi}_K^{-1}$. In Figure 3.3 we show this construction for grad-conforming finite elements, in which $\Psi_K(\hat{v}) = \hat{v} \circ \boldsymbol{\Phi}_K^{-1}$.

3. The set of DOFs in the physical space is defined as

$$\Sigma \doteq \{\hat{\sigma} \circ \Psi_K^{-1} \ : \ \hat{\sigma} \in \hat{\Sigma}\}.$$

In Figure 3.4 we show this idea for grad-conforming finite elements. The idea is to take a function defined in the physical space $u(x)$ and transform it to the reference space $\hat{u}(\hat{x}) \doteq \Psi_K^{-1}(u)(\hat{x}) = u \circ \boldsymbol{\Phi}_K(\hat{\boldsymbol{x}})$. Now we can apply the degrees of freedom on the resulting function

$$\hat{\sigma}_i\,(u \circ \boldsymbol{\Phi}_K) = u \circ \boldsymbol{\Phi}_K(\hat{\boldsymbol{x}}_i) = u(\boldsymbol{x}_i).$$

[5] $\boldsymbol{\Phi}$ must be a smooth bijective mapping with smooth inverse

[6] This is the general definition for a finite element in the physical space. Nevertheless, the last ingredient is not required for the finite element spaces considered herein, Lagrangian finite element spaces for problems in $H^1(\Omega)$, a.k.a. *grad-conforming*[7] finite elements. As a result, you can think this operator $\hat{\Psi}_K$ *is just the identity*.

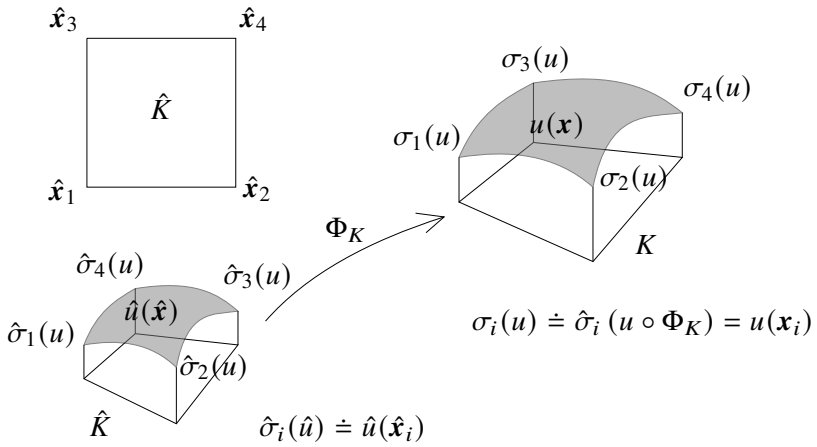


Figure 3.4: In this plot, we show the degrees of freedom for a linear finite element in two-dimensions. In Lagrangian finite elements, all degrees of freedom have a geometrical interpretation, i.e., they are linked to a node (point). For linear elements, the nodes are the vertices of the quadrilateral (idem for triangles). In the reference cell $\hat{K}$, given a function $u \in C^0(\overline{\hat{K}})$, the degrees of freedom are just the evaluations of the function in the corresponding nodes, i.e., $\hat{\sigma}_i(\hat{u}) \doteq \hat{u}(\hat{x}_i)$. In the physical space, we compose the solution in $C^0(\overline{K})$ with Ψ_K^{-1} and now we can apply the reference degree of freedom, i.e., $\sigma_i(u) \doteq \hat{\sigma}_i\left(\Psi_K^{-1}(u)\right) = \hat{\sigma}_i\,(u \circ \Phi_K) = (u \circ \Phi_K)(\hat{x}_i) = u(x_i)$.

The reference finite element space $\hat{\mathcal{V}}$ is usually a polynomial space. Thus, the first ingredient is to define bases of polynomials, e.g., monomial bases; see Sect. 3.4. In Sect. 3.5, we show how to compute the basis of shape functions out of whatever basis $\hat{\mathcal{V}}$ in the reference space. Given the set of shape functions $\{\hat{\phi}^i : i \in \mathcal{N}_{\hat{\Sigma}}\}$ in the reference finite element, it is easy to check that $\{\phi^i_K \doteq \Psi_K(\hat{\phi}^i) : i \in \mathcal{N}_{\hat{\Sigma}}\}$ are the set of shape functions of the finite element in the physical space.

The definition of degrees of freedom in the physical space is essential to project functions in an infinite dimensional space, e.g., $C^0\left(\overline{K}\right)$, into $\mathcal{V}$. Given a function v, we define the *local interpolator* for the finite element at hand as

$$\pi_K(v) \doteq \sum_{i \in \mathcal{N}_\Sigma} \sigma_i(v)\phi^i.$$

One can check that the interpolation operator is in fact a projection. This operation is, e.g., used to project Dirichlet boundary conditions into the finite element space using the offset function method.

Since the following exposition is restricted to Lagrangian FEs, we will not use $\hat{\Psi}$ (it is just the identity) and we will always use $\Psi_K(\hat{v}) = \hat{v} \circ \mathbf{\Phi}_K^{-1}$.

3.4 Construction of polynomial spaces

Local finite element spaces are usually polynomial spaces. Given an order $k \in \mathbb{N}$ and a set $\mathcal{N}_k$ of distinct points (nodes) in $\mathbb{R}$ (we will indistinctly represent nodes by their index i or position x_i), we define the corresponding set of Lagrangian polynomials $\{\ell_0^k, \ldots, \ell_k^k\}$ as:

$$\ell_m^k(x) \doteq \frac{\Pi_{n \in \mathcal{N}_k \setminus \{m\}}(x - x_n)}{\Pi_{n \in \mathcal{N}_k \setminus \{m\}}(x_m - x_n)}.$$

We can also define the Lagrangian basis $\mathcal{L}^k = \{\ell_i^k \,:\, 0 \le i \le k\}$. This set of polynomials is a basis for k-th order polynomials. We note that $\ell_m^k(x_l) = \delta_{ml}$, for $0 \le m,\, l \le k$.

For multi-dimensional spaces, we can define the set of nodes as the tensor product of 1D nodes. Given a d-tuple order $\boldsymbol{k}$, we define the corresponding set of nodes for d-cubes as: $\mathcal{N}^{\boldsymbol{k}} \doteq \mathcal{N}^{k_1} \times \cdots \times \mathcal{N}^{k_d}$ (see Figure 3.5).

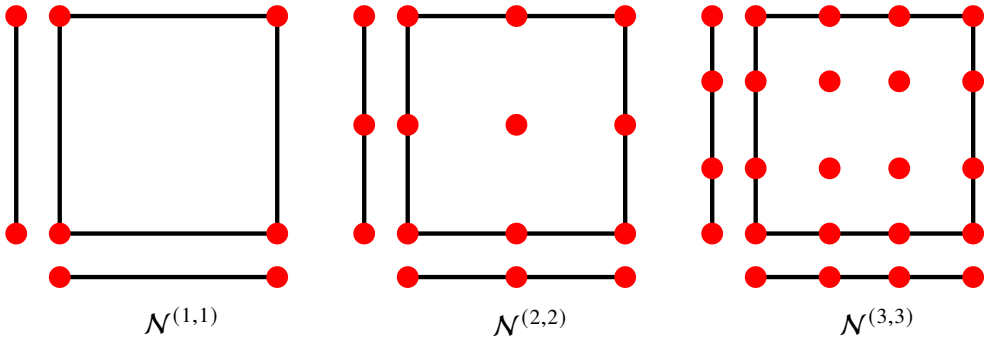


Figure 3.5: Equidistant nodes for two-dimensional Lagrangian polynomial spaces $\mathcal{Q}_{(1,1)}$, $\mathcal{Q}_{(2,2)}$ and $\mathcal{Q}_{(3,3)}$ obtained as tensor product of their one-dimensional counterparts. In short, these isotropic order spaces are expressed as $\mathcal{Q}_1$, $\mathcal{Q}_2$ and $\mathcal{Q}_3$, resp.

Analogously, we define the multi-dimensional Lagrange basis

$$\mathcal{L}^{\boldsymbol{k}} = \{\ell^{\boldsymbol{k}}_{\boldsymbol{m}} : \boldsymbol{m} \in \mathcal{N}^{\boldsymbol{k}}\}, \qquad \text{where} \quad \ell^{\boldsymbol{k}}_{\boldsymbol{m}}(\boldsymbol{x}) \doteq \Pi_{i=1}^{d} \ell^{k_i}_{m_i}(\boldsymbol{x} \cdot \boldsymbol{e}_i).$$

Clearly, $\ell^{\boldsymbol{k}}_{\boldsymbol{t}}(\boldsymbol{x}_{\boldsymbol{s}}) = \delta_{\boldsymbol{st}}$, for $\boldsymbol{s}, \boldsymbol{t} \in \mathcal{N}^{\boldsymbol{k}}$. [8]

This tensor product construction leads to a basis for the space of polynomials that are of degree less or equal to k with respect to each variable $x_1, \ldots, x_d$. We can define monomials by a d-tuple $\boldsymbol{\alpha}$ as $p_{\boldsymbol{\alpha}}(\boldsymbol{x}) \doteq \Pi_{i=1}^{d} x_i^{\alpha_i}$, and the polynomial space of order $\boldsymbol{k}$ as $\mathcal{Q}_{\boldsymbol{k}} = \text{span}\{p_{\boldsymbol{\alpha}}(\boldsymbol{x}) : \boldsymbol{\alpha} \in \mathcal{N}^{\boldsymbol{k}}\}$. This space is also spanned by the Lagrangian basis $\mathcal{Q}_{\boldsymbol{k}} = \text{span}\{\boldsymbol{\ell} : \boldsymbol{\ell} \in \mathcal{L}^{\boldsymbol{k}}\}$. For isotropic spaces in which the same order $p \in \mathbb{N}$ is used in all dimensions we simply write $\mathcal{Q}_p \doteq \mathcal{Q}_{p\mathbf{1}}$ where $p\mathbf{1} \doteq (p, \ldots, p)$.

3.4.1 Local finite element space in cubes

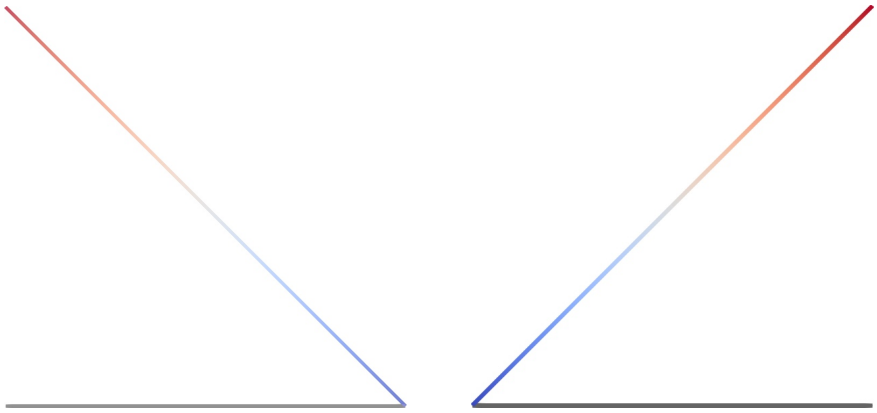
When the reference geometry $\hat{K}$ is an d-cube, we define the reference finite element space as $\mathcal{V}_k \doteq \mathcal{Q}_{k\mathbf{1}}$ with the same order k for all components. The set of nodes $\mathcal{N}^{k\mathbf{1}}$ can be generated, e.g., from the equidistant Lagrangian nodes. Let us define the bijective mapping $\mathtt{i}(\cdot)$ from the set of nodes $\mathcal{N}^{k\mathbf{1}}$ to $\{1, \ldots, |\mathcal{N}^{k\mathbf{1}}|\} \equiv \mathcal{N}_\Sigma$, i.e., the local node numbering. The set of local DOFs $\mathcal{N}_{\Sigma_K}$ are the nodal values, i.e., $\sigma_{\mathtt{i}(\boldsymbol{s})} \doteq v(\boldsymbol{x}_{\boldsymbol{s}})$, for $\boldsymbol{s} \in \mathcal{N}^{\boldsymbol{k}}$. Clearly, the reference finite element shape functions related to these DOFs are $\phi^{\mathtt{i}(\boldsymbol{s})} \doteq \ell^{k\mathbf{1}}_{\boldsymbol{s}}$.

Let us start with the one-dimensional spaces (see Figure 3.6 for linear and Figure 3.7 for quadratic shape functions, resp.). These Lagrangian bases span $\mathcal{Q}_1 \doteq \text{span}\,\{1, x\}$ and $\mathcal{Q}_2 \doteq \text{span}\,\{1, x, x^2\}$. Using the tensor product definition we can readily construct higher-dimensional shape functions. The isotropic space obtained as tensor product of linear polynomials in two-dimensions is the bilinear space $\mathcal{Q}_1 = \text{span}\,\{1, x, y, xy\}$. The one for second order polynomials is

$$\mathcal{Q}_2 = \text{span}\,\{1, x, y, xy, x^2, y^2, x^2y, xy^2, x^2y^2\}\,.$$

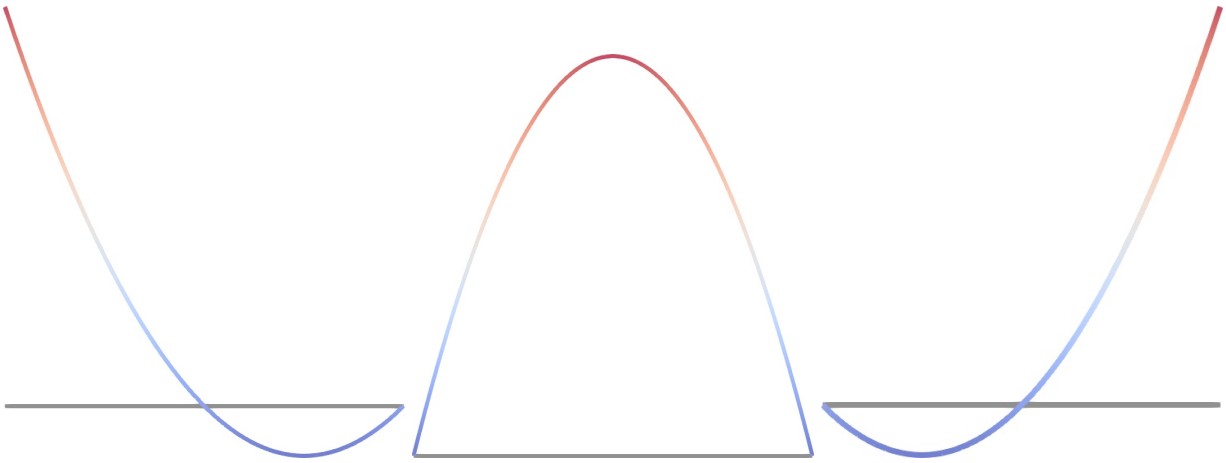
The corresponding Lagrangian bases (of shape functions) are collected in Figure 3.8 for linear and Figure 3.9 for quadratic shape functions, resp.).

[8] A different polynomial order in each space dimension (anisotropic spaces) can be required when building some finite element spaces. However, for the case we want to consider in this text, i.e., grad-conforming finite elements, we will use the same order in all dimensions (isotropic spaces). E.g., $\boldsymbol{k} \doteq (p, p)$ in two-dimensions and $\boldsymbol{k} \doteq (p, p, p)$ in three-dimensions, for $p \in \mathbb{N}$.



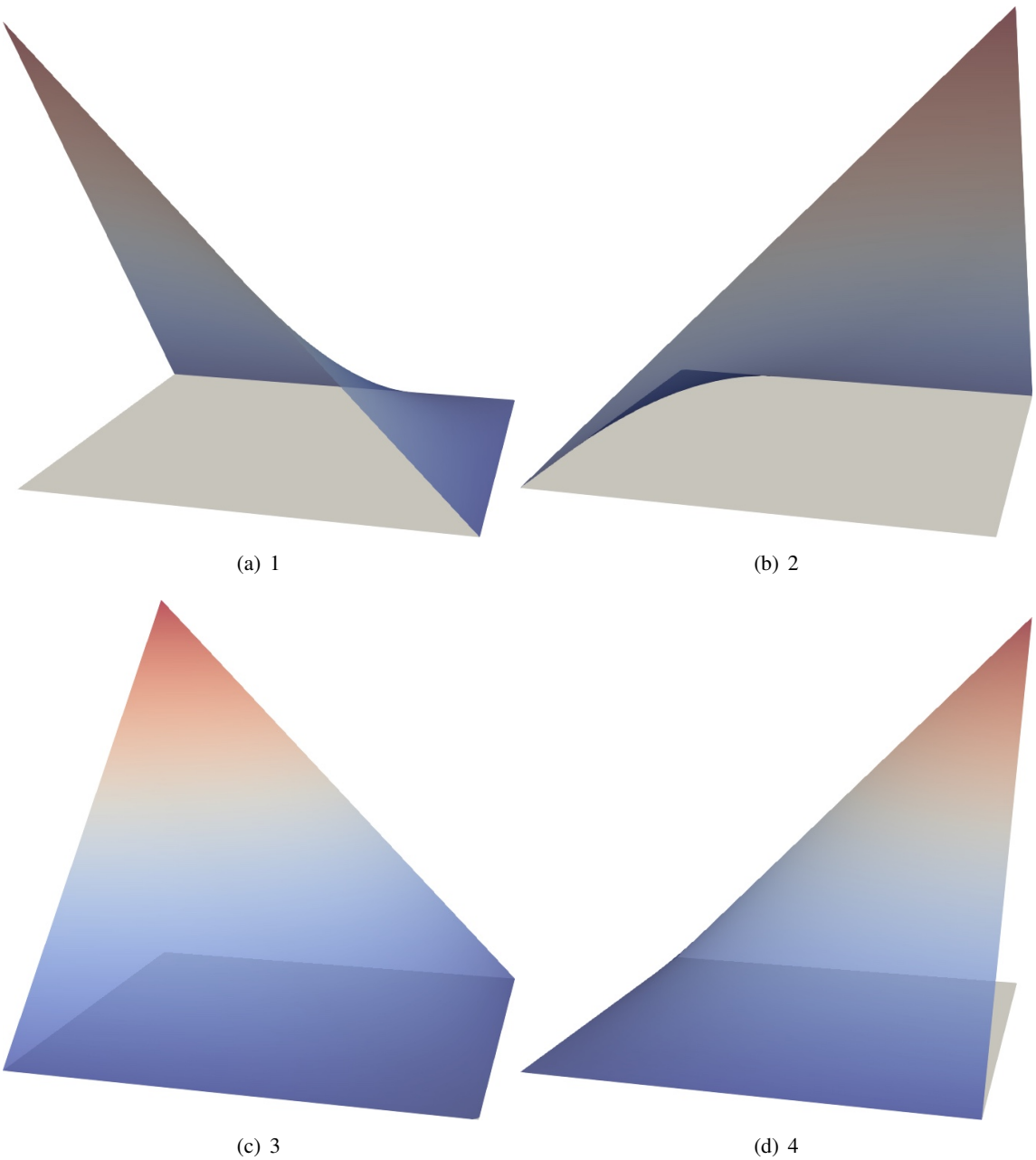
(a) 1 (b) 2

Figure 3.6: Shape functions for the one-dimensional linear finite element space $\mathcal{P}_1 = \mathcal{Q}_1$ in the reference segment. The label says which is the related Lagrangian node in $\mathcal{N}_1$.



(a) 1 (b) 2 (c) 3

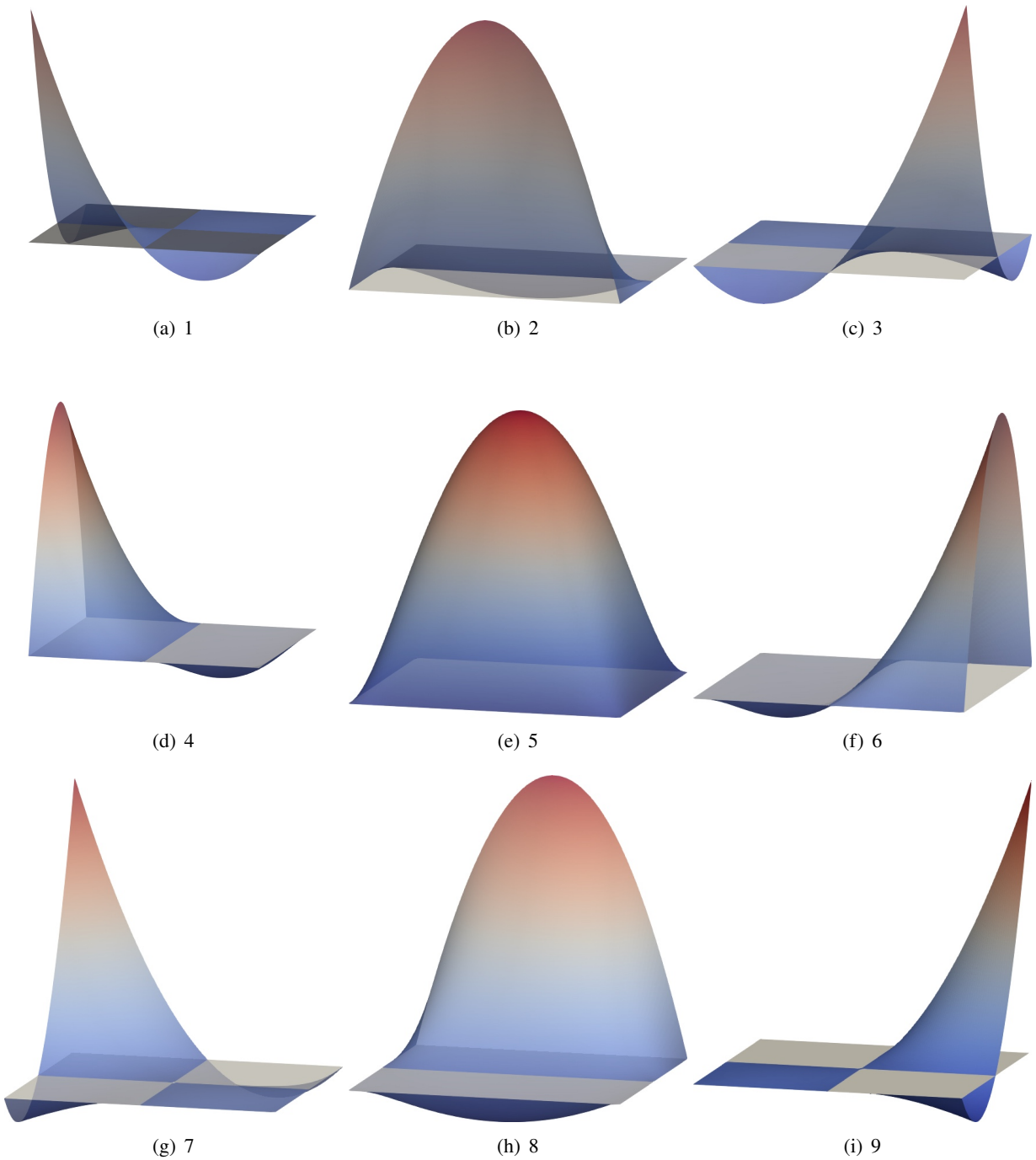
Figure 3.7: Shape functions for the one-dimensional quadratic finite element space $\mathcal{P}_2 = \mathcal{Q}_2$ in the reference segment. The label says which is the related Lagrangian node in $\mathcal{N}_2$.



(a) 1 (b) 2

(c) 3 (d) 4

Figure 3.8: Shape functions for the two-dimensional bilinear finite element space Q_1 in the reference square. The label says which is the related Lagrangian node in the set $\mathcal{N}^{(1,1)}$ (see Figure 3.5).



(a) 1 (b) 2 (c) 3

(d) 4 (e) 5 (f) 6

(g) 7 (h) 8 (i) 9

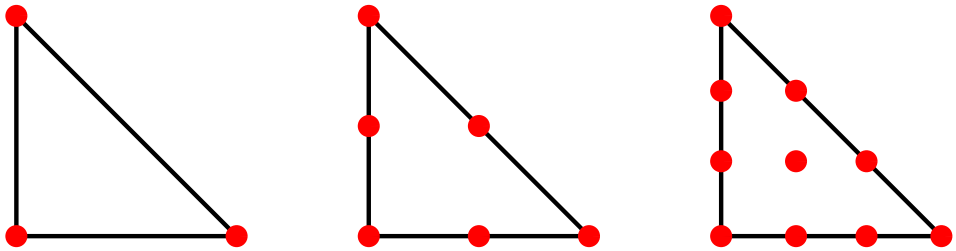
Figure 3.9: Shape functions for the two-dimensional biquadratic finite element space Q_2 in the reference square. The label says which is the related Lagrangian node in the set $\mathcal{N}^{(2,2)}$ (see Figure 3.5).

3.4.2 Local finite element space in simplices

The definition of polynomial spaces on d-simplices is slightly different. It requires the definition of the space of polynomials of degree equal or less than k in the variables $x_1, \ldots, x_d$. It does not involve a full tensor product of 1D Lagrange polynomials (or monomials) but a truncated space, i.e., the corresponding polynomial space of polynomials up to order k, which is $\mathcal{P}_k = \text{span}\{p_\alpha(\boldsymbol{x}) : |\alpha| \leq k\}$, with $|\alpha| \doteq \sum_{i=1}^{d} \alpha_i$.

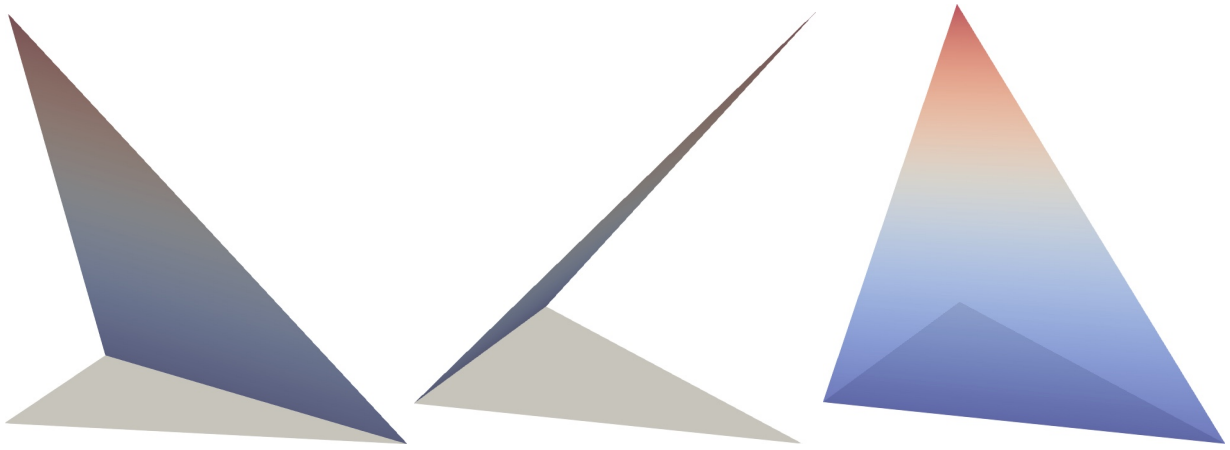
As an example, in two-dimensions, the linear space is $\mathcal{P}_1 \doteq \text{span}\{1, x, y\}$ whereas the quadratic space is $\mathcal{P}_2 \doteq \text{span}\{1, x, y, x^2, xy, y^2\}$.

Analogously as for d-cubes, we can define a basis of the dual space, i.e., the degrees of freedom as nodal evaluations. A basis for the dual space of $\mathcal{P}_k$ are the values at the set of nodes $\tilde{\mathcal{N}}^k \doteq \{\boldsymbol{s} \in \mathcal{N}^{k\mathbf{1}} : |\boldsymbol{s}| \leq k\}$ (see Figure 3.10). It generates the typical grad-conforming finite elements on d-simplices. Whereas the construction of the shape function bases is somewhat easy for quad meshes, using the tensor product of the Lagrangian basis, the situation is more complicated for tet meshes. In the following section, we show an easy method to transform a monomial basis into the shape functions basis in a general way. The shape functions for linear and quadratic elements can be found in Figures 3.11 and 3.12, resp.



$\tilde{\mathcal{N}}^1$ $\tilde{\mathcal{N}}^2$ $\tilde{\mathcal{N}}^3$

Figure 3.10: Equidistant nodes for two-dimensional Lagrangian polynomial spaces $\mathcal{P}_1$, $\mathcal{P}_2$ and $\mathcal{P}_3$, obtained by truncation of the tensor-product node sets ($|\boldsymbol{s}| \leq k$).

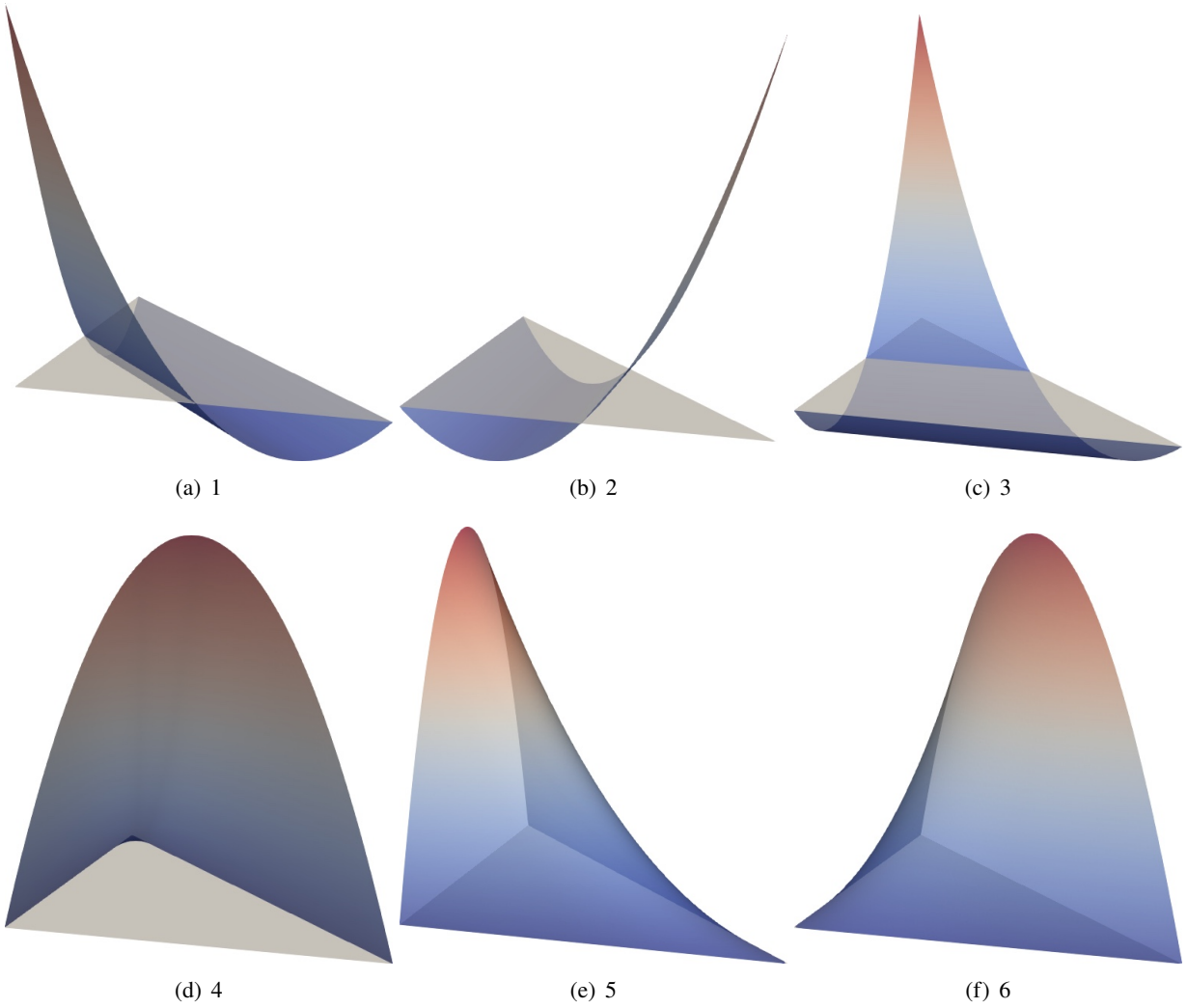


(a) 1 (b) 2 (c) 3

Figure 3.11: Shape functions for the two-dimensional linear finite element space $\mathcal{P}_1$ in the reference triangle. The label says which is the related Lagrangian node in $\tilde{\mathcal{N}}^1$ (see Figure 3.10).

3.4.3 Shape functions in the physical space

Let us consider quadrilateral finite elements. For the computation of the geometrical map Φ_K, we can consider a d-linear approximation determined by the position of the vertices in the physical domain. Thus,



(a) 1 (b) 2 (c) 3

(d) 4 (e) 5 (f) 6

Figure 3.12: Shape functions for the two-dimensional quadratic finite element space $\mathcal{P}_2$ in the reference triangle. The label says which is the related Lagrangian node in the set $\tilde{\mathcal{N}}^2$ (see Figure 3.10).

we can use all the finite element machinery so far to compute this geometrical map. Given the nodal values, i.e., the position of the vertices $\boldsymbol{x}^i$ in the physical space, we create the map as follows:

$$\boldsymbol{x} = \Phi_K(\hat{\boldsymbol{x}}) \doteq \sum_{i=1}^{n_\Sigma} \hat{\phi}_1^i(\hat{\boldsymbol{x}})\boldsymbol{x}^i,$$

where $\{\hat{\phi}_1^i\}$ are the shape functions for $\hat{Q}_1$. It is easy to check from the definition of shape functions and degrees of freedom for Lagrangian elements that Φ_K maps every vertex $\hat{\boldsymbol{x}}_i$ in the reference cell $\hat{K}$ to the corresponding vertex $\boldsymbol{x}^i$ in the physical cell K.

Using what we already know for quadrilateral finite elements, each component of $\boldsymbol{\Phi}_K$ belongs to $\hat{Q}_1$; e.g., in two dimensions the physical coordinates can be expressed as a bilinear polynomial in terms of the reference coordinates. As a result the map is nonlinear (due to the $\hat{x}\hat{y}$ term).[9]

Thus, the space spanned by $\{\phi^i\}$ is not the space $Q_1(K)$ in general. The following result describes which polynomials the physical space contains, and when it is a polynomial space at all.

Lemma 3.4.1: Polynomial spaces on a mapped quadrilateral

Let K be a quadrilateral cell with bilinear geometrical map Φ_K, let $p \geq 1$, and let $\mathcal{V} \doteq \{\hat{v} \circ \Phi_K^{-1} : \hat{v} \in \hat{Q}_p\}$ be the physical space of the Q_p Lagrangian element. Then:

(i) $\mathcal{P}_p(K) \subset \mathcal{V}$ for every quadrilateral K;

(ii) $\mathcal{V} = Q_p(K)$ if K is an axis-aligned rectangle, i.e., if Φ_K is affine, $\Phi_K^{-1}(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x} + \mathbf{b}$, with $\mathbf{A}$ diagonal (up to a permutation of the axes); for any other affine map, $\mathcal{V}$ is a space of polynomials different from $Q_p(K)$;

(iii) if Φ_K is not affine, $\mathcal{V}$ contains functions that are not polynomials; in particular, the physical shape functions are not all polynomials. (We prove it in two dimensions; the result holds in any dimension $d \geq 2$.)

Proof. (i) A function w on K belongs to $\mathcal{V}$ if and only if $w \circ \Phi_K \in \hat{Q}_p$, by definition. Let $w = x_1^{\alpha_1} \cdots x_d^{\alpha_d}$ be a monomial of degree $|\alpha| \leq p$. Then

$$w \circ \Phi_K = \prod_{j=1}^{d} \left(\Phi_{K,j}\right)^{\alpha_j}$$

is a product of $|\alpha|$ components of the bilinear map, each of them in $\hat{Q}_1$, hence a polynomial of degree at most $|\alpha| \leq p$ in each reference variable, i.e., $w \circ \Phi_K \in \hat{Q}_p$. All monomials of degree at most p, and therefore $\mathcal{P}_p(K)$, belong to $\mathcal{V}$.
(ii) If $\mathbf{A}$ is diagonal, $\hat{x}_j = a_j x_j + b_j$, so for $\hat{v} \in \hat{Q}_p$ the function $\hat{v}(\mathbf{A}\boldsymbol{x} + \mathbf{b})$ has degree at most p in each physical variable, i.e., $\mathcal{V} \subset Q_p(K)$; both spaces have dimension $(p+1)^d$, so they coincide (the same holds if $\mathbf{A}$ is a diagonal matrix times a permutation). If $\mathbf{A}$ is not of this form, since it is invertible, some column k of $\mathbf{A}$ has two nonzero entries, in rows $j \neq l$, say. The function $\hat{x}_1 \cdots \hat{x}_d \in \hat{Q}_p$ corresponds in $\mathcal{V}$ to $\prod_m (\mathbf{A}_{m,:}\boldsymbol{x} + b_m)$, which contains the monomial x_k^2 (from the factors j and l) multiplied by a nonzero polynomial in the other variables: its degree in x_k exceeds one, which is impossible for $p = 1$, and, taking instead $\hat{x}_1^p \cdots \hat{x}_d^p$, its degree in x_k is $2p > p$: the function is not in $Q_p(K)$. Thus $\mathcal{V} \neq Q_p(K)$, although $\mathcal{V}$ consists of polynomials (compositions of polynomials with an affine map).
(iii) The components of Φ_K^{-1} belong to $\mathcal{V}$, since $\hat{x}_j \in \hat{Q}_1 \subset \hat{Q}_p$. Suppose they were polynomials. Differentiating $\Phi_K \circ \Phi_K^{-1} = \mathrm{id}$ gives $\det \boldsymbol{J}_{\Phi_K} \det \boldsymbol{J}_{\Phi_K^{-1}} = 1$ with both determinants polynomials, so both must be constant. But for a bilinear map in two dimensions, $\det \boldsymbol{J}_{\Phi_K}$ is an affine function of $\hat{\boldsymbol{x}}$ (the reader can check it), which is constant only when the $\hat{x}\hat{y}$ terms of Φ_K vanish, i.e., when Φ_K is affine; the same

[9] In general, the cell K will not have flat faces in three-dimensions because we cannot find a plane that contains four points unless they are aligned. In two dimensions, the edges will be straight because two distinct points define a line.

computation, with a determinant of higher degree, works for the d-linear map in any dimension. Hence, for a non-affine map, Φ_K^{-1} is not polynomial and $\mathcal{V}$ contains non-polynomial functions; since the ϕ^i span $\mathcal{V}$, at least one of them is not a polynomial. □

Property (i) is the one that matters for approximation. Since $\mathcal{P}_p(K)$ is contained in the physical space, the local interpolant reproduces polynomials of degree p on every quadrilateral, and the approximation properties of Section 3.12 survive on distorted meshes. For $p = 1$ the inclusion can be seen directly: the geometrical map is built from the shape functions, $\Phi_K(\hat{\boldsymbol{x}}) = \sum_i \boldsymbol{x}^i \hat{\phi}^i(\hat{\boldsymbol{x}})$, so that, evaluating at $\hat{\boldsymbol{x}} = \Phi_K^{-1}(\boldsymbol{x})$,

$$\boldsymbol{x} = \Phi_K\big(\Phi_K^{-1}(\boldsymbol{x})\big) = \sum_i \boldsymbol{x}^i\, \hat{\phi}^i\big(\Phi_K^{-1}(\boldsymbol{x})\big) = \sum_i \boldsymbol{x}^i\, \phi^i(\boldsymbol{x}), \qquad \forall \boldsymbol{x} \in K,$$

and, since the physical shape functions form a partition of unity ($\sum_i \hat{\phi}^i = 1$ on $\hat{K}$ and $\phi^i = \hat{\phi}^i \circ \Phi_K^{-1}$, so $\sum_i \phi^i = 1$ on K whatever the map), any linear polynomial can be written as

$$a + \boldsymbol{b}\cdot\boldsymbol{x} = a \sum_i \phi^i + \boldsymbol{b}\cdot \sum_i \boldsymbol{x}^i \phi^i = \sum_i \underbrace{\left(a + \boldsymbol{b}\cdot\boldsymbol{x}^i\right)}_{\text{nodal values}} \phi^i,$$

i.e., it is the combination of the physical shape functions with its own nodal values as coefficients, even though the ϕ^i themselves are rational functions on a general quadrilateral.

3.5 Construction of the shape functions basis

The analytical expression of shape functions can become very complicated for high-order finite elements and non-trivial definitions of DOFs, e.g., for electromagnetic applications (see below). Furthermore, we require an automatic generator of shape functions to have a code that provides bases for arbitrarily high orders. When the explicit construction of the shape functions is not apparent, we proceed as follows.

Let us consider a finite element defined by $\{K, \mathcal{V}, \Sigma\}$.[10] First, we generate a *pre-basis* $\{\psi^j\}_{j\in\mathcal{N}_\Sigma}$ that spans the local finite element space $\mathcal{V}$, e.g., a Lagrangian polynomial basis (see Sect. 3.4). On the other hand, given the set of local DOFs, we proceed as follows. The shape functions can be written as $\phi^i = \sum_{j\in\mathcal{N}_\Sigma} \Xi_{ij}\psi^j$, where ψ^j are the elements of the pre-basis. By definition, the shape functions must satisfy $\sigma_i(\phi^j) = \delta_{ij}$ for $i, j \in \mathcal{N}_\Sigma$. As a result, let us define $\mathbf{C}_{ij} \doteq \sigma_i(\psi^j)$. We have (using Einstein's notation):

$$\delta_{ij} = \sigma_i(\phi^j) = \sigma_i(\Xi_{jk}\psi^k) = \sigma_i(\psi^k)\Xi_{jk},$$

or, in matrix form, $\mathbf{C}\Xi^T = I$, and thus $\Xi^T = \mathbf{C}^{-1}$. As a result, $\Xi_{ij} = (\mathbf{C}^{-1})_{ji}$. The shape functions are computed as a linear combination of the pre-basis functions.

3.6 Global finite element space and conformity

Now that we have defined the finite element spaces at the local level, we have to glue together these pieces keeping C^0 continuity. In this section, we will consider how to build global (and conforming) finite element spaces. Next, we will learn how to integrate the bilinear forms in the corresponding weak formulation in Sect. 3.9.

Let us define the *global* finite element space. Conforming finite element spaces are defined as: $\mathcal{X}_h \doteq \{v \in \mathcal{X} : v|_K \in \mathcal{V}\}$. The main complication in this definition is to enforce the conformity of the finite element space, i.e., $\mathcal{X}_h \subset \mathcal{X}$. Since we want a conforming finite element space $\mathcal{X}_h \subset \mathcal{X}$, we must keep some continuity across inter-cell boundaries. E.g., for problems posed in $H^1(\Omega)$, it implies full continuity ($C^0(\Omega)$ piecewise polynomials are in $H^1(\Omega)$).

In fact, the conformity constraint is the one that motivates the choice of $\hat{\Sigma}$ and Ψ, and as a consequence, Σ. In practice, the conformity constraint must be re-stated as a continuity constraint over finite element DOFs.

[10] In this section, we do not make difference between reference and physical spaces, e.g., using the $\hat{}$ symbol. In any case, all the following developments are usually performed at the reference finite element level.

For conforming meshes, these constraints are implicitly enforced via a global DOF numbering, even though it is not possible in general for adaptive schemes with non-conforming meshes or cell-dependent order, which require more involved constraints.

The idea of global DOF numbering is the following. We start with the cell-local finite element spaces we already know how to define. At every cell, we have a set of local DOFs. At the discrete level, we want to express that continuity in terms of DOFs, gluing together (enforcing the same values) on pairs of DOFs at different cells. Before providing the abstract definition, let us understand what it means in one dimension. In one dimension, let us consider two consecutive cell (segments) $[x_{i-1}, x_i]$ and $[x_i, x_{i+1}]$. We can define a local Lagrangian basis of order p and its corresponding local $p+1$ nodes at every cell. In both cells, a node is geometrically located at x_i. If we do not enforce any continuity, functions could have a jump at x_i. To enforce continuity, we must enforce these two local nodes to have the same value. Conceptually, they represent the same *global* DOFs.

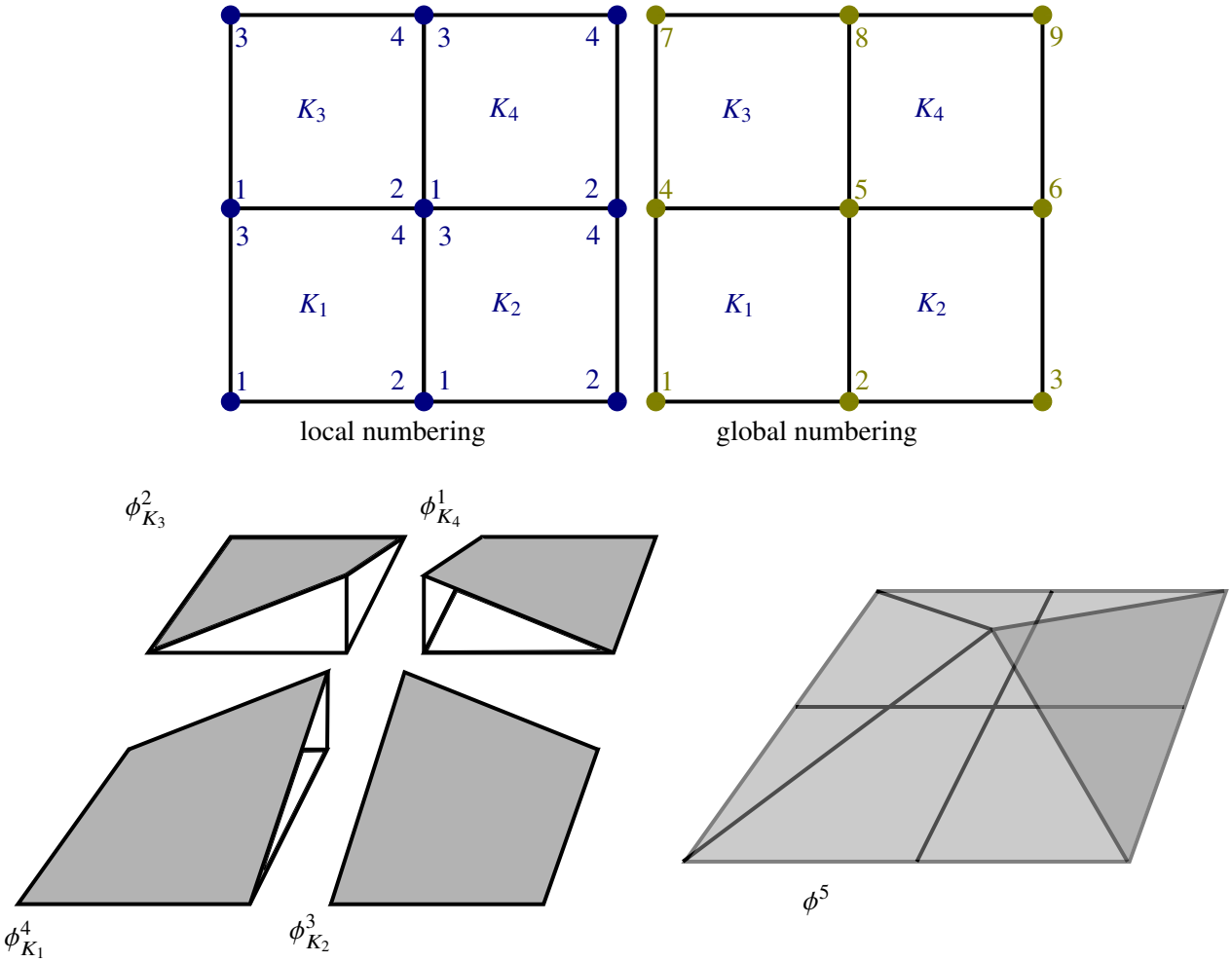


Figure 3.13: In the top-left corner we show the local numbering for each cell in a mesh. The top-right corner shows the global numbering for the same mesh. We can observe, e.g., that the local degrees of freedom (nodes) $\sigma^4_{K_1}$, $\sigma^3_{K_2}$, $\sigma^2_{K_3}$ and $\sigma^1_{K_4}$ correspond to the same global degree of freedom (node) σ_5. In the bottom-left part we show the corresponding local shape functions for these local degrees of freedom and the global shape function corresponding to the global degree of freedom.

This idea can be stated in an abstract way. Let us define by $\mathcal{M}_h \doteq \{(j, K) : j \in \mathcal{N}_{\Sigma_K}, K \in \mathcal{T}_h\}$ the Cartesian product of local DOFs for all cells. We define the global DOFs as the quotient space of $\mathcal{M}_h$ by an equivalence relation $\sim$. Using standard notation, given $\sim$, the equivalence class of $a \in \mathcal{M}_h$ with respect to $\sim$ is represented with $[a] \doteq \{b \in \mathcal{M}_h : a \sim b\}$, and the corresponding quotient set is $\mathcal{N}_h \doteq \{[a] : a \in \mathcal{M}_h\}$.

The set $\mathcal{N}_h$ is the set of global DOF and $[\cdot]$ represents the *local-to-global* DOF map. Using the one-to-one mapping between DOFs and shape functions, the same operator allows one to define global shape functions $\phi^a = \sum_{(i,K)\sim a} \phi^i_K$. Typically, ϕ^a "contains" up to one local shape function from a given element K, but may contain local shape functions from several elements. This construction should lead to conformity, i.e., $\mathcal{X}_h = \text{span}\{\phi^a\}_{a\in\mathcal{N}_h} \subset \mathcal{X}$.

For grad-conforming finite elements, the global finite element space is determined by the following equivalence relation. The set of local DOFs for d-cubes is $\mathcal{M}_h \doteq \{(\boldsymbol{s}, K) : \boldsymbol{s} \in \mathcal{N}^{k\mathbf{1}}, K \in \mathcal{T}_h\}$ due to the one-to-one mapping between DOFs and nodes; we replace the set of nodes by $\tilde{\mathcal{N}}^k$ for d-simplices. Furthermore, we say that $(\boldsymbol{s}, K) \sim (\boldsymbol{s}', K')$ iff $\boldsymbol{x}_{\boldsymbol{s}} = \boldsymbol{x}_{\boldsymbol{s}'}$, i.e., *two local degrees of freedom in two different cells are the same degree of freedom in the global space if their corresponding nodes are in the same spatial point in* Ω. The implementation of this equivalence relation, and thus, the global numbering, relies on the ownership relation between d-faces and DOFs. An illustration of the local-to-global map for grad-conforming finite elements in two dimensions is provided in Figure 3.13. In one dimension, we recover the *hat function* definition in the previous section.

With such global DOF definition, it is easy to check that the global finite element space is grad-conforming, i.e., $\mathcal{V} \subset C^0(\overline{\Omega}) \subset H^1(\Omega)$. The idea is to check that, at any d-face (edge in two dimensions or face in three dimensions), the restriction of the finite element function to that d-face is a polynomial of the same order. The nodes on the closure of the d-face uniquely determine the polynomial on the d-face. Since these nodes are the same for all cells sharing the d-face, the function is continuous at the d-face.

3.7 Interpolant

Let us consider an infinite-dimensional space $\tilde{\mathcal{X}}$ such that 1) $\mathcal{X}_h \subset \tilde{\mathcal{X}} \subset \mathcal{X}$ and 2) for every function $v \in \tilde{\mathcal{X}}$ and global DOF $a \in \mathcal{N}_h$, all the local DOFs $i,\ j \in [a]$ are such that $\sigma_i(v) = \sigma_j(v)$, i.e., local DOFs related to the same global DOF are continuous among cells. The *global interpolator* is defined as:

$$\pi_{\mathcal{X}_h}(v) \doteq \sum_{K\in\mathcal{T}_h} \pi_K(v) = \sum_{K\in\mathcal{T}_h} \sum_{i\in\mathcal{N}_{\Sigma_K}} \sigma_i(v)\phi^i_K, \qquad \text{for } v \in \tilde{\mathcal{X}}. \tag{3.5}$$

Under the two hypotheses above, this interpolation operator is also equal to $\pi_{\mathcal{X}_h}(v) = \sum_{a\in\mathcal{N}_h} \sigma_a(v)\phi^a$, which is the reason why they were taken in the first place.

Figure 3.13 shows this process at one physical cell, which for grad-conforming Lagrangian finite elements simply implies computing the nodal values (local degrees of freedom) at each cell and using the local-to-global map. It is easy to check that it is, in fact, a projector (using the definition of shape function). In any case, we use *projection operator* to refer to other projectors that involve the solution of a global finite element system, e.g., based on the minimization of the L^2 or H^1 norm.

Since Lagrangian DOFs involve point-wise evaluations of functions and $H^1_0(\Omega) \not\subset C^0(\Omega)$ for $d > 1$, the interpolator (3.5) is not defined in such space. This is only true in one-dimension, the case considered in the previous section. Instead, we consider that functions to be interpolated belong, e.g., to the space $\tilde{\mathcal{X}} \doteq C^0(\Omega)$. There are slightly more involved interpolators for grad-conforming finite element spaces that are bounded in $H^1(\Omega)$ for any space dimension but we do not consider them here for simplicity.

3.8 Assembly and linear system

Once we have defined a basis for the finite element space $\mathcal{X}_h$ using the finite element machinery presented above, every finite element function u_h can be uniquely represented by a vector $\mathbf{u} \in \mathbb{R}^{|\mathcal{N}_h|}$ as $u_h = \sum_{b\in\mathcal{N}_h} \phi^b \mathbf{u}_b$. In fact, problem (3.3) can be re-stated as: find $\mathbf{u} \in \mathbb{R}^{|\mathcal{N}_h|}$ such that

$$a(\phi^b, \phi^a)\mathbf{u}_b = \ell_h(\phi^a), \qquad \text{for any } a \in \mathcal{N}_h.$$

We have ended up with a finite-dimensional linear problem, i.e., a linear system. In matrix form, the problem can be stated as:

$$\text{Solve} \quad \mathbf{A}\mathbf{u} = \mathbf{f}, \qquad \text{with} \quad \mathbf{A}_{ab} \doteq a(\phi^b, \phi^a), \quad \mathbf{f}_a \doteq \ell_h(\phi^a).$$

Assuming that the bilinear form can be split into cell contributions as $a(\cdot,\cdot) = \sum_{K\in\mathcal{T}_h} a_K(\cdot,\cdot)$, e.g., by replacing $\int_\Omega$ by $\sum_{K\in\mathcal{T}_h}\int_K$, the construction of the matrix is implemented through a *cell-wise assembly process*[11], as follows:

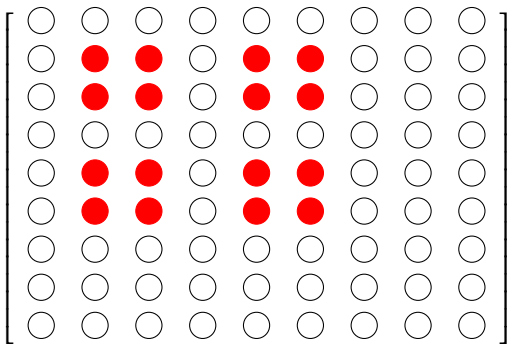
$$\mathbf{A}_{ab} = \sum_{K\in\mathcal{T}_h} \sum_{\substack{(i,K)\in[a],\\(j,K)\in[b]}} \mathbf{A}^K_{ij} \doteq \sum_{K\in\mathcal{T}_h} \sum_{\substack{(i,K)\in[a],\\(j,K)\in[b]}} a_K(\phi^j_K, \phi^i_K).$$

We remind that a, b are global indices in $\mathcal{N}_h$, and $i, j \in \mathcal{N}_{\Sigma_K}$ are the local indices to the cell K. The finite element affine operator (3.4) can be represented as $\mathcal{F}_h(u_h) \doteq \mathbf{Au} - \mathbf{f}$, i.e., it can be represented with a matrix and a vector of size $|\mathcal{N}_h|$.

Let us consider a practical example of this process. Let us assume that we are integrating the entries for cell K_2 in Figure 3.13. Thus, the element-local matrix reads:

$$\mathbf{A}^K \doteq \begin{bmatrix} a(\phi^1_{K_2},\phi^1_{K_2}) & a(\phi^1_{K_2},\phi^2_{K_2}) & a(\phi^1_{K_2},\phi^3_{K_2}) & a(\phi^1_{K_2},\phi^4_{K_2}) \\ a(\phi^2_{K_2},\phi^1_{K_2}) & a(\phi^2_{K_2},\phi^2_{K_2}) & a(\phi^2_{K_2},\phi^3_{K_2}) & a(\phi^2_{K_2},\phi^4_{K_2}) \\ a(\phi^3_{K_2},\phi^1_{K_2}) & a(\phi^3_{K_2},\phi^2_{K_2}) & a(\phi^3_{K_2},\phi^3_{K_2}) & a(\phi^3_{K_2},\phi^4_{K_2}) \\ a(\phi^4_{K_2},\phi^1_{K_2}) & a(\phi^4_{K_2},\phi^2_{K_2}) & a(\phi^4_{K_2},\phi^3_{K_2}) & a(\phi^4_{K_2},\phi^4_{K_2}) \end{bmatrix}$$

Now, we have to assemble the local matrix in the global one using the local to global map. The local degrees of freedom 1, 2, 3 and 4 correspond to the global ones 2, 3, 5 and 6. Thus, these entries will be added to the corresponding entries (in red) of the global matrix

$\mathbf{A}+=$ 

The operator + = means add the object on the right to the one on the left.

3.9 Numerical integration

In general, the local bilinear form can be stated as:

$$a_K(\phi^j_K, \phi^i_K) \doteq \int_K \mathcal{I}(\boldsymbol{x})\mathrm{d}\Omega,$$

for some integrand $\mathcal{I}(x)$, where the evaluation of $\mathcal{I}(\boldsymbol{x})$ involves the evaluation of shape function derivatives. Let us represent the Jacobian of the geometrical mapping with $\boldsymbol{J}_K \doteq \frac{\partial \boldsymbol{\Phi}_K}{\partial \hat{\boldsymbol{x}}}$. We can rewrite the cell integration in the reference cell, and next consider a quadrature rule Q defined by a set of points/weights $(\hat{\boldsymbol{x}}_{\mathrm{gp}}, \mathrm{w}_{\mathrm{gp}})$, as follows:

$$\int_K \mathcal{I}(\boldsymbol{x})\mathrm{d}\Omega = \int_{\hat{K}} \mathcal{I}\circ\boldsymbol{\Phi}_K(\hat{\boldsymbol{x}})|\boldsymbol{J}_K|\mathrm{d}\hat{\Omega} = \sum_{\hat{\boldsymbol{x}}_{\mathrm{gp}}\in\mathrm{Q}} \mathcal{I}\circ\boldsymbol{\Phi}_K(\hat{\boldsymbol{x}}_{\mathrm{gp}})\mathrm{w}(\hat{\boldsymbol{x}}_{\mathrm{gp}})|\boldsymbol{J}_K(\hat{\boldsymbol{x}}_{\mathrm{gp}})|.$$

[11] The implementation of finite element methods is cell-based, i.e., all the computations are at the cell level, and then, using the local-to-global map (the equivalence class) assembled in the global linear system. This is different from finite difference methods, that do not share the concept of cell and are nodal-based.

Here, the main complication is the evaluation of $\mathcal{I} \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}})$. The evaluation of this functional requires the evaluation of $\partial_{\boldsymbol{\alpha}} \phi_K^j \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}})$ for some values of the multi-index $\boldsymbol{\alpha}$ (idem for the test functions). Usually, $|\boldsymbol{\alpha}| \leq 1$ in C^0 finite elements, since higher-order derivatives would require higher inter-cell continuity.[12]

Let us consider the case of zero and first derivatives, i.e., the evaluation of $\phi_K^j \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}})$ and $\boldsymbol{\nabla} \phi_K^j \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}})$. $\boldsymbol{\nabla}_{\hat{\boldsymbol{x}}}$ represents the gradient in the reference space. The values of the shape functions (times the geometrical mapping) on the quadrature points is determined as follows for grad-conforming (Lagrangian) finite elements:

$$\phi_K^j \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}}) = \hat{\phi}^j(\hat{\boldsymbol{x}}_{\text{gp}}),$$

whereas shape function gradients are computed as:

$$\begin{aligned} \boldsymbol{\nabla} \phi_K^j \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}}) &= \boldsymbol{\nabla}(\hat{\phi}^j \circ \mathbf{\Phi}_K^{-1}) \circ \mathbf{\Phi}_K(\hat{\boldsymbol{x}}_{\text{gp}}) \\ &= \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{\phi}^j(\hat{\boldsymbol{x}}_{\text{gp}}) \cdot \boldsymbol{J}_K^{-1}(\hat{\boldsymbol{x}}_{\text{gp}}), \end{aligned}$$

where we have used some elementary differentiation rules and the inverse function theorem in the last equality:

$$\boldsymbol{\nabla} f(x) = \boldsymbol{\nabla}\left(\hat{f} \circ \Phi_K^{-1}\right) = \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{f} \cdot \boldsymbol{\nabla} \Phi_K^{-1} = \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{f} \cdot \boldsymbol{J}_K^{-1} \circ \Phi_K^{-1}.$$

Using matrix notation, the term $\boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{f} \cdot \boldsymbol{J}_K^{-1}$ is usually written as $\boldsymbol{J}_K^{-T} \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{f}$. Thus, one only needs to provide the values of the Jacobian matrix, its inverse, and its determinant from one side, and the value of the shape functions $\hat{\phi}^j$ and their gradients $\boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{\phi}^j$ in the reference space, on the other side, at all quadrature points, to compute all the entries of the finite element matrices; second-order derivatives can be treated analogously.

As an example, a mass matrix[13] would be computed as

$$\begin{aligned} \int_K \phi^i(\boldsymbol{x})\, \phi^j(\boldsymbol{x})\, \mathrm{d}\Omega &= \int_{\hat{K}} \hat{\phi}^i(\hat{\boldsymbol{x}})\, \hat{\phi}^j(\hat{\boldsymbol{x}})\, |\boldsymbol{J}_K(\hat{\boldsymbol{x}})| \mathrm{d}\hat{\Omega} \\ &= \sum_{\hat{\boldsymbol{x}}_{\text{gp}} \in \mathrm{Q}} \hat{\phi}^i(\hat{\boldsymbol{x}}_{\text{gp}})\, \hat{\phi}^j(\hat{\boldsymbol{x}}_{\text{gp}})\, \mathrm{w}(\hat{\boldsymbol{x}}_{\text{gp}}) |\boldsymbol{J}_K(\hat{\boldsymbol{x}}_{\text{gp}})|. \end{aligned}$$

On the other hand, the Laplacian matrix would be computed as

$$\begin{aligned} &\int_K \boldsymbol{\nabla} \phi^i(\boldsymbol{x}) \cdot \boldsymbol{\nabla} \phi^j(\boldsymbol{x})\, \mathrm{d}\Omega \\ &= \int_{\hat{K}} [\boldsymbol{J}_K^{-T} \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{\phi}^i](\hat{\boldsymbol{x}}) \cdot [\boldsymbol{J}_K^{-T} \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{\phi}^j](\hat{\boldsymbol{x}})\, |\boldsymbol{J}_K(\hat{\boldsymbol{x}})| \mathrm{d}\hat{\Omega} \\ &= \sum_{\hat{\boldsymbol{x}}_{\text{gp}} \in \mathrm{Q}} [\boldsymbol{J}_K^{-T} \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{\phi}^i](\hat{\boldsymbol{x}}_{\text{gp}}) \cdot [\boldsymbol{J}_K^{-T} \boldsymbol{\nabla}_{\hat{\boldsymbol{x}}} \hat{\phi}^j](\hat{\boldsymbol{x}}_{\text{gp}}) \mathrm{w}(\hat{\boldsymbol{x}}_{\text{gp}}) |\boldsymbol{J}_K(\hat{\boldsymbol{x}}_{\text{gp}})|. \end{aligned}$$

Quadrature rules for $\hat{K}$ being an d-cube can be obtained as a tensor product of a 1D quadrature rule, e.g., the Gauss-Legendre quadrature. As it is well known, considering d-cube topologies for $\hat{K}$, Gauss quadratures with n points per direction can integrate *exactly* $2n-1$ order polynomials. E.g., for a Lagrangian reference finite element of order p and an affine geometrical map, we choose $n = p+1$ per direction to integrate exactly a mass matrix (the integrand has order $2p$ per direction).

Remark 3.9.1: Accuracy of a quadrature and exactness degree

The exactness degree of a quadrature dictates its accuracy. Let $Q_m^{[a,b]}$ be a rule on $[a,b] \subset \mathbb{R}$ with positive weights that is exact for polynomials up to degree q, and let f be smooth. We can split

[12] There exist C^1 finite element methods that can approximate fourth order problems, e.g., the bi-harmonic problem, but we are not going to consider them here.

[13] The mass matrix is the one that arises from a zero-order (a.k.a. reaction) term, e.g., the first term in $u - \Delta u = 0$. In weak form, it leads to a term $\int_\Omega uv \mathrm{d}\Omega$. The origin of the name comes from the fact that this term arises after the finite difference discretisation of a time derivative, and the time derivative term is the so-called inertia term.

$f = p + r$, where p is the Taylor polynomial of degree q of f at the midpoint of the interval, and the remainder satisfies $\max_{[a,b]} |r| \leq C\,(b-a)^{q+1} \max_{[a,b]} |f^{(q+1)}|$. The rule makes no error on p, which is what exactness means, so the whole error comes from the remainder,

$$E(f) \doteq \int_a^b f(x)\,\mathrm{d}x - Q_m^{[a,b]}(f) = \int_a^b r(x)\,\mathrm{d}x - Q_m^{[a,b]}(r).$$

Both terms are bounded by $(b-a)\max_{[a,b]}|r|$. For the integral this is trivial. For the quadrature, the weights are positive and, by exactness on constants, they add up to $\sum_{j=1}^m \omega_j^m = Q_m^{[a,b]}(1) = b-a$, so

$$\left|Q_m^{[a,b]}(r)\right| = \left|\sum_{j=1}^m \omega_j^m\, r(\xi_j^m)\right| \leq \sum_{j=1}^m \omega_j^m \left|r(\xi_j^m)\right| \leq \left(\sum_{j=1}^m \omega_j^m\right) \max_{[a,b]} |r| = (b-a) \max_{[a,b]} |r|.$$

Thus,

$$|E(f)| \leq C\,(b-a)^{q+2} \max_{[a,b]} |f^{(q+1)}|.$$

For the m-point Gauss rule, $q = 2m-1$, and the error is of order $(b-a)^{2m+1}$; each extra quadrature point buys two orders of accuracy. E.g., the midpoint rule ($m = 1$) satisfies $E(f) = \frac{(b-a)^3}{24} f''(\xi)$ for some $\xi \in (a,b)$. Tensor-product rules on d-cubes inherit this accuracy per direction. In the finite element method, integrals are computed cell-wise on cells of size h, so the quadrature error per cell scales like h^{q+2}; summed over the $\sim 1/h$ cells of a 1D mesh, the total quadrature error scales like h^{q+1}, to be compared with the discretisation error of the method.

Symmetric quadrature rules on triangles and tetrahedra for different orders can also be constructed. In any case, to create arbitrarily large quadrature rules for d-simplices, one can consider the so-called Duffy transformation. The latter is a change of variables that transform our d-simplex integration domain into an d-cube, and integrate on the d-cube using tensor product quadratures.

Let us finally mention that in some instances, e.g., for integrating Neumann boundary condition terms of the type $\int_{\Gamma_N} h(x)v(x)\mathrm{d}F$, we can use the same ideas above. Instead of integrating over the cell, we would integrate on the faces. We can consider a reference facet $\hat{F}$, and a mapping $\boldsymbol{\Phi}_F : \hat{F} \to F$ from the reference to the physical space. Let us represent the Jacobian of the geometrical mapping with $\boldsymbol{J}_F \doteq \frac{\partial \boldsymbol{\Phi}_F}{\partial \hat{\boldsymbol{x}}}$, which has values in $\mathbb{R}^{d\times(d-1)}$. We can rewrite the facet integral in the reference facet, and next consider a quadrature rule Q on $\hat{F}$ defined by a set of points/weights $(\hat{\boldsymbol{x}}_{\mathrm{gp}}, \mathrm{w}_{\mathrm{gp}})$, as follows:

$$\int_F \mathcal{I}(\boldsymbol{x})\mathrm{d}\Omega = \int_{\hat{F}} \mathcal{I}\circ\boldsymbol{\Phi}_F(\boldsymbol{x})|\boldsymbol{J}_F|\mathrm{d}F = \sum_{\hat{\boldsymbol{x}}_{\mathrm{gp}}\in \mathrm{Q}} \mathcal{I}\circ\boldsymbol{\Phi}_F(\hat{\boldsymbol{x}}_{\mathrm{gp}})\mathrm{w}(\hat{\boldsymbol{x}}_{\mathrm{gp}})|\boldsymbol{J}_F(\hat{\boldsymbol{x}}_{\mathrm{gp}})|.$$

$|\boldsymbol{J}_F|$ is defined as:

$$|\boldsymbol{J}_F| = \left\|\frac{\mathrm{d}\boldsymbol{\Phi}_F}{\mathrm{d}\hat{\boldsymbol{x}}}\right\|_2 \quad \text{and} \quad |\boldsymbol{J}_F| = \left\|\frac{\partial\boldsymbol{\Phi}_F}{\partial\hat{x}_1} \times \frac{\partial\boldsymbol{\Phi}_F}{\partial\hat{x}_2}\right\|_2,$$

for $d = 2, 3$, respectively. The facet map can easily be computed using similar ideas as in Sect. 3.4.3. The idea is to consider the shape functions related to the nodes on the closure of $\hat{F}$ and their position in the physical space:

$$\boldsymbol{\Phi}_F(\hat{\boldsymbol{x}}) \doteq \sum_{i\in\overline{\hat{F}}} \hat{\phi}_1^i(\hat{\boldsymbol{x}})\boldsymbol{x}^i,$$

where $\{\hat{\phi}_1^i\}$ are the shape functions for $\hat{Q}_1$.

3.10 Grad-conforming finite elements for vector fields

When one has to deal with vector or tensor fields, we can generate them as a Cartesian product of scalar spaces as follows. We define the local finite element space $\boldsymbol{\mathcal{V}}_k \doteq [Q_k]^d$ (or $[\mathcal{P}_k]^d$). The local degrees of

freedom are nodal values for a given component, i.e.,

$$\sigma_i^\alpha(\boldsymbol{u}) = \boldsymbol{u}_\alpha(\boldsymbol{x}_i) \qquad \text{for } i = 1, \dots, n_\Sigma, \quad \alpha = 1, \dots, d.$$

The equivalence class for the local-to-global map is such that two local degrees of freedom represent the same global one if the nodes are at the same point in the physical space and they are related to the same component. Analogously, shape functions are computed as $\phi^a \doteq \sum_{(i,s,K)\sim a} \ell_s^{k1} \vec{e}_i$; $\vec{e}_i$ represents the i-th canonical basis vector of $\mathbb{R}^d$. E.g., for a two-dimensional vector field, the bi-linear local finite element space reads

$$\mathcal{V} \doteq \text{span}\left\{\left(\phi^1, 0\right)^T, \dots, \left(\phi^4, 0\right)^T, \left(0, \phi^1\right)^T, \dots, \left(0, \phi^4\right)^T\right\}.$$

We proceed analogously for d-simplices.

3.11 Cartesian product of finite elements for multi-field problems

Many problems governed by PDEs involve more than one field, e.g., the Navier-Stokes equations or any multi-physics problem. Let us consider a PDE that involves a set of unknown fields $(\boldsymbol{u}_1, \dots, \boldsymbol{u}_n) \in \mathcal{X}^1 \times \dots \times \mathcal{X}^n$, defined as the Cartesian product of functional spaces. We can proceed as above, and define a finite element space for every field space separately, leading to a global finite element space $\mathcal{X}_h^1 \times \dots \times \mathcal{X}_h^n$ defined by Cartesian product of finite element spaces. To define the global numbering of DOFs in the multi-field case, we consider that two DOFs are equivalent if they are related to the same field and satisfy the equivalence relation of the finite element space of this field.

The Cartesian product of finite element spaces is enough to define volume-coupling multi-physics problems governed on the same physical domain, i.e., the different physics are defined on the whole domain and coupled through volume terms in the formulation. However, many multi-physics problems are interface-based, i.e., the coupling between different physics that are defined on different subdomains is through transmission conditions on the interface. It is the case, e.g., of fluid-structure problems. In these cases, different finite element spaces could be defined on different parts of the global mesh, i.e., one must describe the set of subdomains $(\Omega_1, \dots, \Omega_n)$ of the whole domain Ω in which the corresponding finite element spaces are defined and enforce the *transmission* conditions on the interface (usually, continuity of the unknown and its flux).

3.12 Approximation properties

The same error estimates we have proven for one-dimensional spaces hold in the multi-dimensional case. The proof is more technical and is out of this course's scope. With these results about the approximability properties of the polynomial spaces considered so far, we can readily obtain bounds for the error committed by the finite element method by recalling Cea's lemma in the previous chapter.

Before showing error estimates, we need some properties for the family of meshes being used in the h-refinement process. As in the previous chapter, let us consider a parameter N that determines the level of refinement in our mesh; e.g., N can represent the number of uniform refinements through bisection for a given initial mesh $\mathcal{T}_0$.

Definition 3.12.1: Shape regular and quasi-uniform mesh

A family of meshes (a.k.a. triangulations) $\{\mathcal{T}_N\}$ is shape regular if there exists a positive constant $c > 0$ independent of N such that

$$\max_{\tau \in \mathcal{T}_N} \frac{\text{diam}(\tau)^d}{|\tau|} \leq c, \qquad \forall N.$$

We can define the cell size $h_\tau = |\tau|^{\frac{1}{d}}$ for $\tau \in \mathcal{T}_N$. The family is also quasi-uniform if

$$\frac{\max_{\tau\in\mathcal{T}_N} |\tau|}{\min_{\tau\in\mathcal{T}_N} |\tau|} \leq \rho, \qquad \forall N.$$

For quasi-uniform families, we can define a mesh width $h_N \doteq \max_{\tau\in\mathcal{T}_N} h_\tau$.

Shape regularity means cells cannot be arbitrarily anisotropic as we increase refinement. E.g., let us consider $\mathcal{T}_0$ to be a rectangle. At each level of refinement rectangles are split into two rectangles by a horizontal cut. This way, we can keep conforming meshes at all levels, but as $N \to \infty$ the vertical dimension of the cell goes to zero, whereas the horizontal remains constant. Thus, such a family of meshes is not shape-regular. Shape regularity prevents *flat* elements as $N \to \infty$ and permits the definition of a meaningful length size. Meshes constructed by uniform refinement in all directions, e.g., splitting the rectangle into four scaled rectangles makes the shape regularity coefficient constant with respect to N.

Quasi-uniformity means that the size of all the cells in a mesh must be *similar*. We cannot refine some cells while keeping others untouched. It gives us a meaningful definition of mesh width in the multidimensional case. Quasi-uniformity is an essential ingredient for the statement of the multi-dimensional interpolation error estimates.

Lemma 3.12.2: Error bounds for the interpolant

Given a finite element space of order p, $\mathcal{Q}_p$ (for quad meshes) or $\mathcal{P}_p$ (for tet meshes), on a family of shape-regular and quasi-uniform meshes with mesh width h, the interpolant π^p satisfies:

$$\|v - \pi^p(v)\|_{L^2(\Omega)} \leq Ch^{p+1}|v|_{H^{p+1}(\Omega)}, \qquad |v - \pi^p(v)|_{H^1(\Omega)} \leq Ch^p|v|_{H^{p+1}(\Omega)},$$

for any $v \in H^{p+1}(\Omega)$, with a constant C independent of h and v.

3.13 Tutorials

1. Consider a linear reference finite element $[-1, 1]^2$, use as prebasis of the local space the space of monomials that span $\mathcal{Q}_1$, and apply the change of basis to get the expression of the shape functions.

2. Let us consider the problem $-\Delta u = f$ on $\Omega \subset (0, 1)^2$. Can you compute the shape function gradients at the reference finite element $[-1, 1]^2$ space for a linear finite element (using the result from the previous exercise)? Can you compute the corresponding Jacobian in element $(0.0, 0.5)^2$? Can you compute the linear system matrix in that element? Let us consider the element with vertices $(0.0, 0.0)$, $(0.5, 0.0)$, $(0.0, 0.5)$ and $(0.6, 0.5)$. Can you compute the Jacobian in this case?

3. Consider a structured 2×2 uniform mesh of a unit square with four bilinear finite elements. Can you compute the global system matrix using this local-to-global numbering and the matrix in the previous exercise? Now consider biquadratic finite elements on the same mesh, i.e., $\mathcal{Q}_2$. Provide a local and global numbering using a drawing.

4. Can you prove why grad-conforming Lagrangian finite elements lead to continuous solutions across cells after *gluing* DOFs?

4 Convection-dominated problems

So far, we have applied the FE method to problems governed by elliptic PDEs, e.g., the Poisson equation, with very satisfactory results. Elliptic problems are related to coercive and continuous bilinear forms in the weak statement of the problem, as we have previously seen. Now, we focus on hyperbolic PDEs or so-called singular limits of elliptic PDEs. We note these problems are prevalent in physics, e.g., in fluid mechanics or transport problems.

Unfortunately, the Galerkin approximation of hyperbolic problems with standard finite element methods does not yield satisfactory results. Hyperbolic problems usually have solutions with jumps (or discontinuities). Furthermore, standard FE methods exhibit large oscillations in these cases. Fortunately, there are different ways to overcome these shortcomings. In this chapter, we will introduce so-called *stabilisation* techniques that add additional terms to the numerical approximation to recover optimal stability and convergence properties in these situations.

4.1 The convection-diffusion problem

In this section, we consider the convection-diffusion-reaction equation, which in strong form reads:

$$\begin{aligned} \boldsymbol{\nabla} \cdot (\boldsymbol{\beta} u) - \boldsymbol{\nabla} \cdot \mu \boldsymbol{\nabla} u + \sigma u &= f, && \text{in } \Omega, \\ u &= g, && \text{on } \Gamma_D, \\ \boldsymbol{n} \cdot \mu \boldsymbol{\nabla} u &= q, && \text{on } \Gamma_N. \end{aligned} \tag{4.1}$$

This problem governs the concentration $u(\boldsymbol{x})$ of a contaminant, which is transported by chemical diffusion (μ being the diffusion coefficient), convective transport ($\boldsymbol{\beta}$ being, e.g., the fluid velocity) and possibly changes due to chemical reactions (σ being the reaction coefficient). f is a forcing term (a well or sink), g is the prescribed Dirichlet value on the boundary, and q a prescribed flux on the Neumann boundary.

The problem above is of mixed elliptic-hyperbolic nature, depending on the relative value of the physical coefficients ($\boldsymbol{\beta}$, μ, σ). When μ is *dominant*, it is an elliptic problem for which the FE methods work well. However, when the diffusive term vanishes, the problem is purely hyperbolic. We will see in this section that standard FE methods do not work well for this kind of PDEs. Thus, the limit of zero viscosity is problematic because, as $\mu \to 0$, the nature of the problem changes from elliptic to hyperbolic.

Let us consider the purely hyperbolic limit ($\mu = 0$) and assume that $\boldsymbol{\nabla} \cdot \boldsymbol{\beta} = 0$ (divergence-free convective field). We have the following equation:

$$(\boldsymbol{\beta} \cdot \boldsymbol{\nabla}) u + \sigma u = f, \qquad \text{in } \Omega, \tag{4.2}$$

where $(\boldsymbol{\beta} \cdot \boldsymbol{\nabla}) \doteq \boldsymbol{\nabla}_{\boldsymbol{\beta}}$ is the directional derivative in the direction of β. Dirichlet boundary conditions for the hyperbolic problem can only be enforced in the inflow:

$$u = g \quad \text{on } \Gamma_-, \qquad \Gamma_- \doteq \{\boldsymbol{x} \in \partial\Omega \ : \ \boldsymbol{n}(\boldsymbol{x}) \cdot \boldsymbol{\beta} < 0\}.$$

In order to provide some intuition about the solution of this equation, let us introduce the concept of *characteristics*. Given $\boldsymbol{x}_- \in \Gamma_-$, we can define the characteristic passing through $\boldsymbol{x}_-$ as follows. Let us consider a curve (the *characteristic*) $\boldsymbol{X}(s)$ for a given parameter $s \in [0, +\infty)$ such that:

$$\boldsymbol{X}(0) = \boldsymbol{x}_-, \qquad \frac{d\boldsymbol{X}}{ds} = \boldsymbol{\beta}(\boldsymbol{X}(s)).$$

Using the chain rule, we readily get

$$\frac{d}{ds}u(\boldsymbol{X}(s)) = \boldsymbol{\nabla} u \cdot \frac{d\boldsymbol{X}}{ds} = \boldsymbol{\beta} \cdot \boldsymbol{\nabla} u = f - \sigma u.$$

Thus, the solution can be computed independently for each characteristic. As a result, if the inflow data g is discontinuous, the solution u is discontinuous across characteristics.

When we add a diffusive term $-\boldsymbol{\nabla} \cdot \mu \boldsymbol{\nabla} u$ to (4.2) with $\mu > 0$, the problem becomes elliptic and its solution is smooth (for smooth data); discontinuities can no longer appear. In this case, diffusion smears out discontinuities on g across characteristics on regions of width $O(\mu^{1/2})$, to make the solution continuous in Ω. Besides, when $\mu > 0$, one can impose Dirichlet boundary conditions on the outflow $\Gamma_+ \doteq \partial\Omega \setminus \Gamma_-$. If the boundary data is not compatible with the purely hyperbolic solution, it will also produce a *boundary layer* of size $O(\mu)$.

From the discussion above, one can observe that the hyperbolic and diffusive regimes have a different nature. On one side, hyperbolic problems admit solutions with shocks and inflow-only Dirichlet boundary conditions. On the other side, diffusive problems (also called viscous, due to the similar nature of viscosity in fluids) cannot exhibit discontinuities and admit Dirichlet boundary conditions on the outflow. The limit $\mu \to 0$ is called a *singularly perturbed limit*, in which the nature of the underlying PDE changes. As one can expect, approximating problems in this limit will be challenging.

4.2 Weak formulation and stability

In this section, we derive the weak formulation of (4.1) and analyse its stability. This analysis will be the basis for the convergence study of the Galerkin method in the next section, which will provide a clear insight into why the standard discretisation is unsuitable for convection-dominated problems.

For simplicity, we assume homogeneous Dirichlet conditions, i.e., $g = 0$, $\Gamma_D = \partial\Omega$ and $\Gamma_N = \emptyset$. The extension to other boundary conditions can be considered as explained in the previous sections. The weak formulation reads:

$$u \in H_0^1(\Omega) : \int_\Omega \mu \boldsymbol{\nabla} u \cdot \boldsymbol{\nabla} v \mathrm{d}\Omega + \int_\Omega v\,(\sigma u + (\boldsymbol{\beta} \cdot \boldsymbol{\nabla})u)\,\mathrm{d}\Omega = \int_\Omega f v \mathrm{d}\Omega, \tag{4.3}$$

for all $v \in H_0^1(\Omega)$. The first result we want to prove is the stability of the continuous problem. In the following, we assume for simplicity that $\mu > 0$ and $\sigma \geq 0$ are constant and $\boldsymbol{\nabla} \cdot \boldsymbol{\beta} = 0$.

Theorem 4.2.1: Stability of convection-diffusion problem

The solution u of (4.3) satisfies the following stability bound:

$$\frac{\mu}{2}|u|^2_{H^1(\Omega)} + \sigma\|u\|^2_{L^2(\Omega)} \leq \frac{C_P^2}{2\mu}\|f\|^2_{H^{-1}(\Omega)}.$$

Proof. Let us take $v = u$ in (4.3). We have:

$$\int_\Omega \mu|\boldsymbol{\nabla} u|^2 \mathrm{d}\Omega + \int_\Omega \sigma u^2 \mathrm{d}\Omega + \int_\Omega u((\boldsymbol{\beta} \cdot \boldsymbol{\nabla} u))\mathrm{d}\Omega = \int_\Omega f u \mathrm{d}\Omega.$$

We can manipulate the convective term using integration by parts and the fact that u vanishes on the boundary as follows:

$$\int_\Omega u(\boldsymbol{\beta} \cdot \boldsymbol{\nabla})u \mathrm{d}\Omega = \frac{1}{2}\int_\Omega \boldsymbol{\beta} \cdot \boldsymbol{\nabla}(u^2)\mathrm{d}\Omega = -\frac{1}{2}\int_\Omega (\boldsymbol{\nabla} \cdot \boldsymbol{\beta})u^2 \mathrm{d}\Omega = 0.$$

The forcing term can be bounded using the duality norm, Poincaré inequality $\|u\|_{H^1(\Omega)} \leq C_P|u|_{H^1(\Omega)}$, and

Young inequality:

$$\int_\Omega f u \mathrm{d}\Omega \leq \mu^{-1/2}\|f\|_{H^{-1}(\Omega)}\mu^{1/2}\|u\|_{H^1(\Omega)}$$
$$\leq C_P\mu^{-1/2}\|f\|_{H^{-1}(\Omega)}\mu^{1/2}|u|_{H^1(\Omega)} \leq \frac{C_P^2}{2\mu}\|f\|^2_{H^{-1}(\Omega)} + \frac{\mu}{2}|u|^2_{H^1(\Omega)}.$$

Combining these results, we get:

$$\frac{\mu}{2}|u|^2_{H^1(\Omega)} + \sigma\|u\|^2_{L^2(\Omega)} \leq \frac{C_P^2}{2\mu}\|f\|^2_{H^{-1}(\Omega)}.$$

□

4.3 Galerkin approximation

The Galerkin approximation of the problem above can readily be obtained by replacing the functional spaces in (4.3) by discrete FE spaces:

$$u_h \in V_h : \int_\Omega \mu\boldsymbol{\nabla}u_h\cdot\boldsymbol{\nabla}v_h\mathrm{d}\Omega + \int_\Omega v_h\left(\sigma u_h + (\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h\right)\mathrm{d}\Omega = \int_\Omega f v_h\mathrm{d}\Omega, \tag{4.4}$$

for all $v_h \in V_h$. The discrete problem satisfies the same bound as in Theorem 4.2.1. However, we have discussed in the introduction that the results obtained with this formulation are not accurate. In order to show why this is the case, let us analyse the error committed by the FE solution.

Theorem 4.3.1: Error estimates for convection-diffusion-reaction

The solution u of (4.3) and the FE solution u_h of (4.4) satisfy the following error estimate:

$$\mu|u-u_h|^2_{H^1(\Omega)} + \sigma\|u-u_h\|^2_{L^2(\Omega)}$$
$$\leq C\left[\mu h^{2(r-1)}|u|^2_{H^r(\Omega)} + \sigma h^{2r}|u|^2_{H^r(\Omega)}\right.$$
$$\left. + \min\left(\frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}}{\sigma^2}\sigma h^{2(r-1)}, \frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}h^2}{\mu^2}\mu h^{2(r-1)}\right)|u|^2_{H^r(\Omega)}\right]$$

for $u \in H^r(\Omega)$ with $1 < r \leq p+1$, p being the order of approximation.

Proof. First, we use the fact that the total error $e \doteq u - u_h$ can be split into two parts, the interpolation error $e_{Ih} \doteq u - I_h(u)$ and the approximation error $e_h \doteq I_h(u) - u_h$, i.e., $e = e_{Ih} + e_h$. $I_h(u)$ is an optimal interpolant of u (see Lemma 2.5.6).
Let us denote the bilinear form on the left-hand side of (4.4) by $a(u,v)$. Now, we use first the Galerkin orthogonality, to get:

$$a(u-u_h, v_h) = a(I_h(u)-u_h, v_h) + a(u-I_h(u), v_h)$$
$$= a(e_h, v_h) + a(e_{Ih}, v_h) = 0, \qquad \forall v_h \in V_h.$$

We can now take $v_h = e_h$ and proceed as in the proof of Theorem 4.2.1 (the convective term vanishes when both arguments are e_h), to get:

$$\mu|e_h|^2_{H^1(\Omega)} + \sigma\|e_h\|^2_{L^2(\Omega)} = a(e_h, e_h) = -a(e_{Ih}, e_h).$$

Next, we must bound the right-hand side of the previous equation. We have:

$$-a(e_{Ih}, e_h) \leq \left|\int_\Omega \mu\boldsymbol{\nabla}e_{Ih}\cdot\boldsymbol{\nabla}e_h\mathrm{d}\Omega\right| + \left|\int_\Omega \sigma e_{Ih}e_h\mathrm{d}\Omega\right| + \left|\int_\Omega e_h((\boldsymbol{\beta}\cdot\boldsymbol{\nabla}e_{Ih}))\mathrm{d}\Omega\right|.$$

We can bound the first two terms as follows:

$$\begin{aligned}&\Big|\int_\Omega \mu\boldsymbol{\nabla} e_{Ih}\cdot\boldsymbol{\nabla} e_h \mathrm{d}\Omega\Big| + \Big|\int_\Omega \sigma e_{Ih} e_h \mathrm{d}\Omega\Big| \\ &\leq \mu|e_{Ih}|^2_{H^1(\Omega)} + \sigma\|e_{Ih}\|^2_{L^2(\Omega)} + \frac{\mu}{4}|e_h|^2_{H^1(\Omega)} + \frac{\sigma}{4}\|e_h\|^2_{L^2(\Omega)}.\end{aligned}$$

The convective term can be treated as follows:

$$\begin{aligned}\int_\Omega e_h(\boldsymbol{\beta}\cdot\boldsymbol{\nabla} e_{Ih})\mathrm{d}\Omega &= -\int_\Omega e_{Ih}(\boldsymbol{\beta}\cdot\boldsymbol{\nabla} e_h)\mathrm{d}\Omega \\ &\leq \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}\|e_{Ih}\|_{L^2(\Omega)}\|\boldsymbol{\nabla} e_h\| \\ &\leq \frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}}{\mu}\|e_{Ih}\|^2_{L^2(\Omega)} + \frac{\mu}{4}\|\boldsymbol{\nabla} e_h\|^2.\end{aligned}$$

Alternatively, if $\sigma > 0$, we can also bound the convective term as follows:

$$\int_\Omega e_h(\boldsymbol{\beta}\cdot\boldsymbol{\nabla} e_{Ih})\mathrm{d}\Omega \leq \frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}}{\sigma}\|e_{Ih}\|^2_{H^1(\Omega)} + \frac{\sigma}{4}\|e_h\|^2_{L^2(\Omega)}.$$

Thus, we get

$$\begin{aligned}&\frac{\mu}{2}|e_h|^2_{H^1(\Omega)} + \frac{\sigma}{2}\|e_h\|^2_{L^2(\Omega)} \\ &\leq \mu|e_{Ih}|^2_{H^1(\Omega)} + \sigma\|e_{Ih}\|^2_{L^2(\Omega)} \\ &\quad + \min\left(\frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}}{\sigma}\|e_{Ih}\|^2_{H^1(\Omega)}, \frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}}{\mu}\|e_{Ih}\|^2_{L^2(\Omega)}\right)\end{aligned}$$

Using the interpolation properties of FE spaces, assuming that the order of approximation is $p > 0$ and that the exact solution $u \in H^r(\Omega)$ for $1 < r \leq p+1$, we get

$$|e_{Ih}|_{H^1(\Omega)} \leq Ch^{r-1}|u|_{H^r(\Omega)}, \qquad \|e_{Ih}\|_{L^2(\Omega)} \leq Ch^r|u|_{H^r(\Omega)}.$$

Using these interpolation error bounds, we finally get:

$$\begin{aligned}&\frac{\mu}{2}|e_h|^2_{H^1(\Omega)} + \frac{\sigma}{2}\|e_h\|^2_{L^2(\Omega)} \\ &\leq C\left[\mu h^{2(r-1)}|u|^2_{H^r(\Omega)} + \sigma h^{2r}|u|^2_{H^r(\Omega)}\right. \\ &\left.\quad + \min\left(\frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}}{\sigma^2 h^2}\sigma h^{2r}|u|^2_{H^r(\Omega)}, \frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}h^2}{\mu^2}\mu h^{2(r-1)}|u|^2_{H^r(\Omega)}\right)\right].\end{aligned}$$

We can prove the final result using the triangle inequality and the interpolation error. □

The last bound shows how the convective term affects the accuracy of the FE method. Let us consider the case $\sigma = 0$ for simplicity. In this case, the convective term cannot be bounded by the reaction term, and we must use the diffusive term in the proof above. In this case, we get

$$\frac{\mu}{2}|e|^2_{H^1(\Omega)} \leq C\left(1 + \frac{\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}h^2}{\mu^2}\right)\frac{\mu}{2}h^{2(r-1)}|u|^2_{H^r(\Omega)} \tag{4.5}$$

for a constant $C > 0$.

The constant in the error blows up as $\|\boldsymbol{\beta}\|_{L^\infty(\Omega)} \to \infty$. As a result, as convection becomes more dominant for a fixed mesh, we should expect a loss of accuracy in the FE method. However, the problem can be *solved* by refining the mesh, recovering good results for $h \sim \frac{\mu}{\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}}$. Unfortunately, in many cases, this mesh size

is not practical (especially in 3D) due to limited computational resources. In the following sections, we will present two methods that aim to solve this issue by adding new *stabilisation* terms to the discrete formulation.

Remark 4.3.2: Relation to turbulent flows

We note that the discussion is very similar when approximating fluid problems using the Navier-Stokes equations. The limit of high convection represents a highly turbulent, very chaotic flow. The approximation using FE methods is not accurate unless the mesh size can capture *viscous* scales. The Navier-Stokes equations are nonlinear and indefinite, so we will not consider them in this introductory course.

4.4 Artificial diffusion

The artificial diffusion method was the first to tackle the problem described above. The method dates back to 1950, proposed by Von Neumann and Richtmyer, who worked at Los Alamos National Labs for the Manhattan project. The idea is simple. Looking at (4.5), it is clear that if $\mu \sim \|\boldsymbol{\beta}\|_{L^\infty(\Omega)} h$, the problem is solved, i.e., the error for the highly convective regime will be similar to the one at the viscous regime. Thus, they proposed to add an artificial diffusion $\mu_{AD} = \|\boldsymbol{\beta}\|_{L^\infty(\Omega)} h$ to the problem. Let us define $\bar{\mu} \doteq \mu + \mu_{AD}$. The discretisation of the problem with artificial diffusion reads:

$$u_h \in V_h : \int_\Omega \bar{\mu} \boldsymbol{\nabla} u_h \cdot \boldsymbol{\nabla} v_h \mathrm{d}\Omega + \int_\Omega v_h \left(\sigma u_h + (\boldsymbol{\beta} \cdot \boldsymbol{\nabla}) u_h\right) \mathrm{d}\Omega = \int_\Omega f v_h \mathrm{d}\Omega. \tag{4.6}$$

Thus, this method is not a Galerkin method anymore since we have added a new term to the Galerkin formulation to fix the accuracy issues for convection-dominant problems.

The stability of the method is obtained as in Theorem 4.2.1, the only difference being the value of the diffusion. Its accuracy is the subject of the next section.

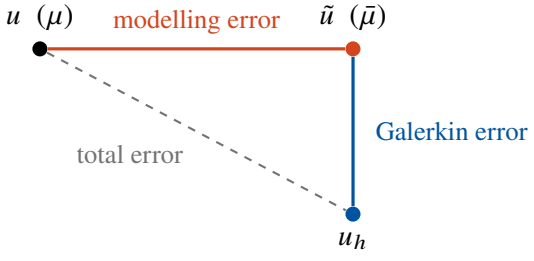
4.4.1 Error analysis for artificial diffusion

Since the method (4.6) is not a Galerkin approximation of (4.3), the error analysis of Theorem 4.3.1 does not apply to it directly. The key observation is that (4.6) *is* a Galerkin method, only for a different problem: it is the Galerkin approximation of the convection-diffusion problem with viscosity $\bar{\mu}$ in place of μ. This suggests splitting the error into a part due to the change of problem and a part due to the discretisation, and treating each one with a result we already have. To make this precise, let us write the bilinear form with an explicit viscosity,

$$a(\nu; w, v) \doteq \int_\Omega \nu \boldsymbol{\nabla} w \cdot \boldsymbol{\nabla} v \mathrm{d}\Omega + \int_\Omega v \left(\sigma w + (\boldsymbol{\beta} \cdot \boldsymbol{\nabla}) w\right) \mathrm{d}\Omega, \qquad \ell(v) \doteq \int_\Omega f v \mathrm{d}\Omega,$$

so that the exact problem (4.3) reads $a(\mu; u, v) = \ell(v)$ for all $v \in H^1_0(\Omega)$ and the artificial diffusion method (4.6) reads $a(\bar{\mu}; u_h, v_h) = \ell(v_h)$ for all $v_h \in V_h$. Alongside u and u_h we now introduce a third function, the solution of the *continuous* problem with the enlarged viscosity:

$$\tilde{u} \in H^1_0(\Omega) : \quad a(\bar{\mu}; \tilde{u}, v) = \ell(v), \qquad \forall v \in H^1_0(\Omega). \tag{4.7}$$



Comparing (4.7) with (4.6), we see that u_h is nothing but the *plain Galerkin approximation of* $\tilde{u}$: same bilinear form, same right-hand side, the space $H^1_0(\Omega)$ replaced by V_h. We may therefore split the error into two pieces of a completely different nature,

$$u - u_h = \underbrace{(u - \tilde{u})}_{\textit{modelling}\text{ error}} + \underbrace{(\tilde{u} - u_h)}_{\textit{discretisation}\text{ error}}, \tag{4.8}$$

We consider the two contributions in turn.

Since u_h is the Galerkin approximation of $\tilde{u}$, Theorem 4.3.1 applies verbatim, with μ replaced by $\bar{\mu}$. Writing the estimate as in (4.5) in terms of the cell Péclet number, we get, for $\sigma = 0$,

$$\bar{\mu}\,|\tilde{u} - u_h|^2_{H^1(\Omega)} \le C\left(1 + \overline{\mathrm{Pe}}_h^2\right)\bar{\mu}\,h^{2(r-1)}|\tilde{u}|^2_{H^r(\Omega)}, \qquad \overline{\mathrm{Pe}}_h \doteq \frac{\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h}{\bar{\mu}}.$$

The choice $\mu_{AD} = \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h$ is precisely what makes this constant harmless. Indeed,

$$\overline{\mathrm{Pe}}_h = \frac{\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h}{\mu + \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h} < 1 \qquad \text{for every } h > 0 \text{ and every } \mu > 0,$$

so the constant that blew up in Theorem 4.3.1 is now bounded by $2C$, uniformly in μ. The artificial diffusion method has removed the mesh restriction $h \lesssim \mu/\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}$ altogether: we can take whatever mesh we can afford.

We turn to the second contribution in (4.8). What we have not yet checked is how far $\tilde{u}$ is from u, i.e., how much we have changed the problem. Since $a(\bar{\mu}; w, v) = a(\mu; w, v) + \mu_{AD}\int_\Omega \boldsymbol{\nabla} w \cdot \boldsymbol{\nabla} v \mathrm{d}\Omega$, the exact solution satisfies

$$a(\bar{\mu}; u, v) = \ell(v) + \mu_{AD}\int_\Omega \boldsymbol{\nabla} u \cdot \boldsymbol{\nabla} v \,\mathrm{d}\Omega, \qquad \forall v \in H^1_0(\Omega).$$

Subtracting (4.7) from this identity, we obtain the equation satisfied by the modelling error:

$$a(\bar{\mu}; u - \tilde{u}, v) = \mu_{AD}\int_\Omega \boldsymbol{\nabla} u \cdot \boldsymbol{\nabla} v \,\mathrm{d}\Omega, \qquad \forall v \in H^1_0(\Omega).$$

We now take $v = u - \tilde{u}$. As in the proof of Theorem 4.2.1, the convective term vanishes, and we are left with

$$\bar{\mu}\,|u - \tilde{u}|^2_{H^1(\Omega)} + \sigma\|u - \tilde{u}\|^2_{L^2(\Omega)} \le \mu_{AD}\,|u|_{H^1(\Omega)}|u - \tilde{u}|_{H^1(\Omega)}.$$

Young's inequality $ab \le \frac{a^2}{2\bar{\mu}} + \frac{\bar{\mu}\,b^2}{2}$ applied to the right-hand side gives

$$\frac{\bar{\mu}}{2}|u - \tilde{u}|^2_{H^1(\Omega)} \le \frac{\mu^2_{AD}}{2\bar{\mu}}|u|^2_{H^1(\Omega)}, \qquad \text{i.e.,} \qquad |u - \tilde{u}|_{H^1(\Omega)} \le \frac{\mu_{AD}}{\bar{\mu}}|u|_{H^1(\Omega)}.$$

Since $\mu_{AD} \le \bar{\mu}$, multiplying by $\bar{\mu}$ we get the bound in the form we need for (4.8):

$$\bar{\mu}\,|u - \tilde{u}|^2_{H^1(\Omega)} \le \mu_{AD}|u|^2_{H^1(\Omega)} = \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}\,h\,|u|^2_{H^1(\Omega)}.$$

Two features of this bound are worth stressing: it does not involve μ^{-1}, and it does not involve h^r. It is $O(h)$, and it does not improve when a higher-order finite element space is used.

Combining the two contributions with the triangle inequality, we obtain the following estimate.

Theorem 4.4.1: Error estimate for artificial diffusion

Let $\sigma = 0$, let u be the solution of (4.3) and u_h the solution of the artificial diffusion method (4.6), with $\mu_{AD} = \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h$ and $\bar{\mu} = \mu + \mu_{AD}$. Assume that the solution $\tilde{u}$ of (4.7) belongs to $H^r(\Omega)$, with

$1 < r \leq p+1$, *and that* $|\tilde{u}|_{H^r(\Omega)} \leq C|u|_{H^r(\Omega)}$ *uniformly in* h. *Then*

$$\bar{\mu}\,|u-u_h|^2_{H^1(\Omega)} \leq C\Big[\underbrace{\bar{\mu}\,h^{2(r-1)}|u|^2_{H^r(\Omega)}}_{\text{discretisation}} + \underbrace{\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}\,h\,|u|^2_{H^1(\Omega)}}_{\text{modelling}}\Big], \tag{4.9}$$

with a constant $C > 0$ *independent of* h, μ *and* $\boldsymbol{\beta}$.

Remark 4.4.2: Robustness and order of convergence

Two things have happened at once, and it is worth separating them.

- The method is *robust* in μ: no constant in (4.9) degenerates as $\mu \to 0$, in sharp contrast with (4.5). This is the property that motivates the method.
- The method is *at most first order in* h, whatever the order p of the finite element space. The first term in (4.9) improves as we increase p; the second one is independent of r and eventually dominates. There is therefore no benefit in combining artificial diffusion with $\mathcal{P}_2$ or higher-order spaces.
- Physically, we have replaced the viscosity μ by $\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h$, which smears every layer of the solution over a few cells, and it does so in every direction, including the crosswind direction, where no stabilisation is needed. Section 4.5 addresses this last point.

4.5 Streamline diffusion

A slightly less dissipative method can be defined by adding the following *streamline* diffusion to the discrete Galerkin formulation in (4.4):

$$\tau\int_\Omega(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})v_h\mathrm{d}\Omega, \qquad \text{with } \tau \doteq \frac{h}{\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}} \tag{4.10}$$

being a numerical coefficient whose choice will become clear when carrying out the error analysis of the method. However, it can already be seen that this choice leads to a dimensionally correct term, i.e., all the terms in the equation (e.g., compare against the convective term) have the same dimension. Dimension consistency is a necessary condition for a good numerical stabilisation. In order to check the dimensional equivalence, we note that, due to the discrete inverse inequality (up to a constant), $\|\boldsymbol{\nabla}u_h\| \leq h^{-1}\|u_h\|$ and so, we can understand $\boldsymbol{\nabla}$ over FE functions as h^{-1}. It can be shown that this term also fixes the accuracy problem of the Galerkin formulation, but it is still a first-order method.

We can now see how effective this method is. The stability analysis follows the same line as above. Taking $v_h = u_h$, we have the additional stability

$$\tau\|\boldsymbol{\beta}\cdot\boldsymbol{\nabla}u_h\|^2.$$

Let us proceed to the error analysis. We only provide the bounds that are different from the Galerkin analysis. We can now handle the error related to the convective term as follows, taking benefit of the additional stability:

$$\begin{aligned}\int_\Omega e_h(\boldsymbol{\beta}\cdot\boldsymbol{\nabla}e_{Ih})\mathrm{d}\Omega = -\int_\Omega e_{Ih}(\boldsymbol{\beta}\cdot\boldsymbol{\nabla}e_h)\mathrm{d}\Omega &\leq \|e_{Ih}\|_{L^2(\Omega)}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_h\| \\ &\leq \tau^{-1}\|e_{Ih}\|^2_{L^2(\Omega)} + \frac{\tau}{4}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_h\|^2.\end{aligned}$$

$$\tau\int_\Omega(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_{Ih}(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_h\mathrm{d}\Omega \leq \tau\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_{Ih}\|^2 + \frac{\tau}{4}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_h\|^2.$$

Now, we can check that the two new interpolation error terms can be bounded as follows:

$$\tau^{-1}\|e_{Ih}\|^2_{L^2(\Omega)} \leq C\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h^{2(r-1/2)}|u|^2_{H^r(\Omega)},$$
$$\tau\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_{Ih}\|^2 \leq C\|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h^{2(r-1/2)}|u|^2_{H^r(\Omega)}.$$

However, the consistency error for this problem can only be bounded as:

$$\tau\int_\Omega(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_h\mathrm{d}\Omega \leq h\|\boldsymbol{\nabla}u\|\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_h\|.$$

Again, the method can only be first order, regardless of the order of the FE approximation.

4.6 SUPG stabilisation

In this section, we will finally see how to fix the Galerkin method in a consistent way. We will define a method with no consistency error, i.e., the exact solution is also the solution of the discrete method (even though not Galerkin). The proposed method will be optimally convergent in terms of h. In this section, we consider $\sigma = 0$ for simplicity.

The question is how to add the *magic* streamline diffusion term in (4.10) without introducing a consistency error. The idea can be to add a term that involves the residual of the PDE in strong form. However, second derivatives of FE functions are not well-defined (these functions are only C^0). Instead, we will add the residual at the interior of each cell in a *broken* cell-wise way. This is the idea behind the Streamline-upwinding Petrov-Galerkin (SUPG) method, which adds the following term:

$$\sum_{K\in\mathcal{T}_h}\tau_K\int_K(-\boldsymbol{\nabla}\cdot(\mu\boldsymbol{\nabla}u_h)+(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h-f)((\boldsymbol{\beta}\cdot\boldsymbol{\nabla})v_h)\mathrm{d}\Omega. \tag{4.11}$$

Two key observations apply. First, the term (4.10) that provides streamline diffusion is part of (4.11). Second, this term vanishes for a smooth enough solution $u \in H^2(\Omega)$. Thus, it introduces no consistency error.

Now, let us analyse the stability of this method. We have, for $v_h = u_h$:

$$\sum_{K\in\mathcal{T}_h}\tau_K\int_K(-\boldsymbol{\nabla}\cdot(\mu\boldsymbol{\nabla}u_h)+(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h-f)((\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h)\mathrm{d}\Omega$$
$$=\sum_{K\in\mathcal{T}_h}\tau_K\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h\|^2_{L^2(K)}-\sum_{K\in\mathcal{T}_h}\tau_K\int_K(\boldsymbol{\nabla}\cdot(\mu\boldsymbol{\nabla}u_h)+f)((\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h)\mathrm{d}\Omega.$$

Let us bound the last term in the expression above. On one side, we can bound the forcing term

$$\sum_{K\in\mathcal{T}_h}\tau_K\int_K f((\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h)\mathrm{d}\Omega \leq \sum_{K\in\mathcal{T}_h}4\tau_K\|f\|^2_{L^2(K)}+\sum_{K\in\mathcal{T}_h}\frac{\tau_K}{4}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h\|^2_{L^2(K)}.$$

Next, we bound the stabilisation term related to the Laplacian. We recall that we assume that μ is constant for simplicity. The viscous term in the SUPG stabilisation reads:

$$-\sum_{K\in\mathcal{T}_h}\tau_K\int_K(\mu\boldsymbol{\nabla}\cdot\boldsymbol{\nabla}u_h)((\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h)\mathrm{d}\Omega.$$

We note that this term vanishes for first order FEs on simplicial meshes (triangles or tetrahedra), i.e., when the local FE spaces are $\mathcal{P}_1(K)$. For hex meshes (quadrilateral or hexahedra), this term only vanishes for first order spaces, i.e., for the local FE space $\mathcal{Q}_1(K)$. In any other situation, the bound for this term is more involved. Let us consider the following discrete inverse inequality for a given FE space:

$$\|\Delta u_h\|_K \leq C_{inv}h_K^{-1}\|\boldsymbol{\nabla}u_h\|_K,$$

where the constant C_{inv} depends on the order p of the FE space (it can be proved that the constant is of order $O(p^2)$). Using this inequality, we get

$$
\begin{aligned}
&- \sum_{K\in\mathcal{T}_h} \tau_K \int_K (\mu\boldsymbol{\nabla}\cdot\boldsymbol{\nabla}u_h)((\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h)\mathrm{d}\Omega \\
&\leq \sum_{K\in\mathcal{T}_h} \tau_K^{1/2} h^{-1}\mu C_{inv}\|\boldsymbol{\nabla}u_h\|_{L^2(K)}\tau_K^{1/2}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h\|_{L^2(K)} \\
&\leq \sum_{K\in\mathcal{T}_h} \tau_K h^{-2}\mu^2 C_{inv}^2\|\boldsymbol{\nabla}u_h\|_{L^2(K)}^2 + \frac{\tau_K}{4}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h\|_{L^2(K)}^2.
\end{aligned}
$$

Under the assumption that

$$
\tau_K < \frac{h^2}{4\mu C_{inv}^2},
$$

we can prove the stability of the method. For example, a typical choice of τ that satisfies the requirements is the following:

$$
\tau_K \doteq \left(\frac{4\mu C_{inv}^2}{h^2} + \frac{\|\boldsymbol{\beta}\|_{L^\infty(K)}}{h}\right)^{-1}.
$$

It provides the desired dissipation in the limit of zero diffusion and satisfies the upper bound required for stability. Combining these results, we can obtain the stability of the SUPG method.

Lemma 4.6.1: Stability of SUPG

The solution u_h of the SUPG stabilisation of (4.4) satisfies the following stability bound:

$$
\frac{\mu}{4}|u_h|_{H^1(\Omega)}^2 + \sum_{K\in\mathcal{T}_h}\frac{\tau_K}{2}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})u_h\|_{L^2(K)}^2 \leq \frac{C_P^2}{2\mu}\|f\|_{H^{-1}(\Omega)}^2 + \sum_{K\in\mathcal{T}_h} 4\tau_K\|f\|_{L^2(K)}^2,
$$

under the assumption that $\tau_K < \frac{h^2}{4\mu C_{inv}^2}$.

We note that, in general, the value of C_{inv} depends on not only the local FE space but also the cell shape. It can be computed locally by solving an eigenvalue problem. However, this is quite expensive. For structured meshes (all cells have the same shape), these constants can be computed beforehand though.

We are now in a position to do what we could not do for the previous two methods: state an error estimate whose order follows the order of the finite element space.

4.6.1 Error analysis for SUPG

The stability result of Lemma 4.6.1 suggests the norm in which the method is naturally analysed. Let us define the *SUPG norm*

$$
|||v|||^2 \doteq \mu\,|v|_{H^1(\Omega)}^2 + \sum_{K\in\mathcal{T}_h}\tau_K\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})v\|_{L^2(K)}^2.
$$

The first term is the control we already had from the Galerkin method, and it degenerates as $\mu \to 0$; the second one is the control the stabilisation has bought us, and it survives that limit. Denoting by $a_h(\cdot,\cdot)$ the SUPG bilinear form, i.e., the left-hand side of (4.4) plus the term (4.11) without the forcing contribution, and by $\ell_h(\cdot)$ the corresponding right-hand side, the computations preceding Lemma 4.6.1 show the coercivity

$$
a_h(v_h, v_h) \geq \frac{\mu}{4}|v_h|_{H^1(\Omega)}^2 + \sum_{K\in\mathcal{T}_h}\frac{\tau_K}{2}\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})v_h\|_{L^2(K)}^2 \geq \frac{1}{4}|||v_h|||^2, \tag{4.12}
$$

for all $v_h \in V_h$, under the assumption $\tau_K < \frac{h^2}{4\mu C_{inv}^2}$.

Theorem 4.6.2: Convergence of SUPG

Let $\sigma = 0$ and let $u \in H^r(\Omega)$, with $2 \le r \le p+1$, be the solution of (4.3), and let u_h be the solution of the SUPG method. Assume that the stabilisation parameter satisfies

$$\tau_K \le \frac{h^2}{4\mu C_{inv}^2} \qquad \text{and} \qquad \tau_K \le \frac{h}{\|\boldsymbol{\beta}\|_{L^\infty(K)}},$$

as does, e.g., the choice $\tau_K = \left(\frac{4\mu C_{inv}^2}{h^2} + \frac{\|\boldsymbol{\beta}\|_{L^\infty(K)}}{h}\right)^{-1}$ proposed above. Then

$$|||u - u_h|||^2 \le C\left(\mu\, h^{2(r-1)} + \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}\, h^{2r-1}\right) |u|^2_{H^r(\Omega)}, \tag{4.13}$$

with $C > 0$ independent of h, μ and $\boldsymbol{\beta}$.

Proof (advanced). Since $u \in H^2(\Omega)$, the residual $-\nabla\cdot(\mu\nabla u) + (\boldsymbol{\beta}\cdot\nabla)u - f$ vanishes almost everywhere, so the stabilisation term (4.11) evaluated at u is zero and $a_h(u, v_h) = \ell_h(v_h)$ for all $v_h \in V_h$. The method is *consistent*, and subtracting the discrete equation we recover Galerkin orthogonality,

$$a_h(u - u_h, v_h) = 0, \qquad \forall v_h \in V_h.$$

This is the step that failed for artificial and streamline diffusion, and it is the reason why no $O(h)$ term will appear below.

Splitting $u - u_h = e_{Ih} + e_h$ with $e_{Ih} \doteq u - I_h(u)$ and $e_h \doteq I_h(u) - u_h \in V_h$ as in the proof of Theorem 4.3.1, orthogonality with $v_h = e_h$ and the coercivity (4.12) give

$$\frac{1}{4}|||e_h|||^2 \le a_h(e_h, e_h) = -a_h(e_{Ih}, e_h). \tag{4.14}$$

It remains to bound the four contributions to $a_h(e_{Ih}, e_h)$. Throughout, we write $\|\cdot\|_K \doteq \|\cdot\|_{L^2(K)}$ and use Young's inequality with a weight chosen so that every term in $|||e_h|||^2$ produced on the right is absorbed by the left-hand side of (4.14).

The diffusive term. By Cauchy–Schwarz and Young,

$$\mu\int_\Omega \nabla e_{Ih}\cdot\nabla e_h \,\mathrm{d}\Omega \le C\mu|e_{Ih}|^2_{H^1(\Omega)} + \frac{\mu}{16}|e_h|^2_{H^1(\Omega)}.$$

The convective term. Integrating by parts as in Theorem 4.3.1 and then splitting the integral cell by cell, we may now use the streamline control instead of the diffusive one:

$$\int_\Omega e_h\,(\boldsymbol{\beta}\cdot\nabla)e_{Ih}\,\mathrm{d}\Omega = -\sum_K \int_K e_{Ih}\,(\boldsymbol{\beta}\cdot\nabla)e_h\,\mathrm{d}\Omega \le \sum_K C\tau_K^{-1}\|e_{Ih}\|_K^2 + \sum_K \frac{\tau_K}{16}\|(\boldsymbol{\beta}\cdot\nabla)e_h\|_K^2.$$

This is the crucial difference with the Galerkin analysis: there, the only way to absorb this term was through $\mu|e_h|^2_{H^1}$, which produced the factor $\|\boldsymbol{\beta}\|^2_{L^\infty(\Omega)}/\mu$.

The streamline part of the stabilisation. Likewise,

$$\sum_K \tau_K \int_K (\boldsymbol{\beta}\cdot\nabla)e_{Ih}\,(\boldsymbol{\beta}\cdot\nabla)e_h\,\mathrm{d}\Omega \le \sum_K C\tau_K\|(\boldsymbol{\beta}\cdot\nabla)e_{Ih}\|_K^2 + \sum_K \frac{\tau_K}{16}\|(\boldsymbol{\beta}\cdot\nabla)e_h\|_K^2.$$

The diffusive part of the stabilisation. Finally,

$$-\sum_K \tau_K \int_K \mu\Delta e_{Ih}\,(\boldsymbol{\beta}\cdot\nabla)e_h\,\mathrm{d}\Omega \le \sum_K C\tau_K\mu^2\|\Delta e_{Ih}\|_K^2 + \sum_K \frac{\tau_K}{16}\|(\boldsymbol{\beta}\cdot\nabla)e_h\|_K^2,$$

and here the upper bound on τ_K enters: $\tau_K\mu^2 \le \frac{\mu h^2}{4C_{inv}^2}$, so that $\tau_K\mu^2\|\Delta e_{Ih}\|_K^2 \le C\mu h^2\|\Delta e_{Ih}\|_K^2$.

Collecting the four bounds in (4.14), the terms carrying a factor $\frac{1}{16}$ add up to at most $\frac{1}{8}|||e_h|||^2$ and are absorbed by the left-hand side, leaving

$$|||e_h|||^2 \leq C\Big[\mu|e_{Ih}|^2_{H^1(\Omega)} + \sum_K \Big(\tau_K^{-1}\|e_{Ih}\|^2_K + \tau_K\|(\boldsymbol{\beta}\cdot\boldsymbol{\nabla})e_{Ih}\|^2_K + \mu h^2\|\Delta e_{Ih}\|^2_K\Big)\Big].$$

We now insert the interpolation estimates of Lemma 2.5.6, namely $\|e_{Ih}\|_K \leq Ch^r|u|_{H^r(K)}$, $|e_{Ih}|_{H^1(K)} \leq Ch^{r-1}|u|_{H^r(K)}$ and, for $r \geq 2$, $\|\Delta e_{Ih}\|_K \leq Ch^{r-2}|u|_{H^r(K)}$. Using $\tau_K^{-1} \leq \frac{4\mu C_{inv}^2}{h^2} + \frac{\|\boldsymbol{\beta}\|_{L^\infty(K)}}{h}$ for the second term and $\tau_K \leq h/\|\boldsymbol{\beta}\|_{L^\infty(K)}$ for the third, the four contributions become, respectively,

$$\mu h^{2(r-1)}, \qquad \mu h^{2(r-1)} + \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h^{2r-1}, \qquad \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h^{2r-1}, \qquad \mu h^{2(r-1)},$$

each multiplied by $|u|^2_{H^r(\Omega)}$ after summing over the cells. Hence

$$|||e_h|||^2 \leq C\left(\mu h^{2(r-1)} + \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h^{2r-1}\right)|u|^2_{H^r(\Omega)}.$$

The interpolation error satisfies the same bound in the SUPG norm, by the same estimates, and the triangle inequality $|||u-u_h||| \leq |||e_{Ih}||| + |||e_h|||$ gives (4.13). The last statement follows by keeping only the streamline term on the left and using $\mu \lesssim \|\boldsymbol{\beta}\|_{L^\infty(\Omega)}h$ on the right. □

4.7 Tutorials

1. Consider the one-dimensional problem

   $$-\mu u'' + \beta u' = 0, \quad 0 < x < 1, \quad u(0) = 1, u(1) = 0.$$

   The analytical solution of this problem (for $\beta = 1$) reads

   $$u(x) = (1 - e^{-\frac{1}{\mu}})^{-1}(1 - e^{-\frac{1-x}{\mu}}).$$

   Write the linear equation corresponding to interior nodes in the mesh (the end-point values are known) for a linear FE approximation of this problem. Here we are asking for the expression of the rows in the linear system arising from the discretisation. Since all the rows are identical, we only ask you to provide one. E.g., for the node i in the mesh, the equation must be in terms of the nodal values U^{i-1}, U^i, U^{i+1}.

2. Consider the nodal equation above and take the limit when $\mu \to 0$. Assume that N is odd. In the limit, which solution do you get?

3. In the problem above, take $\beta = 1$ and $\mu = 1/10$. First, choose a value of h for which you would expect reasonably good behavior of the Galerkin method. Justify your choice.

4. Define a (consistent) SUPG method for the problem with reaction. Perform the stability analysis. Which additional requirements do you need on τ_K to get stability? Can you define an expression of τ that satisfies this other requirement and the previous ones (for absorbing the diffusive term and for having the right amount of added diffusion in the convection-dominated case)?

5. Next, use the code provided in the lectures for this problem (using Gridap) with the values in the previous question. Observe what happens when $\mu = 10^{-k}$, $k = 1, 2, 3, 4$. Compute the error $\|u - u_h\|_{H^1(\Omega)}$ with respect to μ. Explain what you observe. (Provide figures or tables with the error and plots of obtained solutions).

6. Take $\mu = 10^{-3}$. Observe what happens when reducing $h = 10^{-l}$, $l = 1, 2, 3$. Compute the error $\|u - u_h\|_{H^1(\Omega)}$ with respect to h. Justify what you observe. (Provide figures or tables).

7. Repeat the last two questions for the streamline diffusion and SUPG method.

8. Now, consider the previous question for quadratic elements instead of linear. What do you observe? Can you explain the results?

5 Parabolic equations

5.1 Introduction

In this section, we analyse time-dependent problems. We will work with the heat equation as a model problem, even though we can readily use the techniques proposed herein for other parabolic problems, e.g., the transient convection-diffusion equation.

We will introduce the method of lines for the discretisation of parabolic PDEs. The idea is to carry out the spatial discretisation using FEs but considering the DOFs values as time-dependent. After this step, the problem will be discrete in space but continuous in time. More specifically, we end up with a system of ordinary differential equations (ODEs) for the DOF values of the FE approximation. Next, we will use an ODE solver for this problem.

This unit does not focus on providing a detailed presentation of numerical methods for ODEs, which is the main ingredient we will need in this chapter. Instead, we rely on the material from unit MTH2051. We refer the interested student to the lecture notes of this unit for more details. We will focus on the θ-method for simplicity, which involves both explicit and implicit methods and first and second-order (in time) schemes.

We will do the stability and convergence analysis of the semi-discrete problem in space. We will also discuss the stability of the fully discrete problem. The convergence analysis of the fully discrete problem is quite technical and not considered.

5.2 The heat equation

Let us consider the heat equation, i.e., the transient version of the Poisson problem. We consider a physical domain in space Ω and the time interval $I \doteq [0, T]$. The strong form of this problem reads:

$$
\begin{aligned}
u_t - \boldsymbol{\nabla} \cdot \mu \boldsymbol{\nabla} u &= f, && \text{in } \Omega \times I \\
u &= g, && \text{on } \Gamma_D \times I \\
\mu \boldsymbol{n} \cdot \boldsymbol{\nabla} u &= q, && \text{on } \Gamma_N \times I \\
u(\boldsymbol{x}, 0) &= u^0(\boldsymbol{x}), && \boldsymbol{x} \in \Omega,
\end{aligned}
$$

where we have considered Dirichlet and Neumann boundary conditions and an *initial condition* u^0 at $t = 0$. The imposition of an initial condition is essential for the problem to be well-posed. In the following, we consider $g = 0$, $\Gamma_D = \partial\Omega$ and $\Gamma_N = \emptyset$ (i.e., homogeneous Dirichlet conditions) for simplicity. In any case, the extension can be carried out using the same techniques as in previous sections.

Let us make some comments about the nature of this system. We can observe that the time derivative term is a *convective* term. If we consider the $d + 1$ space-time domain $\Omega \times I$, $u_t = \boldsymbol{\beta} \cdot \boldsymbol{\nabla}_{(\boldsymbol{x},t)} u$ for $\boldsymbol{\beta} = (\boldsymbol{0}, 1)^T$. On the other hand, we only have diffusion in the spatial direction. This observation tells us that a standard Galerkin FE approximation of this problem in space-time is not going to work.

Another critical observation is time *causality*, the solution at a given time depends on past time values but not future time values. The *arrow of time* should be preserved at the discrete level since it is very beneficial from a computational point of view. Thus, standard FE approximations of the space-time problem (possibly with stabilisation in the time direction convection) are not a clever choice. They couple all the time values in all directions.

The methods proposed below preserve time causality and can be stable. We consider first space discretisation and move next to time discretisation.

5.3 Space discretisation using FEs

To discretise the problem in space, we start writing it in weak form for almost every $t \in I$:

$$u(t) \in H_0^1(\Omega) \ : \ \int_\Omega u_t v \mathrm{d}\Omega + \int_\Omega \mu \boldsymbol{\nabla} u \cdot \boldsymbol{\nabla} v \mathrm{d}\Omega = \int_\Omega f v \mathrm{d}\Omega, \tag{5.1}$$

for any $v \in H_0^1(\Omega)$. The equation is only required for a.e. $t \in I$: the solution is a function of time with values in $H_0^1(\Omega)$, defined up to sets of zero measure in I, and u_t is understood in $L^2(I; H^{-1}(\Omega))$.

Let us introduce some notation. The space of functions $L^2(I;V)$ is the one that contains functions such that

$$\|u\|^2_{L^2(I;V)} \doteq \int_0^T \|u\|_V^2 \mathrm{d}t < \infty.$$

E.g., V can be $L^2(\Omega)$ or $H^1(\Omega)$. We also define the norms

$$\|u\|^2_{C^j(I;V)} \doteq \sup_{t\in I} \sum_{i=0}^{j} \|d_t^i u\|_V^2.$$

This norm requires the function (and its j derivatives) to be continuous in time.

With this notation, we can state some properties of (5.1). The solution of this problem satisfies $u \in C^0(I; L^2(\Omega)) \cap L^2(I; H^1(\Omega))$. We will show this stability at the discrete level below.

We now consider the method of lines, in which we consider a spatial FE space V_h (the grad-conforming FE space introduced in previous sections), but consider the DOFs to be time-dependent: $u_h(\boldsymbol{x}, t) \doteq \sum_{a=1}^N \varphi^a(\boldsymbol{x}) U^a(t)$, where $V_h \doteq \mathrm{span}\{\varphi^1, \dots, \varphi^N\}$ and $U(t) \in \mathbb{R}^N$ is the vector of DOF values. Using the Galerkin method at every time value $t \in I$, we obtain:

$$u_h(t) \in V_h \ : \ \int_\Omega (u_h)_t v_h \mathrm{d}\Omega + \int_\Omega \mu \boldsymbol{\nabla} u_h \cdot \boldsymbol{\nabla} v_h \mathrm{d}\Omega = \int_\Omega f v_h \mathrm{d}\Omega, \tag{5.2}$$

for any $v_h \in V_h$. This is a semi-discrete problem in time, since we have performed the discretisation in space via FE method but not in time. We can now proceed as for steady problems and end up with a system of ODEs:

$$\left[\int_\Omega \varphi^a(\boldsymbol{x}) \varphi^b(\boldsymbol{x}) \mathrm{d}\Omega\right] U_t^b(t) + \left[\int_\Omega \mu \boldsymbol{\nabla} \varphi^a(\boldsymbol{x}) \cdot \boldsymbol{\nabla} \varphi^b(\boldsymbol{x}) \mathrm{d}\Omega\right] U^b(t) = \int_\Omega f \varphi^a \mathrm{d}\Omega,$$

for any $a = 1, \dots, N$. Using compact matrix notation, we have the ODE system:

$$MU_t + KU = F, \qquad \text{or} \qquad U_t + M^{-1}KU = M^{-1}F,$$

where we have used the fact that the mass matrix is non-singular. The analysis of this ODE system has been studied in MTH2051. We make use of Picard-Lindelöf theorem to prove the existence and uniqueness of a vector $U(t)$ with $C^1(I)$ entries.

To prove error estimates, we will make use of the Gronwall inequality.

Lemma 5.3.1: Gronwall inequality

Let $\phi \in C^1(I)$ and $g \in C^0(I)$ such that $\phi_t \le \beta\phi + g$, for $\beta \in \mathbb{R}$. Then, it holds

$$\phi(t) \le e^{\beta t}\phi(0) + \int_0^t e^{\beta(t-s)} g(s) \mathrm{d}s, \qquad \forall t \in I.$$

Proof. Multiply the inequality by $e^{-\beta t}$ and integrate with respect to t. □

Theorem 5.3.2: Stability of semi-discrete heat equation

The solution u_h of (5.2) satisfies the stability bound for any $s \in I$:

$$\|u_h(s)\|^2_{L^2(\Omega)} + \int_0^s \mu \|\boldsymbol{\nabla} u_h\|^2_{L^2(\Omega)} \mathrm{d}t \leq \int_0^s \frac{C_P^2}{\mu} \|f(t)\|^2_{H^{-1}(\Omega)} \mathrm{d}t + \|u_h(0)\|^2_{L^2(\Omega)}.$$

Thus, $u_h \in C^0(I; L^2(\Omega)) \cap L^2(I; H^1(\Omega))$.

Proof. Let us take $v_h = u_h$ in (5.2). Treating the diffusive term and forcing term as for the steady case, we obtain:

$$\int_\Omega u_h (u_h)_t \mathrm{d}\Omega + \frac{\mu}{2} \|\boldsymbol{\nabla} u_h\|^2_{L^2(\Omega)} \leq \frac{C_P^2}{2\mu} \|f\|^2_{H^{-1}(\Omega)}.$$

For the time derivative term, we use:

$$\int_\Omega u_h (u_h)_t \mathrm{d}\Omega = \frac{1}{2} \int_\Omega (u_h^2)_t \mathrm{d}\Omega = \frac{1}{2} d_t \|u_h\|^2_{L^2(\Omega)}.$$

Combining these two equations, integrating from $t = 0$ to $t = s \in I$, and multiplying by two, we get the desired result. □

Now, let us prove error estimates in space for the semi-discrete problem. The proof is quite long but accessible. It relies on a specific projector that allows us to eliminate the interpolation error related to the diffusive term to obtain optimal error bounds in the $C^0(I; L^2(\Omega))$ norm. We make use of the so-called Ritz projector. Let us consider at each time $s \in I$ the solution of the following problem:

$$\mathcal{R}_h(u) \in V_h \ : \ \int_\Omega (\boldsymbol{\nabla} u - \boldsymbol{\nabla} \mathcal{R}_h(u)) \cdot \boldsymbol{\nabla} v_h \mathrm{d}\Omega = 0, \qquad \forall v_h \in V_h. \tag{5.3}$$

This is a Poisson problem at each time. The $H^1(\Omega)$ part of the estimate below follows from Céa's lemma and the interpolation bounds of Lemma 3.12.2; the $L^2(\Omega)$ part requires the duality argument mentioned in Chapter 2 (and H^2-regularity of the Poisson problem, e.g., a convex domain), which we do not prove:

$$\|u - \mathcal{R}_h u\|_{L^2(\Omega)} + h \|u - \mathcal{R}_h u\|_{H^1(\Omega)} \leq C h^{r+1} |u|_{H^{r+1}(\Omega)}. \tag{5.4}$$

Using this projector we can eliminate the interpolation error for the viscous term (that can only be bounded by h^r) and obtain optimal error estimates in $C^0(I; L^2(\Omega))$ that decrease with h^{r+1} (see proof below). Optimal error estimates of order h^r in the norm $L^2(I; H^1(\Omega))$ could be obtained without this projector, in a more standard way. In the proof below, we use $e_h \doteq \mathcal{R}_h u - u_h$.

Theorem 5.3.3: Error estimates for the semi-discrete heat equation

Let us assume that the solution u of (5.1) is $u \in C^1(I; H^{r+1}(\Omega))$ and the solution u_h of the semi-discrete counterpart (5.2) for a FE space of order r, with the discrete initial condition $u_h(0) \doteq \mathcal{R}_h u(0)$ or any

choice satisfying $\|u(0)-u_h(0)\|_{L^2(\Omega)} \le Ch^{r+1}|u(0)|_{H^{r+1}(\Omega)}$. *They satisfy the following error bound:*

$$\begin{aligned}\|u-u_h\|_{C^0(I;L^2(\Omega))} \le & C\Bigg[h^{r+1}|u(0)|_{H^{r+1}(\Omega)} \\ & + h^{r+1}\|u\|_{C^0(I;H^{r+1}(\Omega))} + \frac{C_P^2 h^{r+1}}{\mu}\|u_t\|_{C^0(I;H^{r+1}(\Omega))}\Bigg], \\ \mu^{\frac{1}{2}}\|u-u_h\|_{L^2(I;H^1(\Omega))} \le & C\Bigg[h^{r+1}|u(0)|_{H^{r+1}(\Omega)} \\ & + \frac{\sqrt{T}}{\sqrt{\mu}}h^{r+1}\|u_t\|_{C^0(I;H^{r+1}(\Omega))} + \sqrt{T}\,\mu^{\frac{1}{2}}h^r\|u\|_{C^0(I;H^{r+1}(\Omega))}\Bigg].\end{aligned}$$

Proof. In the following, $(\cdot,\cdot)$ denotes the $L^2(\Omega)$ inner product. We can use Galerkin orthogonality and the definition of $\mathcal{R}_h$ in (5.3) to get:

$$\begin{aligned}&((\mathcal{R}_h u-u_h)_t, v_h) + (\mu\boldsymbol{\nabla}(\mathcal{R}_h u-u_h), \boldsymbol{\nabla} v_h)\\ &= ((\mathcal{R}_h u-u)_t, v_h) + (\mu\boldsymbol{\nabla}(\mathcal{R}_h u-u), \boldsymbol{\nabla} v_h)\\ &= ((\mathcal{R}_h u-u)_t, v_h), \qquad \forall v_h \in V_h.\end{aligned}$$

Now, we can take $v_h = \mathcal{R}_h u - u_h$ to get, proceeding as in the proof of Theorem 5.3.2:

$$\begin{aligned}&\frac{1}{2}\,d_t\|\mathcal{R}_h u-u_h\|^2_{L^2(\Omega)} + \mu\|\boldsymbol{\nabla}(\mathcal{R}_h u-u_h)\|^2_{L^2(\Omega)}\\ &\le ((\mathcal{R}_h u-u)_t, \mathcal{R}_h u-u_h).\end{aligned} \tag{5.5}$$

We split the proof into two parts. We obtain $L^2(\Omega)$ errors first, and $H^1(\Omega)$ errors next.
(1.) $L^2(\Omega)$ error: We can treat the right-hand side as follows:

$$\begin{aligned}&((\mathcal{R}_h u-u)_t, \mathcal{R}_h u-u_h)\\ &\le \frac{C_P^2}{2\mu}\|(\mathcal{R}_h u-u)_t\|^2_{L^2(\Omega)} + \frac{\mu}{2C_P^2}\|\mathcal{R}_h u-u_h\|^2_{L^2(\Omega)},\end{aligned}$$

Using the Poincaré inequality $\|v\|_{L^2(\Omega)} \le C_P|v|_{H^1(\Omega)}$ (with the same constant as in Theorem 4.2.1) on the left-hand side of (5.5), we get:

$$d_t\|\mathcal{R}_h u-u_h\|^2_{L^2(\Omega)} + \frac{\mu}{C_P^2}\|\mathcal{R}_h u-u_h\|^2_{L^2(\Omega)} \le \frac{C_P^2}{\mu}\|(\mathcal{R}_h u-u)_t\|^2_{L^2(\Omega)}.$$

Gronwall's inequality leads to (for $e_h \doteq \mathcal{R}_h u - u_h$):

$$\begin{aligned}\|e_h(t)\|^2_{L^2(\Omega)} \le & \|e_h(0)\|^2_{L^2(\Omega)} e^{-\mu C_P^{-2}t}\\ & + \frac{C_P^2}{\mu}\int_0^t e^{-\mu C_P^{-2}(t-s)}\|(\mathcal{R}_h u-u)_t\|^2_{L^2(\Omega)}\mathrm{d}s.\end{aligned}$$

We can treat the last term as:

$$\begin{aligned}&\int_0^t e^{-\mu C_P^{-2}(t-s)}\|(\mathcal{R}_h u-u)_t\|^2_{L^2(\Omega)}\mathrm{d}s\\ &\le \max_{s\in[0,t]}\|(\mathcal{R}_h u-u)_t\|^2_{L^2(\Omega)}\int_0^t e^{-\mu C_P^{-2}(t-s)}\mathrm{d}s\\ &\le \frac{C_P^2}{\mu}(1-e^{-\mu C_P^{-2}t})\max_{s\in[0,t]}\|(\mathcal{R}_h u-u)_t\|^2_{L^2(\Omega)}.\end{aligned}$$

Using the triangle inequality, we finally get:

$$\|u(t) - u_h(t)\|_{L^2(\Omega)} \leq \|e_h(0)\|_{L^2(\Omega)} e^{-\mu C_P^{-2} \frac{t}{2}} + \frac{C_P^2}{\mu} \max_{s\in[0,t]} \left(\|(\mathcal{R}_h u - u)_t\|_{L^2(\Omega)} \right) + \|\mathcal{R}_h u(t) - u(t)\|_{L^2(\Omega)}.$$

Using now approximability results for the initial error and the Ritz projector in (5.4) (note that $\mathcal{R}_h$ commutes with ∂_t, so $(\mathcal{R}_h u - u)_t = \mathcal{R}_h u_t - u_t$), and bounding the decaying exponential by one, we obtain the desired bound

$$\|u - u_h\|_{C^0(I;L^2(\Omega))} \leq C \left[h^{r+1} |u(0)|_{H^{r+1}(\Omega)} + h^{r+1} \|u\|_{C^0(I;H^{r+1}(\Omega))} + \frac{C_P^2 h^{r+1}}{\mu} \|u_t\|_{C^0(I;H^{r+1}(\Omega))} \right].$$

(2.) $H^1(\Omega)$ error: We can also obtain H^1 bounds for the error as follows. We start again from (5.5), but now treat the right-hand side as follows:

$$((\mathcal{R}_h u - u)_t, \mathcal{R}_h u - u_h) \leq \frac{1}{2\mu\alpha} \|(\mathcal{R}_h u - u)_t\|^2_{H^{-1}(\Omega)} + \frac{\mu\alpha}{2} \|\mathcal{R}_h u - u_h\|^2_{H^1(\Omega)}$$

for an arbitrary $\alpha > 0$. Combining this result with (5.5), using Poincaré and picking α small enough, we get:

$$d_t \|\mathcal{R}_h u - u_h\|^2_{L^2(\Omega)} + \mu |\mathcal{R}_h u - u_h|^2_{H^1(\Omega)} \leq \frac{C}{\mu} \|(\mathcal{R}_h u - u)_t\|^2_{H^{-1}}.$$

Integrating in time this inequality, we get:

$$\mu \int_0^t \|\nabla e_h\|^2_{L^2(\Omega)} \mathrm{d}s \leq \frac{C}{\mu} \int_0^t \|(\mathcal{R}_h u - u)_t\|^2_{H^{-1}(\Omega)} \mathrm{d}s + \|e_h(0)\|^2_{L^2(\Omega)}.$$

As above, we can use error bounds for the Ritz projector (5.4), to get:

$$\mu^{\frac{1}{2}} \|u - u_h\|_{L^2(I;H^1(\Omega))} \leq C \left[h^{r+1} |u(0)|_{H^{r+1}(\Omega)} + \frac{\sqrt{T}}{\sqrt{\mu}} h^{r+1} \|u_t\|_{C^0(I;H^{r+1}(\Omega))} + \sqrt{T}\, \mu^{\frac{1}{2}} h^r \|u\|_{C^0(I;H^{r+1}(\Omega))} \right].$$

□

5.4 Time discretisation

Now, we must discretise (5.2) in time. For that, we can use an ODE solver, e.g., θ-method, Runge-Kutta, etc. If you have done or are doing MTH2051, you have already seen these methods before. We will focus on the θ-method.

In the θ-method, we use a finite difference method. In general, the θ-method splits the time interval $I \doteq [0, T]$ using a partition $0 \doteq t_0 < t_1 < \ldots < t_{M-1} < t_M \doteq T$. We denote the time step size $\tau_i \doteq t_{i+1} - t_i$. For simplicity, we can assume that the time step size is the same at all steps. At each segment $[t_i, t_{i+1}]$, $i = 0, \ldots, M-1$, we consider a linear variation of the unknown $U(t)$, i.e.,

$$U_t = \frac{U^{i+1} - U^i}{\tau_i}, \quad t \in [t_i, t_{i+1}], \quad U(t^i + \theta\tau_i) = \theta U^{i+1} + (1-\theta) U^i,$$

and solve the ODE at the value $t_i + \theta\tau_i$:

$$M\frac{U^{i+1} - U^i}{\tau_i} + KU^{i+\theta} = F^{i+\theta},$$

where we assume that U^i is known. We note that we can make use of *causality* in time. At each time step, we know U^i from the previous time step (or initial condition) and can compute for the next value U^{i+1}. That is the reason why this kind of methods are usually called *time marching* methods. This fact has very important consequences in the reduction of the computational cost of transient problems.

The most common values for θ are the following. The case $\theta = 1$ is the so-called backward Euler method (BE). In this case, we get the following discrete problem at each time value:

$$(M + \tau_i K)U^{i+1} = \tau_i F^{i+1} + MU^i. \tag{5.6}$$

Using the fact that the matrices M and K are positive-definite, we will readily check the stability of this method (see below).

The case $\theta = 0$ is the so-called forward Euler method (FwE). In this case, we get the following discrete problem at each time value:

$$MU^{i+1} = \tau_i F^i + (M - \tau_i K)U^i. \tag{5.7}$$

This method is explicit since all terms but the time derivative term are treated using the expression from the previous time step. Thus, the problem to be solved at each time step involves a mass matrix, which is very easy to invert (spectrally equivalent to an identity matrix). However, the *big* price to pay is a loss of unconditional stability. The stability of explicit methods relies on a so-called CFL (for Courant-Friedrichs-Lewy) condition. I.e., the time step size must be small *enough* for stability to hold.

The two methods above are at most first order in time. The error only decreases linearly with τ. A quadratic method is obtained by picking $\theta = \frac{1}{2}$, which is known as the Crank-Nicolson (CN) scheme (for the linear problem at hand, and with $F^{i+\frac{1}{2}}$ the forcing at the mid-point, it coincides with the implicit mid-point rule). Doing some algebraic manipulations, we finally get:

$$(M + \frac{1}{2}\tau_i K)U^{i+1} = \tau_i F^{i+\frac{1}{2}} + (M - \frac{1}{2}\tau_i K)U^i. \tag{5.8}$$

Again, this method is unconditionally stable and second order in time.

Let us now prove the stability of these methods. We use the following matrix-norm notation, $\|U\|_X^2 = U^T XU$.

Lemma 5.4.1: Stability of BE-FE method

The solution of the BE discretisation in time of the heat equation in (5.6) satisfies the following bound for $m = 1, \dots, M$:

$$\|U^m\|_M^2 + \sum_{i=1}^{m} \|U^i - U^{i-1}\|_M^2 + \sum_{i=1}^{m} \tau\|U^i\|_K^2 \leq \sum_{i=1}^{m} \tau\|F^i\|_{K^{-1}}^2 + \|U^0\|_M^2.$$

Proof. In order to prove stability of the Backward-Euler method, we use the following trick:

$$(a - b)a = \frac{1}{2}(a^2 - b^2 + (a - b)^2).$$

Now, we pre-multiply the equation by $(U^{i+1})^T$, use the formula above, bound the forcing term as $\tau(U^{i+1})^T F^{i+1} \leq \frac{\tau}{2}\|F^{i+1}\|_{K^{-1}}^2 + \frac{\tau}{2}\|U^{i+1}\|_K^2$, multiply by two and add up to time value m to get the desired result. □

Remark 5.4.2: Numerical diffusion of backward Euler

We note that the first and third terms on the left-hand side of the stability bound in the previous lemma are the L^2 and H^1 stability terms we obtained at the semi-discrete level. However, after BE integration, we have a new stability term from the discrete integration. This term comes from the *numerical diffusion* introduced by this method. The method adds artificial diffusion to the problem at hand, which can negatively affect the energy conservation of the scheme (it is unphysically dissipating energy).

Now, let us look at the CN scheme stability.

Lemma 5.4.3: Stability of CN-FE method

The solution of the CN discretisation in time of the heat equation in (5.8) satisfies the following bound for $m = 1, \dots, M$:

$$\|U^m\|_M^2 + \sum_{i=1}^{m} \tau \|U^{i-\frac{1}{2}}\|_K^2 \leq \sum_{i=1}^{m} \tau \|F^{i-\frac{1}{2}}\|_{K^{-1}}^2 + \|U^0\|_M^2.$$

Proof. For CN, we have to use the expression:

$$(a-b)(a+b) = a^2 - b^2.$$

In this case, we pre-multiply by $(U^{i+\frac{1}{2}})^T$, where $U^{i+\frac{1}{2}} \doteq \frac{1}{2}U^{i+1} + \frac{1}{2}U^i$. Using the expression above, we readily get the desired result. □

Thus, we can observe that this method introduces no numerical dissipation at all but is stable. Furthermore, one can prove this method is second-order accurate in time.

The last method to analyse is FwE. We note that this method is not unconditionally stable. Instead, the stability of this scheme depends on the size of the time step value.

Lemma 5.4.4: Stability of FwE-FE method

The solution of the FwE discretisation in time of the heat equation in (5.7) satisfies the following bound for $m = 1, \dots, M$:

$$\|U^m\|_M^2 + C \sum_{i=1}^{m} \|U^i - U^{i-1}\|_M^2 + \frac{1}{2} \sum_{i=1}^{m} \tau \|U^i\|_K^2 \leq 2 \sum_{i=1}^{m} \tau \|F^{i-1}\|_{K^{-1}}^2 + \|U^0\|_M^2,$$

assuming that the CFL condition $\tau < \frac{h^2}{C_{inv}^2 \mu}$ holds, with $C \doteq 1 - C_{inv}^2 \frac{\tau\mu}{h^2} > 0$. Note that C degenerates as τ approaches the CFL limit; e.g., $\tau \leq \frac{h^2}{2C_{inv}^2 \mu}$ gives $C \geq \frac{1}{2}$.

Proof. For FwE, we can do the same as for BE for the time derivative. The problem is that instead of $U^{i+1}\tau K U^{i+1}$, we now have $U^{i+1}\tau K U^i$. We can do the following:

$$\begin{aligned} U^{i+1}\tau K U^i &= U^{i+1}\tau K U^{i+1} - U^{i+1}\tau K(U^{i+1} - U^i) \\ &= \tau \|U^{i+1}\|_K^2 - U^{i+1}\tau K(U^{i+1} - U^i) \\ &\geq \frac{\tau}{2}\|U^{i+1}\|_K^2 - \frac{\tau}{2}\|U^{i+1} - U^i\|_K^2. \end{aligned}$$

The forcing term is bounded with unequal weights, $\tau(U^{i+1})^T F^i \leq \tau \|F^i\|_{K^{-1}}^2 + \frac{\tau}{4}\|U^{i+1}\|_K^2$, so that half of the stiffness term survives on the left-hand side. We can now try to absorb the last term with $\frac{1}{2}\|U^{i+1} - U^i\|_M^2$.

We know that

$$\|V\|_M^2 = \|v_h\|_{L^2(\Omega)}^2, \qquad \|V\|_K^2 = \mu\|\boldsymbol{\nabla} v_h\|_{L^2(\Omega)}^2.$$

Using the inverse inequality $\|\boldsymbol{\nabla} v_h\|_{L^2(\Omega)} \leq C_{inv} h^{-1}\|v_h\|_{L^2(\Omega)}$, valid for FE functions on quasi-uniform meshes, we get:

$$\tau\|V\|_K^2 = \tau\mu\|\boldsymbol{\nabla} v_h\|_{L^2(\Omega)}^2 \leq C_{inv}^2 \frac{\tau\mu}{h^2}\|v_h\|_{L^2(\Omega)}^2 = C_{inv}^2 \frac{\tau\mu}{h^2}\|V\|_M^2.$$

This inequality bounds a first derivative by the function itself, whereas the one used in Section 4.6 bounds a second derivative (the Laplacian) by a first derivative, cell by cell. Both are instances of the same mechanism: on a space of piecewise polynomials, one derivative can be traded for one power of h^{-1}, i.e., $|v_h|_{H^m(K)} \leq C h_K^{-(m-l)}|v_h|_{H^l(K)}$ for $0 \leq l \leq m$; here $(m,l) = (1,0)$, in Section 4.6 $(m,l) = (2,1)$. The global form used here requires a quasi-uniform mesh, so that the cell estimates can be added with a single h. Neither inequality holds for general $H^1(\Omega)$ functions, which can oscillate below the mesh scale. Assuming that $\tau < \frac{h^2}{C_{inv}^2\mu}$ holds, i.e., the CFL condition, we can readily get the desired result, after multiplying by two and adding up. □

The CFL condition $\tau < \frac{h^2}{C_{inv}^2\mu}$ implies that the time step size must decrease with h^2 as $h \to 0$, which is not acceptable in general. Second-order space derivatives are rarely treated explicitly due to this quadratic relation with the time step size. Only convective terms (first order) are suitable for explicit treatment since the corresponding CFL is milder, $\tau \sim h$. As a result, a semi-implicit method (that considers zero and first-order terms explicitly and second-order terms implicitly) is an excellent choice in these situations.

5.5 Total error

We know from the numerical analysis of ODEs that the BE method is a first order scheme in time, while the CN scheme is second-order. Combining these results with the semi-discrete error estimates for the heat equation in Theorem 5.3.3, we get the space-time errors below. We do not prove them for the sake of brevity.

Theorem 5.5.1: Error estimates for the fully discrete heat equation

Let us assume that the solution u of (5.1) is $u \in C^1(I; H^{r+1}(\Omega))$, with $u_{tt} \in C^0(I; L^2(\Omega))$ for $\theta = 1$ and $u_{ttt} \in C^0(I; L^2(\Omega))$ for $\theta = \frac{1}{2}$, and the solution u_h of the fully-discrete counterpart (5.6) or (5.8) for a FE space of order r. We denote by $u_h^i \in V_h$ the FE function with DOF vector U^i. They satisfy the following error bound:

$$\|u(t_i) - u_h^i\|_{L^2(\Omega)} \leq C(h^{r+1} + \tau^\beta), \qquad i = 1, \ldots, M,$$

$$\left(\sum_{n=1}^{M} \tau\,\mu\|u(t_n) - u_h^n\|_{H^1(\Omega)}^2\right)^{\frac{1}{2}} \leq C(h^r + \tau^\beta),$$

where $\beta = 1$ for $\theta = 1$ and $\beta = 2$ for $\theta = 0.5$.

5.6 Tutorial

1. Can you write the CN method in terms of $U^{n+\frac{1}{2}}$ and U^n. Compare the cost of BE and CN schemes.

2. Write the discrete problem for the transient convection-diffusion equation using θ-method. Can you provide a CFL condition for the FwE method applied to this problem? Next, we consider a semi-implicit method in which the diffusive term is evaluated implicitly (at $n+1$, like in BE) and the convective term explicitly (at t^n, like in FwE). Can you provide the CFL condition for this scheme?

3. Some of the schemes above can be understood using Petrov-Galerkin schemes and FE spaces, using a weak space-time form. Prove that the Crank-Nicolson method for the heat equation can be recovered using a tensor product space $V_h \times \mathcal{P}_1(I)$ for the trial space and $V_h \times \mathcal{P}_0(I)$ for the test space.

6 Solving the linear system

6.1 Introduction

Any physical problem which involves PDEs (solid mechanics, fluid mechanics, aeroelasticity, etc.) can be discretized (by finite elements, finite volumes or finite differences). After linearisation (for nonlinear problems) e.g., using Newton's method, we end up with a linear system to be solved, as we have seen in the previous chapters for finite element discretisations. Assuming that we have n DOFs in the FE problem, we have to solve a system

$$Au = f, \qquad A \in \mathbb{R}^{n\times n}, \qquad f \in \mathbb{R}^n,$$

and obtain the vector $u \in \mathbb{R}^n$ of DOFs. The finite element solution is the linear combination of the FE shape function basis for these values of u, i.e.,

$$u_h(x) = \sum_{a=1}^{n} u^a \phi^a(x).$$

By construction, A is a square matrix and, assuming that the problem is well-posed, non-singular, as we have already proved. Thus, this problem admits a unique solution. Finally, we note that while u_h is independent of the basis chosen to describe our FE space, the specific expression of the linear system, i.e., A, f and u, do depend on the basis.

This linear system size increases with mesh refinement. Larger linear systems translate to higher computational power requirements.

The main objective of this chapter is to provide a short introduction about how to solve linear systems arising from FE discretizations.

6.2 Preliminaries

6.2.1 Eigenvalues and eigenvectors

Given a square matrix $A \in \mathbb{R}^{n\times n}$, its eigenvalues $\lambda \in \mathbb{C}$ and eigenvectors $u \in \mathbb{C}^n \backslash \{0\}$ are solutions of

$$Au = \lambda u.$$

We make use of the following definitions:

1. The set of eigenvalues of A is called spectrum $\sigma(A)$ of A. It is a set in the complex plane.
2. The *spectral radius* is $\rho(A) \doteq max_{\lambda\in\sigma(A)}|\lambda|$, i.e. the maximum of the modulus of the eigenvalues.
3. The *kernel* is $Ker(A) \doteq \{u \in \mathbb{R}^n : Au = 0\}$. Thus, a singular matrix has a non-trivial kernel, i.e., it contains non-zero vectors.
4. The *matrix norm* of $B \in \mathbb{R}^{n\times n}$ is

$$\|B\|_* \doteq \max_{u\in\mathbb{R}^n\backslash\{0\}} \frac{\|Bu\|_*}{\|u\|_*}$$

where $\|\cdot\|_*$ is a norm, i.e. $\|u\|_2 = \sqrt{u^T u}$ (Euclidean norm) or $\|u\|_A \doteq \sqrt{u^T Au}$ (norm of a vector induced by an s.p.d. matrix).

5. The *condition number* is defined by

$$\kappa_*(A) \doteq \|A\|_*\|A^{-1}\|_*.$$

If $\kappa_*(A) \to \infty$ the matrix is becoming closer to singular; $\kappa_*(A) = \infty$ for a singular matrix. The condition number ($\kappa_*(A)$) is a measure of how difficult it is to solve a linear system with the matrix A, in terms of the propagation of rounding errors and, as we will see, the convergence of iterative solvers.

6. A is *positive definite* if $u^T Au > 0$ for every $u \in \mathbb{R}^n \setminus \{0\}$ (note that $u^T Au$ is a scalar).

7. If in addition A is *symmetric*, its eigenvalues are real and strictly positive, i.e., the eigenvalues lie on the positive part of the real line.

8. If the matrix A is positive definite and symmetric (s.p.d.), there exists an orthonormal basis of eigenvectors, i.e. linearly independent $\{v_1, ..., v_n\}$ with $v_i^T v_j = \delta_{ij}$ and $Av_i = \lambda_i v_i$. A is diagonalizable.

9. In a s.p.d. matrix A, the condition number (with respect to the Euclidean norm) is $\kappa(A) \doteq \kappa_2(A) \doteq \frac{\lambda_{max}(A)}{\lambda_{min}(A)}$. Which can be proved as follows,

$$\|A\|_2 = \max_{u\in\mathbb{R}^n\setminus\{0\}} \frac{\|Au\|_2}{\|u\|_2} = \max_{u\in\mathbb{R}^n\setminus\{0\}} \frac{\|\sum_i \alpha_i\lambda_i v_i\|_2}{\|\sum_i \alpha_i v_i\|_2} = \lambda_{max},$$

where $u = \sum_i \alpha_i v_i$, since $\{v_1, \ldots, v_n\}$ is a basis and $Av_i = \lambda_i v_i$; the maximum is attained at the eigenvector of λ_{max}. Moreover, we readily get $\frac{1}{\lambda_i} v_i = A^{-1} v_i$, i.e., the eigenvectors of A^{-1} are the same as the ones of A and its eigenvalues the inverse of the ones of A. As a result, we get

$$\|A^{-1}\|_2 = \sup_{u\in\mathbb{R}^n\setminus\{0\}} \frac{\|A^{-1}u\|_2}{\|u\|_2} = \sup_{u\in\mathbb{R}^n\setminus\{0\}} \frac{\|\sum_i \alpha_i \frac{1}{\lambda_i} v_i\|_2}{\|\sum_i \alpha_i v_i\|_2} = \frac{1}{\lambda_{min}}.$$

Hence,

$$\kappa_2(A) \doteq \|A\|_2\|A^{-1}\|_2 = \lambda_{max} \cdot \frac{1}{\lambda_{min}}.$$

10. If the condition number κ is close to 1, the matrix is *well-conditioned*. As $\kappa \to \infty$, the matrix is *ill-conditioned*. Thus, if $\lambda_{min} \to 0$ or $\lambda_{max} \to \infty$ the matrix becomes harder to solve (since $\kappa \geq 1$ as $\lambda_{max} \geq \lambda_{min}$).

11. We can also use the Courant-Fischer (also called min-max) theorem to characterise the extreme eigenvalues of a symmetric matrix. We can define the Rayleigh quotient R_A of a symmetric matrix:

$$R_A(u) = \frac{(Au, u)}{(u, u)}.$$

Then, the extreme eigenvalues of the matrix are

$$\lambda_{max} \doteq \max\{R_A(x) : x \neq 0\}, \qquad \lambda_{min} \doteq \min\{R_A(x) : x \neq 0\}.$$

Lemma 6.2.1: Generalised eigenvalue problems

Let A and B be two s.p.d. matrices. The n eigenvalues of the following problems coincide:

$$Au = \lambda Bu,$$
$$B^{-1}Au = \lambda u,$$
$$B^{-\frac{1}{2}}AB^{-\frac{1}{2}}u = \lambda u.$$

Thus, $\kappa(B^{-1}A) = \kappa(B^{-\frac{1}{2}}AB^{-\frac{1}{2}})$. Furthermore, the extreme generalised eigenvalues are characterised

by the Rayleigh quotients

$$\lambda_{min} = \min_{u \in \mathbb{R}^n \setminus \{0\}} \frac{u^T A u}{u^T B u},$$
$$\lambda_{max} = \max_{u \in \mathbb{R}^n \setminus \{0\}} \frac{u^T A u}{u^T B u}.$$

The matrix $B^{-\frac{1}{2}}$ exists since B^{-1} is s.p.d. and can be diagonalised. (We note that we can take the square root of a matrix by a change-of-basis transformation onto the one that diagonalises it, which exists due to the fact that B is s.p.d. Then, we take the square root of the eigenvalues, which are all positive. Then, we transform the resulting matrix back to the original basis.)

6.3 Finite Element Matrices

In this section, we want to analyse the condition number of finite element matrices. We will consider the matrices that arise from a reaction term (mass matrix) and a Laplacian term:

$$\int_\Omega u_h v_h \mathrm{d}\Omega, \qquad \int_\Omega \nabla u_h \cdot \nabla v_h \mathrm{d}\Omega, \qquad u_h,\ v_h \in V_h.$$

We consider the basis of shape functions, such that $V_h \doteq \mathrm{span}\{\phi^1, \dots, \phi^n\}$. The corresponding matrices thus read:

$$M_{ab} \doteq \int_\Omega \phi^a \phi^b \mathrm{d}\Omega, \qquad L_{ab} \doteq \int_\Omega \nabla \phi^a \cdot \nabla \phi^b \mathrm{d}\Omega, \qquad a,\ b \in \{1, \dots, n\}.$$

We note that any $u_h \in V_h$ can be written as $\sum_{a \in \{1,\dots,n\}} \phi^a u^a$. Thus, we have:

$$u^T M v = \langle Mv, u \rangle = \int_\Omega u_h v_h \mathrm{d}\Omega,$$

which is the inner product in $L^2(\Omega)$. Analogously,

$$u^T L v = \langle Lv, u \rangle = \int_\Omega \nabla u_h \cdot \nabla v_h \mathrm{d}\Omega,$$

which is an inner product in $H^1_0(\Omega)$. As a result,

$$u^T (M + L) v = \int_\Omega u_h v_h \mathrm{d}\Omega + \int_\Omega \nabla u_h \cdot \nabla v_h \mathrm{d}\Omega,$$

is an inner product in $H^1(\Omega)$. If we replace v by u above, we recover the corresponding (squared) norms. E.g.,

$$u^T M u = \|u_h\|^2_{L^2(\Omega)}, \quad u^T L u = \|\nabla u_h\|^2_{L^2(\Omega)} = |u_h|^2_{H^1(\Omega)},$$
$$u^T (M + L) u = \|u_h\|^2_{H^1(\Omega)}.$$

6.3.1 Mass matrix

In this section, we will check that the mass matrix is conditioned as follows:

$$c h^d \leq \frac{\langle Mv, v \rangle}{\langle v, v \rangle} \leq C h^d, \quad \forall v \in \mathbb{R}^n \setminus \{0\}.$$

Using the expression of Rayleigh quotients above and the relation with the eigenvalues, this is equivalent to saying that λ_{min} and λ_{max} are equal to a constant times h^d.

In order to prove these bounds, we can decompose the integral into cell components $K \in \mathcal{T}_h$ ($\mathcal{T}_h$ is the mesh) and write the integral in the reference element:

$$\int_\Omega u_h^2 = \sum_{a,b\in(1,\dots,n)} u^a u^b \sum_{K\in\mathcal{T}_h} \int_{\hat{K}} \hat{\phi}^{gl(a,K)} \cdot \hat{\phi}^{gl(b,K)} |J_K| \mathrm{d}\hat{K},$$

where gl denotes the re-index from global to local numbering. We have that $|J_K| = Ch^d$ by a scaling argument, since the geometrical map transforms $\hat{K}$ (of diameter 1) into K (of diameter h). So, we only need to look at the eigenvalues of the reference FE mass matrix

$$\hat{M} = \int_{\hat{K}} \hat{\phi}^a \hat{\phi}^b \mathrm{d}\hat{K}, \quad a,\ b \in \{1,\dots,\hat{n}\},$$

where $\hat{n}$ are the shape functions in the reference element (e.g., $\hat{n}$ is 2 for a linear element in 1D).

Denoting its minimum eigenvalue by $\lambda_{min}(\hat{M})$ and using again the fact that the number of neighbours is bounded, we readily obtain:

$$\int_\Omega u_h^2 \geq Ch^d \lambda_{min}(\hat{M}) \langle u, u\rangle .$$

We can use the same idea to bound the maximum eigenvalue, i.e.,

$$\int_\Omega u_h^2 \leq Ch^d \lambda_{max}(\hat{M}) \langle u, u\rangle .$$

We can readily check that the maximum and minimum eigenvalues of $\hat{M}$ are constants. They cannot depend on h, since they are defined at the reference element. The eigenvalues can be explicitly computed but the specific value is not important here. It is easy to check that the reference mass matrix is also non-singular, since it is a s.p.d. matrix. Thus, we have proved the desired bound on the Rayleigh quotient and proved that the eigenvalues are $O(h^d)$.

As a result, the condition number is $\kappa(M) = O(1)$. It does not depend on h. Thus, mesh refinement does not negatively affect the condition number.

6.3.2 Laplacian matrix

We proceed similarly for the Laplacian matrix. We want to check the following bound:

$$ch^d \leq \frac{\langle Lv, v\rangle}{\langle v, v\rangle} \leq Ch^{d-2}, \quad \forall v \in \mathbb{R}^n \setminus \{0\}.$$

It is equivalent to saying that $\lambda_{\min} \geq ch^d$ and $\lambda_{\max} \leq Ch^{d-2}$. The proof of the upper bound is a direct consequence of the inverse inequality and the previous estimates for the mass matrix:

$$\langle Lu, u\rangle = \|\boldsymbol{\nabla} u_h\|^2 \leq Ch^{-2}\|u_h\|^2 \leq Ch^{d-2} \langle u, u\rangle .$$

The lower bound relies on the Poincaré inequality and the mass matrix bounds:

$$\langle Lu, u\rangle = \|\boldsymbol{\nabla} u_h\|^2 \geq C\|u_h\|^2 \geq Ch^d \langle u, u\rangle .$$

So, the condition number $\kappa(L) = O(h^{-2})$. As a result, as we refine the mesh and reduce the FE characteristic size, the condition number increases quadratically with $1/h$. Thus, as we refine the mesh the problem becomes more ill-conditioned and it will be harder to solve.

In the following sections, we briefly introduce two different types of algorithms to solve linear systems, i.e., direct and iterative methods.

6.4 Direct methods

Direct methods are very robust but computationally intensive. These methods are applicable to any matrix, not only FE matrices. Most of them are based on *Gaussian elimination*. The Gaussian elimination process works as follows: Given $A := A^{(0)}$, compute

$$\begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \dots & a_{1n}^{(0)} \\ a_{21}^{(0)} & a_{22}^{(0)} & \dots & a_{2n}^{(0)} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1}^{(0)} & a_{n2}^{(0)} & \dots & a_{nn}^{(0)} \end{bmatrix} \rightarrow \begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \dots & a_{1n}^{(0)} \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 0 & a_{n2}^{(1)} & \dots & a_{nn}^{(1)} \end{bmatrix} \dots$$
$$\rightarrow \begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \dots & a_{1n}^{(0)} \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & a_{nn}^{(n-1)} \end{bmatrix}.$$

The first step (related to the first column) can be represented as

$$A(2,:) - A(1,:) \times \frac{a_{21}}{a_{11}}, \qquad \dots, \qquad A(n,:) - A(1,:) \times \frac{a_{n1}}{a_{11}},$$

which can be written as $A_{(1)} = L_{(1)}^{-1} A$, where

$$L_{(1)}^{-1} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ -\frac{a_{21}^{(0)}}{a_{11}^{(0)}} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ -\frac{a_{n1}^{(0)}}{a_{11}^{(0)}} & 0 & \dots & 1 \end{bmatrix},$$

with inverse

$$L_{(1)} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ \frac{a_{21}^{(0)}}{a_{11}^{(0)}} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ \frac{a_{n1}^{(0)}}{a_{11}^{(0)}} & 0 & \dots & 1 \end{bmatrix}.$$

Then, we proceed for all columns (in a compact form), $A_{(n-1)} \doteq U = L_{(n-1)}^{-1} \dots L_{(1)}^{-1} A$. Finally, we get the LU factorization of $A = LU$, with

$$L = L_{(1)} \dots L_{(n-1)} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ \frac{a_{21}^{(0)}}{a_{11}^{(0)}} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ \frac{a_{n1}^{(0)}}{a_{11}^{(0)}} & \frac{a_{n2}^{(1)}}{a_{22}^{(1)}} & \dots & 1 \end{bmatrix}, \quad U = \begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \dots & a_{1n}^{(0)} \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & a_{nn}^{(n-1)} \end{bmatrix}$$

where L is a lower triangular matrix composed of the product of $L_{(i)}$ matrices as $L = \prod_{i=1}^{n-1} L_{(i)}$ and U is the resulting upper triangular matrix. Then, we can solve $Ax = f$ via forward ($Ly = f$) and backward ($Ux = y$) substitution, which are very fast algorithms. However, the computational cost is the computation of L and U matrices. The system is solved as follows,

$$Ax = f \quad \rightarrow \quad L(Ux) = f \quad \rightarrow \quad Ly = f, \quad Ux = y.$$

Some properties of direct methods are stated below.

First, we note that the complexity is $O(n^3)$ (number of floating-point operations) and $O(n^2)$ (required memory) for dense matrices. The complexity estimates can be obtained by counting the number of operations (numbers to be stored) in the Gaussian elimination process.

However, FE matrices are highly sparse ($nnz << n$, where nnz is the number of non-zeros per row). The number of non-zeros depends on the spatial dimension and the order of discretization, i.e. it depends on the local DOFs. For sparse matrices, using appropriate orderings (e.g., nested dissection), the complexity is reduced to $O(n^2)$ flops and $O(n^{4/3})$ memory for 3D FE matrices (the constants depend on nnz). For a sparse matrix, one can readily check that the memory is $O(n)$ (more specifically, $n \times nnz$) but the *fill-in* of L and U increases the memory up to the bound above.

As indicated above, direct methods have high computational and memory demands. Direct methods are also very hard to parallelise (not scalable on more than $O(100)$ processors on distributed memory machines). As a result, they are not acceptable for large scale simulations of 3D complex physical phenomena or realistic engineering applications. In order to alleviate this problem, we usually rely on iterative solvers with robust preconditioners in these situations, which we briefly describe below.

6.5 Iterative solvers

Iterative solvers approximate the solution of the linear system in an iterative way and up to some tolerance for a given convergence criterion. The most advanced iterative algorithms (e.g., multigrid algorithms) can attain linear complexity in terms of flops and memory consumption (with respect to the problem size), which is a dramatic improvement with respect to direct methods. Furthermore, they are more amenable for parallel computations on distributed memory computers. Other very effective methods are the so-called domain decomposition methods, which are based on a divide-and-conquer approach, and thus, very well-suited for large scale parallel computations on millions of processors. However, the multigrid and domain decomposition methods are quite involved, and we do not cover them in this unit. Instead, we consider some basic iterative solvers that are building blocks for multigrid methods.

6.5.1 Richardson method

Let us consider a matrix B such that,

$$B = A + E,$$

where E is the perturbation matrix, i.e. the error of approximating A by using B. We call B a *preconditioner* for A. The ideal preconditioner B should be such that the action of B^{-1} is cheap to compute and/or easy to parallelise and B is close to A ($B^{-1}A$ is close to the identity matrix, i.e., its condition number is close to one).

We aim at solving $Au = f$, which is now equivalent to $Bu = f + Eu$. In the (preconditioned) Richardson method, we consider the following iterative procedure: Given u^0, compute for $k = 0, \ldots$

$$Bu^{k+1} = f + Eu^k.$$

If $B = I$, the solver is called *Richardson* method. Otherwise, we call it the *preconditioned Richardson method*.

First, we can check that, if the iterative process converges, then it converges to u. Let us define the so-called residual $r^k \doteq f - Au^k$. Since

$$Bu^{k+1} = f + Eu^k = f - Au^k + Bu^k = r^k + Bu^k,$$

we can write each iterate as

$$u^{k+1} = u^k + B^{-1}r^k = u^k + B^{-1}(f - Au^k).$$

The method stops when r^k is zero, i.e., u^k is a solution of the linear system. Since we assume that A is non-singular, $u^k = u$. In practice, the residual never vanishes exactly, and the iteration is stopped when it is small enough, typically when the relative residual satisfies $\|r^k\| \leq \text{tol}\,\|f\|$ for a *tolerance* $\text{tol} > 0$ provided by the user. The tolerance is an input of every iterative solver; it should be chosen in accordance with the

accuracy of the discretisation, since there is no point in solving the linear system much more accurately than the discretisation error.

In order to improve the convergence of the method, we usually make us of a *relaxation*, i.e., we compute

$$u^{k+1} = u^k + \alpha B^{-1}(f - Au^k), \qquad \alpha > 0, \tag{6.1}$$

where $\alpha > 0$ is the *relaxation factor*. It only takes part of the proposed correction to improve convergence. α small provides a more stable algorithm but it can also make convergence slower in some cases (think what happens when $\alpha = 0$).

Error of Richardson method

Let us analyse the error $e^k = u - u^k$ at each iterate. We subtract u on both sides of (6.1) to get:

$$u^{k+1} - u = u^k - u + \alpha B^{-1}(f - Au^k).$$

Then, since $f = Au$, we get:

$$e^{k+1} = e^k - \alpha B^{-1}(Ae^k) = (I - \alpha B^{-1}A)e^k.$$

Let us assume that A and B are s.p.d. matrices, and let us denote the *iteration matrix* by $G \doteq I - \alpha B^{-1}A$. From Lemma 6.2.1, the eigenvalues $\lambda_i(B^{-1}A)$ of $B^{-1}A$ are real and positive, and thus the eigenvalues of G are $1 - \alpha\lambda_i(B^{-1}A)$, which are real. We want to bound $\|e^{k+1}\| \leq \|G\|\,\|e^k\|$ in a norm in which $\|G\|$ is as small as possible, namely equal to its spectral radius $\rho(G) = \max_i |1 - \alpha\lambda_i(B^{-1}A)|$. The Euclidean norm is *not* such a norm: G is not symmetric in general ($B^{-1}A$ is symmetric only when A and B commute), so $\|G\|_2$ is not the maximum modulus of its eigenvalues; in general $\|G\|_2 > \rho(G)$, and $\|G\|_2$ can even be larger than one when all $|1 - \alpha\lambda_i(B^{-1}A)| < 1$.

Instead, we measure the error in the norm induced by B,

$$\|v\|_B \doteq \sqrt{v^T Bv} = \|B^{1/2}v\|_2,$$

where $B^{1/2}$ is the s.p.d. square root of B (see the proof of Lemma 6.2.1). The matrix norm induced by $\|\cdot\|_B$ is, with the change of variable $w = B^{1/2}v$,

$$\|G\|_B \doteq \max_{v\neq 0} \frac{\|Gv\|_B}{\|v\|_B} = \max_{v\neq 0} \frac{\|B^{1/2}Gv\|_2}{\|B^{1/2}v\|_2} = \max_{w\neq 0} \frac{\|B^{1/2}GB^{-1/2}w\|_2}{\|w\|_2} = \|B^{1/2}GB^{-1/2}\|_2.$$

Now, $B^{1/2}GB^{-1/2} = I - \alpha B^{-1/2}AB^{-1/2}$ *is* symmetric, so its Euclidean norm is the maximum modulus of its eigenvalues; and its eigenvalues are those of G, since the two matrices are similar (equivalently, by Lemma 6.2.1, the eigenvalues of $B^{-1/2}AB^{-1/2}$ are those of $B^{-1}A$). As a result,

$$\|G\|_B = \max_i |1 - \alpha\lambda_i(B^{-1}A)| = \rho(G),$$

and we have

$$\|e^{k+1}\|_B = \|Ge^k\|_B \leq \|G\|_B\|e^k\|_B = \rho(I - \alpha B^{-1}A)\|e^k\|_B.$$

For the Richardson method without preconditioner, $B = I$, the B-norm is the Euclidean norm and $G = I - \alpha A$ is symmetric, so the argument above is not needed.

Using recursion, we get

$$\|e^{k+1}\|_B \leq \rho(I - \alpha B^{-1}A)^{k+1}\|e^0\|_B.$$

We can readily observe that the method will converge if $\rho(I - \alpha B^{-1}A) < 1$. However, in general, it does not converge. Besides, one can also check that

$$\begin{aligned}\rho(I - \alpha B^{-1}A) &= \max_i |1 - \alpha\lambda_i(B^{-1}A)| \\ &= \max(|1 - \alpha\lambda_{\min}(B^{-1}A)|, |1 - \alpha\lambda_{\max}(B^{-1}A)|).\end{aligned}$$

As a result, it is easy to check that the method will always converge when $0 < \alpha < \frac{2}{\lambda_{max}}$, whereas the optimal choice is $\alpha = \frac{2}{\lambda_{max}+\lambda_{min}}$. However, the computing of λ_{max} and λ_{min} is usually more expensive than solving the system and this optimal value cannot be used in practise.

In summary, this method is simple but does not converge in general. With relaxation, the convergence is slow. The value of α that ensures convergence requires knowing $\sigma(B^{-1}A)$, which is far more expensive than solving the linear system itself. Besides, there is no maximum bound for the number of iterations required for convergence. As we can see, there is room for improvement.

6.5.2 Steepest descent method

This method is based on the fact that the solution of $Ax = f$ is the minimum of

$$\Phi(x) = \frac{1}{2}x^T Ax - f^T x$$

for A s.p.d., since $\partial_x \Phi(x) = Ax - f \doteq -r$. Considering the unpreconditioned Richardson method,

$$u^{k+1} = u^k + \alpha_k r^k,$$

we can compute the value of α that minimises the functional. We can check that (try to prove it!)

$$\alpha_k = \text{argmin}_{\alpha\in\mathbb{R}}\Phi(u^k + \alpha r^k) = \frac{\langle r^k, r^k\rangle}{\langle r^k, Ar^k\rangle}.$$

Then, the *steepest descent method* is defined by: Given u^0, compute for $k = 0, \ldots$

$$u^{k+1} = u^k + \alpha_k(f - Au^k),$$

with $\alpha_k = \frac{\langle r^k, r^k\rangle}{\langle r^k, Ar^k\rangle}$.

This method represents an improvement with respect to Richardson method. No knowledge of the spectrum of A ($\sigma(A)$) is required. One can prove that the error is reduced as

$$\|e^k\|_A \leq \left(\frac{\kappa(A) - 1}{\kappa(A) + 1}\right)^k \|e^0\|_A,$$

which implies convergence. The computational cost per iteration is the same as Richardson (plus some inner products). At each iteration, we perform matrix-vector products with sparse matrices, which have a complexity $O(n)$. However, the number of iterations required to attain a given tolerance grows with $\kappa(A)$, and thus with mesh refinement for FE matrices. As a result, the total cost, which is $O(n)$ times the number of iterations, deteriorates as we refine the mesh.

6.5.3 Conjugate Gradient

In the steepest descent algorithm, the distance can be optimally chosen but the direction is not. In order to get an optimal direction, assume we have a basis of *conjugate directions* $\{p^k\}_{i=0}^{n-1}$, i.e.

$$\langle p^i, Ap^j\rangle = 0, \qquad i \neq j$$

i.e., an A-orthogonal basis. Then, we compute for $k = 0, \ldots, n-1$

$$u^{k+1} = u^k + \alpha_k p^k$$

with $\alpha^k = \frac{\langle r^k, p^k\rangle}{\langle p^k, Ap^k\rangle}$ as above. It is easy to check that this algorithm converges to the solution in a number of iterations at most equal to the dimension of the problem. In fact, CG was originally proposed as a direct solver. However, the algorithm is severely affected by rounding errors and the residual may not be close to zero up to machine precision after n iterations. Years after it was originally proposed, the method was

rediscovered as a powerful iterative method, probably the most used method in practice. The reason is that after far less iterations than the total size n, one can get residual below acceptable tolerances for most practical applications.

The open question is how to build the A-orthogonal basis. The CG method performs this computation in an iterative way. Given the residual r^k, we can compute:

$$p^k = r^k + \sum_{i=0}^{k-1} \beta^i p^i,$$

with $\beta^i = -\frac{\langle Ar^k, p^i \rangle}{\langle Ap^i, p^i \rangle}$, i.e. β^i s.t. $\langle Ap^k, p^i \rangle = 0$.

This approach requires full orthogonalization with respect to the previous directions, i.e. in each step it has to orthogonalise with all the previous directions. All the previous directions have to be stored in memory and the computational cost increases with k.

The good news is that full orthogonalization is not required (in exact arithmetic). The *key Property of CG* is that we only need to orthogonalize with respect to the previous direction. Hence, there is no need to store all the previous directions and the computational cost is dramatically reduced. The algorithm is shown in Algorithm 6.1, with x^0 as input and x as output.

Algorithm 6.1: Conjugate gradient (CG)

$p^0 := r^0 := f - Ax^0$
for $j = 0, \ldots,$ till CONV **do**
  $s^{j+1} = Ap^j$
  $\alpha_j := (r^j, r^j)/(s^{j+1}, p^j)$
  $x^{j+1} := x^j + \alpha_j p^j$
  $r^{j+1} := r^j - \alpha_j s^{j+1}$
  $\beta_j := (r^{j+1}, r^{j+1})/(r^j, r^j)$
  $p^{j+1} := r^{j+1} + \beta_j p^j$
end for

We stress that this algorithm is only applicable on s.p.d. matrices, since it relies on the re-statement of the problem as a quadratic form. The convergence analysis of the method provides the following bound:

$$\|e^k\|_A \leq 2\left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1}\right)^k \|e^0\|_A.$$

We observe that this method always converges (in exact arithmetic), and, compared with the steepest descent bound, the convergence rate depends on $\sqrt{\kappa(A)}$ instead of $\kappa(A)$.

6.5.4 Preconditioned Conjugate Gradient

Looking at the convergence estimates, in order to reduce the number of iterations of CG, we would like $\kappa(A) \sim 1$. Since we cannot change the matrix, what we can do is to precondition the system. Let us consider a s.p.d. B preconditioner for A. In this situation, one can compute $B^{1/2}$ (using diagonalisation and taking the square root of the eigenvalues). Next, we can define the equivalent system:

$$Au = f \quad \rightarrow \quad B^{-1/2}AB^{-1/2}v = B^{-1/2}f, \quad u = B^{-1/2}v.$$

If B is a good approximation of A, $\kappa(B^{-1/2}AB^{-1/2}) \sim 1$ and the CG method will converge very fast. We note, as explained above, that $\kappa(B^{-1/2}AB^{-1/2}) = \kappa(B^{-1}A)$. Hence, we have that

$$\|e^k\|_A \leq 2\left(\frac{\sqrt{\kappa(B^{-1}A)} - 1}{\sqrt{\kappa(B^{-1}A)} + 1}\right)^k \|e^0\|_A.$$

CG can readily be applied for the preconditioned system above, with system matrix $B^{-1/2}AB^{-1/2}$, since the matrix is s.p.d. However, the computation of $B^{-1/2}$ is very costly and complex. Instead, we would rather use the (left) preconditioned system

$$Au = f \rightarrow B^{-1}Au = B^{-1}f.$$

Unfortunately, $B^{-1}A$ is not symmetric and CG cannot be applied. Fortunately, we can re-arrange the algorithm in such a way that we can still use CG without the need to apply $B^{-1/2}$. The preconditioned CG method in Algorithm 6.2 only requires the action of B^{-1} but is equivalent to CG for $B^{-1/2}AB^{-1/2}$.

Algorithm 6.2: Preconditioned conjugate gradient (PCG)

$r^0 := f - Ax^0$, $z^0 = B^{-1}r^0$, $p^0 := z^0$
for $j = 0, \ldots,$ till CONV **do**
  $s^{j+1} = Ap^j$
  $\alpha_j := (z^j, r^j)/(s^{j+1}, p^j)$
  $x^{j+1} := x^j + \alpha_j p^j$
  $r^{j+1} := r^j - \alpha_j s^{j+1}$
  $z^{j+1} := B^{-1}r^{j+1}$
  $\beta_j := (z^{j+1}, r^{j+1})/(z^j, r^j)$
  $p^{j+1} := z^{j+1} + \beta_j p^j$
end for

The computational cost per PCG iteration is a matrix-vector product with A and the action of B^{-1}. Now, the open question is how to define a cheap and robust preconditioner B for A. And more specifically, how to do that for A coming from the FE discretisation of physical problems. As commented above, two kinds of techniques are multigrid and domain decomposition methods. This is an active field of research and there is no time to explore the design of preconditioners in this unit.

7 Neural networks for PDE approximation

In the previous chapters, we have built a complete toolbox for the numerical approximation of partial differential equations. We have derived weak formulations, discretised them with the Galerkin method on finite element spaces, analysed the error of the resulting approximations, and learned how to solve the linear systems they produce. In this chapter, we look at this toolbox from a distance and identify one structural feature shared by all these methods. They are *linear* approximation methods, in the sense that the discrete space is fixed *before* we know anything about the solution. We will discuss the limits that this choice imposes, and introduce a family of *nonlinear* approximation methods based on neural networks (NNs) that has attracted an enormous interest in the last years. We will define NNs, connect them rigorously with the finite element spaces of this course, describe how they are *trained*, and present the most popular scheme in this field, the physics-informed neural network (PINN) method [18]. We will pay special attention to the pathologies of these methods (non-convex optimisation, numerical integration, imposition of boundary conditions), since they are essential to understand when NNs can and cannot compete with the methods of the previous chapters. We will close the chapter with an introduction to inverse problems, the setting in which these techniques are most attractive.

7.1 The limits of linear approximation

Let us take a step back and look at the structure of the methods in this course. In the Galerkin method of Chapter 2, we pick a finite dimensional space V_h (spectral polynomials, finite elements) and seek the approximation $u_h \in V_h$. The space V_h is a *vector space*, and, crucially, it is chosen *a priori*. It does not depend on the solution u to be approximated. We say that these methods perform *linear approximation*. The same observation applies to finite difference and finite volume methods.

The theory of Chapter 2 tells us the price of this choice. By Cea's lemma, the error of the Galerkin solution is (up to constants) the best approximation error of u in V_h, and the interpolation estimates bound the latter by $Ch^p|u|_{H^{p+1}(\Omega)}$ in the $H^1(\Omega)$ norm for finite elements of order p. Let us translate this bound into computational cost. To reach a target accuracy ε on a quasi-uniform mesh in d dimensions, we need $h \sim \varepsilon^{1/p}$, and thus a number of DOFs

$$N \sim h^{-d} \sim \varepsilon^{-d/p}. \tag{7.1}$$

The cost grows exponentially with the dimension d. This growth is an instance of the so-called *curse of dimensionality*. It is harmless for $d = 1, 2, 3$ and moderate accuracies, which is the regime in which finite elements excel, but it becomes prohibitive for high-dimensional problems (e.g., parametric problems, or equations posed in phase space).

Linear approximation is also inefficient for *multiscale* solutions in low dimensions, since the space cannot exploit the local features of the solution. Let us consider a function on $(0, 1)$ that is essentially constant except for a sharp transition layer of width $\delta \ll 1$, as in Figure 7.1; we have already met such solutions in the convection-dominated problems of Chapter 4. A uniform mesh must satisfy $h \lesssim \delta$ *everywhere* to capture the layer, which requires $N \sim 1/\delta$ cells. However, most of these cells are wasted in regions where the solution is trivial. A mesh with small cells only inside the layer captures the same solution with a number of cells that is essentially independent of δ. The catch is that we need to know *where* the layer sits, i.e., the mesh must depend on the solution. This is no longer linear approximation.

7.1.1 Adaptive finite elements

The classical way to escape linear approximation within the finite element world is *adaptive mesh refinement*. The key tool is an *a posteriori error estimator*, i.e., a quantity $\eta_K(u_h)$ that can be computed from the discrete

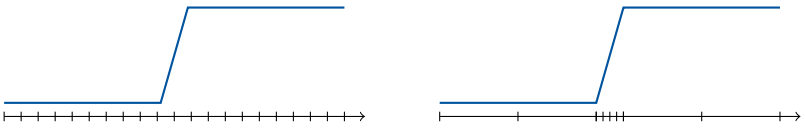


Figure 7.1: A solution with a sharp inner layer of width δ. A uniform mesh resolving the layer wastes most of its cells in the smooth regions, whereas a mesh adapted to the solution captures it with far fewer cells. The adapted mesh depends on the solution, so this is a form of nonlinear approximation.

solution u_h (and the data) on every cell K of the mesh and that estimates the error committed around that cell. Let us make this concrete for the Poisson problem $-\Delta u = f$ in Ω with homogeneous Dirichlet conditions, approximated with linear finite elements. The standard residual-based estimator is

$$\eta_K(u_h)^2 \doteq h_K^2 \, \|f + \Delta u_h\|_{L^2(K)}^2 + \frac{1}{2} \sum_{F \subset \partial K \setminus \partial\Omega} h_F \, \|[\nabla u_h \cdot \boldsymbol{n}]\|_{L^2(F)}^2,$$

where h_F is the size of the facet F and $[q]$ denotes the jump of a quantity q across a facet. The first term measures the strong residual of the equation inside the cell; for linear elements, $\Delta u_h = 0$ on each cell, and it reduces to $h_K \|f\|_{L^2(K)}$. The second term measures the jumps of the normal flux across interior facets. The exact solution has a continuous flux, so these kinks are a computable symptom of error. Two properties make this quantity a genuine error estimator. It is *reliable*, i.e., it bounds the true error from above,

$$|u - u_h|_{H^1(\Omega)} \le C_{\mathrm{rel}} \left(\sum_{K \in \mathcal{T}_h} \eta_K(u_h)^2 \right)^{1/2},$$

and it is (locally) *efficient*, i.e., it bounds the error from below,

$$\eta_K(u_h) \le C_{\mathrm{eff}} \, |u - u_h|_{H^1(\omega_K)} + \text{ data oscillation},$$

where ω_K is the patch of cells sharing a facet with K. The data oscillation term can be made explicit; for linear elements it reads

$$\mathrm{osc}(f, \omega_K) \doteq \left(\sum_{K' \in \omega_K} h_{K'}^2 \, \|f - \bar{f}_{K'}\|_{L^2(K')}^2 \right)^{1/2},$$

where $\bar{f}_{K'}$ denotes the mean value of f on the cell K'. It measures the features of f that live below the mesh scale, which no piecewise polynomial can see; it vanishes for cell-wise constant data, and for smooth f it is of higher order than the estimator itself, $\mathrm{osc}(f, \omega_K) = O(h_K^2)$ against $\eta_K = O(h_K)$. Reliability makes the estimated error trustworthy for stopping the algorithm; efficiency guarantees that we do not refine where the true error is small. The constants depend on the shape regularity of the mesh but not on the mesh size; we refer to [20, 16] for proofs. We stress that the estimator is the key ingredient of the whole approach, and that it is problem-dependent. It is derived from the PDE at hand, and a new estimator must be designed, and proved reliable and efficient, for each new problem class. The derivation and analysis of such estimators is out of the scope of this course. Given an estimator, the adaptive algorithm is a loop over meshes; at each step, we solve the finite element problem, estimate the error cell by cell, and refine the cells that carry the largest error.

Algorithm 7.1: Adaptive finite element method

Given an initial mesh $\mathcal{T}_0$, set $\alpha := 0$
for $\alpha = 0, 1, \ldots,$ till the estimated error is below the tolerance **do**
  SOLVE the finite element problem on $\mathcal{T}_\alpha$ for $u_\alpha \in V_\alpha$
  ESTIMATE the error, computing $\eta_K(u_\alpha)$ for all $K \in \mathcal{T}_\alpha$
  MARK the cells with the largest η_K

REFINE the marked cells to obtain $\mathcal{T}_{\alpha+1}$
end for

We note that the sequence of spaces $V_0, V_1, \ldots$ *does* depend on the solution, through the computed approximations. Each V_α is, of course, still a vector space; the nonlinearity of the method enters through the solution-dependent selection of the next space, not through the spaces themselves. Adaptivity is a nonlinear approximation method built out of linear pieces. The price is a *two-loop* structure. The outer loop generates meshes, and each outer iteration requires the full solution of a finite element problem in the inner loop.

The reward of adaptivity can be quantified. Let us consider a two-dimensional domain with a re-entrant corner, near which the solution behaves like $u \sim r^\lambda$ with $\lambda < 1$ (r is the distance to the corner), while being smooth elsewhere. The regularity $u \in H^{p+1}(\Omega)$ behind the estimate (7.1) is lost, and the convergence rates in terms of the number of DOFs become

$$|u - u_N|_{H^1(\Omega)} \leq C \begin{cases} N^{-\lambda/2} & \text{uniform refinement, any order } p, \\ N^{-p/2} & \text{adaptive refinement, order } p. \end{cases}$$

On uniform meshes, the rate is capped by the regularity of the solution rather than by the order of the method; since $\lambda < 1 \leq p$, increasing the order buys nothing. Adaptivity recovers the rate of the smooth case; meshes graded towards the corner compensate the singularity. This is not a property of hand-tailored meshes only. The adaptive algorithm above, driven by the computable estimators η_K and never seeing u, provably attains the rate of the best possible mesh sequence [16].

7.1.2 Kolmogorov widths

The limits of linear approximation can be made precise. Let us build a concrete example first. Let η be a smooth bump of unit height, supported on an interval of width $\delta \ll 1$. The function $u(x,t) \doteq \eta(x-t)$, i.e., the bump travelling with unit speed, is the solution of the Cauchy problem for the transport equation met in Chapter 4,

$$\partial_t u + \partial_x u = 0 \ \text{ in } \mathbb{R} \times (0,1], \qquad u(x,0) = \eta(x) \ \text{ for } x \in \mathbb{R},$$

posed on the whole line so that no inflow boundary condition is needed. Suppose now that we want a single discrete space V_n able to approximate the solution at *every* time $t \in [0,1]$, i.e., the whole family of snapshots

$$\mathcal{K} \doteq \{\eta(\cdot - t) \ : \ t \in [0,1]\},$$

where $\eta(\cdot - t)$ denotes the function of x alone obtained by freezing the time, $x \mapsto \eta(x-t)$; the dot marks the slot of the free variable. Intuition warns us at once. The basis functions that resolve the bump around one position are useless once the bump has moved away, so resolving *all* the snapshots with one space requires $n \sim 1/\delta$ basis functions, even though each individual snapshot can be captured, on a mesh adapted to its position, with a number of DOFs independent of δ. The *Kolmogorov n-width* makes this intuition rigorous. For a set of functions $\mathcal{K}$ in a normed space V, it is defined as

$$d_n(\mathcal{K}) \doteq \inf_{\substack{V_n \subset V \\ \dim V_n = n}} \sup_{u \in \mathcal{K}} \inf_{v_n \in V_n} \|u - v_n\|,$$

i.e., the best worst-case error achievable by *any* n-dimensional linear space. No linear method, however cleverly its space is chosen, can beat $d_n(\mathcal{K})$ uniformly on $\mathcal{K}$. For the travelling-bump family above, and for transport-dominated solution sets in general, where a sharp feature can sit anywhere in the domain, the width d_n decays very slowly with n. This is often called the *Kolmogorov barrier*. Nonlinear approximation methods are not constrained by this barrier, since they do not draw their approximations from a single linear space; we refer to [7] for a complete treatment of nonlinear approximation.

7.2 Neural networks

NNs entered scientific computing as a flexible family of nonlinear parametrisations of functions. In this section, we define them, and we connect them with the finite element spaces of this course.

Definition 7.2.1: Fully-connected feed-forward neural network

Let $L \in \mathbb{N}$ and let $(n_0, n_1, \dots, n_L) \in \mathbb{N}^{L+1}$ be an *architecture*, with $n_0 = d$ and $n_L = 1$. For each layer $1 \le k \le L$, let $\Theta_k : \mathbb{R}^{n_{k-1}} \to \mathbb{R}^{n_k}$ be the affine map $\Theta_k(\boldsymbol{x}) \doteq W_k \boldsymbol{x} + \boldsymbol{b}_k$, with weight matrix $W_k \in \mathbb{R}^{n_k \times n_{k-1}}$ and bias vector $\boldsymbol{b}_k \in \mathbb{R}^{n_k}$. Given an *activation function* $\rho : \mathbb{R} \to \mathbb{R}$, applied component-wise to vectors, the NN realisation with parameters $\theta \doteq \{W_k, \boldsymbol{b}_k\}_{k=1}^{L}$ is the function

$$u_\theta \doteq \Theta_L \circ \rho \circ \Theta_{L-1} \circ \cdots \circ \rho \circ \Theta_1.$$

The layers $1, \dots, L-1$ are called *hidden layers* and n_k is the number of *neurons* of layer k. The number of trainable parameters is $P \doteq \sum_{k=1}^{L} n_k(n_{k-1}+1)$.

We note that the activation function is not applied after the last layer, so as not to constrain the range of the model. Typical activation functions are the rectified linear unit and the hyperbolic tangent,

$$\mathrm{ReLU}(x) \doteq \max(0, x), \qquad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}},$$

depicted in Figure 7.2. The regularity of u_θ is dictated by the regularity of ρ. Networks activated by ReLU are continuous and piecewise linear, whereas networks activated by tanh are C^∞ functions.

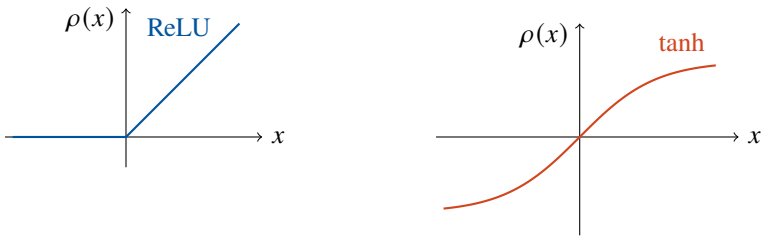


Figure 7.2: The two most common activation functions. ReLU produces continuous piecewise linear networks; tanh produces smooth networks.

The central structural observation of this chapter is the following. The set of all realisations of a fixed architecture,

$$\mathcal{N} \doteq \left\{ u_\theta \ : \ \theta \in \mathbb{R}^P \right\}, \tag{7.2}$$

is *not* a vector space. The parameters of the hidden layers enter the realisation *nonlinearly*, through compositions with the activation function. Only the parameters of the last layer act linearly. The set $\mathcal{N}$ is a nonlinear, finite-dimensional *family* of functions, parameterised by θ. In particular, the sum of two realisations of the same architecture is, in general, not a realisation of that architecture (see the Tutorial). Approximating in $\mathcal{N}$ is nonlinear approximation, and the parameters can *adapt* to the function being approximated, exactly as the adapted mesh of Figure 7.1 adapts to the layer.

7.2.1 Neural networks can emulate finite elements

At first sight, NNs look like a family of approximants completely alien to this course. They are not. Let us show that networks activated by ReLU contain the linear finite element spaces of Chapter 2. We start by building a hat function.

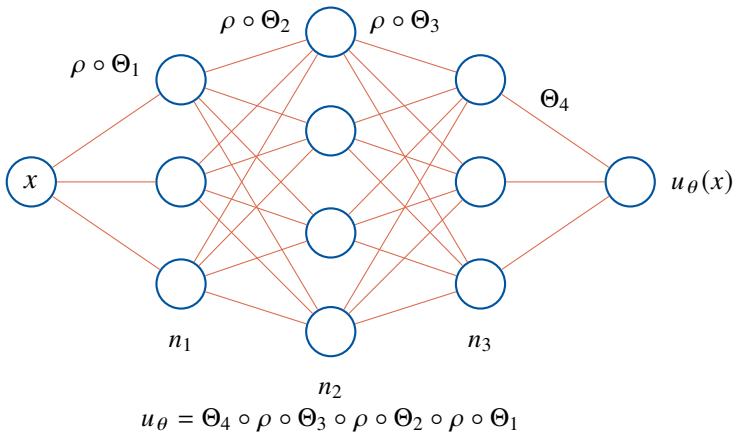


Figure 7.3: A fully-connected feed-forward network with architecture $(1, 3, 4, 3, 1)$. Each edge carries a weight, each neuron adds a bias and applies the activation function (except in the last layer). The affine maps Θ_k of Definition 7.2.1 act between consecutive layers, and the activation ρ is applied inside each hidden neuron.

Example 7.2.2: A hat function out of three ReLUs

Let $x_{j-1} < x_j < x_{j+1}$ be three consecutive nodes of a mesh of $\mathbb{R}$, with cell sizes $h_j \doteq x_j - x_{j-1}$ and $h_{j+1} \doteq x_{j+1} - x_j$. Then, the hat function ϕ_j of Definition 2.3.2 satisfies, for every $x \in \mathbb{R}$,

$$\phi_j(x) = \frac{1}{h_j}\mathrm{ReLU}(x - x_{j-1}) - \left(\frac{1}{h_j} + \frac{1}{h_{j+1}}\right)\mathrm{ReLU}(x - x_j) + \frac{1}{h_{j+1}}\mathrm{ReLU}(x - x_{j+1}). \tag{7.3}$$

The check is elementary. For $x \le x_{j-1}$ all three terms vanish. On $[x_{j-1}, x_j]$ only the first term is active, and it ramps linearly from 0 to 1. On $[x_j, x_{j+1}]$ the second term activates, the total slope becomes $\frac{1}{h_j} - \frac{1}{h_j} - \frac{1}{h_{j+1}} = -\frac{1}{h_{j+1}}$, and the function ramps back to 0. For $x \ge x_{j+1}$ the third term activates and the total slope is zero. The right-hand side of (7.3) is a NN with architecture $(1, 3, 1)$; the hidden weights are all equal to one, the hidden biases are $-x_{j-1}, -x_j, -x_{j+1}$, and the output weights are the three coefficients. Shape functions are, quite literally, small neural networks. The construction is depicted in Figure 7.4.

Summing hat functions, any linear finite element function can be realised by a shallow network. In fact, a more economical representation exists, with one neuron per cell.

Theorem 7.2.3: Finite element emulation in 1D

Let V_h be the linear finite element space on a mesh of $\Omega = [a, b]$ with M cells and nodes $a = x_0 < x_1 < \ldots < x_M = b$. Let $\mathcal{N}$ be the set of realisations of the architecture $(1, n_1, 1)$ with activation ReLU *and $n_1 \ge M$. Then, $V_h \subset \mathcal{N}$ on Ω, i.e., every $v_h \in V_h$ coincides on Ω with the realisation of a one-hidden-layer network with M neurons.*

Proof. Let $v_h \in V_h$ and let s_j denote its (constant) slope on the cell (x_{j-1}, x_j), for $j = 1, \ldots, M$. We claim that, for every $x \in [a, b]$,

$$v_h(x) = v_h(x_0) + s_1\,\mathrm{ReLU}(x - x_0) + \sum_{j=1}^{M-1}(s_{j+1} - s_j)\,\mathrm{ReLU}(x - x_j). \tag{7.4}$$

Both sides are continuous and piecewise linear with breakpoints contained in $\{x_0, \ldots, x_{M-1}\}$, and they take the same value at x_0. It thus suffices to check that the slopes coincide on every cell. On (x_{k-1}, x_k), the active terms of the right-hand side are those with $j \le k - 1$, and its slope is $s_1 + \sum_{j=1}^{k-1}(s_{j+1} - s_j) = s_k$,

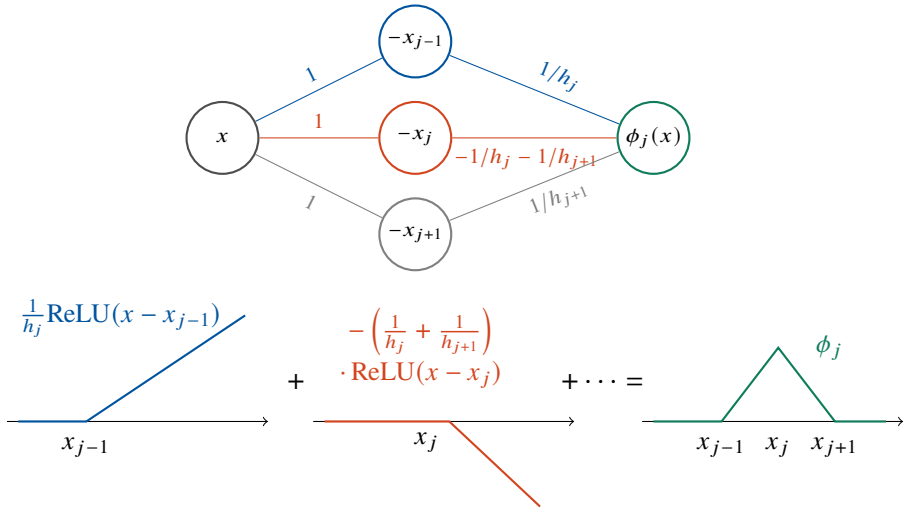


Figure 7.4: The hat function ϕ_j as the $(1, 3, 1)$ network of Example 7.2.2. Top, the network with its explicit parameters; each hidden neuron has unit input weight and carries one of the biases $-x_{j-1}, -x_j, -x_{j+1}$, and the output layer combines the neurons with the slope coefficients. Bottom, the ridges realised by the first two hidden neurons and their sum with the third one (in grey in the network, not shown as a ridge), which cancels the residual slope to the right of x_{j+1}. Moving the biases moves the kinks, i.e., the mesh nodes; changing the output weights changes the slopes; replacing ReLU by a smooth activation rounds off the kinks.

which is the slope of v_h. The right-hand side of (7.4) is the realisation of the architecture $(1, M, 1)$ with hidden weights equal to one, hidden biases $-x_0, \ldots, -x_{M-1}$, output weights $s_1, s_2 - s_1, \ldots, s_M - s_{M-1}$ and output bias $v_h(x_0)$; see Figure 7.5. Unused neurons, if $n_1 > M$, can be switched off by setting their output weights to zero. □

This result has an important consequence for approximation. Since the family $\mathcal{N}$ *contains* V_h, the best approximation error over $\mathcal{N}$ is at most the best approximation error over V_h,

$$\inf_{v \in \mathcal{N}} \|u - v\| \le \inf_{v_h \in V_h} \|u - v_h\| \le C h^p \, |u|_{H^{p+1}(\Omega)} \quad (p = 1), \tag{7.5}$$

by the interpolation estimates of Chapter 2. NNs are *at least as expressive* as the finite element spaces they can emulate. We stress that this is a *linear* lower bound on a nonlinear family, and it is far from sharp. The breakpoints of the network in (7.4) are its hidden biases, which are trainable. A one-hidden-layer ReLU network in 1D is exactly a continuous piecewise linear function with n_1 *free* breakpoints, i.e., a free-knot linear spline. Optimising the parameters moves the mesh; training a NN contains, as a special case, the mesh adaptation of Section 7.1.1, without an a posteriori estimator and without a two-loop algorithm. There is more. Since the breakpoints are free, the same architecture contains the linear finite element space on *every* mesh of Ω with M cells. The bound (7.5) can thus be taken over all such meshes at once, and the best approximation over $\mathcal{N}$ is at least as good as the best *adaptive* finite element approximation with M cells of Section 7.1.1, a much stronger statement than the fixed-mesh bound (7.5). This is the precise sense in which NNs perform nonlinear approximation.

7.2.2 Higher dimensions and the need for depth

The situation in more than one dimension is more subtle, and it reveals the role of depth. A network with a *single* hidden layer is a linear combination of *ridge functions* $\rho(\boldsymbol{w} \cdot \boldsymbol{x} + b)$, i.e., functions of the single scalar variable $\boldsymbol{w} \cdot \boldsymbol{x} + b$, which are constant along the hyperplanes $\boldsymbol{w} \cdot \boldsymbol{x} + b = c$. For the ReLU activation, such a function vanishes on one side of the hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ and is linear on the other, so its only kink, which we call a *crease*, is that hyperplane, which extends across the whole domain. A hat function in 2D, instead, has creases along *bounded* segments (the edges of the mesh around its node) that terminate at vertices. A

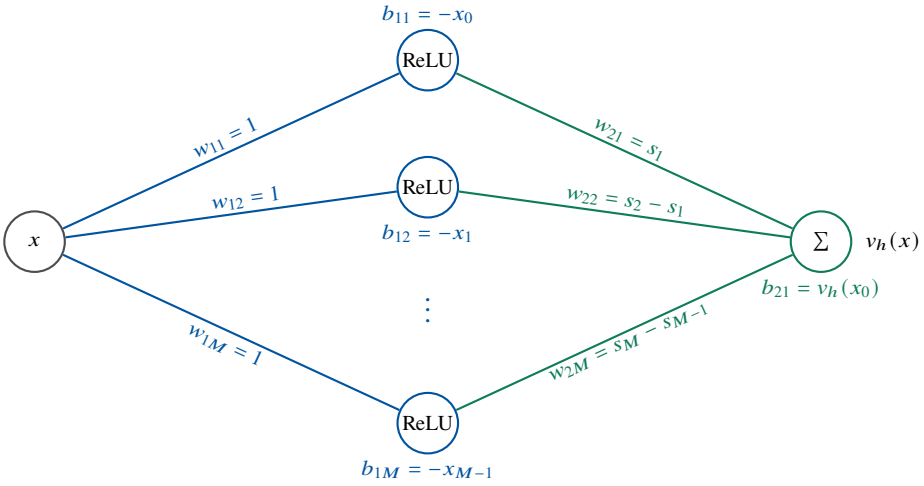


Figure 7.5: The $(1, M, 1)$ network of Theorem 7.2.3 realising a linear finite element function v_h with slopes $s_1, \dots, s_M$ on the M cells; w_{ki} and b_{ki} denote the entries of W_k and $\boldsymbol{b}_k$ in Definition 7.2.1. Weights sit on the edges and each neuron carries one bias: hidden neuron j computes $\mathrm{ReLU}(x - x_{j-1})$, i.e., it switches on at the node x_{j-1}, and the output neuron adds the contributions with the slope jumps $s_j - s_{j-1}$ (with $s_0 \doteq 0$) as weights and the value $v_h(x_0)$ as bias, without activation. The mesh nodes are the hidden biases, hence trainable parameters.

finite sum of full-hyperplane creases cannot produce creases that stop; one hidden layer cannot represent a single 2D hat function, no matter how many neurons it has. With *two* hidden layers, the compositions of the second layer can cut the creases of the first, and bounded features become representable; see Figure 7.6.

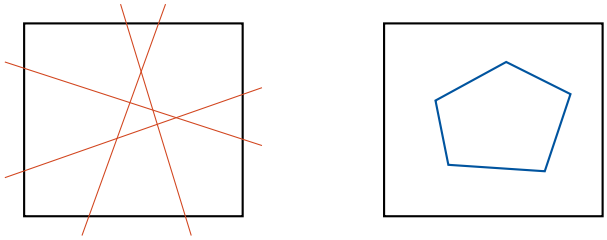


Figure 7.6: Creases of ReLU networks in 2D. With one hidden layer, every crease is a full straight line across the domain, and no bounded feature (e.g., the support of a hat function) can be delimited. With two hidden layers, creases can terminate and bounded regions can be carved.

The connection with finite elements survives in higher dimensions at the price of depth. It can be proved that every continuous piecewise linear function on $\mathbb{R}^d$ is the realisation of a ReLU network with at most $\lceil \log_2(d+1) \rceil$ hidden layers [1], and that linear finite element functions on general simplicial meshes in 2D and 3D can be emulated with two hidden layers, with a number of neurons proportional to the number of DOFs (with a constant that depends on the mesh) [9]. The bound (7.5) thus extends to higher dimensions for deep enough networks.

For completeness, we also state the classical density result of the field. Its proof is out of the scope of this course.

Theorem 7.2.4: Universal approximation [6, 17]

Let $\rho \in C^0(\mathbb{R})$ and let $K \subset \mathbb{R}^d$ be compact. The set of one-hidden-layer network realisations with activation ρ (and arbitrary width) is dense in $C^0(K)$ with the maximum norm if and only if ρ is not a polynomial.

This theorem is reassuring but much weaker than it looks. It is a density statement, like the density of polynomials in C^0. It says nothing about *rates*, i.e., about how many neurons are needed for a given accuracy, and nothing about how to find the parameters. The finite element theory of Chapter 2 provides both an approximation rate and an algorithm (solve a linear system) that attains it. For NNs, the emulation results above provide non-optimal rates, and the algorithmic question leads us to *training*, where the real difficulties of the field live.

7.3 Training

Let us first consider the task for which NNs were primarily developed and are most successful, i.e., *data fitting*, a form of nonlinear regression. We want to approximate an unknown function from samples $(\boldsymbol{x}_i, y_i)$, $i = 1, \dots, N_s$. We measure the misfit of a network with a *loss* (or cost) function, e.g., the least-squares loss

$$\mathcal{L}(\theta) \doteq \frac{1}{N_s} \sum_{i=1}^{N_s} |u_\theta(\boldsymbol{x}_i) - y_i|^2, \tag{7.6}$$

and we define *training* as the minimisation problem

$$\theta^* \in \arg\min_{\theta \in \mathbb{R}^P} \mathcal{L}(\theta). \tag{7.7}$$

Training is performed with gradient-based optimisation. The simplest scheme is gradient descent, which is the nonlinear analogue of the Richardson iteration of Chapter 6; the *learning rate* plays the role of the relaxation parameter. Figure 7.7 depicts the iteration on a convex loss with two parameters; the gradient is orthogonal to the level sets, and the iterates zigzag towards the minimiser.

Algorithm 7.2: Gradient descent training

Given an initial guess θ^0 (usually random) and a learning rate $\alpha > 0$
for $k = 0, 1, \dots,$ till convergence **do**
  compute the gradient $\boldsymbol{g}^k := \nabla_\theta \mathcal{L}(\theta^k)$
  update $\theta^{k+1} := \theta^k - \alpha\, \boldsymbol{g}^k$
end for

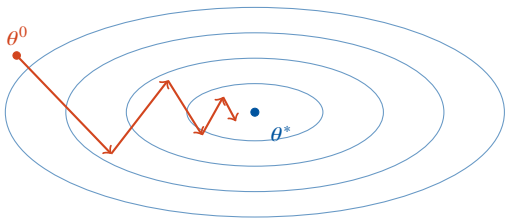
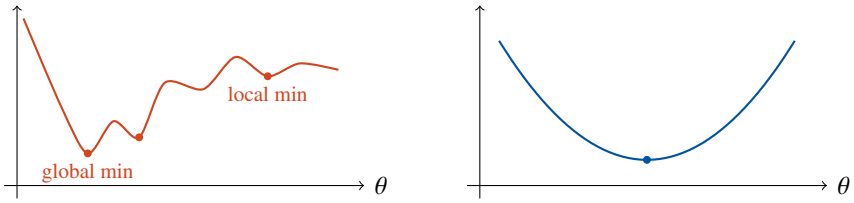


Figure 7.7: Gradient descent on a convex loss with two parameters. The blue curves are level sets of the loss; the gradient is orthogonal to them, and the iterates zigzag towards the unique minimiser θ^*. On a non-convex landscape, such as the left one in Figure 7.8, the same dynamics can instead stall at a local minimum or a saddle point.

In practice, one uses variants with adaptive step sizes and momentum, e.g., the Adam optimiser [11], possibly evaluating the loss on random subsets of the data (*stochastic* gradient descent). The gradients $\nabla_\theta \mathcal{L}$, and later the spatial derivatives $\partial_{\boldsymbol{x}} u_\theta$ appearing in PDE residuals, are computed exactly (to machine precision) by *automatic differentiation*, which propagates the chain rule through the compositions in Definition 7.2.1; see [4]. No finite differences are involved in the differentiation.

The essential difference with everything we have done in this course lies in the structure of (7.7). For the Galerkin method applied to the linear coercive problems of Chapter 1, the discrete energy is a quadratic

functional of the DOFs with a symmetric positive definite matrix, and, in particular, a smooth *convex* function of the DOFs. It has a unique global minimum, which we can compute by solving a linear system (Theorem 1.2.14). On the contrary, the loss (7.6) is a *non-convex* function of θ, because the parameters enter nonlinearly. The landscape has many local minima, saddle points and flat regions; see Figure 7.8. Gradient-based optimisers can stall at any of them, the computed solution depends on the (random) initialisation, and, in general, *no convergence guarantee to the global minimum exists*. This is the price of nonlinear approximation. The approximation power of the family and the computability of the minimiser pull in opposite directions.



nonlinear approximation | linear approximation (e.g., FEM)

Figure 7.8: Training a NN is a non-convex optimisation problem (left), to be contrasted with the convex quadratic minimisation behind the Galerkin method for coercive problems (right, see Chapter 1). Gradient-based methods can get stuck at local minima, and the result depends on the initialisation.

7.4 Physics-informed neural networks

We can now put the pieces together and approximate PDEs. Let us consider the abstract boundary value problem of Chapter 1,

$$\mathcal{L}u = f \ \text{ in } \Omega, \qquad u = u_D \ \text{ on } \Gamma_D, \qquad \boldsymbol{n} \cdot \mathcal{F}(u) = g_N \ \text{ on } \Gamma_N,$$

where $\mathcal{L}$ is a differential operator (e.g., $\mathcal{L}u = -\Delta u$) and $\mathcal{F}$ the associated flux. The idea is disarmingly simple. We take the ansatz $u_\theta \in \mathcal{N}$ and we train it to minimise the *strong residual* of the PDE, with the boundary conditions imposed by penalty,

$$\mathcal{L}(\theta) \doteq \int_\Omega (\mathcal{L}u_\theta - f)^2 \,\mathrm{d}x + \beta_D \int_{\Gamma_D} (u_\theta - u_D)^2 \,\mathrm{d}s + \beta_N \int_{\Gamma_N} (\boldsymbol{n} \cdot \mathcal{F}(u_\theta) - g_N)^2 \,\mathrm{d}s, \tag{7.8}$$

with penalty coefficients $\beta_D, \beta_N > 0$. If the boundary value problem admits a classical solution, this solution is a zero, hence a global minimiser, of this non-negative functional. We must resist the temptation to claim more. The loss measures the residuals in L^2-type norms, which are *not* the norms in which the problem is well posed; the natural setting of Chapter 1 controls the solution in $H^1(\Omega)$, and the data enter through dual and fractional norms, such as $H^{-1}(\Omega)$ for the forcing and $H^{1/2}(\Gamma_D)$ for the Dirichlet datum. No stability of the minimisers with respect to perturbations of the loss follows from this construction, and no well-posedness theory backs it [3]. From a numerical analysis standpoint, this is a naive formulation of the problem. We note that the loss involves the residual in $L^2(\Omega)$, so it is only meaningful when $\mathcal{L}u_\theta - f \in L^2(\Omega)$; for a second-order operator, this is a stronger requirement than the weak setting of Chapter 1, which only involves first derivatives. Networks with smooth activations satisfy it automatically.

Definition 7.4.1: Monte Carlo integration

Let $g : \Omega \to \mathbb{R}$ be integrable and let $\{\boldsymbol{p}_i\}_{i=1}^N$ be independent random points, uniformly distributed in Ω. The *Monte Carlo* approximation of the integral of g is

$$\int_\Omega g \,\mathrm{d}x \approx \frac{|\Omega|}{N} \sum_{i=1}^N g(\boldsymbol{p}_i).$$

The approximation is unbiased, and its root mean square error is $|\Omega|\,\sigma(g)\,N^{-1/2}$, where $\sigma(g)^2$ is the variance of g over Ω,

$$\sigma(g)^2 \doteq \frac{1}{|\Omega|}\int_\Omega (g-\bar g)^2\,\mathrm{d}x, \qquad \bar g \doteq \frac{1}{|\Omega|}\int_\Omega g\,\mathrm{d}x,$$

i.e., the mean square deviation of g from its mean value $\bar g$ over Ω; it vanishes for constant g and measures how much the integrand varies. The rate $N^{-1/2}$ is independent of the dimension d, but it is very slow; each extra digit of accuracy costs a hundred times more points.

The integrals in (7.8) cannot be computed in closed form. The PINN method [18] approximates them by Monte Carlo (Definition 7.4.1), i.e., by averaging the integrand at randomly sampled *collocation points* $\{\boldsymbol{p}_i\}_{i=1}^{N_p} \subset \Omega$ and $\{\boldsymbol{q}_i\}_{i=1}^{N_\partial} \subset \partial\Omega$,

$$\mathcal{L}_N(\theta) \doteq \frac{1}{N_p}\sum_{i=1}^{N_p} |\mathcal{L}u_\theta(\boldsymbol{p}_i) - f(\boldsymbol{p}_i)|^2 + \frac{\gamma}{N_\partial}\sum_{i=1}^{N_\partial} |u_\theta(\boldsymbol{q}_i) - u_D(\boldsymbol{q}_i)|^2\,, \tag{7.9}$$

where all the derivatives of u_θ are computed by automatic differentiation. The method is mesh-free, a few lines of code in a machine learning framework, and applies verbatim to nonlinear operators. This simplicity explains its enormous popularity. The idea itself is not new; training NNs on PDE residuals at collocation points was already proposed in the 1990s [12], and its revival [18] is due to the maturity of automatic differentiation frameworks and hardware.

Let us now examine the method with the critical eyes that this course has trained. Three independent difficulties deserve attention.

7.4.1 The integration problem

Monte Carlo integration converges as $N_p^{-1/2}$, regardless of the dimension. This robustness in high dimension is its *raison d'être*, but in the low dimensions of engineering practice it is dramatically inferior to the quadratures of Chapter 3, and it has a more insidious consequence. The discrete loss (7.9) only *feels* the residual at the sampling points. Its global minimisers need not be close to the solution at all.

Example 7.4.2: Zero loss, wrong solution

Let us consider $-u'' = 0$ in $(0,1)$ with $u(0) = 0$ and $u(1) = 1$, whose solution is $u(x) = x$, and the discrete loss

$$\mathcal{L}_N(v) = \frac{1}{N_p}\sum_{i=1}^{N_p} |v''(p_i)|^2 + |v(0)|^2 + |v(1)-1|^2$$

for fixed, distinct collocation points $p_1, \dots, p_{N_p} \in (0,1)$. Let us define, for an arbitrary amplitude $A \in \mathbb{R}$,

$$v_A(x) \doteq x + A\,x^2(1-x)^2\prod_{i=1}^{N_p}(x-p_i)^3.$$

Every v_A satisfies the boundary conditions exactly, and the perturbation and its first two derivatives vanish at every p_i (each factor $(x-p_i)^3$ has a triple zero). Thus, $\mathcal{L}_N(v_A) = 0$ for every A, while v_A is arbitrarily far from the solution as $|A|$ grows; see Figure 7.9. A sufficiently expressive network can approximate any v_A, so the discrete loss possesses near-zero-loss configurations that are wildly wrong. The continuous loss (7.8) would reject them; the failure is entirely due to the pointwise sampling of the residual.

Practical remedies include using many more points than parameters, resampling the collocation points during training, and adding points where the pointwise residual is large. None of these remedies, however,

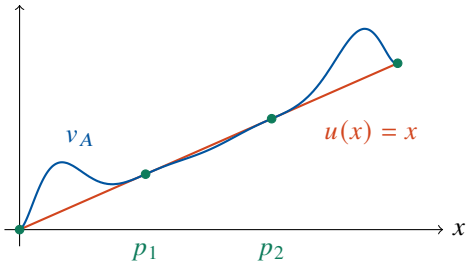


Figure 7.9: The spurious minimiser of Example 7.4.2, plotted for $N_p = 2$ collocation points $p_1 = 1/3$, $p_2 = 2/3$ and a large amplitude A. Both functions have zero discrete loss: the boundary values match, and at the collocation points the perturbation vanishes together with its first and second derivatives (triple zeros), so v_A is tangent to u there and $v_A''(p_i) = 0$.

restores the convergence guarantees and the integration error bounds that standard quadrature theory provides. More systematic routes exist, based on quadratures adapted to the network; we refer to [14, 3] for adaptive quadratures for NN losses with integration error control.

7.4.2 The boundary condition problem

In the finite element method, Dirichlet boundary conditions are imposed *strongly*, by construction of the space (Chapter 2). On the NN family there is no such handle, and the penalty terms in (7.8) are a weak surrogate with an unpleasant trade-off. A small β_D tolerates boundary violations; a large β_D makes the loss stiff and slows down training. In practice, the boundary terms are a major source of training difficulties. There is also a deeper, structural issue. The Dirichlet datum lives in the fractional trace space $H^{1/2}(\Gamma_D)$, and the natural measure of the boundary misfit is the corresponding fractional norm; dual norms appear likewise for Neumann data, in $H^{-1/2}(\Gamma_N)$. Penalising the boundary misfit in $L^2(\Gamma_D)$, as in (7.8), is not consistent with the well-posedness setting of the problem, and this norm mismatch pollutes the approximation; see [3].

An alternative is to impose the boundary conditions exactly by construction. Assume $\Omega = [0,1]^d$ and homogeneous Dirichlet conditions, and let

$$\xi(\boldsymbol{x}) \doteq \prod_{i=1}^{d} x_i(1 - x_i),$$

which vanishes on $\partial\Omega$ and is positive inside. The modified ansatz $\tilde{u}_\theta \doteq \xi\, u_\theta$ satisfies the boundary condition *strongly*, for every θ, and the boundary terms disappear from the loss (non-homogeneous data can be handled by adding a lifting of u_D, as with the offset function method of Chapter 1). Training improves markedly. The weakness of this trick is geometrical; for a general domain, a smooth, explicitly computable function that vanishes exactly on $\partial\Omega$ (and only there) is not available, and approximate distance functions bring their own errors. Strong imposition remains awkward on the NN family, in stark contrast with finite element spaces.

7.4.3 An honest assessment

It is natural to ask how PINNs compare with the methods of this course on the problems of this course, i.e., forward, low-dimensional PDEs. The literature contains thousands of reported success stories, but it is hard to find any that survives a fair comparison, and the reader should treat comparisons against weak or misconfigured baselines with suspicion. An instructive account is the experience report [5], in which a PINN required about 32 hours of training to match the accuracy that a standard solver delivered in about 20 seconds on a 2D flow benchmark, and failed to reproduce the physics (vortex shedding) on a second one. There is no contradiction with the theory we have developed. For forward low-dimensional problems, the Galerkin method solves a convex problem with a guaranteed error, optimal complexity and decades of numerical maturity; a PINN attacks a harder (non-convex) optimisation problem to approximate the same solution, with inexact integration and penalised boundary conditions, and without error control. To this day,

no one has convincingly demonstrated the effectiveness of NN approaches for the forward approximation of low-dimensional PDEs. The situation is somewhat better for inverse problems. The potential interest of NN discretisations lies elsewhere; in high-dimensional and parametric problems, where linear methods face the curse of dimensionality (7.1), and in *inverse* problems, where the flexibility of the loss and the differentiability of the whole pipeline help. We come back to the latter in Section 7.6.

7.5 Variational losses

The strong residual in (7.8) is only one possible loss. The variational structure developed in Chapter 1 suggests weaker, and often better behaved, alternatives.

The most elegant one is the *deep Ritz method* [8]. For a symmetric coercive problem, Chapter 1 characterises the solution as the unique global minimiser of the energy $J(v) = \frac{1}{2}a(v,v) - \ell(v)$ over the space V. The deep Ritz method simply minimises the *same* energy over the NN family,

$$\min_{\theta\in\mathbb{R}^P} J(u_\theta) + \beta_D \int_{\Gamma_D} (u_\theta - u_D)^2 \,\mathrm{d}s. \tag{7.10}$$

This is precisely the quadratic minimisation problem of Definition 1.2.2 posed on $\mathcal{N}$ instead of a vector space. The energy only involves first derivatives, so the regularity demanded of the network is milder than for the strong residual. Two warnings from Chapter 1 carry over with new force. First, J restricted to the family is not convex in θ, so we are back to non-convex optimisation. Second, existence of a minimiser over $\mathcal{N}$ is not guaranteed; the family is not a closed subspace, and minimising sequences of networks can degenerate (weights blowing up while the realisations converge to something outside $\mathcal{N}$), in the same spirit as the non-existence example of Chapter 1. Infima need not be attained. There is a third annoyance, of a more practical kind. Unlike the residual loss (7.8), the energy is not signed; its minimum value is negative and unknown a priori. During training, the loss value therefore carries no information about the quality of the approximation, and it cannot be used as a stopping criterion or as an error indicator [3].

Other members of this family include the deep Galerkin method [19], variational PINNs, which test the weak residual against a finite set of test functions [10], and the deep Nitsche method, which imposes Dirichlet conditions weakly through a Nitsche-type variational form [13]. Minimal-residual formulations measure the weak residual in a dual norm, whose evaluation requires a supremum over test functions; realising the test space by a second, adversarially trained network is possible, but it further increases the number of parameters and the brittleness of training. For coercive symmetric problems, the test function can be taken equal to the trial one, and one recovers energy minimisation.

7.5.1 A one-parameter tour of the loss landscape

The following example, adapted from [14], shows that the pathologies of training appear already with a *single* parameter. Let us consider the Poisson problem $u'' = f''$ on $\Omega = (-1,1)$ with $u = f$ on the boundary and manufactured solution $f = \mathrm{ReLU}$. As a model, we take the one-parameter, one-neuron network

$$u_c(x) \doteq \mathrm{ReLU}(x + c), \qquad c \in \mathbb{R},$$

so that the exact solution corresponds to $c = 0$. As a loss, we take the deep Ritz energy (7.10), i.e., the energy of Chapter 1 with the Dirichlet conditions imposed by a penalty term. For a Poisson problem $-v'' = \tilde{f}$ in $(-1,1)$ with Dirichlet conditions $v(\pm 1) = g(\pm 1)$, it reads

$$\mathcal{J}(v) \doteq \frac{1}{2}\int_{-1}^{1} (v')^2 \,\mathrm{d}x - \int_{-1}^{1} \tilde{f}\, v \,\mathrm{d}x + \frac{\beta}{2} \sum_{b\in\{-1,1\}} (v(b) - g(b))^2, \qquad \beta > 0.$$

The first two terms are the energy of Chapter 1, and the last one penalises the violation of the Dirichlet conditions, weighted by β. We note that $\mathcal{J}$ is bounded from below and convex in v, as the energies of

Chapter 1. For our data, f'' is a Dirac delta at the origin, so that $-\int \tilde{f}\, v = v(0)$, and a direct computation gives

$$\mathcal{J}(c) = \begin{cases} \frac{\beta}{2} & \text{if } c < -1, \\ \frac{1}{2}\left(\beta c^2 + c + 1\right) + \max(c, 0) & \text{if } -1 \le c \le 1, \\ 1 + c + \frac{\beta}{2}\left((c-1)^2 + c^2\right) & \text{if } c > 1. \end{cases}$$

On $[-1, 0]$ the energy is a parabola with vertex at $c^* = -\frac{1}{2\beta}$, and on $[0, 1]$ it is increasing, so c^* is the global minimiser (for $\beta > \frac{1}{2}$). This tiny landscape already exhibits several diseases. First, $\mathcal{J}$ is *constant* for $c < -1$; the kink of the network leaves the domain, the model becomes identically zero on Ω, and the loss is blind to c (a flat region). The gradient vanishes there, so an unlucky initialisation ($c < -1$) never recovers; the useful basin of attraction is only part of the parameter line. Second, the global minimiser is *not* the exact solution; $c^* = -\frac{1}{2\beta} \neq 0$. The penalty term makes the loss *inconsistent*; the exact solution is not a stationary point of $\mathcal{J}$ for any finite β, only in the limit $\beta \to \infty$, and the error of the minimiser is $O(1/\beta)$. Third, to reduce the consistency error we would like to take $\beta \to \infty$, but the curvature of $\mathcal{J}$ around the minimum is β, so gradient descent $c \mapsto c - \alpha \mathcal{J}'(c)$ converges only for $\alpha\beta < 2$; a larger β forces smaller learning rates and more iterations. This is the trade-off of weakly imposed boundary conditions discussed in Section 7.4.2. Fourth, the gradient of $\mathcal{J}$ *jumps* at $c = 0$ (from $\frac{1}{2}$ to $\frac{3}{2}$), because ReLU is not differentiable at the origin. Here the kink is harmless, since it does not sit at the minimum, but this is a coincidence of the penalty formulation; with a consistent loss, which moves the minimum to $c = 0$, the kink sits at the minimum and gradient descent does not converge at all, hopping between the two sides of the kink forever (see [14]). With a full network, these effects multiply across thousands of parameters.

The cure for the last disease is regularity. If the activation function is C^1, the derivative and the integral signs can be interchanged, and if it is C^2, the gradient of a loss involving first spatial derivatives is continuous. This is why PDE approximation with NNs is performed in practice with smooth activations (tanh and relatives), or with smoothed versions of ReLU [14], even though the finite element connection of Section 7.2.1 is a ReLU story.

7.6 Inverse problems

We close the chapter with the setting in which NN discretisations are most attractive. In a *forward* problem, the data (coefficients, forcing, boundary values) are known and the solution is sought. In an *inverse* problem, part of the data is unknown, and we are given instead partial observations of the solution. A model example is the identification of a diffusion coefficient,

$$-\nabla \cdot (k \nabla u) = f \ \text{ in } \Omega, \qquad u = u_D \ \text{ on } \partial\Omega, \qquad \mathcal{M}(u) = \boldsymbol{d},$$

where k is unknown, $\mathcal{M}$ is an *observation operator* (e.g., the values of u at a few sensor locations) and $\boldsymbol{d}$ is the measured data. Such problems are typically ill-posed (many coefficients can explain the same few observations) and are regularised by adding prior information.

7.6.1 PDE-constrained optimisation

The classical computational approach is PDE-constrained optimisation. One minimises the misfit $\|\boldsymbol{d} - \mathcal{M}(u(k))\|^2$ (plus regularisation) over the coefficient k, where $u(k)$ denotes the finite element solution of the forward problem for that coefficient. Let us state the problem in matrix form, after a finite element discretisation of both fields. Let $\boldsymbol{k} \in \mathbb{R}^m$ collect the DOFs of the coefficient and $\boldsymbol{u} \in \mathbb{R}^N$ those of the state, linked by the discrete forward problem $A(\boldsymbol{k})\boldsymbol{u} = \boldsymbol{f}$, where $A(\boldsymbol{k})$ is the stiffness matrix for the coefficient $\boldsymbol{k}$. For linear observations, $\mathcal{M}(u) = M\boldsymbol{u}$, we seek

$$\boldsymbol{k}^* \in \operatorname*{argmin}_{\boldsymbol{k} \in \mathbb{R}^m} G(\boldsymbol{k}), \qquad G(\boldsymbol{k}) \doteq \frac{1}{2}\left|\boldsymbol{d} - M\boldsymbol{u}(\boldsymbol{k})\right|^2 + \frac{\gamma}{2}\left|R\boldsymbol{k}\right|^2,$$

where the second term is a *regularisation* term with weight $\gamma > 0$ (e.g., R a discrete gradient), which penalises irregular coefficients and compensates the ill-posedness of the problem. The map $\boldsymbol{k} \mapsto \boldsymbol{u}(\boldsymbol{k}) = A(\boldsymbol{k})^{-1}\boldsymbol{f}$ is nonlinear, so G is non-convex; non-convexity is intrinsic to inverse problems, it is not brought in by NNs. The gradient of G follows from the chain rule and the identity $\partial \boldsymbol{u}/\partial k_j = -A^{-1}(\partial A/\partial k_j)\boldsymbol{u}$,

$$\frac{\partial G}{\partial k_j} = \boldsymbol{z}^T \frac{\partial A}{\partial k_j}\boldsymbol{u} + \gamma \left(R^T R \boldsymbol{k}\right)_j, \qquad \text{with } A(\boldsymbol{k})^T \boldsymbol{z} = M^T (\boldsymbol{d} - M\boldsymbol{u}).$$

The auxiliary vector $\boldsymbol{z}$ is the *adjoint state*. It solves one linear system with the *transpose* of the stiffness matrix (the same matrix, for our symmetric problem) and the observation misfit as right-hand side. One gradient evaluation thus costs one forward and one adjoint solve, independently of the number m of coefficient DOFs. The result is a *two-loop* algorithm (when using iterative linear solvers); an optimisation outer loop around two linear-solve inner loops. It is accurate and well understood, but each outer iteration is expensive.

7.6.2 Neural network parametrisations

We saw in Section 7.3 that fitting data is the native task of NNs, and the misfit term above is nothing but a data-fitting loss constrained by physics. This makes inverse problems a natural candidate for the neural framework. The NN formulation collapses the two loops into one. We parameterise *both* unknown fields with networks, u_{θ_u} and k_{θ_k}, and minimise a single loss combining all the information we have,

$$\mathcal{L}(\theta_u, \theta_k) \doteq \left|\boldsymbol{d} - \mathcal{M}(u_{\theta_u})\right|^2 + w_R \left\|\boldsymbol{\nabla} \cdot (k_{\theta_k} \boldsymbol{\nabla} u_{\theta_u}) + f\right\|^2 + w_D \left\|u_{\theta_u} - u_D\right\|^2_{\partial\Omega}, \tag{7.11}$$

with weights $w_R, w_D > 0$. The residual term in (7.11) is the strong PINN-type choice, adopted here for concreteness; any formulation of this chapter, e.g., e.g., the deep Ritz energy, can play this role. The PDE is no longer solved at each step; it is a *soft constraint*, satisfied increasingly well as training proceeds, while the observations pull the pair (u, k) towards the measurements. Training is harder than for a forward problem (the loss couples two networks and the constraint is enforced only approximately), but the marginal cost of going from forward to inverse is small; one more network and one more term in the loss. This is the opposite of the classical situation, in which the inverse problem is dramatically more expensive than the forward one.

There is a second, more subtle reason why NNs fit inverse problems well. One could try the same one-loop trick with finite element parametrisations of u and k. It performs poorly. The DOFs of a finite element function are *local*; a sensor only constrains the few basis functions whose support contains it, and away from the sensors the coefficient is essentially unconstrained and the reconstruction is wildly irregular. The parameters of a NN are *global*; every weight influences the realisation everywhere, and the architecture itself acts as an implicit smoothness prior that regularises the reconstruction between observations. Interpolating the networks onto finite element spaces, so that the residual in (7.11) becomes a finite element residual with exact integration and strong boundary conditions, further improves the reconstruction [2].

7.7 Summary

Let us summarise the chapter. The methods of this course are linear approximation methods; their spaces are chosen a priori, which caps their efficiency for multiscale solutions and dooms them in high dimension. NNs are nonlinear parametrisations that contain finite element spaces (exactly for ReLU activations, with enough depth in several dimensions) and can adapt to the solution, playing the role of an adaptive mesh without a mesh. The price is steep; training is a non-convex optimisation problem without guarantees, the loss integrals must be approximated (and naive sampling admits spurious minimisers), and essential boundary conditions lack a natural strong imposition. For forward low-dimensional problems, these drawbacks make NNs uncompetitive against the finite element method. The clearest added value of the neural framework is in inverse problems, where a single training loop replaces the classical nested optimisation, and the global, smooth parametrisation regularises the reconstruction.

The field is not that young anymore. The current wave is about a decade old, and the idea dates back to the 1990s; thousands of researchers and tens of thousands of papers later, these methods remain of

dubious practical value for forward PDE approximation, and one will not find them behind real engineering applications. NNs excel in data-rich problems, e.g., weather prediction, where they have been transformative; they struggle in the data-scarce regimes where PDE models are most valuable, e.g., climate projection. Machine learning has been sold as a hammer for every scientific nail, and the approximation of PDEs does not appear to be one of them. Will this change in the future? Perhaps, but I would not bet on ever-larger ML-style architectures. Progress will likely require departing from them, towards nonlinear approximations designed for PDEs from the start, with better integration, better-posed minimisation and more interpretable architectures, e.g., the free-knot spline methods of [15].

7.8 Tutorial

1. Express $|x|$, $\max(x, y)$ and $\min(x, y)$ as NN realisations with activation ReLU, specifying an architecture and the values of all weights and biases for each. (Hint; start from $|x| = \mathrm{ReLU}(x) + \mathrm{ReLU}(-x)$ and $\max(x, y) = y + \mathrm{ReLU}(x - y)$.)

2. Consider the quadratic finite element space V_h^2 on a uniform mesh of $[0, 1]$ with M cells. Can its functions be represented *exactly* by ReLU networks? And by networks with activation $\rho(x) = \mathrm{ReLU}(x)^2$? (Hint; determine the regularity of the realisations in each case, and compare it with the regularity of V_h^2 functions.)

3. Show that the set $\mathcal{N}$ in (7.2) is not a vector space. More precisely, show that it is closed under multiplication by scalars, and exhibit two realisations of the architecture $(1, 1, 1)$ with activation ReLU whose sum is not a realisation of $(1, 1, 1)$, but is one of $(1, 2, 1)$. Generalise; the sum of two realisations of an architecture with hidden widths $(n_1, \dots, n_{L-1})$ is a realisation of the architecture with hidden widths $(2n_1, \dots, 2n_{L-1})$.

4. In Example 7.4.2, verify explicitly that the perturbation $\delta(x) \doteq x^2(1 - x)^2 \prod_i (x - p_i)^3$ satisfies $\delta(0) = \delta(1) = 0$ and $\delta''(p_i) = 0$ for every collocation point. Suppose now that the collocation points are resampled at every training iteration, uniformly at random in $(0, 1)$. Explain qualitatively why the spurious minimisers of Example 7.4.2 are no longer stationary configurations of the training process.

5. Consider a symmetric coercive problem with energy $J(v) = \frac{1}{2}a(v, v) - \ell(v)$, and a ReLU architecture $\mathcal{N}$ that emulates the finite element space V_h as in Theorem 7.2.3 (with homogeneous Dirichlet conditions built into the space). Show that $\inf_{v \in \mathcal{N}} J(v) \le J(u_h)$, where u_h is the Galerkin solution in $V_{h,0}$, and combine this bound with Theorem 2.4.2 to conclude that an exact minimiser of the deep Ritz loss over $\mathcal{N}$, if it exists, satisfies an energy-norm error estimate no worse than that of the finite element method.

6. (Continuation of Exercise 2.) Construct the analogue of the representation (7.4) for C^1 piecewise quadratic functions using neurons with activation $\rho(x) = \mathrm{ReLU}(x)^2$, and count the neurons. Explain which additional neurons are needed to capture the derivative jumps of Lagrange quadratic elements.

Bibliography

[1] R. Arora, A. Basu, P. Mianjy, and A. Mukherjee. Understanding deep neural networks with rectified linear units. In *International Conference on Learning Representations (ICLR)*, 2018.

[2] S. Badia, W. Li, and A. F. Martín. Finite element interpolated neural networks for solving forward and inverse problems. *Computer Methods in Applied Mechanics and Engineering*, 418:116505, 2023.

[3] S. Badia and K. Nori. Adaptive anisotropic composite quadratures for residual minimisation in neural PDE approximations. *arXiv preprint arXiv:2605.00308*, 2026.

[4] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[5] P.-Y. Chuang and L. A. Barba. Experience report of physics-informed neural networks in fluid simulations: pitfalls and frustration. *arXiv preprint arXiv:2205.14249*, 2022.

[6] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[7] R. A. DeVore. Nonlinear approximation. *Acta Numerica*, 7:51–150, 1998.

[8] W. E and B. Yu. The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018.

[9] J. He, L. Li, J. Xu, and C. Zheng. ReLU deep neural networks and linear finite elements. *Journal of Computational Mathematics*, 38(3):502–527, 2020.

[10] E. Kharazmi, Z. Zhang, and G. E. Karniadakis. hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, 374:113547, 2021.

[11] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[12] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[13] Y. Liao and P. Ming. Deep Nitsche method: Deep Ritz method with essential boundary conditions. *Communications in Computational Physics*, 29(5):1365–1384, 2021.

[14] A. Magueresse and S. Badia. Adaptive quadratures for nonlinear approximation of low-dimensional PDEs using smooth neural networks. *Computers & Mathematics with Applications*, 162:1–21, 2024.

[15] A. Magueresse and S. Badia. Energy minimisation using overlapping tensor-product free-knot B-splines. *arXiv preprint arXiv:2508.17705*, 2025.

[16] R. H. Nochetto, K. G. Siebert, and A. Veeser. Theory of adaptive finite element methods: An introduction. In *Multiscale, Nonlinear and Adaptive Approximation*, pages 409–542. Springer, 2009.

[17] A. Pinkus. Approximation theory of the MLP model in neural networks. *Acta Numerica*, 8:143–195, 1999.

[18] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[19] J. Sirignano and K. Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018.

[20] R. Verfürth. *A Posteriori Error Estimation Techniques for Finite Element Methods*. Oxford University Press, 2013.